CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

🚀 Unidad 1. Introducción a la algorítmica y pseudocódigo

Fundamentos del pensamiento computacional, arquitectura y ciclo de vida del software, evolución de los lenguajes, paradigmas de programación, diagramas de flujo ANSI, pseudocódigo estricto con PSeInt, modularidad y trazas de ejecución paso a paso.



MÓDULO PROFESIONAL
Programación (0485) — 1º DAM

RESULTADOS DE APRENDIZAJE
RA1 (Fundamentos del Software y Algorítmica)

DURACIÓN ESTIMADA
8 horas lectivas

ENTORNO OFICIAL
IntelliJ IDEA / PSeInt / Java LTS

¡Te damos la bienvenida al módulo profesional de **Programación (código 0485)** del CFGS de Desarrollo de Aplicaciones Multiplataforma (DAM)!

Antes de abrir el entorno de desarrollo IntelliJ IDEA y sumergirnos en la sintaxis de Java, es imperativo construir la habilidad más determinante en la carrera de cualquier desarrollador de software: el **pensamiento algorítmico**. Un ordenador actual es una máquina prodigiosamente rápida, capaz de realizar miles de millones de operaciones por segundo sin fatigarse, pero carece por completo de intuición, criterio o sentido común. La computadora hará con absoluta fidelidad y a la velocidad de la luz exactamente lo que le indiques; si diseñas una solución lógica defectuosa, la máquina ejecutará tu error sin pestañear.

En esta primera unidad didáctica aprenderás a pensar de manera estructurada, analítica y modular, transformando problemas del mundo real en algoritmos rigurosos, verificables e independientes del lenguaje final.

> 📥 **Recursos y Descargas en PDF de la Unidad 1**
>
> Puedes estudiar esta unidad tanto en la plataforma web como en formato documento digital/impreso:
>
> - 📄 **Descargar Teoría Completa de la Unidad 1 en PDF** (25 páginas)
> - 📝 **Boletín Inicial:** Descargar Propuestos (PDF) · Descargar Resueltos (PDF)
> - 📝 **Boletín Intermedio:** Descargar Propuestos (PDF) · Descargar Resueltos (PDF)
> - 🧠 **Retos y Extras:** Descargar Retos (PDF)

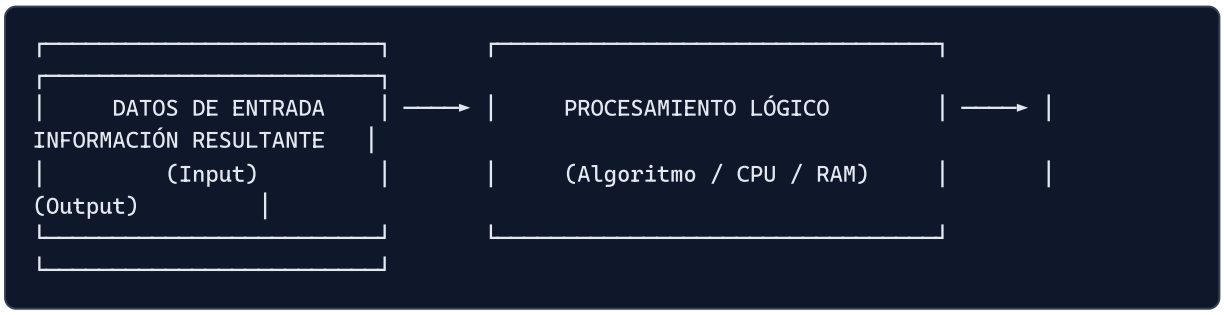
---

1. Fundamentos: Programas, Procesos y Tipología del Software

1.1 El Enfoque de Caja Negra (*Black Box*)

La Real Academia Española define un programa como un *"conjunto de instrucciones que permite a un ordenador realizar funciones diversas"*. Dicho de forma más intuitiva: un programa es una **receta** que un ordenador sigue al pie de la letra, sin criterio propio. Hace exactamente lo que le indicas, ni más ni menos (y esa fidelidad ciega es la causa de que los errores o *bugs* existan).

Desde el punto de vista de la ingeniería informática, cualquier programa puede modelarse conceptualmente como una **caja negra**: un sistema cerrado del cual nos interesa qué entra, qué sale y qué transformaciones realiza, aislando temporalmente la complejidad interna.



1. **Entrada (Input):** Datos iniciales no procesados procedentes de periféricos (teclado, ratón), medios de almacenamiento secundario (ficheros, bases de datos) o interfaces de red (llamadas a APIs, sockets).
2. **Procesamiento:** Secuencia lógica y ordenada de instrucciones que transforman, comparan, calculan o combinan los datos recibidos haciendo uso de la memoria principal (RAM) y la Unidad Central de Procesamiento (CPU).

3. **Salida (Output):** Información útil estructurada y comunicada al usuario a través de periféricos de salida (pantalla, altavoces), persistida en disco o transmitida telemáticamente a otro sistema.

1.2 Distinción Conceptual Rigurosa

En el argot profesional a menudo se confunden términos que tienen matices técnicos muy concretos:

- **Algoritmo:** Procedimiento computacional paso a paso, formal y abstracto, que toma un conjunto de valores como entrada y produce un resultado como salida. Es completamente independiente del hardware y del lenguaje de programación.
- **Programa:** Codificación concreta de uno o varios algoritmos en un lenguaje de programación formal (Java, C, Python), almacenada como texto fuente legible o compilada en disco.
- **Proceso:** Instancia de un programa en ejecución en la memoria RAM del ordenador, con su propio espacio de direcciones, contador de programa, registros y recursos concedidos por el Sistema Operativo.
- **Aplicación / Software:** Sistema informático integral formado por múltiples programas, bibliotecas (*frameworks* / *APIs*), interfaces gráficas o de consola, ficheros de configuración, recursos multimedia y documentación técnica.

1.3 Clasificación del Software según su Entorno de Ejecución

En el ecosistema profesional actual, el software que desarrollamos adopta diversas arquitecturas según las necesidades del usuario final:

Tipo de Aplicación	Descripción y Características	Ejemplos Habituales
Aplicaciones de Consola (*CLI / Terminal*)	Interactúan exclusivamente en modo texto a través de la terminal o símbolo del sistema, sin interfaz gráfica de ventanas. Es el formato idóneo para iniciarse en la programación y centrarse en la lógica pura sin distracciones visuales (el punto de partida de este curso).	Comandos del sistema operativo, scripts de administración, utilidades de diagnóstico.
Aplicaciones de Escritorio (*Desktop / GUI*)	Se instalan y ejecutan localmente en el sistema operativo del usuario (Windows, Linux, macOS) ofreciendo ventanas, botones y menús gráficos interactivos. Tienen acceso directo al hardware, máxima fluidez y pueden funcionar desconectadas de la red.	IntelliJ IDEA, LibreOffice, Photoshop, VLC Media Player.
Aplicaciones Web	Funcionan bajo la arquitectura cliente-servidor. El cliente (navegador web) ejecuta la interfaz (HTML, CSS, JavaScript) mientras que la lógica de negocio y los datos residen en servidores remotos (*backend*).	Gmail, Spotify Web, plataformas bancarias en línea.
Aplicaciones Móviles	Diseñadas específicamente para dispositivos móviles (teléfonos, tablets) bajo Android o iOS. Pueden ser nativas (Java/Kotlin, Swift) o multiplataforma (Flutter, React Native).	WhatsApp, Instagram, Google Maps móvil.
Servicios / Demonios (*Daemons / Background Services*)	Procesos que se ejecutan en segundo plano de manera desatendida, sin interacción directa con el usuario, procesando tareas programadas o eventos de sistema.	Servicios de sincronización en la nube, indexadores de búsqueda, servidores web en segundo plano.
Sistemas Embebidos / Empotrados (*Embedded*)	Software integrado en dispositivos de hardware dedicados con recursos de memoria y procesador muy restringidos y requisitos de tiempo real estricto.	Centralitas de automóviles (ECU), microondas, routers domésticos, domótica IoT.
Servicios Web y APIs REST	Aplicaciones sin interfaz visual que ofrecen servicios de procesamiento o intercambio de datos a otras aplicaciones a través de protocolos de red (JSON, XML).	Pasarelas de pago (Stripe, PayPal), APIs meteorológicas.

---

2. Evolución Histórica de los Lenguajes de Programación

Para comunicarnos con un procesador electrónico debemos salvar una enorme distancia conceptual: los seres humanos pensamos con lenguaje natural, conceptos abstractos y ambigüedades, mientras que los circuitos integrados de una CPU solo comprenden transistores encendidos o apagados (diferencias de potencial eléctrico representadas como `1` y `0`).

A lo largo de la historia de la computación, los lenguajes han evolucionado a través de cinco generaciones (GL: *Generation Language*), incrementando progresivamente su nivel de abstracción:

```
Nivel de Abstracción Humana
 ▲
 |  5GL: Lenguajes basados en Inteligencia Artificial y Reglas Lógicas (Prolog,
Mercury)
 |  4GL: Lenguajes Declarativos orientados a Datos y Consultas (SQL, R)
 |  3GL: Lenguajes de Alto Nivel Estructurados y Orientados a Objetos (Java, C, C++,
C#, Python)
 |  2GL: Lenguajes Ensambladores (Assembly x86, ARM)
 |  1GL: Código Máquina Binario (Instrucciones en ceros y unos directas para la CPU)
 ▼
Nivel de Hardware / Circuitos Físicos
```

2.1 Primera Generación (1GL): Lenguaje Máquina

- Las primeras computadoras de propósito único (como el COLOSSUS en la Segunda Guerra Mundial) se "programaban" modificando manualmente cables e interruptores en paneles físicos.
- Con la llegada de la arquitectura Von Neumann (ENIAC), las instrucciones pasaron a residir en memoria como cadenas binarias de ceros y unos (`0` y `1`).
- Cada instrucción codifica directamente la operación del microprocesador (*OpCode*) y las direcciones físicas de memoria de los operandos.
- **Inconvenientes:** Extremadamente difícil de programar, nula legibilidad para humanos, facilidad pasmosa para cometer errores irreversibles y dependencia total de la arquitectura del chip: un programa máquina escrito para Intel x86 no funciona en procesadores ARM ni PowerPC.

2.2 Segunda Generación (2GL): Lenguaje Ensamblador (*Assembler*)

- Sustituye las cadenas binarias por códigos nemotécnicos legibles por humanos (por ejemplo, `MOV AX, 1` para mover un dato a un registro o `ADD AX, BX` para sumar).
- Se introduce el programa **Ensamblador**, encargado de traducir el código nemotécnico a código máquina puro.
- **Inconvenientes:** Aunque mejora la legibilidad, sigue estando estrechamente acoplado a los registros físicos y a la arquitectura interna de un procesador específico. Escribir algoritmos matemáticos complejos requiere cientos de instrucciones de bajo nivel.

2.3 Tercera Generación (3GL): Lenguajes de Alto Nivel

- Nacen a finales de los años 50 con la aparición de **FORTRAN** (1957, considerado el primer lenguaje de alto nivel para cálculo científico), seguido por **LISP** (1958, cálculo simbólico e IA inicial), **COBOL** (1960, aplicaciones de gestión empresarial), **BASIC** (1964, diseñado con fines pedagógicos), **C** (1972, Dennis Ritchie en los Laboratorios Bell) y **Pascal** (Niklaus Wirth, concebido para la enseñanza rigurosa de la algorítmica).
- Más adelante emergen los pioneros de la orientación a objetos: **Simula** (1967, primer lenguaje con noción de clase y objeto), **C++** (Bjarne Stroustrup, 1983), **Java** (James Gosling / Sun Microsystems, 1995) y **C#** (Microsoft, 2000).
- Emplean estructuras lógicas comprensibles, palabras en inglés (`if` , `while` , `class`) y notación matemática estándar (`total = precio * 1.21`).
- **Ventaja colosal:** Son **portables**; un mismo algoritmo puede ejecutarse en múltiples plataformas hardware sin reescribir la lógica, siempre que exista un compilador o intérprete adaptado al sistema de destino.

> **El mismo "Hola, mundo" a través de la historia**
>
> Compara cómo ha cambiado la forma de expresar la misma tarea elemental según la época y el nivel de abstracción del lenguaje:
>
> ▶ Ensamblador (Intel 8086 con DOS int 21h)
> ▶ FORTRAN (1957)
> ▶ COBOL (1960)
> ▶ C (1972)
> ▶ Pascal
> ▶ Java (1995)
> ▶ Python
>
> Fíjate en la clara tendencia evolutiva: cada nueva generación de lenguajes necesita **menos líneas accesorias** y su código se aproxima **mucho más al lenguaje humano** para alcanzar exactamente el mismo resultado.

2.4 Cuarta Generación (4GL) y Quinta Generación (5GL)

- **4GL (Lenguajes Declarativos):** Se centran en indicar **QUÉ** resultado se desea obtener y no **CÓMO** calcularlo paso a paso. El ejemplo paradigmático es **SQL** (*Structured Query Language*), donde especificamos qué registros deseamos recuperar de una base de datos y el motor optimizador decide la mejor estrategia interna de acceso.
- **5GL (Inteligencia Artificial y Resolución Lógica):** Lenguajes basados en restricciones, hechos y motores de inferencia lógica formal (como **Prolog**), donde el programador suministra las reglas del dominio y el motor resuelve problemas deductivos.

---

3. Traducción de Programas: Compilación, Interpretación y la Máquina Virtual de Java

Un ordenador no puede ejecutar directamente texto fuente de alto nivel (`.java`, `.c`, `.py`). Debe someterse a un proceso de traducción técnica. Existen tres filosofías principales:

3.1 El Modelo Compilado (Tradicional)

Un programa especializado denominado **compilador** analiza todo el código fuente en bloque antes de su puesta en marcha:

1. **Análisis léxico, sintáctico y semántico:** Comprueba que el código cumpla escrupulosamente las normas gramaticales del lenguaje y no existan incongruencias de tipos.
2. **Generación de código objeto:** Transforma el código fuente en código máquina intermedio (`.obj` u `.o`).
3. **Enlazado (*Linking*):** El enlazador (*linker*) combina el código objeto con las bibliotecas estándar del sistema para producir un único fichero binario ejecutable (`.exe` en Windows, binario ELF en Linux).

- **Ventaja:** Máxima velocidad de ejecución nativa en tiempo de producción.
- **Inconveniente:** Si compilas en Windows para arquitectura x64, el ejecutable generado no funcionará en Linux ni en una CPU ARM sin recompilar todo el proyecto. Lenguajes típicos: C, C++, Rust, Go.

3.2 El Modelo Interpretado

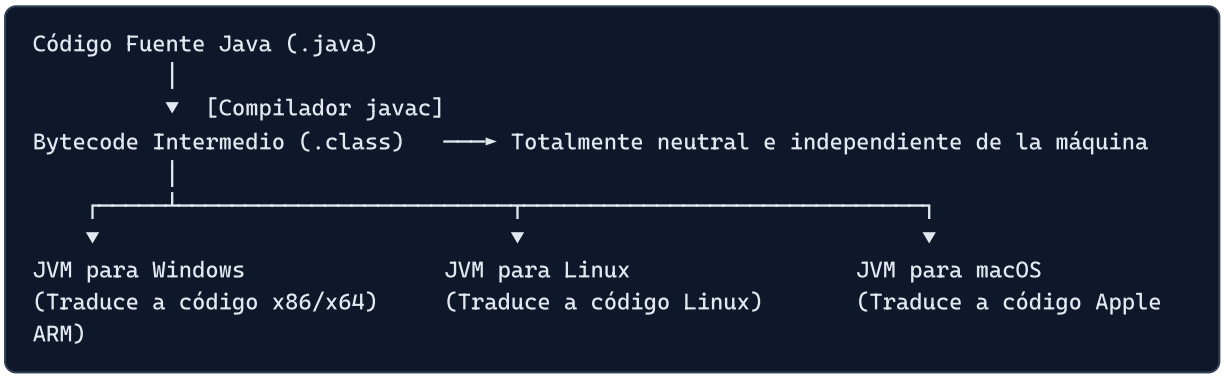
No existe un paso previo de generación de binario ejecutable. Un programa llamado **intérprete** lee el código fuente línea por línea, analiza su sintaxis y la traduce inmediatamente a órdenes del procesador sobre la marcha:

- **Ventaja:** Gran flexibilidad y ciclo interactivo rápido; ideal para scripts y pruebas dinámicas.
- **Inconveniente:** La ejecución es significativamente más lenta, ya que la traducción se realiza en tiempo real en cada ejecución. Si existe un error sintáctico en la línea 500, el programa fallará tras haber ejecutado las 499 anteriores. Lenguajes típicos: JavaScript en navegadores, PHP, Python puro.

3.3 El Modelo Híbrido de Java: Bytecode y la Máquina Virtual (JVM)

Los creadores de Java (James Gosling y Sun Microsystems) revolucionaron la industria del software en 1995 introduciendo una solución intermedia que resuelve el dilema entre velocidad y portabilidad bajo el lema:

> *"Write Once, Run Anywhere" (Escribe una vez, ejecuta en cualquier parte).*



1. El compilador de Java (`javac`) no genera código máquina nativo para un procesador físico, sino **Bytecode**: un conjunto de instrucciones de muy bajo nivel optimizadas para una computadora conceptual e ideal: la **Máquina Virtual de Java (JVM)**.
2. Cada sistema operativo (Windows, Linux, macOS) cuenta con su propia implementación de la JVM.
3. La JVM carga el fichero `.class`, verifica su seguridad y lo ejecuta mediante un motor mixto que combina interpretación inicial con un sofisticado compilador en tiempo de ejecución denominado **JIT (*Just-In-Time Compiler*)**, el cual detecta los fragmentos de código más ejecutados (*hotspots*) y los convierte en código máquina ultrarrápido idéntico al de un lenguaje compilado nativo.

3.4 Las Herramientas del Desarrollador

Para transformar una idea abstracta en una aplicación plenamente operativa y libre de errores, un programador recurre a un ecosistema de herramientas especializadas:

- **Editor de código fuente:** Aplicación de texto avanzada con coloreado sintáctico, autocompletado inteligente de palabras clave, indentación automática y detección temprana de erratas.
- **Compilador / Intérprete:** El motor de traducción que convierte el código de alto nivel en binario nativo o bytecode.
- **Depurador (*Debugger*):** Herramienta crucial que permite detener la ejecución en cualquier punto (*breakpoints* o puntos de interrupción), avanzar instrucción por instrucción inspeccionando el contenido exacto de las variables en la memoria RAM y verificar el flujo del programa en tiempo real.
- **Entorno de Desarrollo Integrado (IDE - *Integrated Development Environment*):** Una suite profesional unificada que aglutina editor, compilador, depurador, herramientas de refactorización de código, gestor de dependencias, terminal integrada y control de versiones (Git) bajo una misma interfaz de trabajo.

> **Nuestro IDE Oficial: IntelliJ IDEA**
> En este curso utilizaremos **IntelliJ IDEA** como IDE oficial. Todas las explicaciones prácticas, capturas de pantalla, navegación por menús, atajos de teclado, refactorizaciones y sesiones de depuración (*debugging*) estarán basadas exclusivamente en IntelliJ IDEA. Es una de las herramientas profesionales más potentes y demandadas en la industria del software empresarial internacional.

---

4. Paradigmas de Programación

Un paradigma de programación es un enfoque metodológico, un estilo fundamental o una filosofía para estructurar y conceptualizar la resolución de problemas mediante software:

4.1 Paradigma Imperativo / Estructurado

Concibe el programa como una serie secuencial de instrucciones que van alterando progresivamente el estado global de la memoria (variables), describiendo al milímetro **CÓMO** resolver el problema.

- **Programación Desestructurada (Histórica):** Predominaba en los primeros lenguajes comerciales. Carecía de bloques formales y abusaba de la instrucción de salto incondicional `GOTO`. Conforme el código crecía, el flujo saltaba erráticamente de una línea a otra, creando el temido **código espagueti**, imposible de mantener o depurar:

```
10 SUMA = 0
20 PRINT "Introduce un número positivo para sumar (negativo para salir):"
30 INPUT N
40 IF N < 0 THEN GOTO 70
50 SUMA = SUMA + N
60 GOTO 20
70 PRINT "La suma acumulada es: " SUMA
```

- **Programación Estructurada:** Se fundamenta en el **Teorema de la Programación Estructurada** (Böhm y Jacopini), que demostró que cualquier algoritmo computable puede resolverse combinando únicamente tres estructuras de control: **secuencia**, **selección** e **iteración**, eliminando definitivamente los saltos `GOTO`:

```
PROGRAM SumadorEstructurado;
VAR
  suma, n: INTEGER;
BEGIN
  suma := 0;
  REPEAT
    WRITE('Introduce un número positivo: ');
    READ(n);
    IF n ≥ 0 THEN
      suma := suma + n;
  UNTIL n < 0;
  WRITELN('La suma acumulada es: ', suma);
END.
```

- **Programación Modular:** Da un paso más allá al descomponer el algoritmo en subprogramas o funciones independientes especializadas (apartado 11).

4.2 Paradigma Orientado a Objetos (POO)

El pilar central de nuestro curso y de Java. En lugar de mantener datos por un lado y funciones sueltas por otro, la POO encapsula **estado** (atributos/variables) y **comportamiento** (métodos/funciones) dentro de entidades coherentes llamadas **Objetos**, que dialogan enviándose mensajes:

```
public class EjemploPOO {
    public static void main(String[] args) {
        // Creamos una instancia u objeto con su propio estado
        Saludo saludoEspanol = new Saludo("Hola");
        saludoEspanol.saludar("María"); // Salida: ¡Hola, María!
    }
}

class Saludo {
    private String prefijo; // Estado interno encapsulado

    public Saludo(String prefijo) {
        this.prefijo = prefijo;
    }

    public void saludar(String nombrePersona) { // Comportamiento
        System.out.println("¡" + this.prefijo + ", " + nombrePersona + "!");
    }
}
```

Sus cuatro pilares fundamentales son: **Abstracción**, **Encapsulamiento**, **Herencia** y **Polimorfismo** (profundizaremos en ellos exhaustivamente en las Unidades 4 y 5).

4.3 Paradigma Declarativo

En contraposición al imperativo, aquí no indicamos *cómo* realizar el cálculo paso a paso, sino **QUÉ** resultado o condición final deseamos obtener; el motor interno se encarga de determinar el plan de ejecución óptimo:

- **Lenguajes de Consulta (SQL):** En lugar de escribir bucles manuales para recorrer tablas y comparar celdas en memoria, declaramos el conjunto deseado:

```
SELECT nombre, departamento, salario
FROM empleados
WHERE salario > 2500.00
ORDER BY salario DESC;
```

- **Programación Funcional:** Trata la computación como la evaluación de funciones matemáticas puras sin estado mutable ni efectos colaterales. A partir de Java 8, Java incorporó expresiones lambda y la API Streams para adoptar este paradigma (Unidad 7).
- **Programación Lógica (Prolog):** Define una base de conocimiento basada en hechos y reglas lógicas, delegando en un motor de inferencia la resolución de hipótesis.

4.4 Paradigma Reactivo

Se basa en flujos de datos asíncronos (*streams*) y en la propagación automática de cambios (patrón observador). Los componentes se suscriben a eventos o emisiones de información y reaccionan en tiempo real ante su llegada (muy utilizado en arquitecturas de alta concurrencia con RxJava o Project Reactor).

> ☕ **Java es un lenguaje multiparadigma**
> Aunque Java es fundamentalmente orientado a objetos, en sus versiones LTS actuales combina con total fluidez programación imperativa estructurada, componentes declarativos y programación funcional con lambdas y streams.

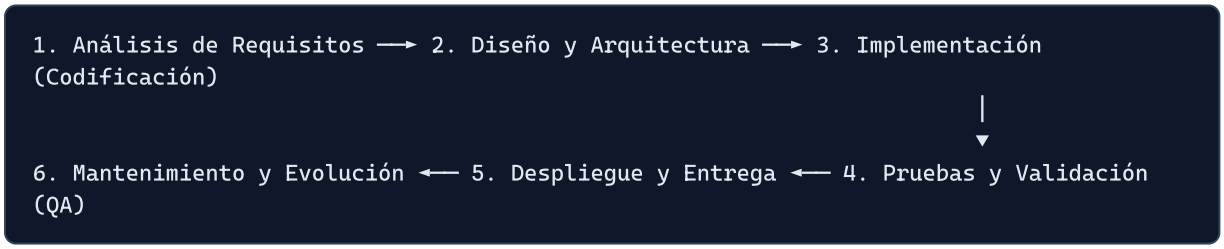
---

5. El Ciclo de Vida del Software y Modelado con UML

Crear software comercial de calidad no consiste en sentarse a teclear código improvisado. Requiere seguir un riguroso **ciclo de vida de ingeniería**.

Una analogía muy gráfica para comprender este proceso es **el encargo de una tarta de bodas**:

1. **Análisis de Requisitos:** Hablas en profundidad con los novios: *"¿Cuántos invitados vendrán?"*, *"¿Qué sabores prefieren?"*, *"¿Hay alérgicos al gluten?"*. Es la fase que los programadores novatos más tienden a ignorar, y la que más caro cuesta si se hace mal.
2. **Diseño y Arquitectura:** Dibujas el boceto, calculas los pisos, la estructura de soporte y la combinación de ingredientes antes de tocar la cocina. Es el plano técnico de la solución.
3. **Implementación (Codificación):** Por fin mezclas los ingredientes, horneas y montas la tarta en la cocina siguiendo fielmente la receta y el plano (traducción al lenguaje de programación).
4. **Pruebas y Verificación (QA):** Pruebas una porción de prueba, verificas la textura, la estabilidad y la temperatura antes del evento.
5. **Despliegue y Entrega:** Transportas la tarta y la presentas impecablemente en el salón de bodas ante los comensales.
6. **Mantenimiento:** Te aseguras de que el corte se realice sin incidencias y atiendes peticiones de última hora.



> ⭐ **Be the Code: La importancia de la documentación y el código limpio**
> La inmensa mayoría del código que vas a mantener y depurar a lo largo de tu trayectoria profesional **no lo habrás escrito tú**. Escribir código limpio, usar identificadores claros y documentar concienzudamente no es un capricho académico: es pensar con empatía en tu compañero de equipo (o en ti mismo dentro de seis meses, cuando ya no recuerdes por qué pusiste esa condición).
>
> En un proyecto profesional se distinguen dos tipos de documentación:
>
> - **Documentación Interna:** Comentarios dentro del propio código fuente (`//` , `/* */` y comentarios de documentación `javadoc /** */`) explicando el *porqué* de las decisiones complejas y las interfaces de los métodos.
> - **Documentación Externa:** Manuales de usuario, especificaciones técnicas de requisitos, diagramas de arquitectura UML y guías de despliegue.

5.1 Tipología de Errores en Programación

Durante las fases de codificación y pruebas nos enfrentamos inevitablemente a tres tipos de errores cualitativamente muy distintos:

- **1. Errores de Compilación (Sintácticos):**
  - El código viola las reglas gramaticales del lenguaje (ejemplo: escribir `pritnln` en vez de `println`, olvidar un punto y coma, o intentar asignar un texto a una variable de tipo entero).
  - El compilador rechaza el archivo y no genera el ejecutable. Son los más fáciles de localizar y corregir porque el propio entorno (IDE) indica la línea exacta del fallo.
- **2. Errores en Tiempo de Ejecución (*Runtime Errors* / Excepciones):**
  - El código compila con éxito porque sintácticamente es correcto, pero durante la ejecución se produce una situación anómala imprevista que el procesador no puede resolver (ejemplo: dividir un número entre cero, intentar abrir un fichero inexistente en el disco duro o agotar la memoria RAM).
  - Si el programa no gestiona la anomalía mediante control de excepciones (`try-catch`), el sistema operativo aborta bruscamente la aplicación.
- **3. Errores Lógicos o Semánticos (*Bugs*):**
  - El programa compila sin advertencias y se ejecuta sin abortar, pero **el resultado producido es matemáticamente incorrecto o incoherente con el objetivo planteado** (ejemplo: aplicar una fórmula de descuento sumando en lugar de restar, o plantear mal la condición de un bucle provocando que itere una vez de menos o caiga en un bucle infinito).
  - Son los errores más costosos y difíciles de detectar, y requieren depuración manual (*debugging*) y trazado meticuloso.

5.2 Introducción al Modelado con UML (*Unified Modeling Language*)

Para plasmar el diseño de un sistema informático de forma universal e inequívoca, los ingenieros recurren a **UML**, un estándar internacional de modelado visual que divide los diagramas en dos grandes familias:

- **Diagramas Estructurales:** Representan la estructura estática del sistema. El más utilizado en programación es el **Diagrama de Clases**, que detalla qué clases componen la aplicación, sus atributos, métodos y cómo se relacionan entre sí (herencia, asociación, agregación y composición).
- **Diagramas de Comportamiento:** Muestran la dinámica y el flujo de ejecución a lo largo del tiempo. Destacan el **Diagrama de Casos de Uso** (qué acciones puede realizar cada actor o usuario), el **Diagrama de Actividades** (flujo paso a paso de un proceso de negocio, muy similar a un diagrama de flujo) y el **Diagrama de Secuencia** (cómo dialogan y se envían mensajes los objetos cronológicamente).

---

6. Algoritmos: Definición, Propiedades y Buenas Prácticas

Un **algoritmo** es un conjunto ordenado, no ambiguo y finito de instrucciones que describe el procedimiento necesario para transformar los datos de partida en el resultado deseado.

Para visualizarlo, una analogía clásica y tangible es una receta de cocina de la vida diaria:

> ***Freír un huevo perfecto***
>
> - ***Entrada (Input):*** *1 huevo, 50 ml de aceite de oliva, una pizca de sal, sartén, fuente de calor (fuego).*
> - ***Procesamiento:***
>
> *1. Verter el aceite en la sartén.*

> *2. Encender el fuego y calentar la sartén.*
> *3. Cuando el aceite alcance la temperatura adecuada, cascar el huevo y verterlo con cuidado.*
> *4. Bañar la yema con aceite caliente usando una espumadera durante 40 segundos.*
> *5. Cuando la clara esté cuajada, retirar el huevo del fuego y añadir la sal.*
>
> - ***Salida (Output):*** *Huevo frito listo para servir.*

6.1 Propiedades de un Algoritmo Correcto

En la teoría computacional, distinguimos entre condiciones **obligatorias** (sin las cuales la secuencia no califica como algoritmo) y cualidades **deseables**:

Condiciones Obligatorias:

1. **Resuelve el problema planteado:** Produce la solución exacta y esperada según los requisitos iniciales.
2. **Independencia de la plataforma:** No depende de ningún sistema operativo, arquitectura de procesador ni lenguaje de programación específico.
3. **Precisión:** Cada instrucción debe estar especificada con total exactitud matemática, sin ambigüedades interpretativas.
4. **Finitud:** El algoritmo debe terminar obligatoriamente tras un número finito y computable de pasos; no puede quedar atrapado en un ciclo infinito.
5. **Determinismo (Repetibilidad):** Ante los mismos datos de entrada, el algoritmo debe arrojar exactamente el mismo resultado en todas y cada una de sus ejecuciones.
6. **Entrada y Salida bien delimitadas:** Define con claridad qué datos recibe del exterior y qué información o efecto observable devuelve.

Cualidades Deseables:

- **Válido:** Ausencia de defectos lógicos y robustez probada ante entradas límite o atípicas (*edge cases*).
- **Eficiente:** Resuelve el problema optimizando el tiempo de procesamiento de la CPU y reduciendo el consumo de memoria RAM.
- **Óptimo:** Es la solución más rápida y ligera matemáticamente demostrable de entre todos los algoritmos conocidos para ese problema.

6.2 Reglas de Nombrado de Identificadores (*Naming Conventions*)

Los nombres que asignamos a nuestras variables, constantes y subalgoritmos se denominan **identificadores**. Un código profesional debe ser autoexplicativo (*código limpio*):

- Deben comenzar obligatoriamente por una letra del alfabeto (o guion bajo `_`), nunca por un número.
- No deben contener espacios en blanco, operadores matemáticos (`+`, `-`, `*`, `/`) ni caracteres especiales conflictivos (`!`, `@`, `#`, `$`, `%`).
- Se desaconseja el uso de tildes y eñes para evitar problemas de codificación de caracteres al exportar código entre distintos sistemas operativos.
- **Convención camelCase:** Si el identificador está formado por varias palabras, la primera palabra va en minúsculas y las subsiguientes con su inicial en mayúscula: `salarioNetoMensual`, `horasTrabajadas`, `precioUnitario`.
- **Convención UPPER_SNAKE_CASE para constantes:** Las constantes se escriben íntegramente en mayúsculas separadas por guiones bajos: `PI`, `TASA_IVA_GENERAL`, `MAX_INTENTOS_ACCESO`.

- **Claridad semántica:** Evita a toda costa nombres crípticos o vagos como `x`, `a1`, `temp2`, `cosa`. Usa nombres que describan inequívocamente su función: `contadorIntentos`, `acumuladorGasto`, `distanciaKilometros`.

---

7. Técnicas de Representación de Algoritmos

Existen tres formas habituales de plasmar un algoritmo: el lenguaje natural humano, los diagramas de flujo y el pseudocódigo.

7.1 Lenguaje Natural: El Peligro de la Ambigüedad

Si intentamos describir un algoritmo complejo en español común, tropezamos inmediatamente con la polisemia, las interpretaciones subjetivas y la falta de rigor formal:

> *"Pela las patatas, córtalas finas y fríelas en aceite caliente hasta que doren. Añade huevo y cuájalo."*

¿Cuánto de finas? ¿A qué temperatura exacta debe estar el aceite? ¿Cuántos huevos? El lenguaje natural es excelente para la literatura, pero pésimo para dar órdenes a una máquina.

7.2 Diagramas de Flujo (Ordinogramas) según la Norma ANSI/ISO

Un diagrama de flujo representa gráficamente la secuencia lógica de un algoritmo empleando formas geométricas estandarizadas por la norma **ANSI X3.5**:

Símbolo	Nombre ANSI	Función y Significado
⬭ (Óvalo / Rectángulo redondeado)	**Terminal**	Representa el punto de arranque (`Inicio`) o de conclusión (`Fin`) del algoritmo o subproceso.
▭ (Rectángulo)	**Proceso**	Operación interna que modifica datos en memoria: cálculos aritméticos, asignaciones de variables (`total ← base × 1.21`).
▱ (Paralelogramo)	**Entrada / Salida**	Operación de lectura de información externa (`Leer variable`) o de visualización de datos al usuario (`Escribir resultado`).
◇ (Rombo)	**Decisión**	Evalúa una condición lógica booleana. Cuenta obligatoriamente con dos o más caminos de salida etiquetados (`Verdadero` / `Falso` o `Sí` / `No`).
○ (Círculo pequeño)	**Conector Interno**	Enlaza partes del diagrama dentro de la misma página para evitar un cruce caótico de líneas.
⌂ (Pentágono invertido)	**Conector de Página**	Conecta flujos entre páginas o láminas diferentes de un documento.
→ ← ↓ ↑ (Líneas de flujo)	**Líneas de Flujo**	Indican la dirección obligatoria del flujo de ejecución. Nunca deben cruzarse entre sí sin conector y no pueden apuntar al vacío.

7.3 Pseudocódigo Estructurado con PSeInt

El pseudocódigo es un lenguaje intermedio artificial que adopta la estructura formal de un lenguaje de programación real pero utilizando términos en lenguaje humano comprensible.

En nuestro curso nos basamos en las convenciones formales de **PSeInt** con perfil estricto:

- Todo algoritmo comienza con la palabra reservada `Algoritmo NombreAlgoritmo` y finaliza con `FinAlgoritmo`.
- Toda variable debe ser declarada explícitamente al inicio mediante `Definir identificador Como TipoDato`.
- La asignación se denota con la flecha direccional `←` para no confundirla con la comparación de igualdad.

---

8. Elementos del Pseudocódigo: Tipos, Operadores y Lógica Booleana

8.1 Tipos de Datos Primitivos

- **Entero:** Representa números enteros sin componente decimal, tanto positivos como negativos (`-10` , `0` , `450`).
- **Real:** Representa números continuos con parte fraccionaria o decimal (`3.14159` , `-0.005` , `12.0`).
- **Carácter:** Almacena un único símbolo tipográfico (letra, dígito o signo de puntuación) delimitado por comillas simples (`'A'` , `'9'` , `'#'`).
- **Cadena:** Secuencia ordenada de caracteres delimitada por comillas dobles (`"Programación Java"` , `"IES Serra Perenxisa"`).
- **Lógico (Booleano):** Tipo algebraico binario que solo puede admitir uno de dos valores posibles: `Verdadero` o `Falso` .

8.2 Operadores Aritméticos y División

- **Potencia:** `^` (ejemplo: `2 ^ 3 = 8`).
- **Multiplicación:** `*` o `×` .
- **División real:** `/` (produce siempre un resultado con decimales: `7.0 / 2.0 = 3.5`).
- **División entera truncada:** `/` sobre operandos enteros (`7 / 2 = 3`).
- **Módulo / Resto:** `MOD` o `%` :
  - La operación `A MOD B` calcula el residuo que sobra tras realizar la división entera entre `A` y `B` .
  - *Ejemplo:* Si dividimos `17` entre `5` , el cociente entero es `3` (5 × 3 = 15) y el resto que sobra es `2` . Por tanto: `17 MOD 5 = 2` .
  - El operador módulo es crucial en programación para detectar números pares o impares (`n MOD 2 = 0`), comprobar divisibilidad o reiniciar contadores circulares.
- **Suma (`+`) y Resta (`-`).**

8.3 Operadores Relacionales (Comparación)

Permiten comparar dos valores del mismo tipo de dato, produciendo siempre un resultado de tipo `Lógico` :

- Igual a: `=` (o `==` en Java).
- Distinto de: `≠` o `<>` (o `≠` en Java).
- Menor que: `<` | Menor o igual que: `≤` o `≤` .
- Mayor que: `>` | Mayor o igual que: `≥` o `≥` .

8.4 Operadores Lógicos y Álgebra de Boole

Permiten combinar condiciones lógicas complejas:

- **Negación (`NO` / `NOT`):** Operador unario que invierte el valor de verdad.

- **Conjunción (`Y` / `AND`):** Devuelve `Verdadero` **únicamente si ambas** expresiones operadas son verdaderas simultáneamente.
- **Disyunción Inclusiva (`O` / `OR`):** Devuelve `Verdadero` si **al menos una** de las expresiones operadas es verdadera.
- **Disyunción Exclusiva (`OEX` / `XOR`):** Devuelve `Verdadero` si una expresión es verdadera y la otra es falsa, pero no si ambas lo son a la vez.

Tablas de Verdad Completas

Proposición A	Proposición B	NO A	A Y B	A O B	A XOR B
`Falso`	`Falso`	`Verdadero`	`Falso`	`Falso`	`Falso`
`Falso`	`Verdadero`	`Verdadero`	`Falso`	`Verdadero`	`Verdadero`
`Verdadero`	`Falso`	`Falso`	`Falso`	`Verdadero`	`Verdadero`
`Verdadero`	`Verdadero`	`Falso`	`Verdadero`	`Verdadero`	`Falso`

> **Un caso clásico que engaña: La hora de tutoría**
>
> Imaginemos la siguiente regla de un centro educativo: *"La tutoría del profesor es los lunes y martes a las 16 horas"*. ¿Cuál de las siguientes expresiones lógicas traduce fielmente la norma?
>
> - **Opción A:** `dia = "lunes" O dia = "martes" Y hora = 16`
> - **Opción B:** `(dia = "lunes" O dia = "martes") Y hora = 16`
>
> Aunque en el lenguaje cotidiano suenen parecidas, en programación son radicalmente distintas. Como el operador lógico `Y` tiene mayor precedencia que `O` (exactamente igual que la multiplicación prima sobre la suma), la Opción A se evalúa como:
>
> `dia = "lunes" O (dia = "martes" Y hora = 16)`
>
> ¡Lo que significa que la condición sería válida **a cualquier hora de los lunes**! La opción correcta es la **Opción B**, usando paréntesis explícitos para agrupar los días lectivos. Ante la menor duda en una expresión lógica compuesta, utiliza siempre paréntesis.

Leyes de De Morgan

Son dos reglas lógicas fundamentales para simplificar condiciones complejas y diseñar pruebas unitarias:

1. `NO (A Y B)` es lógicamente equivalente a `(NO A) O (NO B)`.
2. `NO (A O B)` es lógicamente equivalente a `(NO A) Y (NO B)`.

8.5 Jerarquía Universal de Precedencia

Cuando una expresión contiene múltiples operadores, el computador los evalúa siguiendo un estricto orden matemático:

1. **Paréntesis:** Expresiones delimitadas por `( ... )`, evaluadas desde el par más interno hacia el exterior.
2. **Potenciación:** `^`.
3. **Operadores unarios:** Signo negativo unario (`-x`) y negación lógica (`NO`).
4. **Multiplicación, División y Módulo:** `*`, `/`, `MOD` (se evalúan de izquierda a derecha en caso de empate).
5. **Suma y Resta:** `+`, `-` (de izquierda a derecha).

6. **Operadores Relacionales:** `<`, `>`, `≤`, `≥`, `=`, `≠`.
7. **Conjunción Lógica:** `Y`.
8. **Disyunción Lógica:** `O` y `XOR`.

> **¡Cuidado con la precedencia!**
> La expresión `9 + 6 / 3` evalúa a **11**, no a **5**, porque la división tiene prioridad estricta sobre la suma. Si deseamos sumar primero, es indispensable forzarlo con paréntesis: `(9 + 6) / 3 = 5`. Emplear paréntesis siempre que tengas dudas cuesta una sola pulsación de tecla y previene fallos lógicos muy difíciles de rastrear.

---

9. Estructuras de Control Algorítmicas en Detalle

9.1 Estructura Secuencial

El flujo de control avanza linealmente de arriba a abajo, ejecutando una instrucción detrás de otra:

```
Algoritmo FacturacionSimple
    Definir baseImponible, porcentajeIva, importeIva, totalPagar Como Real

    Escribir "Introduce el importe de la base imponible en euros:"
    Leer baseImponible

    porcentajeIva ← 21.0
    importeIva ← baseImponible × (porcentajeIva / 100.0)
    totalPagar ← baseImponible + importeIva

    Escribir "Base imponible: ", baseImponible, " €"
    Escribir "IVA (21%):      ", importeIva, " €"
    Escribir "Total factura:  ", totalPagar, " €"
FinAlgoritmo
```

9.2 Estructuras de Selección (Condicionales)

Selección Simple y Doble (`Si - Entonces - SiNo`)

Evalúa una condición booleana para bifurcar la ejecución en dos caminos alternativos y excluyentes:

```
Algoritmo DescuentoComercial
    Definir totalCompra, totalConDescuento, rebaja Como Real

    Escribir "Introduce el importe total de la compra:"
    Leer totalCompra

    Si totalCompra > 100.0 Entonces
        rebaja ← totalCompra × 0.10 // 10% de descuento para compras superiores a 100 €
        totalConDescuento ← totalCompra - rebaja
        Escribir "¡Enhorabuena! Tienes un descuento de: ", rebaja, " €"
        Escribir "Importe final a pagar: ", totalConDescuento, " €"
    SiNo
        Escribir "No se aplica descuento (compra ≤ 100 €). Total a pagar: ",
totalCompra, " €"
    FinSi
FinAlgoritmo
```

Selección Anidada

Cuando la toma de decisiones requiere evaluar múltiples escenarios escalonados:

```
Algoritmo ClasificacionClimatica
    Definir temperatura Como Real
    Escribir "Introduce la temperatura en °C:"
    Leer temperatura

    Si temperatura < 0.0 Entonces
        Escribir "Estado: Congelación / Nieve"
    SiNo
        Si temperatura < 18.0 Entonces
            Escribir "Estado: Ambiente frío"
        SiNo
            Si temperatura < 28.0 Entonces
                Escribir "Estado: Ambiente templado / Agradable"
            SiNo
                Escribir "Estado: Ambiente caluroso / Canícula"
            FinSi
        FinSi
    FinSi
FinAlgoritmo
```

Selección Múltiple (`Segun - Hacer` / *Switch*)

Especialmente indicada cuando una variable entera o de carácter se contrasta frente a un catálogo discreto de valores posibles (como los menús interactivos):

```
Algoritmo MenuOperaciones
    Definir opcion Como Entero
    Definir num1, num2, resultado Como Real

    Escribir "═══════════════════════════"
    Escribir "   CALCULADORA ARITMÉTICA "
    Escribir "═══════════════════════════"
    Escribir "1. Sumar"
    Escribir "2. Restar"
    Escribir "3. Multiplicar"
    Escribir "4. Dividir"
    Escribir "Selecciona una opción (1-4):"
    Leer opcion

    Escribir "Introduce el primer número:"
    Leer num1
    Escribir "Introduce el segundo número:"
    Leer num2

    Segun opcion Hacer
        1:
            resultado ← num1 + num2
            Escribir "Resultado de la suma: ", resultado
        2:
            resultado ← num1 - num2
            Escribir "Resultado de la resta: ", resultado
        3:
            resultado ← num1 × num2
            Escribir "Resultado de la multiplicación: ", resultado
        4:
            Si num2 ≠ 0 Entonces
                resultado ← num1 / num2
                Escribir "Resultado de la división: ", resultado
            SiNo
                Escribir "Error matemático: No es posible dividir entre cero."
            FinSi
        De Otro Modo:
            Escribir "Opción no reconocida en el menú."
    FinSegun
FinAlgoritmo
```

9.3 Estructuras Iterativas (Bucles)

Los bucles permiten repetir un bloque de instrucciones múltiples veces. En programación estructurada existen tres modalidades fundamentales:

Bucle `Mientras` (Bucle con Pre-condición)

- Evalúa la condición lógica **antes** de ejecutar cada iteración.
- **Comportamiento clave:** Si la condición se evalúa como `Falsa` desde el inicio, el cuerpo del bucle **no se ejecutará ninguna vez (0 veces)**.
- Requiere que dentro del bucle exista al menos una instrucción que modifique las variables de la condición; de lo contrario, caerá en un **bucle infinito**.

```
Algoritmo MediaCalificaciones
    Definir sumaNotas, notaActual, media Como Real
    Definir totalAlumnos, contador Como Entero

    Escribir "¿Cuántos alumnos hay en el aula?"
    Leer totalAlumnos

    sumaNotas ← 0.0
    contador ← 1

    Mientras contador ≤ totalAlumnos Hacer
        Escribir "Introduce la nota del alumno número ", contador, ":"
        Leer notaActual
        sumaNotas ← sumaNotas + notaActual
        contador ← contador + 1 // Instrucción de avance indispensable
    FinMientras

    Si totalAlumnos > 0 Entonces
        media ← sumaNotas / totalAlumnos
        Escribir "La media de la clase es: ", media
    SiNo
        Escribir "No se han introducido alumnos."
    FinSi
FinAlgoritmo
```

Bucle `Repetir - Hasta Que` (Bucle con Post-condición)

- Ejecuta primero el bloque de instrucciones y comprueba la condición al final.
- **Comportamiento clave:** El cuerpo del bucle se ejecutará **como mínimo una vez (≥ 1 vez)**, independientemente del estado de la condición.
- En PSeInt, la iteración continúa mientras la condición sea falsa y **se detiene en el instante en que la condición sea verdadera**.
- Es el patrón canónico para la **validación y saneamiento de entradas de usuario**:

```
Algoritmo ValidacionEdadSegura
    Definir edad Como Entero

    Repetir
        Escribir "Introduce una edad válida para el registro laboral [16 - 65 años]:"
        Leer edad
        Si edad < 16 O edad > 65 Entonces
            Escribir "Error: La edad no cumple los criterios legales. Inténtalo de
nuevo."
        FinSi
    Hasta Que edad ≥ 16 Y edad ≤ 65

    Escribir "Edad validada y registrada correctamente: ", edad, " años."
FinAlgoritmo
```

Bucle `Para` (Bucle de Contador Automático)

- Se utiliza cuando el número exacto de iteraciones se conoce de antemano antes de entrar al bucle.
- Gestiona automáticamente la inicialización de la variable de control, la comprobación del límite y el incremento (o decremento si el paso es negativo):

```
Algoritmo CuentaAtrasYPaso
    Definir i Como Entero

    Escribir "--- Conteo decreciente de dos en dos ---"
    Para i ← 10 Hasta 0 Con Paso -2 Hacer
        Escribir "Valor actual de i: ", i
    FinPara
    Escribir "¡Final del bucle Para!"
FinAlgoritmo
```

Bucles Anidados

Un bucle puede alojarse dentro de otro. En cada iteración completa del bucle exterior, el bucle interior ejecutará todas sus iteraciones desde el principio hasta el final:

```
Algoritmo DibujarRectanguloAsteriscos
    Definir filas, columnas, f, c Como Entero
    filas ← 4
    columnas ← 6

    Para f ← 1 Hasta filas Con Paso 1 Hacer
        Para c ← 1 Hasta columnas Con Paso 1 Hacer
            Escribir Sin Bajar "*"
        FinPara
        Escribir "" // Salto de línea al terminar cada fila
    FinPara
FinAlgoritmo
```

---

10. ⭐ Be the Code: Metodología Formal de la Traza

> *En programación, "**Be the Code**" es la disciplina de simular mentalmente el papel de la CPU, ejecutando el algoritmo paso a paso sobre una libreta física. La tabla de traza (también llamada prueba de escritorio o dry run) es la herramienta más potente para erradicar errores de lógica sin necesidad de adivinar.*

Analicemos minuciosamente un algoritmo de búsqueda y cálculo de divisores: el **Algoritmo de Euclides para el Máximo Común Divisor (MCD)** con dos números de entrada: `A = 30` y `B = 12`.

```
Algoritmo TrazaEuclides
    Definir a, b, resto Como Entero
    a ← 30
    b ← 12

    Mientras b ≠ 0 Hacer
        resto ← a MOD b
        a ← b
        b ← resto
    FinMientras

    Escribir "El MCD obtenido es: ", a
FinAlgoritmo
```

Tabla de Seguimiento de Estados de Memoria

Paso	Línea de Código	`a`	`b`	`resto`	Evaluación de la Condición	Efecto en Memoria / Salida
1	`a ← 30`	**30**	*indefinido*	*indefinido*	—	Asignación inicial de variable `a`
2	`b ← 12`	30	**12**	*indefinido*	—	Asignación inicial de variable `b`
3	`Mientras b ≠ 0`	30	12	*indefinido*	`12 ≠ 0` (**Verdadero**)	La condición se cumple: entramos a la **Iteración 1**
4	`resto ← a MOD b`	30	12	**6**	—	`30 MOD 12 = 6` (porque 30 = 12 × 2 + 6)
5	`a ← b`	**12**	12	6	—	`a` adopta el valor de `b` (pasa a valer 12)
6	`b ← resto`	12	**6**	6	—	`b` adopta el valor de `resto` (pasa a valer 6)
7	`Mientras b ≠ 0`	12	6	6	`6 ≠ 0` (**Verdadero**)	La condición se cumple: entramos a la **Iteración 2**
8	`resto ← a MOD b`	12	6	**0**	—	`12 MOD 6 = 0` (división exacta: 12 = 6 × 2 + 0)
9	`a ← b`	**6**	6	0	—	`a` adopta el valor de `b` (pasa a valer 6)
10	`b ← resto`	6	**0**	0	—	`b` adopta el valor de `resto` (pasa a valer 0)
11	`Mientras b ≠ 0`	6	0	0	`0 ≠ 0` (**Falso**)	La condición falla: **sale inmediatamente del bucle**
12	`Escribir "El MCD ... ", a`	6	0	0	—	**Emisión de salida: "El MCD obtenido es: 6"**

> 💡 **Consejo para depurar: La prueba de escritorio**
>
> Cuando un programa produzca un resultado inesperado durante este curso, no cambies líneas impulsivamente en el IDE. Coge papel y bolígrafo, dibuja una tabla con una columna para cada variable y evalúa cada línea exactamente como lo haría el procesador. Detectarás la discrepancia en segundos.

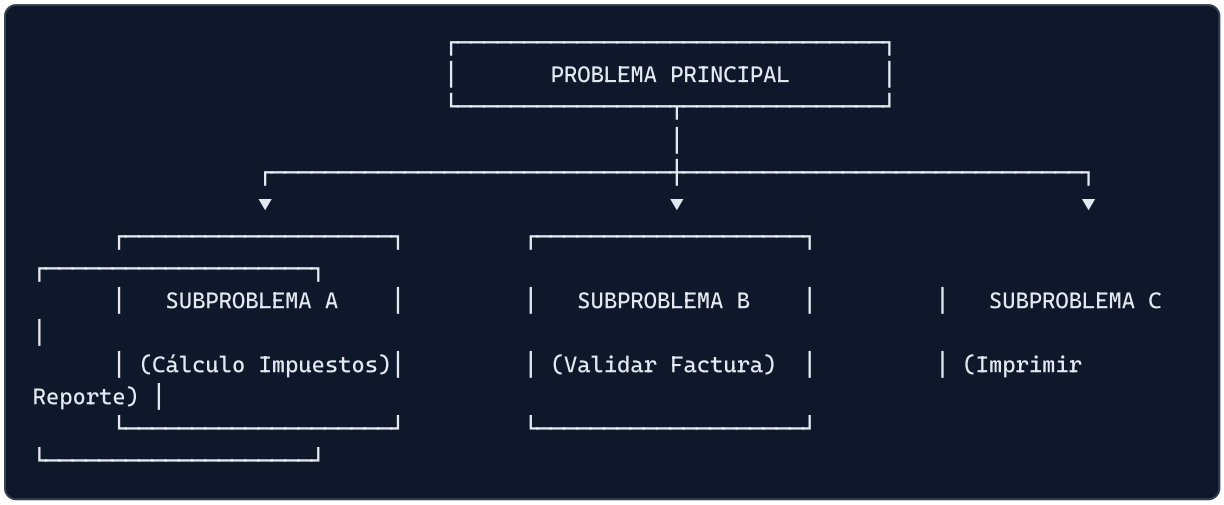
---

11. Programación Modular y Subalgoritmos

Conforme un problema real crece en envergadura, escribir cientos de líneas en un único bloque lineal desemboca en un código monolítico, redundante y frágil.

Imagina una tarea de 20 instrucciones que necesitas repetir en 7 lugares distintos de tu aplicación. Copiar y pegar esas 20 líneas siete veces puede parecer rápido al principio, pero es una pésima práctica de ingeniería: el código se vuelve kilométrico, ilegible y, si dentro de dos meses descubres un fallo o cambia la normativa fiscal, tendrás que localizar y modificar los 7 puntos idénticos (arriesgándote a olvidar uno).

Para solucionarlo aplicamos el principio de la ingeniería clásica **"Divide y Vencerás"** (*Top-Down Design* o descomposición modular descendente):



> **El error más habitual al empezar**
>
> Cuando se empieza a programar, la tentación inmediata es escribirlo todo en un único bloque continuo enorme. Aunque "funcione" en ejercicios sencillos de diez líneas, se vuelve inmanejable y propenso a catástrofes en proyectos reales. Acostúmbrate desde el primer día a pensar en subalgoritmos pequeños, cada uno con una única responsabilidad clara y bien delimitada.

11.1 Procedimientos vs Funciones

Un subalgoritmo es un módulo de código independiente dotado de un nombre propio que resuelve una tarea especializada:

- **Procedimiento:** Subalgoritmo que ejecuta una serie de acciones o efectos secundarios (por ejemplo, limpiar la pantalla, imprimir un informe formateado o escribir en disco) sin retornar formalmente un valor de resultado a la instrucción llamante.
- **Función:** Subalgoritmo que recibe cero o más datos de entrada, procesa un cálculo y **devuelve obligatoriamente un único valor de resultado** al punto donde fue invocada.

```
// DEFINICIÓN DE UNA FUNCIÓN MATEMÁTICA
Funcion retorno ← CalcularAreaCirculo(radio)
    Definir retorno, constantePi Como Real
    constantePi ← 3.14159265
    retorno ← constantePi × (radio × radio)
FinFuncion

// DEFINICIÓN DE UN PROCEDIMIENTO VISUAL
SubProceso ImprimirCabecera(titulo)
    Escribir "═══════════════════════════════════════"
    Escribir "   ", titulo
    Escribir "═══════════════════════════════════════"
FinSubProceso

// ALGORITMO PRINCIPAL CONSUMIDOR
Algoritmo GeometriaModular
    Definir r, area Como Real

    ImprimirCabecera("SISTEMA DE CÁLCULO GEOMÉTRICO")
    r ← 5.0
    area ← CalcularAreaCirculo(r) // Llamada a la función
    Escribir "El área para radio ", r, " es: ", area
FinAlgoritmo
```

11.2 Parámetros Formales vs Parámetros Actuales (Argumentos)

- **Parámetros Formales:** Son las variables ficticias declaradas en la cabecera del subalgoritmo que definen qué datos necesita recibir para operar (en el ejemplo anterior: `radio` o `titulo`).
- **Parámetros Actuales (o Argumentos):** Son los valores, expresiones o variables reales que el programa principal transfiere al subalgoritmo en el momento exacto de la llamada (en el ejemplo: el valor `r` o el texto `"SISTEMA ... "`).

11.3 Mecanismos de Paso de Parámetros

Existen dos formas fundamentales de transmitir información a un subalgoritmo:

1. **Paso por Valor:**
    - El subalgoritmo recibe una **copia exacta e independiente** del contenido de la variable original.
    - Cualquier modificación que el subalgoritmo realice sobre su parámetro formal **NO alterará** el valor de la variable original del llamante.
    - *Ejemplo:* Si le dejas una fotocopia de tus apuntes a un compañero y él pinta sobre la fotocopia, tu folio original permanece inalterado.
2. **Paso por Referencia:**
    - El subalgoritmo no recibe una copia, sino la **dirección de memoria física** donde reside la variable original.
    - Cualquier alteración que sufra el parámetro dentro del subalgoritmo **modificará directamente y de forma permanente la variable original** del programa principal.

> ⚠️ **Advertencia crucial: El paso de parámetros en Java**
> En Java, todos los parámetros se pasan estrictamente **por valor**. Sin embargo, cuando pasamos un objeto o un array, el valor copiado es la propia referencia (puntero) a la memoria heap, lo cual genera un comportamiento equivalente a la referencia sobre los atributos internos del objeto. Analizaremos este mecanismo crucial en la Unidad 4.

11.4 Ámbito y Visibilidad de Variables (*Scope*)

- **Variables Locales:** Declaradas dentro de un subalgoritmo o bloque específico. Solo existen en la memoria mientras dicho bloque se está ejecutando. Al terminar la función, se destruyen. Ninguna otra parte del programa puede acceder a ellas.
- **Variables Globales:** Declaradas en el ámbito raíz del programa, accesibles y modificables desde cualquier subalgoritmo en cualquier momento.
- **Buena práctica de ingeniería:** Debe evitarse el uso de variables globales. Provocan dependencias ocultas, acoplamiento dañino y dificultan enormemente la detección de fallos. La información debe fluir siempre a través de parámetros y valores de retorno.

11.5 Cohesión y Acoplamiento

El diseño de software modular se guía por dos métricas de calidad esenciales:

- **Alta Cohesión (Deseable):** Cada módulo, subalgoritmo o función debe tener una única responsabilidad bien delimitada y cumplirla a la perfección.
- **Bajo Acoplamiento (Deseable):** Los módulos deben ser lo más independientes entre sí posible, comunicándose únicamente mediante interfaces públicas claras (parámetros y retornos), minimizando las dependencias compartidas.

---

12. ¡No Hay Preguntas Tontas! 💡

¿Por qué estudiamos pseudocódigo y diagramas de flujo si en el trabajo nos van a pedir Java, C# o Python?

> *Cuando aprendes a conducir, primero aprendes las normas viales, la interpretación de señales y la dinámica de anticipación del tráfico. Si intentaras aprender todo eso subido a un coche deportivo de competición a 200 km/h bajo la lluvia, te estrellarías en la primera curva sin saber si falló el embrague o si tomaste mal la rotonda.*
>
> *Java es un lenguaje fuertemente tipado con una sintaxis rigurosa que exige clases, métodos estáticos, puntos y comas, paquetes y llaves `{}`. Si cometes un error siendo principiante, es muy difícil discernir si tu fallo es de lógica matemática o un simple despiste gramatical. El pseudocódigo aísla tu cerebro de la sintaxis y te enseña a pensar con lógica algorítmica pura. Un buen algoritmo en pseudocódigo se traduce a Java en diez minutos; un mal algoritmo no funcionará en ningún lenguaje del planeta.*

¿Por qué `17 / 4` puede dar `4` en lugar de `4.25`?

> *En los ordenadores, los tipos de datos condicionan el tipo de operación aritmética. Si ambos operandos son de tipo **Entero**, la máquina aplica **división entera truncada**, descartando la fracción decimal: `17 / 4 = 4`. Para obtener el resultado decimal exacto (`4.25`), al menos uno de los operandos debe declararse explícitamente como número **Real***

> *(`17.0 / 4` o `17 / 4.0`). El residuo exacto que se descartó en la división entera se recupera mediante la operación módulo: `17 MOD 4 = 1`.*

¿Cuál es la diferencia fundamental entre el operador `=` y el operador `←` o `==`?

> *En matemáticas, el símbolo `=` representa una equivalencia simétrica y pasiva: si decimos que `x = 5`, estamos afirmando un hecho invariable. En programación, la **asignación** (`←` o `=` en Java) es una acción direccional y destructiva: significa "toma el valor calculado a la derecha y guárdalo dentro de la celda de memoria identificada a la izquierda, sobrescribiendo cualquier dato previo". Por eso una instrucción como `x ← x + 1` tiene perfecto sentido en informática (incrementar `x`), mientras que en matemáticas sería una aberración insoluble (`x = x + 1 → 0 = 1`). Para contrastar si dos variables son iguales sin modificarlas, se emplea la comparación de igualdad (`=` en pseudocódigo o `==` en Java).*

¿Por qué los bucles `Mientras` pueden no ejecutarse ninguna vez y los `Repetir` siempre se ejecutan al menos una?

> *Porque el bucle `Mientras` es de **pre-condición**: comprueba la puerta de entrada antes de franquear el paso. Si la condición es falsa antes de empezar, el ordenador salta el bloque completo. Por el contrario, el bucle `Repetir` (o `do-while` en Java) es de **post-condición**: entra al cuerpo del bucle, ejecuta las instrucciones y solo cuando llega al final evalúa si debe repetir o salir. Por esta razón, `Repetir` garantiza siempre un mínimo de una ejecución.*

¿`Repetir ... Hasta Que` y `Mientras` no son lo mismo pero al revés?

> *Casi, pero con una diferencia esencial: **cuándo se evalúa la condición**.*
>
> - *`Mientras` pregunta primero y actúa después (puede que no actúe ninguna vez si la condición inicial ya es falsa).*
> - *`Repetir ... Hasta Que` actúa primero y pregunta después (se ejecuta siempre como mínimo una vez).*
>
> *Además, en pseudocódigo estándar, `Mientras` continúa iterando mientras su expresión evaluada sea **Verdadera**, mientras que `Repetir ... Hasta Que` se mantiene iterando mientras sea **Falsa** y se detiene en el momento justo en que la condición se vuelve **Verdadera**. Por lo tanto, para transformar un bucle en el otro no basta con mover la sentencia: ¡hay que invertir toda la lógica booleana de la expresión!*

---

13. Resumen de la Unidad y Enlace con Java

En esta extensa unidad has adquirido los cimientos sólidos de la ingeniería del software:

- Cómo procesa información un computador bajo el modelo de **caja negra**.
- La evolución de los lenguajes de programación desde el código binario 1GL hasta los lenguajes declarativos 4GL y lógicos 5GL.
- Las diferencias operativas entre **compilación**, **interpretación** y el innovador modelo híbrido de **Java (Bytecode + JVM + JIT)**.
- El ecosistema de herramientas: editores, depuradores, compiladores y el IDE oficial **IntelliJ IDEA**.
- Los cuatro grandes paradigmas computacionales: imperativo estructurado, orientado a objetos, funcional y reactivo.
- El **ciclo de vida del software** (analogía de la tarta de bodas), la importancia de la documentación interna y externa, y la tipología de errores (sintácticos, de ejecución y lógicos).

- Las propiedades formales de un algoritmo: obligatorias (precisión, determinismo, finitud) y deseables (validez, eficiencia, optimización).
- La simbología oficial **ANSI/ISO** para construir diagramas de flujo claros.
- Los tipos primitivos, operadores aritméticos, relacionales y lógicos con sus tablas de verdad y leyes de De Morgan.
- Las tres estructuras de control canónicas: secuencia, selección (`Si-Entonces` , `Segun`) e iteración (`Mientras` , `Repetir` , `Para`).
- La técnica de trazado manual **Be the Code** para depurar algoritmos sobre papel.
- El diseño modular mediante **procedimientos**, **funciones**, paso de parámetros (por valor vs por referencia) y visibilidad de variables.

En la **Unidad 2**, nos trasladaremos a nuestro entorno de desarrollo oficial **IntelliJ IDEA**, configuraremos el Java Development Kit (**JDK**) y traduciremos todos estos fundamentos a la sintaxis real de **Java LTS**.

---

14. Relación Curricular: Resultados de Aprendizaje y Criterios de Evaluación

Esta unidad cubre los contenidos troncales del currículo oficial del módulo **0485 - Programación (DAM)**:

Resultado de Aprendizaje (RA)	Criterios de Evaluación Oficiales Cubiertos
RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	**CE 1.a)** Se ha reconocido la estructura de un programa informático. **CE 1.c)** Se han utilizado entornos integrados de desarrollo (IntelliJ IDEA). **CE 1.i)** Se han introducido comentarios explicativos y documentación adecuada en el código fuente.

---

15. Boletines de Ejercicios y Retos Prácticos

Pon a prueba tu pensamiento algorítmico y consolida lo aprendido con el repertorio de actividades prácticas de la Unidad 1:

🟢 Boletín Inicial (Propuestos) ✅ Boletín Inicial (Resueltos) ⭐ Boletín Intermedio (Propuestos) ✅ Boletín Intermedio (Resueltos) 🔥 Retos y Plataformas Externas

> 🖨️ **Versiones en PDF para imprimir**
>
> - 📄 Boletín Inicial (PDF) · Con Soluciones (PDF)
> - 📄 Boletín Intermedio (PDF) · Con Soluciones (PDF)
> - 📄 Retos y Extras (PDF)

DAM 0485 · Programación · Unidad Didáctica 1

📝 Boletín 01 Inicial — Algorítmica y Pseudocódigo

Colección de 10 ejercicios prácticos fundamentales para dominar la lógica booleana, el trazado de variables en memoria, estructuras de control básicas y modularidad en pseudocódigo.

Este boletín contiene **10 ejercicios prácticos** diseñados para afianzar los conceptos fundamentales de la Unidad 1. Resuélvelos sobre el papel o en la herramienta **PSeInt** antes de consultar el Boletín Inicial Resuelto.

> 📥 **Descarga en PDF**
> Puedes descargar este boletín en PDF para imprimirlo o trabajar sin conexión. También dispones del PDF con las soluciones completas.

Ejercicio 1: Traza manual con variables en memoria y truco del intercambio (*Swap*)

Dadas tres variables enteras `A`, `B` y `C`, cuyos valores iniciales son `A ← 5`, `B ← 2` y `C ← 10`:

1. Indica el valor que contendrá cada una de las tres variables tras ejecutar secuencialmente las siguientes instrucciones rellenando una tabla de traza de variables paso a paso:

```
A ← A + B
B ← C − A
C ← A × B
A ← C MOD B
B ← B + 1
```

2. Analiza ahora el siguiente algoritmo de tres pasos con dos variables iniciales `X ← 7` e `Y ← 4`:

```
X ← X + Y
Y ← X − Y
X ← X − Y
```

   - Elabora la tabla de traza indicando el valor de `X` e `Y` en cada línea.
   - ¿Qué efecto final ha tenido esta secuencia sobre los valores de ambas variables? ¿Por qué es un truco célebre en la historia de la algorítmica?

Ejercicio 2: Evaluación de expresiones aritméticas y precedencia

Evalúa manualmente el resultado exacto de las siguientes expresiones aritméticas respetando la jerarquía universal de operadores (sin utilizar calculadora ni ordenador):

- a) `25 MOD 4`
- b) `7 / 2 + 2.5`
- c) `(4 + 6) × 3 + 2 × (5 − 1)`
- d) `5 / 2 + 17 MOD 3`
- e) `3 + 4 × 2`

- f) `(3 + 4) × 2`
- g) `10 MOD 3 + 1`
- h) `2 ^ 3 + 1`
- i) `37 / 4 × 4 − 2`

Ejercicio 3: Evaluación de expresiones lógicas y álgebra booleana

Determina si cada una de las siguientes expresiones lógicas se evalúa a `Verdadero` o `Falso`, indicando el orden de evaluación:

- a) `7 ≥ 5 O 27 ≠ 8`
- b) `(45 ≤ 7) O NO (5 ≥ 7)`
- c) `(25 ≥ 7) Y NO (7 ≤ 2)`
- d) `('H' < 'J') Y ('9' ≠ '7')`
- e) `10 + 4 < 15 − 3 O 2 × 5 + 1 > 14 − 2 × 2`
- f) `4 × 2 ≤ 8 O 2 × 2 < 5 Y 4 > 3 + 1`
- g) `10 ≤ 2 × 5 Y 3 < 4 O NO (8 > 7) Y 3 × 2 ≤ 4 × 2 − 1`

Ejercicio 4: Algoritmo de la vida cotidiana en lenguaje estructurado

Describe un algoritmo en lenguaje natural estructurado, no ambiguo, preciso y finito para realizar una de las siguientes tareas cotidianas:

- **Opción A:** Cambiar una rueda pinchada de un vehículo (contemplando seguridad, elevación con gato y orden de afloje/apriete en cruz).
- **Opción B:** La preparación completa de una tortilla de patatas tradicional.

Identifica con claridad cuáles son los datos de **Entrada (Input)**, los pasos ordenados del **Procesamiento** y el resultado de **Salida (Output)**.

Ejercicio 5: Cálculo de área y perímetro de un rectángulo

Diseña un algoritmo en pseudocódigo (compatible con PSeInt con perfil estricto) que solicite al usuario por teclado la base y la altura de un rectángulo (números reales positivos). El algoritmo debe calcular y mostrar por pantalla:

1. El perímetro del rectángulo: `2 × (base + altura)`.
2. El área del rectángulo: `base × altura`.

Ejercicio 6: Estructuras condicionales simples y dobles (`Si - Entonces - SiNo`)

Diseña dos algoritmos en pseudocódigo:

1. **Mayor de dos números:** Solicita dos números distintos por teclado (`num1` y `num2`) y muestra por pantalla cuál de los dos es el mayor.
2. **Control de mayoría de edad:** Solicita la edad de una persona (entero) y muestra por pantalla `"Es mayor de edad"` si tiene 18 años o más, o `"Es menor de edad"` en caso contrario.

Ejercicio 7: Selección múltiple (`Según ... Hacer`) para días de la semana

Diseña un algoritmo en pseudocódigo que solicite al usuario un número entero del `1` al `7` correspondiente a un día de la semana y muestre su nombre en texto:

- `1` → Lunes
- `2` → Martes
- `3` → Miércoles
- `4` → Jueves
- `5` → Viernes
- `6` → Sábado
- `7` → Domingo

Si el usuario introduce cualquier otro número fuera de ese rango, el bloque `De Otro Modo` debe mostrar el mensaje de error: `"Error: El valor introducido no corresponde a ningún día válido [1-7]"`.

Ejercicio 8: Comparativa de bucles: Conteo y acumulador (`Mientras` vs `Para`)

1. Escribe un algoritmo en pseudocódigo que, utilizando un bucle `Mientras`, muestre por pantalla los números correlativos del `1` al `10`.
2. Reescribe el mismo algoritmo sustituyendo el bucle `Mientras` por un bucle `Para`. Reflexiona: ¿qué ventajas ofrece el bucle `Para` cuando conocemos de antemano el número exacto de iteraciones?
3. Amplía el algoritmo para que calcule la **suma acumulada** de los primeros `N` números enteros positivos (desde `1` hasta `N`), solicitando el valor de `N` al usuario.

Ejercicio 9: Diseño modular y funciones (`esPar`)

Diseña un programa modular en pseudocódigo compuesto por:

1. Una función o subalgoritmo llamado `esPar` que reciba como parámetro formal un número entero `n` y devuelva el valor booleano `Verdadero` si el número es par o `Falso` si es impar (empleando el operador `MOD`).
2. El algoritmo principal consumidor que, mediante un bucle `Para`, recorra los números del `1` al `5`, invoque a la función `esPar` en cada vuelta y muestre por pantalla mensajes del estilo:
   - `"El número 1 es impar"`
   - `"El número 2 es par"`

Ejercicio 10: Fundamentos de ingeniería del software

Responde brevemente a las siguientes cuestiones conceptuales:

1. **Clasificación de lenguajes:** Clasifica los siguientes tres lenguajes según su nivel de abstracción (alto o bajo nivel) y según su modelo de ejecución (compilado, interpretado o híbrido): **Ensamblador**, **Python** y **Java**.
2. **Ciclo de vida del software:** Ordena cronológicamente las cinco fases clásicas del desarrollo de software: *Pruebas (QA)*, *Análisis de Requisitos*, *Despliegue y Mantenimiento*, *Diseño y Arquitectura*, *Implementación (Codificación)*.

DAM 0485 · Programación · Unidad Didáctica 1

📝 Boletín 01 Inicial (Resuelto) — Algorítmica y Pseudocódigo

Soluciones completas, razonadas y verificadas paso a paso para los 10 ejercicios prácticos del Boletín Inicial de la Unidad 1.

A continuación se detallan las soluciones exhaustivas de los 10 ejercicios propuestos en el Boletín Inicial.

> 📥 **Descarga en PDF**
> Puedes descargar este boletín resuelto en PDF para conservarlo como material de consulta y estudio.

Solución al Ejercicio 1: Traza manual con variables en memoria y truco del intercambio (*Swap*)

1. Traza con tres variables (`A = 5` , `B = 2` , `C = 10`):

Paso	Instrucción ejecutada	`A`	`B`	`C`	Explicación detallada
0	*Estado inicial*	**5**	**2**	**10**	Valores de partida en memoria
1	`A ← A + B`	**7**	2	10	`5 + 2 = 7` (se actualiza `A`)
2	`B ← C − A`	7	**3**	10	`10 − 7 = 3` (se actualiza `B`)
3	`C ← A × B`	7	3	**21**	`7 × 3 = 21` (se actualiza `C`)
4	`A ← C MOD B`	**0**	3	21	`21 MOD 3 = 0` (21 es múltiplo exacto de 3, resto 0)
5	`B ← B + 1`	0	**4**	21	`3 + 1 = 4` (se incrementa `B`)

- **Resultado final en memoria:** `A = 0` , `B = 4` , `C = 21` .

2. Traza del truco del intercambio sin variable auxiliar (`X = 7` , `Y = 4`):

Paso	Instrucción	`X`	`Y`	Explicación matemática
0	*Inicio*	**7**	**4**	Valores iniciales
1	`X ← X + Y`	**11**	4	`X` almacena la suma combinada: `7 + 4 = 11`
2	`Y ← X − Y`	11	**7**	`Y` resta su propio valor a la suma: `11 − 4 = 7` (`Y` **adopta el valor original de** `X`)
3	`X ← X − Y`	**4**	7	`X` resta el nuevo `Y` (el 7 original) a la suma: `11 − 7 = 4` (`X` **adopta el valor original de** `Y`)

- **Conclusión:** Al concluir las tres operaciones, los valores se han **intercambiado exitosamente** (`X = 4` , `Y = 7`) sin haber reservado ninguna variable auxiliar `TEMP` en memoria RAM. Es un truco clásico de optimización

en sistemas de memoria muy restringida.

Solución al Ejercicio 2: Evaluación de expresiones aritméticas y precedencia

- **a)** `25 MOD 4`
  - División entera: `25 = 4 × 6 + 1` → Resto: `1`.
- **b)** `7 / 2 + 2.5`
  - La división `7 / 2` produce `3.5`. Luego: `3.5 + 2.5 =` `6.0`.
- **c)** `(4 + 6) × 3 + 2 × (5 − 1)`
  - Paréntesis primero: `10 × 3 + 2 × 4`
  - Multiplicaciones: `30 + 8 =` `38`.
- **d)** `5 / 2 + 17 MOD 3`
  - `5 / 2 = 2.5`
  - `17 MOD 3 = 2` (porque `17 = 3 × 5 + 2`)
  - Suma final: `2.5 + 2 =` `4.5`.
- **e)** `3 + 4 × 2`
  - La multiplicación prima sobre la suma: `3 + 8 =` `11`.
- **f)** `(3 + 4) × 2`
  - El paréntesis obliga a sumar primero: `7 × 2 =` `14`.
- **g)** `10 MOD 3 + 1`
  - El operador `MOD` tiene la misma jerarquía que la multiplicación: `10 MOD 3 = 1`. Luego: `1 + 1 =` `2`.
- **h)** `2 ^ 3 + 1`
  - La potenciación es prioritaria sobre el resto de operaciones aritméticas: `2³ = 8`. Luego: `8 + 1 =` `9`.
- **i)** `37 / 4 × 4 − 2`
  - Asociatividad de izquierda a derecha para operadores de igual precedencia (`/` y `×`):
  - `(37 / 4) × 4 − 2 = 9.25 × 4 − 2 = 37 − 2 =` `35`.

Solución al Ejercicio 3: Evaluación de expresiones lógicas y álgebra booleana

- **a)** `7 ≥ 5 O 27 ≠ 8`
  - `7 ≥ 5` es `Verdadero`. Por evaluación en cortocircuito del operador `O`, toda la expresión es `Verdadero`.
- **b)** `(45 ≤ 7) O NO (5 ≥ 7)`
  - `45 ≤ 7` es `Falso`.
  - `5 ≥ 7` es `Falso` → `NO Falso` es `Verdadero`.
  - `Falso O Verdadero` → `Verdadero`.
- **c)** `(25 ≥ 7) Y NO (7 ≤ 2)`
  - `25 ≥ 7` es `Verdadero`.
  - `7 ≤ 2` es `Falso` → `NO Falso` es `Verdadero`.
  - `Verdadero Y Verdadero` → `Verdadero`.
- **d)** `('H' < 'J') Y ('9' ≠ '7')`

  - En orden de codificación ASCII, `'H'` precede a `'J'` (`Verdadero`).
  - El carácter `'9'` es distinto de `'7'` (`Verdadero`).
  - `Verdadero Y Verdadero` → **`Verdadero`**.

- **e)** `10 + 4 < 15 - 3 O 2 × 5 + 1 > 14 - 2 × 2`
  - Izquierda: `14 < 12` → `Falso`.
  - Derecha: `10 + 1 > 14 - 4` → `11 > 10` → `Verdadero`.
  - `Falso O Verdadero` → **`Verdadero`**.

- **f)** `4 × 2 ≤ 8 O 2 × 2 < 5 Y 4 > 3 + 1`
  - Aritmética: `8 ≤ 8 O 4 < 5 Y 4 > 4`
  - Relacionales: `Verdadero O Verdadero Y Falso`
  - Como `Y` tiene mayor precedencia que `O`: `Verdadero O (Verdadero Y Falso)` → `Verdadero O Falso` → **`Verdadero`**.

- **g)** `10 ≤ 2 × 5 Y 3 < 4 O NO (8 > 7) Y 3 × 2 ≤ 4 × 2 - 1`
  - Sub-bloque 1: `10 ≤ 10` (`V`) `Y 3 < 4` (`V`) → `Verdadero`.
  - Sub-bloque 2: `NO (8 > 7)` → `NO (Verdadero)` → `Falso`.
  - Evaluación: `Verdadero O (Falso Y ...)` → **`Verdadero`**.

Solución al Ejercicio 4: Algoritmo de la vida cotidiana en lenguaje estructurado

```
Algoritmo CambiarRuedaVehiculo
    // 1. Fase de Seguridad y Señalización
    1. Detener el vehículo fuera de la calzada en terreno plano, firme y seguro.
    2. Aplicar el freno de estacionamiento (freno de mano) e insertar la primera marcha
(o posición P).
    3. Encender las luces de emergencia, colocarse el chaleco reflectante y situar el
dispositivo V-16 o triángulos.

    // 2. Preparación de Herramientas
    4. Extraer del maletero: rueda de repuesto, gato mecánico y llave de tuercas.
    5. Retirar el embellecedor y aflojar media vuelta los tornillos de la rueda con la
llave, SIN extraerlos.

    // 3. Elevación y Sustitución
    6. Colocar el gato en el punto de anclaje reforzado del chasis más cercano a la
rueda averiada.
    7. Accionar la palanca del gato hasta que el neumático quede elevado 4 cm sobre el
asfalto.
    8. Desenroscar completamente los tornillos y retirar la rueda pinchada.
    9. Montar la rueda de repuesto haciendo coincidir los orificios del buje.
    10. Enroscar manualmente los tornillos hasta hacer tope y ajustarlos ligeramente
con la llave.

    // 4. Descenso y Ajuste Final
    11. Bajar el gato lentamente hasta que el neumático apoye con firmeza en el suelo y
retirarlo.
    12. Apretar enérgicamente los tornillos con la llave siguiendo un patrón en cruz
(estrella).
    13. Guardar las herramientas y la rueda pinchada en el maletero.
FinAlgoritmo
```

Solución al Ejercicio 5: Cálculo de área y perímetro de un rectángulo

```
Algoritmo RectanguloAreaPerimetro
    Definir baseRect, alturaRect, perimetro, area Como Real

    Escribir "Introduce la base del rectángulo (en metros):"
    Leer baseRect
    Escribir "Introduce la altura del rectángulo (en metros):"
    Leer alturaRect

    Si baseRect > 0 Y alturaRect > 0 Entonces
        perimetro ← 2 × (baseRect + alturaRect)
        area ← baseRect × alturaRect

        Escribir "═══════════════════════════"
        Escribir "    RESULTADOS GEOMÉTRICOS    "
        Escribir "═══════════════════════════"
        Escribir "Perímetro total: ", perimetro, " m"
        Escribir "Área de superficie: ", area, " m²"
    SiNo
        Escribir "Error: Las dimensiones del rectángulo deben ser estrictamente
positivas."
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 6: Estructuras condicionales simples y dobles (`Si - Entonces - SiNo`)

1. Mayor de dos números:

```
Algoritmo DeterminarMayor
    Definir num1, num2 Como Real

    Escribir "Introduce el primer número:"
    Leer num1
    Escribir "Introduce el segundo número:"
    Leer num2

    Si num1 > num2 Entonces
        Escribir "El número mayor es: ", num1
    SiNo
        Si num2 > num1 Entonces
            Escribir "El número mayor es: ", num2
        SiNo
            Escribir "Ambos números son idénticos: ", num1
        FinSi
    FinSi
FinAlgoritmo
```

2. Control de mayoría de edad:

```
Algoritmo ControlMayoriaEdad
    Definir edad Como Entero

    Escribir "Introduce la edad de la persona:"
    Leer edad

    Si edad ≥ 18 Entonces
        Escribir "Es mayor de edad (acceso permitido)."
    SiNo
        Escribir "Es menor de edad (acceso restringido)."
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 7: Selección múltiple (`Según ... Hacer`) para días de la semana

```
Algoritmo DiaDeLaSemana
    Definir dia Como Entero

    Escribir "Introduce el número de día de la semana [1-7]:"
    Leer dia

    Segun dia Hacer
        1:
            Escribir "Día 1: Lunes"
        2:
            Escribir "Día 2: Martes"
        3:
            Escribir "Día 3: Miércoles"
        4:
            Escribir "Día 4: Jueves"
        5:
            Escribir "Día 5: Viernes"
        6:
            Escribir "Día 6: Sábado"
        7:
            Escribir "Día 7: Domingo"
        De Otro Modo:
            Escribir "Error: El valor introducido (", dia, ") no corresponde a ningún
día válido [1-7]."
    FinSegun
FinAlgoritmo
```

Solución al Ejercicio 8: Comparativa de bucles (`Mientras` vs `Para`)

1. Conteo del 1 al 10 con bucle `Mientras`:

```
Algoritmo ContarConMientras
    Definir i Como Entero
    i ← 1
    Mientras i ≤ 10 Hacer
        Escribir "Número: ", i
        i ← i + 1 // Incremento manual indispensable
    FinMientras
FinAlgoritmo
```

2. Conteo del 1 al 10 con bucle `Para`:

```
Algoritmo ContarConPara
    Definir i Como Entero
    Para i ← 1 Hasta 10 Con Paso 1 Hacer
        Escribir "Número: ", i
    FinPara
FinAlgoritmo
```

- **Ventaja didáctica:** Con `Para`, la inicialización, la condición de parada y el incremento de la variable de control están unificados en una sola línea compacta, eliminando por completo el riesgo de olvidar la instrucción de avance y caer en un bucle infinito.

3. Suma acumulada de 1 a N:

```
Algoritmo SumatorioHastaN
    Definir N, i, sumaAcumulada Como Entero

    Escribir "Introduce el valor de N (entero positivo):"
    Leer N

    Si N > 0 Entonces
        sumaAcumulada ← 0
        Para i ← 1 Hasta N Con Paso 1 Hacer
            sumaAcumulada ← sumaAcumulada + i
        FinPara
        Escribir "La suma de 1 hasta ", N, " es: ", sumaAcumulada
    SiNo
        Escribir "Error: N debe ser mayor que cero."
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 9: Diseño modular y funciones (`esPar`)

```
// SUBALGORITMO / FUNCIÓN PURA
Funcion resultado ← esPar(n)
    Definir resultado Como Logico
    resultado ← (n MOD 2 = 0)
FinFuncion

// ALGORITMO PRINCIPAL CONSUMIDOR
Algoritmo ComprobadorPares
    Definir i Como Entero

    Escribir "═══ COMPROBACIÓN DE PARIDAD [1 - 5] ═══"
    Para i ← 1 Hasta 5 Con Paso 1 Hacer
        Si esPar(i) Entonces
            Escribir "El número ", i, " es PAR."
        SiNo
            Escribir "El número ", i, " es IMPAR."
        FinSi
    FinPara
FinAlgoritmo
```

Solución al Ejercicio 10: Fundamentos de ingeniería del software

1. Clasificación de lenguajes:

Lenguaje	Nivel de Abstracción	Modelo de Traducción y Ejecución
Ensamblador (*Assembly*)	**Bajo Nivel (2GL)**	**Ensamblado:** Un programa ensamblador traduce directamente los nemotécnicos a código máquina nativo 1:1, estrechamente acoplado al procesador.
Python	**Alto Nivel (3GL)**	**Interpretado:** Un intérprete analiza y ejecuta el código fuente instrucción por instrucción en tiempo real.
Java	**Alto Nivel (3GL)**	**Híbrido:** El compilador (`javac`) genera **Bytecode** neutro (`.class`), y la Máquina Virtual de Java (**JVM**) lo ejecuta combinando interpretación y compilación dinámica **JIT**.

2. Orden cronológico de las fases del ciclo de vida del software:

1. **Análisis de Requisitos:** Descubrir qué necesita el cliente y definir las especificaciones.
2. **Diseño y Arquitectura:** Planificar la solución técnica, estructuras de datos y diagramas UML.
3. **Implementación (Codificación):** Escribir el código fuente en el lenguaje de programación elegido.
4. **Pruebas y Verificación (QA):** Detectar y corregir fallos mediante casos de prueba exhaustivos.
5. **Despliegue y Mantenimiento:** Poner en producción el software y mantenerlo operativo en el tiempo.

DAM 0485 · Programación · Unidad Didáctica 1

📝 Boletín 01 Intermedio — Algorítmica y Pseudocódigo

10 retos algorítmicos aplicados que combinan bucles centinela, validaciones robustas con Repetir, modularidad, recursividad, primos, descomposición de dígitos, años bisiestos y arquitectura del software.

Este boletín contiene **10 ejercicios de nivel intermedio** para poner a prueba tu capacidad de estructurar algoritmos modulares, robustos y eficientes en pseudocódigo. Intenta resolverlos antes de consultar el Boletín Intermedio Resuelto.

> 📥 **Descarga en PDF**
> Puedes descargar este boletín intermedio en PDF para imprimirlo o trabajar sin conexión. También dispones del PDF con las soluciones completas.

Ejercicio 1: Validación estricta de entrada con bucle Repetir

Diseña un algoritmo en pseudocódigo que solicite al usuario una calificación académica. El programa debe obligar al usuario a repetir la introducción de la nota mientras el valor introducido esté fuera del rango válido `[0.0, 10.0]`. Una vez introducida una nota correcta, el algoritmo debe clasificarla e imprimir su calificación cualitativa:

- `[0.0, 5.0)`: Suspenso
- `[5.0, 6.0)`: Aprobado
- `[6.0, 7.0)`: Bien
- `[7.0, 9.0)`: Notable
- `[9.0, 10.0]`: Sobresaliente

Ejercicio 2: Bucle con centinela: Contador de pares e impares

Diseña un algoritmo que solicite números enteros al usuario de forma continua hasta que introduzca el número `0` (**valor centinela de parada**). Al terminar la introducción de datos, el programa debe mostrar por pantalla:

1. Cuántos números introducidos eran **pares**.
2. Cuántos números introducidos eran **impares**.

(Nota: El número 0 no debe contabilizarse como dato a evaluar, solo marca el final de la secuencia).

Ejercicio 3: Búsqueda de mayor y menor de 5 números sin arrays

Diseña un algoritmo que pida 5 números enteros cualesquiera por teclado y determine cuál es el **mayor** y cuál es el **menor** de todos ellos, utilizando únicamente variables escalares simples y una estructura `Para` (sin utilizar arrays ni vectores).

- **Trampa didáctica a evitar:** ¿Por qué inicializar las variables `mayor` y `menor` a cero al principio es un grave error de lógica que fallará si el usuario introduce únicamente números negativos? ¿Cómo debe inicializarse correctamente?

Ejercicio 4: Subalgoritmo Factorial iterativo y análisis de traza

1. Diseña un subalgoritmo/función llamado `factorial(n)` que reciba un número entero positivo `n` y calcule su factorial (`n! = 1 × 2 × 3 × ... × n`) utilizando una estructura iterativa `Para`.
2. Elabora la tabla de traza detallada paso a paso para la invocación `factorial(4)`, registrando la evolución en memoria de las variables en cada ciclo del bucle.

Ejercicio 5: Introducción a la recursividad (`factorialRecursivo`)

Un subalgoritmo puede invocarse a sí mismo para resolver un problema dividiéndolo en instancias más pequeñas; esta técnica se denomina **recursividad**:

1. Diseña una versión recursiva llamada `factorialRecursivo(n)`. Identifica con claridad cuál es el **caso base** (la condición que frena la recursión) y cuál es el **caso recursivo**.
2. Compara la solución iterativa del Ejercicio 4 con la versión recursiva: reflexiona sobre la legibilidad, la elegancia del código y el consumo de memoria en la pila (*stack*).

Ejercicio 6: Diseño modular integral: Calculadora de IMC con validación

Diseña una aplicación modular en pseudocódigo estructurada en funciones y procedimientos para calcular el **Índice de Masa Corporal (IMC = peso / altura^2)**:

1. `SubAlgoritmo peso ← pedirPeso()`: Solicita el peso en kilogramos y valida con `Repetir ... Hasta Que` que sea estrictamente positivo (`> 0`).
2. `SubAlgoritmo altura ← pedirAltura()`: Solicita la altura en metros y valida con `Repetir ... Hasta Que` que sea un valor positivo (`> 0`).
3. Algoritmo principal que invoque a los subalgoritmos anteriores, calcule el IMC y clasifique el resultado según las categorías de la OMS:
   - `IMC < 18.5`: Bajo peso
   - `18.5 ≤ IMC < 25.0`: Peso normal / Saludable
   - `25.0 ≤ IMC < 30.0`: Sobrepeso
   - `IMC ≥ 30.0`: Obesidad

Ejercicio 7: Comprobador de números primos optimizado (`esPrimo`)

Un número entero mayor que 1 es **primo** si y solo si es divisible únicamente por 1 y por sí mismo. Diseña un subalgoritmo `resultado ← esPrimo(n)` que reciba un número entero `n` y devuelva `Verdadero` si es primo o `Falso` en caso contrario. Optimiza el bucle para que descarte números menores que 2 y no realice iteraciones redundantes.

Ejercicio 8: Contador y sumador de dígitos de un número entero

Diseña un algoritmo que solicite al usuario un número entero positivo cualquiera (por ejemplo, `4825`) y, mediante un bucle, calcule y muestre:

1. La cantidad total de dígitos que posee (en el ejemplo: `4` dígitos).
2. La suma de todos sus dígitos individuales (`4 + 8 + 2 + 5 = 19`).

Pista: Emplea sucesivas divisiones enteras entre `10` para recortar el último dígito y el operador módulo `MOD 10` para extraerlo.

Ejercicio 9: Determinación de Año Bisiesto (Regla Gregoriana)

En el calendario gregoriano, un año es **bisiesto** (tiene 366 días) si es divisible entre `4`, excepto si es divisible entre `100`, a menos que también sea divisible entre `400`. *Ejemplos:*

- El año `2024` es bisiesto (divisible por 4 y no por 100).
- El año `1900` **no** fue bisiesto (divisible por 100 pero no por 400).
- El año `2000` **sí** fue bisiesto (divisible por 400).

Diseña un algoritmo que pida un año al usuario y determine mediante una **única condición lógica combinada** (usando `MOD`, `Y`, `O`) si el año es bisiesto o común.

Ejercicio 10: Diagnóstico de errores y arquitectura de la JVM

Responde de forma razonada a las dos siguientes cuestiones teóricas de ingeniería:

1. **Tipología de errores:** Clasifica cada uno de los siguientes incidentes como error **sintáctico**, **semántico/lógico** o **en tiempo de ejecución (*runtime*)**, justificando cuándo y cómo se detecta:
   - (a) Olvidar una palabra reservada de cierre como `FinSi` o un punto y coma en el código.
   - (b) Calcular el área de un círculo utilizando por error la fórmula del perímetro (`2 × PI × radio`).
   - (c) Calcular la velocidad media dividiendo la distancia entre el tiempo cuando el usuario introduce un tiempo igual a `0`.
2. **Ventajas del modelo híbrido de Java:** Explica qué ventaja práctica fundamental ofrece el modelo de Bytecode + JVM de Java frente a un lenguaje puramente compilado como C, y qué ventaja ofrece frente a un lenguaje puramente interpretado como Python.

DAM 0485 · Programación · Unidad Didáctica 1

📝 Boletín 01 Intermedio (Resuelto) — Algorítmica y Pseudocódigo

Soluciones completas, razonadas y verificadas en pseudocódigo PSeInt para los 10 ejercicios del Boletín Intermedio de la Unidad 1.

A continuación se detallan las soluciones exhaustivas de los 10 ejercicios planteados en el Boletín Intermedio.

> 📥 **Descarga en PDF**
>
> Puedes descargar este boletín resuelto en PDF para conservarlo como material de consulta y estudio.

Solución al Ejercicio 1: Validación estricta de entrada con bucle Repetir

```
Algoritmo CalificacionCualitativa
    Definir nota Como Real

    // Filtro de validación mediante bucle con post-condición
    Repetir
        Escribir "Introduce la nota académica del alumno [0.0 - 10.0]:"
        Leer nota
        Si nota < 0 O nota > 10 Entonces
            Escribir "Error: La nota debe encontrarse obligatoriamente entre 0.0 y
10.0."
        FinSi
    Hasta Que nota ≥ 0 Y nota ≤ 10

    // Clasificación cualitativa mediante condicionales anidadas
    Si nota < 5.0 Entonces
        Escribir "Calificación obtenida: Suspenso"
    SiNo
        Si nota < 6.0 Entonces
            Escribir "Calificación obtenida: Aprobado"
        SiNo
            Si nota < 7.0 Entonces
                Escribir "Calificación obtenida: Bien"
            SiNo
                Si nota < 9.0 Entonces
                    Escribir "Calificación obtenida: Notable"
                SiNo
                    Escribir "Calificación obtenida: Sobresaliente"
                FinSi
            FinSi
        FinSi
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 2: Bucle con centinela: Contador de pares e impares

```
Algoritmo ContadorParesImparesCentinela
    Definir n, pares, impares Como Entero

    pares ← 0
    impares ← 0

    Escribir "Introduce números enteros (introduce 0 para finalizar):"
    Leer n

    // El bucle Mientras se ejecuta hasta que el usuario introduzca el centinela 0
    Mientras n ≠ 0 Hacer
        Si n MOD 2 = 0 Entonces
            pares ← pares + 1
        SiNo
            impares ← impares + 1
        FinSi

        Escribir "Siguiente número (0 para terminar):"
        Leer n
    FinMientras

    Escribir "═══════════════════════════════"
    Escribir "   RECUENTO FINAL DE DATOS    "
    Escribir "═══════════════════════════════"
    Escribir "Total de números pares:   ", pares
    Escribir "Total de números impares: ", impares
FinAlgoritmo
```

Solución al Ejercicio 3: Búsqueda de mayor y menor de 5 números sin arrays

```
Algoritmo MayorYMenorDeCinco
    Definir i, n, mayor, menor Como Entero

    Escribir "Introduce el 1º número:"
    Leer n

    // ¡CRÍTICO! Inicializamos mayor y menor con el PRIMER valor real introducido.
    // Si hubiéramos puesto "mayor ← 0" y todos los números fuesen negativos (-10, -5,
-20),
    // el algoritmo diría falsamente que el mayor es 0 cuando el usuario jamás lo
introdujo.
    mayor ← n
    menor ← n

    Para i ← 2 Hasta 5 Con Paso 1 Hacer
        Escribir "Introduce el ", i, "º número:"
        Leer n

        Si n > mayor Entonces
            mayor ← n
        FinSi

        Si n < menor Entonces
            menor ← n
        FinSi
    FinPara

    Escribir "————————————————————————"
    Escribir "El número MAYOR introducido es: ", mayor
    Escribir "El número MENOR introducido es: ", menor
FinAlgoritmo
```

Solución al Ejercicio 4: Subalgoritmo Factorial iterativo y análisis de traza

1. Código PSeInt del subalgoritmo y algoritmo principal:

```
Funcion resultado ← factorial(n)
    Definir resultado, i Como Entero
    resultado ← 1
    Para i ← 1 Hasta n Con Paso 1 Hacer
        resultado ← resultado × i
    FinPara
FinFuncion

Algoritmo PruebaFactorial
    Definir num Como Entero
    Escribir "Introduce un entero positivo para calcular su factorial:"
    Leer num
    Si num ≥ 0 Entonces
        Escribir num, "! = ", factorial(num)
    SiNo
        Escribir "El factorial no está definido para números negativos."
    FinSi
FinAlgoritmo
```

2. Tabla de traza para la invocación `factorial(4)`:

Ciclo	Variable `i`	Operación realizada	Valor de `resultado`
Inicio	*indefinido*	`resultado ← 1`	**1**
Iteración 1	`1`	`resultado ← 1 × 1`	**1**
Iteración 2	`2`	`resultado ← 1 × 2`	**2**
Iteración 3	`3`	`resultado ← 2 × 3`	**6**
Iteración 4	`4`	`resultado ← 6 × 4`	**24**
Fin bucle	—	Retorno del subalgoritmo	**24**

Solución al Ejercicio 5: Introducción a la recursividad (`factorialRecursivo`)

```
Funcion resultado ← factorialRecursivo(n)
    Definir resultado Como Entero

    // CASO BASE: Detiene la recursión cuando n alcanza 0 o 1
    Si n ≤ 1 Entonces
        resultado ← 1
    SiNo
        // CASO RECURSIVO: n! = n × (n − 1)!
        resultado ← n × factorialRecursivo(n − 1)
    FinSi
FinFuncion
```

Comparativa teórica (Iterativo vs Recursivo):

- **Legibilidad y elegancia:** La versión recursiva es prácticamente idéntica a la definición matemática formal (`n! = n × (n−1)!`). Una vez dominada la recursividad, resulta conceptualmente muy limpia.
- **Consumo de memoria:** La versión iterativa (`Para`) utiliza un único espacio de memoria fijo. La versión recursiva crea un **marco de pila (*stack frame*)** en la memoria RAM por cada llamada activa, esperando a que los casos más pequeños retornen su valor. Para valores grandes de `n`, una función recursiva sin optimizar puede desbordar la pila (*StackOverflowError*).

Solución al Ejercicio 6: Diseño modular integral: Calculadora de IMC con validación

```
// 1. Subalgoritmo para pedir y validar el peso
SubProceso p ← pedirPeso()
    Definir p Como Real
    Repetir
        Escribir "Introduce el peso en kilogramos (ejemplo: 72.5):"
        Leer p
        Si p ≤ 0 Entonces
            Escribir "Error: El peso debe ser un número estrictamente positivo."
        FinSi
    Hasta Que p > 0
FinSubProceso

// 2. Subalgoritmo para pedir y validar la altura
SubProceso a ← pedirAltura()
    Definir a Como Real
    Repetir
        Escribir "Introduce la altura en metros (ejemplo: 1.75):"
        Leer a
        Si a ≤ 0 O a > 2.80 Entonces
            Escribir "Error: La altura debe ser válida y positiva [0.5 - 2.80 m]."
        FinSi
    Hasta Que a > 0 Y a ≤ 2.80
FinSubProceso

// 3. Algoritmo principal coordinador
Algoritmo CalculoIMCModular
    Definir peso, altura, imc Como Real

    Escribir "═══ SISTEMA CLÍNICO DE CÁLCULO DE IMC ═══"
    peso ← pedirPeso()
    altura ← pedirAltura()

    imc ← peso / (altura × altura)

    Escribir "──────────────────────────────────────────"
    Escribir "Tu Índice de Masa Corporal (IMC) es: ", imc

    Si imc < 18.5 Entonces
        Escribir "Diagnóstico OMS: Bajo peso (insuficiencia ponderal)."
    SiNo
        Si imc < 25.0 Entonces
            Escribir "Diagnóstico OMS: Intervalo normal / Peso saludable."
        SiNo
            Si imc < 30.0 Entonces
                Escribir "Diagnóstico OMS: Sobrepeso."
            SiNo
                Escribir "Diagnóstico OMS: Obesidad."
            FinSi
        FinSi
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 7: Comprobador de números primos optimizado (`esPrimo`)

```
Funcion esP ← esPrimo(n)
    Definir esP Como Logico
    Definir divisor Como Entero

    Si n < 2 Entonces
        esP ← Falso
    SiNo
        esP ← Verdadero
        divisor ← 2
        // Optimización matemática: Si no tiene divisores hasta su raíz cuadrada, es
primo
        Mientras divisor × divisor ≤ n Y esP = Verdadero Hacer
            Si n MOD divisor = 0 Entonces
                esP ← Falso // Divisor exacto encontrado: no es primo
            FinSi
            divisor ← divisor + 1
        FinMientras
    FinSi
FinFuncion

Algoritmo ProbarPrimalidad
    Definir num Como Entero
    Escribir "Introduce un número entero:"
    Leer num
    Si esPrimo(num) Entonces
        Escribir "El número ", num, " es PRIMO."
    SiNo
        Escribir "El número ", num, " es COMPUESTO (no es primo)."
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 8: Contador y sumador de dígitos de un número entero

```
Algoritmo ContadorYSumadorDigitos
    Definir numOriginal, numCopia, ultimoDigito, totalDigitos, sumaDigitos Como Entero

    Escribir "Introduce un número entero positivo:"
    Leer numOriginal

    Si numOriginal < 0 Entonces
        numCopia ← -numOriginal // Tomamos valor absoluto si fuese negativo
    SiNo
        numCopia ← numOriginal
    FinSi

    totalDigitos ← 0
    sumaDigitos ← 0

    Si numCopia = 0 Entonces
        totalDigitos ← 1
        sumaDigitos ← 0
    SiNo
        Mientras numCopia > 0 Hacer
            ultimoDigito ← numCopia MOD 10         // Extraemos el dígito de las
unidades
            sumaDigitos ← sumaDigitos + ultimoDigito
            totalDigitos ← totalDigitos + 1
            numCopia ← trunc(numCopia / 10)         // Recortamos el último dígito
        FinMientras
    FinSi

    Escribir "El número ", numOriginal, " tiene ", totalDigitos, " dígitos."
    Escribir "La suma de todos sus dígitos es: ", sumaDigitos
FinAlgoritmo
```

Solución al Ejercicio 9: Determinación de Año Bisiesto (Regla Gregoriana)

```
Algoritmo AnioBisiestoGregoriano
    Definir anio Como Entero
    Definir esBisiesto Como Logico

    Escribir "Introduce el año a comprobar:"
    Leer anio

    // Regla gregoriana unificada:
    // Divisible por 4 Y NO por 100, O BIEN divisible por 400
    esBisiesto ← (anio MOD 4 = 0 Y anio MOD 100 ≠ 0) O (anio MOD 400 = 0)

    Si esBisiesto Entonces
        Escribir "El año ", anio, " es BISIESTO (febrero tiene 29 días, total 366
días)."
    SiNo
        Escribir "El año ", anio, " es COMÚN (febrero tiene 28 días, total 365 días)."
    FinSi
FinAlgoritmo
```

Solución al Ejercicio 10: Diagnóstico de errores y arquitectura de la JVM

1. Diagnóstico y clasificación de errores:

- **(a) Olvidar una palabra reservada como `FinSi` o un punto y coma:**
  - **Tipo:** Error **Sintáctico** (de compilación).
  - **Momento de detección:** En tiempo de análisis léxico/sintáctico por parte del compilador o del intérprete antes de iniciar la ejecución del programa. El compilador rechaza el archivo y reporta la línea y columna del fallo gramatical.
- **(b) Calcular el área de un círculo usando la fórmula del perímetro (`2 × PI × r`):**
  - **Tipo:** Error **Semántico o Lógico** (*bug*).
  - **Momento de detección:** Durante la fase de pruebas (QA) o en producción. El programa compila sin advertencias y se ejecuta con total normalidad, pero el valor numérico obtenido es incorrecto. Requiere depuración manual o trazado sobre papel.
- **(c) Dividir entre cero cuando la variable tiempo vale `0` :**
  - **Tipo:** Error **en Tiempo de Ejecución** (*Runtime Exception*).
  - **Momento de detección:** En pleno funcionamiento del programa, en el instante exacto en que la CPU intenta ejecutar la operación aritmética inválida. Provoca el cierre abrupto de la aplicación si no se gestiona mediante control de excepciones (`try-catch`).

2. Ventajas del modelo híbrido de Java:

- **Frente a un lenguaje compilado nativo como C:** Ofrece **portabilidad total** (*Write Once, Run Anywhere*). En C, el código binario generado está atado a la arquitectura de CPU (x86, ARM) y al sistema operativo; en Java, el mismo fichero binario intermedio (`.class` o `.jar`) se ejecuta idénticamente en cualquier plataforma sin necesidad de recompilar el proyecto, requiriendo únicamente la JVM de dicho entorno.
- **Frente a un lenguaje puramente interpretado como Python:** Ofrece **mayor rendimiento y verificación temprana**. Al compilarse previamente a Bytecode, se descartan los errores sintácticos y de tipado antes de la ejecución. Además, el motor **JIT** de la JVM compila los métodos más ejecutados a código máquina nativo ultrarrápido en caliente, alcanzando velocidades cercanas a las de C++.

DAM 0485 · Programación · Unidad Didáctica 1

🧠 Boletín 01 Extras — Retos de Pensamiento Computacional

Retos algorítmicos avanzados, acertijos de lógica computacional y preparación para plataformas de programación competitiva.

En las primeras unidades aún no disponemos de sintaxis Java suficiente para enviar soluciones automáticas a jueces en línea como CodeWars o Acepta el Reto (lo cual iniciaremos a partir de la **Unidad 3**). Sin embargo, este boletín presenta **retos clásicos de lógica algorítmica y optimización** que entrenarán tu agilidad mental para la resolución de problemas en competición.

> 📥 **Descarga en PDF**
> Puedes descargar este boletín de retos y extras en PDF para resolverlos en papel o en tus sesiones de entrenamiento lógico.

Reto 1: La Conjetura de Collatz (Secuencia del Granizo)

Enunciado

El matemático Lothar Collatz propuso en 1937 la siguiente regla para cualquier número entero positivo `N`:

- Si el número es **par**, se divide entre 2 (`N ← N / 2`).
- Si el número es **impar**, se multiplica por 3 y se le suma 1 (`N ← 3 × N + 1`).
- El proceso se repite con el nuevo valor obtenido hasta que el número llegue irremediablemente al valor **1**.

Diseña un algoritmo en pseudocódigo que:

1. Pida un número entero positivo inicial `N`.
2. Imprima toda la secuencia de números generados hasta alcanzar el 1.
3. Cuente y muestre la cantidad total de pasos necesarios (longitud de la órbita) y el valor pico más alto alcanzado durante la trayectoria.

> 💡 **Pistas para la resolución**
> - Utiliza un bucle `Mientras N ≠ 1 Hacer`.
> - Mantén una variable `pasos ← 0` y una variable `picoMaximo ← N`.
> - Actualiza `picoMaximo` en cada iteración comparando si el nuevo `N` supera el récord anterior.

Reto 2: El Juego de Adivinar el Número (Búsqueda Binaria)

Enunciado

El ordenador genera un número secreto entero aleatorio entre 1 y 100 (en PSeInt puedes usar `numeroSecreto ← Azar(100) + 1`). El usuario debe intentar adivinarlo en el menor número posible de intentos.

- En cada intento, el usuario introduce un número.
- Si el número del usuario es menor que el secreto, el programa avisa: *"El número secreto es MAYOR"*.
- Si es mayor, avisa: *"El número secreto es MENOR"*.
- Si acierta, el programa le felicita y le indica en cuántos intentos lo ha logrado.

Pregunta algorítmica de análisis: ¿Cuál es el número máximo de intentos que un jugador necesitaría para adivinar **siempre** el número si juega de manera óptima utilizando la técnica de **búsqueda binaria** (dividir el intervalo a la mitad en cada paso)?

> 💡 **Pistas para la resolución**
>
> - Como el rango es de 1 a 100, la búsqueda binaria evalúa `100 / 2 = 50`, luego `25`, `12`, etc.
> - ¿Qué potencia de 2 supera a 100? `2⁶ = 64 < 100`, mientras que `2⁷ = 128 ≥ 100`. ¡El número máximo de intentos necesarios es siempre ≤ 7!

Reto 3: Algoritmo Voraz (Greedy) para el Cambio de Monedas

Enunciado

Una máquina expendedora debe devolver el cambio a un cliente tras una compra. Se dispone de una cantidad infinita de monedas de curso legal de los siguientes valores (en céntimos de euro):

- `200` (2 €), `100` (1 €), `50`, `20`, `10`, `5`, `2` y `1`.

Dado un importe a devolver en céntimos (por ejemplo, `387` céntimos): Diseña un algoritmo que devuelva el cambio utilizando el **mínimo número total de monedas posible**.

> 💡 **Pistas para la resolución**
>
> - Esta estrategia se conoce en algorítmica como **enfoque voraz (greedy)**: en cada paso, selecciona la moneda de mayor valor que no sobrepase la cantidad pendiente de devolución.
> - Puedes resolverlo iterando secuencialmente desde la moneda más grande (200) hacia la más pequeña (1), calculando cuántas monedas caben mediante división entera (`cambio / valorMoneda`) y actualizando el resto mediante módulo (`cambio MOD valorMoneda`).

Reto 4: El Problema Clásico del Cruce del Río

Enunciado

Un granjero debe transportar a través de un río a un **lobo**, una **cabra** y una **lechuga (col)**. Dispone de una barca en la que solo caben él y **uno** de los tres elementos en cada viaje.

- Si el granjero deja solos al lobo y a la cabra en cualquier orilla, el lobo se comerá a la cabra.
- Si deja solas a la cabra y a la lechuga, la cabra se comerá la lechuga.
- El lobo no come vegetales.

Escribe la secuencia exacta y ordenada de viajes (ida y vuelta) que el granjero debe realizar para cruzar los tres elementos a la otra orilla sanos y salvos.

> 💡 **Pistas para la resolución**
>
> - Piensa en qué elemento es el único que puede quedarse temporalmente solo con cualquiera de los otros dos sin peligro.
> - A veces, para avanzar en un algoritmo, es necesario realizar una operación aparentemente contraintuitiva: ¡traerse de vuelta en la barca a un pasajero que ya habías cruzado!

▶ 💡 Solución Razonada al Cruce del Río

Reto 5: El Enigma de los Tres Interruptores y la Bombilla

Enunciado

Te encuentras en la planta baja de una casa frente a un panel con **tres interruptores** (`1`, `2` y `3`) en posición de apagado. En la buhardilla del piso superior hay una única bombilla incandescente tradicional, la cual es invisible desde la planta baja.

- Puedes accionar los interruptores tantas veces como desees.
- Sin embargo, solo puedes subir al piso de arriba a comprobar el estado de la bombilla **una única vez**. ¿Cómo puedes determinar con absoluta certeza matemática cuál de los tres interruptores es el que enciende la bombilla?

> 💡 **Pistas para la resolución**
>
> - Un dispositivo físico no solo emite información lumínica visible por los ojos; también genera otros efectos observables en el entorno físico (como la disipación de energía térmica o calor).

▶ 💡 Solución Razonada de los Interruptores

Reto 6: Las 9 Monedas y la Balanza de Dos Platos (Búsqueda Ternaria)

Enunciado

Dispones de **9 monedas de oro** visualmente idénticas, pero una de ellas es falsa y tiene un peso ligeramente menor que las 8 restantes (las cuales pesan exactamente lo mismo). Cuentas con una **balanza clásica de dos platos** (que únicamente indica si el plato izquierdo pesa más, si pesa más el derecho, o si ambos están equilibrados). ¿Cómo puedes identificar inequívocamente la moneda falsa utilizando la balanza un **máximo de 2 veces**?

> 💡 **Pistas para la resolución**
>
> - En lugar de dividir el conjunto por la mitad (búsqueda binaria, que requeriría 4 pesadas en el peor caso), aprovecha que una balanza de dos platos tiene **tres estados posibles**: inclinación izquierda, inclinación derecha o equilibrio perfecto.
> - Divide las monedas en grupos de 3 (búsqueda ternaria: `9 = 3 × 3`).

▶ 💡 Solución Razonada de las 9 Monedas

Reto 7: La Torre de Hanói y la Estrategia Recursiva

Enunciado

El clásico juego de la **Torre de Hanói** consta de 3 varillas verticales (`Origen` , `Auxiliar` y `Destino`) y `N` discos de radios decrecientes ensartados en la varilla `Origen` . El objetivo es trasladar todos los discos a la varilla `Destino` cumpliendo dos normas inquebrantables:

1. Solo se puede mover un disco en cada paso.
2. Nunca puede colocarse un disco sobre otro de menor tamaño.

Desafío algorítmico: Describe en pseudocódigo un subalgoritmo recursivo `Hanoi(n, origen, destino, auxiliar)` que imprima la secuencia exacta de movimientos necesarios para resolver la torre de `N` discos.

> 💡 **Pistas para la resolución**
>
> Aplica descomposición recursiva (*Divide y Vencerás*):
>
> - Mover `N` discos de `Origen` a `Destino` equivale a:
>   1. Mover los `N - 1` discos superiores de `Origen` a `Auxiliar` .
>   2. Mover el disco grande restante `N` directamente de `Origen` a `Destino` .
>   3. Mover los `N - 1` discos aparcados en `Auxiliar` hacia `Destino` .
> - El caso base se produce cuando `N = 1` (un único movimiento elemental).

▶ 💡 Solución en Pseudocódigo para la Torre de Hanói

CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Unidad 2. Primeros pasos con Java

El ecosistema Java (JDK, JVM, Bytecode), instalación y configuración de IntelliJ IDEA, estructura canónica de un programa, tipos de datos primitivos, variables, constantes, operadores, clases estándar (String, Math, java.time, BigDecimal), sentencias condicionales y entrada/salida por consola.



MÓDULO PROFESIONAL
Programación (0485) — 1º DAM

RESULTADOS DE APRENDIZAJE
RA1, RA4 (Estructura de programas y tipos de datos)

DURACIÓN ESTIMADA
24 horas lectivas

ENTORNO OFICIAL
IntelliJ IDEA / Java 25 LTS

¡Enhorabuena por haber superado la Unidad 1! Ya dominas los cimientos del pensamiento algorítmico, la representación mediante diagramas de flujo ANSI y el diseño estructurado con pseudocódigo. Ahora ha llegado el momento más esperado por todo estudiante de desarrollo de software: **traducir esa lógica conceptual a un lenguaje de programación real, industrial y de propósito general**.

En esta unidad nos sumergiremos en **Java** (en su versión estándar moderna **Java 25 LTS**, manteniendo compatibilidad plena con Java 21 LTS), uno de los lenguajes más robustos, seguros y demandados del panorama tecnológico mundial. Utilizaremos como entorno de trabajo oficial **IntelliJ IDEA**, la herramienta de referencia en la industria del desarrollo de software profesional.

Aprenderás cómo viaja tu código desde el editor hasta los transistores del procesador a través de la Máquina Virtual de Java (JVM), dominarás los 8 tipos primitivos de datos, comprenderás cómo manipular texto, números y fechas con precisión matemática, interactuarás con el usuario mediante la consola y controlarás el flujo de ejecución mediante sentencias condicionales clásicas y modernas.

> 📥 **Recursos y Descargas en PDF de la Unidad 2**
> Puedes estudiar esta unidad tanto en la plataforma web como en formato documento digital/impreso:
>
> - 📄 **Descargar Teoría Completa de la Unidad 2 en PDF** (41 páginas)
> - 📝 **Boletín Inicial:** Descargar Propuestos (PDF) · Descargar Resueltos (PDF)
> - 📝 **Boletín Intermedio:** Descargar Propuestos (PDF) · Descargar Resueltos (PDF)
> - 🧠 **Retos y Extras:** Descargar Retos (PDF)

---

1. El Ecosistema de la Plataforma Java

1.1 Breve Historia y Filosofía de Diseño

Java nació a principios de los años 90 en **Sun Microsystems**, liderado por el ingeniero **James Gosling**. Inicialmente bautizado como *Oak* (por un roble que Gosling veía desde la ventana de su despacho), el proyecto buscaba crear un lenguaje para programar dispositivos electrónicos inteligentes (el proyecto *Star7*). Posteriormente, con el auge de Internet y la World Wide Web en 1995, el lenguaje fue renombrado como **Java** (en homenaje al café originario de la isla de Java en Indonesia) y presentado al mundo con una promesa revolucionaria:

> ***WORA (Write Once, Run Anywhere)***
> *"Escribe tu código una sola vez y ejecútalo en cualquier lugar."*

A diferencia de lenguajes como C o C++, cuyos programas debían recompilarse específicamente para cada sistema operativo y procesador (generando binarios dependientes de la arquitectura x86, ARM, etc.), Java introdujo un nivel de abstracción intermedio: la **Máquina Virtual de Java (JVM)**.

En 2010, **Oracle Corporation** adquirió Sun Microsystems, asumiendo la administración y liderazgo del ecosistema Java, junto con el proceso comunitario abierto **JCP (*Java Community Process*)** y el proyecto de código abierto **OpenJDK**.

> **Duke: La Mascota Oficial de Java**
> La simpática criatura con forma de triángulo blanco y nariz roja que acompaña a la documentación de Java se llama **Duke**. Fue diseñada originalmente por Joe Palrang como el agente interactivo que guiaba al usuario en el prototipo doméstico *Star7*. Hoy en día es el embajador de marca de la comunidad Java en todo el mundo.

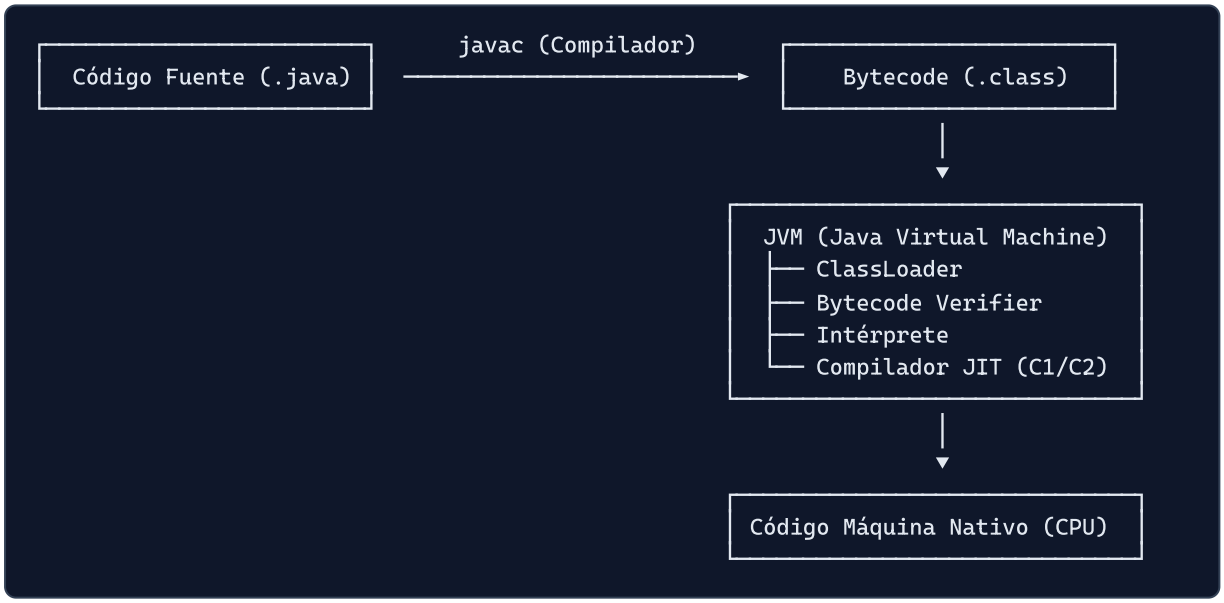
1.2 Tipología de Aplicaciones en Java

Gracias a su versatilidad y rendimiento, Java no se limita a un único entorno, sino que abarca prácticamente todas las ramas de la informática actual:

1. **Aplicaciones de Consola (*CLI*):** Programas basados en texto que interactúan mediante la terminal y flujos estándar (`System.in` , `System.out`), ideales para scripts, herramientas de administración y aprendizaje inicial.
2. **Aplicaciones de Escritorio con Interfaz Gráfica (*GUI*):** Software para sistemas operativos de usuario final creado mediante librerías gráficas como Swing o la moderna **JavaFX**.
3. **Aplicaciones Web y Servicios Backend:** La columna vertebral de la banca, el comercio electrónico y las grandes plataformas (Netflix, Amazon, Twitter). Emplea tecnologías como Servlets, Jakarta EE (anteriormente Java EE) y frameworks líderes como **Spring Boot** y Quarkus.
4. **Aplicaciones Móviles:** El sistema operativo **Android** adoptó Java desde sus inicios como lenguaje oficial para el desarrollo de apps nativas.
5. **Sistemas Embebidos e Internet de las Cosas (*IoT*):** Dispositivos médicos, cajeros automáticos, tarjetas SIM y sensores industriales que ejecutan entornos reducidos de Java.

1.3 La Arquitectura Técnica: Código Fuente, Bytecode y JVM

Para comprender qué ocurre cuando pulsamos el botón "Run" en nuestro entorno, debemos distinguir con total precisión las tres etapas del ciclo de ejecución:



1. **Código Fuente (`.java`):** Es el texto en lenguaje Java comprensible para los seres humanos.
2. **Compilador (`javac`):** No traduce el código fuente a lenguaje máquina directo de la CPU, sino a un código intermedio universal denominado **Bytecode** (almacenado en archivos `.class`). El Bytecode es independiente del procesador y del sistema operativo.
3. **Máquina Virtual de Java (`JVM`):** Es el software instalado en el sistema operativo del cliente que interpreta y ejecuta el Bytecode. Cada plataforma (Windows, macOS, Linux, Android) tiene su propia versión de la JVM adaptada a su hardware, pero todas interpretan exactamente el mismo Bytecode.
4. **Compilador JIT (*Just-In-Time*):** Para evitar la lentitud clásica de los intérpretes puros, las JVM modernas detectan las partes del código que se ejecutan repetidamente (*hot spots*) y las compilan dinámicamente sobre la marcha a código máquina nativo del procesador, logrando un rendimiento cercano al de C++.

5. **Recolector de Basura (*Garbage Collector*):** Un proceso en segundo plano de la JVM que rastrea los objetos en la memoria RAM que ya no se usan y libera su espacio automáticamente, eliminando de raíz las fugas de memoria (*memory leaks*) que plagaban a los lenguajes tradicionales.

1.4 La Tríada Fundamental: JDK, JRE y JVM

Es muy frecuente que los estudiantes principiantes confundan estas tres siglas esenciales:

Sigla	Significado	¿Qué incluye?	¿Para quién es?
JVM	*Java Virtual Machine*	Motor de ejecución, intérprete, compilador JIT y Garbage Collector.	La base técnica de todo el sistema.
JRE	*Java Runtime Environment*	La **JVM** + las **librerías estándar de clases** de Java (`rt.jar` / módulos).	**Usuarios finales** que solo quieren ejecutar programas ya compilados.
JDK	*Java Development Kit*	El **JRE** completo + **herramientas de desarrollo** (`javac` , `javadoc` , depuradores, empaquetador `jar`).	**Desarrolladores de software**. Es lo que nosotros instalaremos obligatoriamente.



1.5 Versiones de Java y Filosofía LTS

Desde 2018, Java adoptó un ritmo de lanzamientos semestral (cada 6 meses, en marzo y septiembre, se publica una nueva versión del lenguaje). Para garantizar la estabilidad industrial sin obligar a las empresas a migrar de versión cada medio año, Oracle introdujo la distinción entre versiones estándar y **LTS (*Long-Term Support*)**:

- **Versiones No-LTS (rápidas):** Tienen soporte oficial de solo 6 meses y se utilizan como banco de pruebas para novedades.
- **Versiones LTS (Soporte Extendido):** Se publican cada dos años y cuentan con un mínimo de 5 a 8 años de parches de seguridad y estabilidad empresarial.
  - **Java 8 LTS:** El hito de la programación funcional (Lambdas y Streams).
  - **Java 11 LTS:** Sistema de módulos e inferencia local con `var` .
  - **Java 17 LTS:** Registros (*records*), clases selladas y nuevo switch.
  - **Java 21 LTS:** Gran salto con hilos virtuales (*Virtual Threads*), pattern matching exhaustivo y máxima optimización.
- **Java 25 LTS (La versión LTS más reciente):** El estándar LTS más actual de la plataforma Java (publicado en septiembre de 2025), consolidando las características modernas, estabilidad empresarial y optimizaciones de la máquina virtual.

OpenJDK vs. Oracle JDK

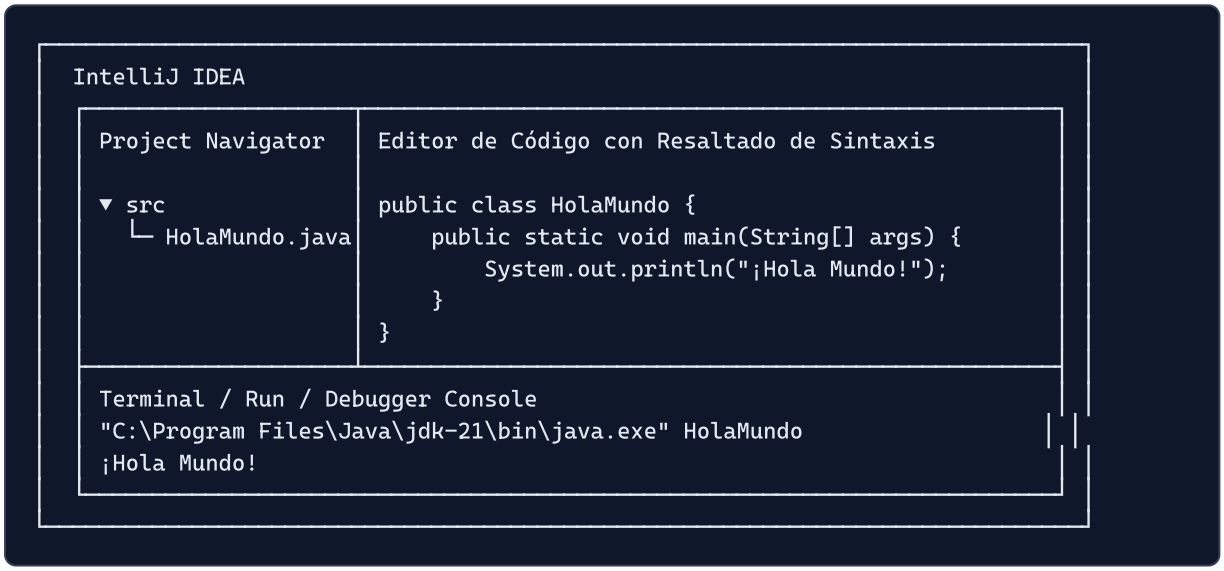
- **OpenJDK:** La implementación oficial de código abierto y libre distribución con licencia GNU GPL v2. Es el estándar sobre el que se basan los compiladores modernos.
- **Oracle JDK:** Distribución comercial de Oracle, con condiciones específicas de licencia para entornos productivos corporativos.
- **Distribuciones comunitarias certificadas:** Existen compilaciones 100% gratuitas, de alto rendimiento y código abierto muy populares como **Eclipse Temurin (Adoptium)**, **Amazon Corretto** o **Azul Zulu**.

---

2. El Entorno de Desarrollo Oficial: IntelliJ IDEA

En el ciclo de DAM profesional, no escribimos código en editores de texto planos sin asistencia. Utilizamos un **Entorno Integrado de Desarrollo (*IDE - Integrated Development Environment*)**.

El IDE oficial para todas las explicaciones, prácticas y proyectos de este curso es **IntelliJ IDEA** (de JetBrains), la herramienta más potente, inteligente y ampliamente adoptada en el sector empresarial de Java.



2.1 Conceptos de Sistema Operativo: PATH, JAVA_HOME y CLASSPATH

Aunque IntelliJ IDEA automatiza la detección del JDK, cualquier técnico superior en informática debe conocer los tres mecanismos con los que el sistema operativo localiza las herramientas de Java:

1. `PATH`**:** Es una variable de entorno del sistema que contiene una lista de directorios donde el sistema operativo busca programas ejecutables cuando se invoca su nombre en la consola (como `javac` o `java`). Para que funcionen desde cualquier carpeta, se debe añadir al PATH la ruta a la subcarpeta `bin` del JDK (por ejemplo, `C:\Program Files\Java\jdk-21\bin` en Windows o `/usr/lib/jvm/jdk-21/bin` en Linux).
2. `JAVA_HOME`**:** Variable de entorno utilizada por servidores de aplicaciones (Tomcat, WildFly) y herramientas de compilación (Maven, Gradle) para saber exactamente en qué carpeta raíz reside el JDK principal del sistema.
3. `CLASSPATH`**:** Variable que indica a la JVM en qué directorios o librerías empaquetadas (`.jar`) debe buscar las clases que necesita un programa para funcionar. Por defecto, siempre incluye el directorio actual representado por un punto (`.`).

> **IntelliJ IDEA se encarga de todo esto por ti**
> En el pasado, los desarrolladores tenían que pelear manualmente configurando estas variables en el registro de Windows o en el fichero `/etc/environment` de Linux. **IntelliJ IDEA gestiona todo esto de forma autónoma**: detecta automáticamente los JDKs instalados en tu equipo o te permite descargar uno nuevo con un simple clic desde su propia interfaz.

2.2 Creación de un Nuevo Proyecto en IntelliJ IDEA

Para crear tu primer proyecto profesional en IntelliJ IDEA Community Edition:

1. Abre IntelliJ IDEA y en la pantalla de bienvenida pulsa sobre **New Project** (o desde el menú: **File** → **New** → **Project...**).
2. En el panel lateral izquierdo, selecciona **Java**.
3. En el campo **Name**, escribe el nombre del proyecto (por ejemplo, `Unidad02_PrimerosPasos`).
4. En **Location**, elige la carpeta de tu disco donde se guardará tu código.
5. En el desplegable **JDK**, selecciona tu versión instalada de **Java 25** (o **Java 21**). Si no aparece ninguna instalada, pulsa sobre **Download JDK...** y elige el proveedor *Eclipse Temurin* o *Amazon Corretto*: IntelliJ lo descargará e instalará automáticamente.
6. Marca la casilla **Add sample code** si deseas que IntelliJ cree automáticamente un archivo de ejemplo con el método `main`.
7. Pulsa **Create**.

Anatomía del Proyecto en el Disco

IntelliJ organiza los archivos de trabajo en una estructura estricta:

- `.idea/`**:** Carpeta oculta donde IntelliJ guarda las preferencias del proyecto, módulos y configuraciones de ejecución (no se debe modificar a mano).
- `src/` **(*Source*):** La carpeta más importante. Aquí residen exclusivamente nuestros archivos de código fuente con extensión `.java`.
- `out/` **(o** `target/` **):** Carpeta donde IntelliJ almacena automáticamente los archivos binarios compilados `.class` listos para ser interpretados por la JVM.

2.3 Atajos de Teclado y Plantillas Vivas (*Live Templates*)

La velocidad y comodidad con la que un programador escribe código en IntelliJ IDEA se debe a sus plantillas automáticas:

Plantilla / Atajo	Acción que realiza	Resultado generado
Escribir `main` o `psvm` + Tab	Genera el punto de entrada de la aplicación.	`public static void main(String[] args) { }`
Escribir `sout` + Tab	Imprime texto por consola con salto de línea.	`System.out.println();`
Escribir `souf` + Tab	Imprime texto con formato enriquecido.	`System.out.printf("");`
`Ctrl + Alt + L`	**Reformat Code:** Limpia e indenta todo el archivo con formato perfecto.	Código limpio y alineado automáticamente.
`Shift + F10`	**Run:** Compila y ejecuta el programa actual.	Abre la consola de salida.
`Shift + F9`	**Debug:** Inicia la ejecución con depurador paso a paso.	Abre el panel de inspección de variables.
`Alt + Intro`	**Show Context Actions:** Soluciona errores y sugiere mejoras.	Corrección guiada de código.

---

3. Anatomía de un Programa en Java

Observemos el programa clásico "Hola Mundo" estructurado conforme a las mejores prácticas de la ingeniería del software:

```
package es.iesperenxisa.primerospasos;

/**
 * Clase principal para ilustrar la anatomía de un programa en Java.
 * @author Docente DAM
 * @version 1.0
 */
public class HolaMundo {

    public static void main(String[] args) {
        // Imprime un saludo por la consola estándar del sistema
        System.out.println("¡Hola, programadores de DAM!");
    }
}
```

3.1 Desglose Riguroso del Código

1. `package es.iesperenxisa.primerospasos;`
   Indica el **paquete** (espacio de nombres) al que pertenece la clase. Los paquetes evitan colisiones de nombres entre clases y organizan el proyecto en carpetas físicas en el disco. La convención internacional de Java exige utilizar el nombre de dominio de la organización en orden inverso (por ejemplo, `es.iesperenxisa`).
2. `public class HolaMundo`
   En Java, **todo el código debe residir obligatoriamente dentro de una clase**. No pueden existir funciones o variables "sueltas" en el archivo.
   - **Regla estricta del compilador:** Si una clase se declara con el modificador `public`, el archivo en el disco **debe llamarse exactamente igual que la clase** y con extensión `.java` (`HolaMundo.java`). Además, solo puede existir una única clase pública por cada archivo físico.

3. `public static void main(String[] args)`
   Es el **punto de entrada universal (*Entry Point*)** de cualquier aplicación ejecutable en Java. Cuando le pedimos a la JVM que lance nuestra aplicación, busca exactamente este método:
   - `public`: Modificador de visibilidad que permite a la JVM invocar el método desde fuera de la clase.
   - `static`: Permite que la JVM invoque el método sin necesidad de crear previamente un objeto de la clase `HolaMundo` en memoria.
   - `void`: Indica que el método realiza una tarea pero no devuelve ningún valor como resultado de su ejecución.
   - `main`: Nombre reservado que busca el lanzador de Java.
   - `String[] args`: Parámetro que representa un array de cadenas de texto para capturar argumentos pasados por línea de comandos al arrancar el programa.
4. `System.out.println( ... )`
   - `System`: Es una clase predefinida en el paquete nuclear de Java (`java.lang`).
   - `out`: Es un objeto estático dentro de `System` que representa el flujo de salida estándar (la consola de texto).
   - `println()`: Es el método que escribe el contenido entre paréntesis en la pantalla y añade automáticamente un salto de línea al final. Su homólogo `System.out.print()` muestra el contenido pero mantiene el cursor en la misma línea.

3.2 Tipología de Comentarios y Generación con JavaDoc

Java soporta tres modalidades de comentarios:

```
// 1. Comentario de una sola línea: ideal para notas breves y directas.

/*
   2. Comentario de bloque o multilínea:
   útil para explicaciones extensas o para
   desactivar temporalmente fragmentos de código.
*/

/**
 * 3. Comentario de documentación técnica (JavaDoc).
 * Describe el propósito de una clase o método y es procesado
 * por la herramienta javadoc para generar páginas web HTML de documentación.
 * @param args Argumentos de la línea de comandos
 */
```

Etiquetas JavaDoc más Comunes:

- `@author Nombre`: Especifica el autor del componente software.
- `@version 1.0`: Indica la versión actual del código.
- `@param nombre Parametro`: Explica qué representa cada parámetro recibido por un método.
- `@return Descripcion`: Detalla qué valor devuelve el método al finalizar.
- `@throws TipoExcepcion`: Documenta qué errores puede lanzar el método si algo falla.

En IntelliJ IDEA, puedes generar la documentación HTML completa de tu proyecto navegando a:
Tools → Generate JavaDoc...

3.3 Compilación y Ejecución Manual desde la Terminal

Aunque el IDE automatiza el proceso, un desarrollador profesional debe comprender cómo interactuar directamente con el compilador desde la consola de comandos de Windows o Linux:

Ventana de terminal

```
# 1. Compilación: Transforma el archivo fuente en Bytecode
# El flag -d indica la carpeta destino donde dejar los binarios compilados
javac -d out/production/Unidad02 src/es/iesperenxisa/primerospasos/HolaMundo.java

# 2. Ejecución: Se lanza la JVM indicando el classpath (-cp) y el nombre canónico de la
clase
java -cp out/production/Unidad02 es.iesperenxisa.primerospasos.HolaMundo

# 3. Novedad de Java moderno (Java 11+): Ejecución directa sin compilar a disco
java src/HolaMundo.java
```

---

4. Elementos Básicos del Lenguaje: Identificadores, Variables y Constantes

Para construir cualquier algoritmo en un lenguaje de programación necesitamos mecanismos para nombrar entidades, almacenar datos temporales en la memoria RAM y garantizar que ciertos valores permanezcan inmutables a lo largo del tiempo.

4.1 Identificadores y Convenciones de Nomenclatura

Un **identificador** es el nombre que el programador asigna a un elemento del código: un paquete, una clase, una variable, una constante o un método.

Reglas Léxicas Obligatorias del Compilador

Si vulneras alguna de estas reglas, el compilador `javac` generará un error de sintaxis y el programa no compilará:

1. Debe comenzar por una letra Unicode (del alfabeto latino básico), un carácter de subrayado (`_`) o el signo de dólar (`$`). **Nunca puede comenzar por un dígito**.
2. Los caracteres posteriores pueden ser letras, dígitos (0-9), subrayados (`_`) o signos de dólar (`$`).
3. Java distingue de manera estricta entre mayúsculas y minúsculas (*case-sensitive*). Por ejemplo, `edad` , `Edad` y `EDAD` son tres identificadores totalmente distintos e independientes.
4. No puede coincidir con ninguna de las **palabras reservadas** (*keywords*) del lenguaje Java (como `class` , `public` , `static` , `void` , `int` , `if` , `while` , `return` , etc.), ni con los literales reservados `true` , `false` y `null` .
5. No puede contener espacios en blanco ni operadores matemáticos o signos de puntuación (`+` , `-` , `*` , `/` , `@` , `#` , `.` , `,`).

> **Atención con tildes y caracteres no ASCII**
>
> Aunque el compilador de Java soporta caracteres Unicode completos en identificadores (lo que permite declarar variables como `año` o `número`), **en el desarrollo profesional está terminantemente desaconsejado**. Utilizar caracteres fuera del estándar ASCII (como la `ñ` , tildes, diéresis o caracteres cirílicos/asiáticos) genera graves problemas de compatibilidad al compartir código entre diferentes sistemas operativos (Windows, Linux, macOS), servidores de integración continua y herramientas de control de versiones como Git. Se debe utilizar siempre el alfabeto inglés sin tildes: `anio` o `year` , `numero` o `number` .

Convenciones de la Comunidad Java (Code Style de Oracle e IntelliJ)

A diferencia de las reglas del compilador (que son obligatorias), las convenciones son acuerdos universales seguidos por millones de desarrolladores para que el código sea inmediatamente legible:

Elemento	Convención de Nomenclatura	Ejemplo Recomendado	Antiejemplo a Evitar
Clases e Interfaces	**PascalCase / UpperCamelCase** (cada palabra empieza en mayúscula)	`CuentaBancaria`, `GestorPedidos`	`cuentaBancaria`, `gestor_pedidos`
Variables y Métodos	**lowerCamelCase** (primera palabra minúscula, siguientes en mayúscula)	`saldoActual`, `calcularInteres()`	`SaldoActual`, `calcular_interes`
Constantes	**UPPER_SNAKE_CASE** (todo mayúsculas separadas por guion bajo)	`PI`, `MAX_INTENTOS_LOGIN`	`maxIntentos`, `max_intentos`
Paquetes	**Todo minúsculas** en orden inverso de dominio de internet	`es.iesperenxisa.primerospasos`	`es.IESPerenxisa.PrimerosPasos`

4.2 Declaración, Inicialización y Ámbito de Variables

Una **variable** es un contenedor con nombre asociado a una posición de la memoria RAM del ordenador, capaz de albergar un dato de un tipo concreto que puede ser modificado durante la ejecución del programa.

En Java, toda variable debe ser declarada especificando obligatoriamente su **tipo de dato** y su **identificador**:

```
// 1. Declaración simple (se reserva el espacio en memoria, sin asignar valor
explícito)
int edadUsuario;
double temperaturaMedia;

// 2. Inicialización posterior (asignación del primer valor)
edadUsuario = 25;
temperaturaMedia = 21.4;

// 3. Declaración e inicialización combinada en una sola línea (recomendado)
int puntos = 100;
boolean usuarioActivo = true;

// 4. Declaración múltiple del mismo tipo (permitido por sintaxis, pero desaconsejado)
int x = 0, y = 10, z = 20;
```

Ámbito (*Scope*) y el Peligro de Variables Locales no Inicializadas

El ámbito de una variable es la región del código fuente comprendida entre las llaves `{ ... }` donde fue declarada. Fuera de ese bloque, la variable deja de existir para el compilador y para la memoria:

```
public void calcular() {
    int total = 10; // Variable local accesible en todo el método

    if (total > 5) {
        int descuento = 2; // Variable local al bloque if
        total -= descuento;
    } // Aquí 'descuento' se destruye y ya no es accesible

    // System.out.println(descuento); // ¡ERROR de compilación! 'descuento' no existe
aquí
}
```

Regla de Oro en Java: Las variables locales NO tienen valor por defecto

A diferencia de los atributos o campos de una clase (que la JVM inicializa automáticamente a `0`, `0.0`, `false` o `null`), **las variables locales declaradas dentro de un método NO reciben ningún valor por defecto**. Si intentas leer o utilizar una variable local sin haberle asignado un valor previamente, el compilador detendrá el proceso arrojando el error:
`java: variable edadUsuario might not have been initialized`

4.3 Constantes con la Palabra Clave `final`

Una **constante** es una variable cuyo valor no puede ser alterado una vez que ha sido asignado por primera vez. En Java se definen anteponiendo el modificador `final`:

```
public class ConstantesDemo {
    public static void main(String[] args) {
        final double TASA_IVA = 0.21;
        final int DIAS_SEMANA = 7;
        final String NOMBRE_APLICACION = "GestorDAM";

        // TASA_IVA = 0.10; // ¡ERROR de compilación! Cannot assign a value to final
variable
    }
}
```

Ventajas de utilizar constantes:

1. **Evitan los "números mágicos"**: Un código con valores como `3.14159` o `86400` repartidos por el código es difícil de entender y mantener. Al bautizarlos como `final double PI` o `final int SEGUNDOS_POR_DIA`, el código se autoexplica.
2. **Seguridad e Integridad**: Garantizan que parámetros críticos del sistema (tasas impositivas, límites de memoria, puertos de red) no puedan ser sobreescritos por accidente por otro método.
3. **Optimizaciones del compilador JIT**: Al saber que el valor jamás cambiará, la JVM puede sustituir la variable directamente por el valor literal en tiempo de ejecución (*inlining*), mejorando el rendimiento.

4.4 Inferencia de Tipos en Variables Locales con `var` (Java 10+)

A partir de Java 10 (2018), el lenguaje incorporó la palabra reservada contextual `var`, permitiendo al compilador inferir automáticamente el tipo de la variable a partir del valor de la expresión a su derecha:

```
// Declaración tradicional con tipo explícito
String mensaje = "Hola Mundo";
int contador = 42;
double precio = 99.95;

// Declaración moderna con inferencia de tipos (var)
var mensajeMod = "Hola Mundo"; // El compilador infiere String
var contadorMod = 42;          // El compilador infiere int
var precioMod = 99.95;         // El compilador infiere double
```

Reglas Estrictas de `var`:

1. **Java sigue siendo un lenguaje fuertemente tipado**: `var` **NO** convierte a Java en JavaScript o Python. El tipo de la variable se fija irrevocablemente durante la compilación. Una vez que `var contadorMod = 42` es compilado como `int`, intentar asignarle un texto (`contadorMod = "Texto"`) causará un error de compilación.
2. **Solo para variables locales con inicializador**: No se puede usar `var` en atributos de clase, ni en parámetros de métodos, ni en tipos de retorno, ni sin asignar un valor en la misma línea (`var x;` es ilegal).
3. **Buenas prácticas**: Usa `var` cuando el tipo sea evidente y reduzca la redundancia visual (por ejemplo al instanciar objetos complejos `var scanner = new Scanner(System.in);`). Evita su uso cuando el tipo de retorno de una función sea ambiguo y dificulte la lectura al desarrollador humano.

---

5. El Sistema de Tipos de Java: Los 8 Tipos Primitivos

Java es un lenguaje de **tipado estático y fuerte**:

- **Estático**: El tipo de cada variable se comprueba y valida en tiempo de compilación.
- **Fuerte**: No se permiten operaciones entre tipos incompatibles sin una conversión explícita.

En el nivel más bajo de la arquitectura de memoria de Java residen los **8 tipos primitivos**, diseñados para ofrecer el máximo rendimiento al mapearse directamente sobre las instrucciones del procesador.

```
Tipos de Datos en Java
├── Primitivos (8 tipos básicos almacenados directamente por valor en Stack)
│   ├── Numéricos Enteros (byte, short, int, long)
│   ├── Numéricos de Coma Flotante (float, double)
│   ├── Carácter (char)
│   └── Booleano (boolean)
└── Referenciados / Objetos (almacenados en Heap, referenciados por puntero)
    ├── Clases (String, Scanner, Integer, etc.)
    ├── Interfaces
    ├── Arrays (vectores y matrices)
    └── Records y Enums
```

5.1 Tabla Comparativa Completa de Tipos Primitivos

Tipo	Categoría	Tamaño	Rango de Valores	Literal / Sufijo	Valor Defecto (en clases)
`byte`	Entero	8 bits (1 byte)	-128 a 127	`(byte) 10`	`0`
`short`	Entero	16 bits (2 bytes)	-32.768 a 32.767	`(short) 3000`	`0`
`int`	Entero	32 bits (4 bytes)	-2.147.483.648 a 2.147.483.647 (-2^{31} a $2^{31}-1$)	`100`	`0`
`long`	Entero	64 bits (8 bytes)	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	`100L` o `100l`	`0L`
`float`	Flotante	32 bits (4 bytes)	$\pm 1.4\times10^{-45}$ a $\pm 3.4028235\times10^{38}$ (~7 dígitos de precisión)	`3.14f` o `3.14F`	`0.0f`
`double`	Flotante	64 bits (8 bytes)	$\pm 4.9\times10^{-324}$ a $\pm 1.7976931348623157\times10^{308}$ (~16 dígitos)	`3.14` o `3.14d`	`0.0d`
`char`	Carácter	16 bits (2 bytes)	Código Unicode UTF-16: `\u0000` (0) a `\uffff` (65.535)	`'A'`, `'\n'`, `'\u0041'`	`'\u0000'`
`boolean`	Lógico	JVM dependiente (~8 bits)	`true` o `false`	`true`, `false`	`false`

Aspectos Críticos a Recordar:

1. **Los enteros literales son siempre `int` por defecto**: Si escribes `3000000000` (3 mil millones), el compilador dará error porque supera el rango de `int`. Para convertirlo en un literal de tipo `long`, debes añadir obligatoriamente la letra `L` al final: `3000000000L` (utiliza siempre `L` mayúscula, ya que `l` minúscula se confunde visualmente con el número `1`).
2. **Los decimales literales son siempre `double` por defecto**: Si escribes `float f = 3.14;`, el compilador emitirá un error indicando pérdida de precisión (un `double` no cabe automáticamente en un `float`). Debes indicar `3.14f`.
3. **El tipo `boolean` es puro**: En Java, a diferencia de C/C++, **un booleano NUNCA equivale a 0 o 1**. `if (1) { ... }` o `boolean flag = 0;` son errores de compilación directos.
4. **El tipo `char` representa números de 16 bits sin signo**: Los caracteres se delimitan con comillas simples (`'A'`), mientras que los textos se delimitan con comillas dobles (`"A"`). Un `char` puede operarse aritméticamente: `'A' + 1` da el número entero `66` (código ASCII de `'B'`).

5.2 Notación de Literales Numéricos y Formatos Alternativos

Java admite representar literales numéricos enteros en distintas bases numéricas y con separadores de legibilidad:

```
// 1. Sistema Decimal (base 10 - habitual)
int decimal = 100;

// 2. Sistema Hexadecimal (base 16 - prefijo 0x o 0X)
int hex = 0x2A; // Equivale a 42 en decimal (2 * 16 + 10)
int colorBlanco = 0xFFFFFF;

// 3. Sistema Binario (base 2 - prefijo 0b o 0B, introducido en Java 7)
int mascaraBinaria = 0b101010; // Equivale a 42 en decimal

// 4. Sistema Octal (base 8 - prefijo 0)
// ¡CUIDADO! Un número que empieza por 0 se interpreta como octal
int octal = 052; // Equivale a 42 en decimal (5 * 8 + 2)

// 5. Separador visual con guion bajo (Java 7+)
// No altera el valor numérico, solo mejora drásticamente la legibilidad humana
long tarjetaCredito = 4552_1100_8923_4321L;
double unMillon = 1_000_000.00;
int bytesMascara = 0b1111_0000_1010_0101;
```

5.3 Desbordamiento Numérico Cíclico (*Integer Overflow*)

Los tipos primitivos enteros en Java utilizan la codificación de **complemento a dos** (*two's complement*) para representar valores positivos y negativos. ¿Qué sucede si sumamos 1 al valor máximo que puede almacenar un tipo entero?

```
byte b = 127; // Máximo valor positivo representable en 8 bits (-128 a 127)
b++;          // Sumamos 1
System.out.println(b); // Imprime: -128
```

> **El Desbordamiento NO lanza ninguna excepción**
>
> En Java, las operaciones aritméticas primitivas que superan los límites de rango **no detienen el programa ni lanzan errores**. Simplemente se produce un desbordamiento cíclico silencioso: el bit de signo cambia y el valor pasa al extremo negativo opuesto.
>
> Si estás trabajando con cálculos matemáticos críticos, acumuladores financieros o conteos masivos, debes utilizar la clase `Math` con sus métodos de comprobación exacta introducidos en Java 8, tales como `Math.addExact(a, b)` o `Math.multiplyExact(a, b)`, los cuales lanzan inmediatamente una excepción `ArithmeticException` si detectan un desbordamiento.

---

6. Conversión de Tipos (*Type Casting*) y Clases Envoltorio (*Wrappers*)

A menudo necesitamos transferir datos entre variables de diferente naturaleza. Dependiendo de si existe o no riesgo de pérdida de información, Java clasifica las conversiones en dos grandes categorías.

6.1 Conversión Implícita o Promoción (*Widening Casting*)

Ocurre de manera totalmente **automática y transparente** cuando se asigna un valor de un tipo con menor capacidad o rango a una variable de un tipo con mayor capacidad. Como no existe peligro de pérdida de magnitud, el compilador lo permite sin advertencias:

```
byte → short → int → long → float → double
         char ↗
```

```
int numeroEntero = 100;
long numeroLargo = numeroEntero;     // Automático: int (32 bits) a long (64 bits)
double numeroReal = numeroLargo;     // Automático: long a double (64 bits coma
flotante)
System.out.println(numeroReal);      // Imprime: 100.0
```

6.2 Conversión Explícita o Truncamiento (*Narrowing Casting*)

Ocurre cuando intentamos guardar un dato de mayor rango o precisión en un contenedor más pequeño. Como existe un **riesgo real de pérdida de información o truncamiento de decimales**, el compilador exige que el programador declare su intención explícitamente mediante el operador de moldeado `(tipo)`:

```
double precioConIva = 99.89;
// int precioEntero = precioConIva; // ¡ERROR de compilación! Possible loss of
precision

// Casting explícito: se descartan por completo los decimales (truncamiento hacia cero)
int precioEntero = (int) precioConIva;
System.out.println(precioEntero); // Imprime: 99 (¡se han perdido los 0.89!)

// Peligro de desbordamiento en narrowing:
int valorGrande = 300;
byte valorByte = (byte) valorGrande; // 300 no cabe en 8 bits (-128 a 127)
System.out.println(valorByte);       // Imprime: 44 (resultado del desbordamiento
binario)
```

6.3 Conversión entre Cadenas (`String`) y Tipos Numéricos

La interacción por consola, archivos o interfaces gráficas siempre se realiza mediante texto (`String`). Para realizar cálculos aritméticos con esos datos, debemos convertirlos a sus tipos primitivos equivalentes mediante los métodos de parseo de las clases envoltorio:

```
// 1. De String a Tipos Primitivos (Parseo)
String textoEdad = "25";
int edad = Integer.parseInt(textoEdad);

String textoSueldo = "1850.75";
double sueldo = Double.parseDouble(textoSueldo);

String textoActivo = "true";
boolean activo = Boolean.parseBoolean(textoActivo);

// 2. De Tipos Primitivos a String
int dias = 365;
String s1 = String.valueOf(dias);    // Forma óptima y recomendada
String s2 = Integer.toString(dias);  // Equivalente directo
String s3 = "" + dias;               // Válido por concatenación, pero menos eficiente
```

La temida excepción `NumberFormatException`

Si intentas parsear una cadena que contiene caracteres no numéricos o espacios en blanco residuales (por ejemplo, `Integer.parseInt(" 25a ")`), la JVM detendrá el programa inmediatamente arrojando un error de tipo `java.lang.NumberFormatException`. Más adelante aprenderemos a capturarlo mediante bloques `try-catch`.

6.4 Clases Envoltorio (*Wrapper Classes*) y Autoboxing/Unboxing

Dado que Java es un lenguaje orientado a objetos pero los tipos primitivos no son objetos (no tienen métodos ni atributos), Java proporciona una **Clase Envoltorio** (*Wrapper*) correspondiente a cada uno de los 8 tipos primitivos dentro del paquete estándar `java.lang`:

Tipo Primitivo	Clase Envoltorio (*Wrapper*)	Métodos y Constantes de Gran Utilidad
`byte`	`Byte`	`Byte.MAX_VALUE`, `Byte.MIN_VALUE`, `Byte.parseByte()`
`short`	`Short`	`Short.MAX_VALUE`, `Short.MIN_VALUE`
`int`	`Integer`	`Integer.MAX_VALUE`, `Integer.toBinaryString()`, `Integer.compare()`
`long`	`Long`	`Long.MAX_VALUE`, `Long.parseLong()`
`float`	`Float`	`Float.isNaN()`, `Float.isInfinite()`
`double`	`Double`	`Double.POSITIVE_INFINITY`, `Double.parseDouble()`
`char`	`Character`	`Character.isDigit()`, `Character.isLetter()`, `Character.toUpperCase()`
`boolean`	`Boolean`	`Boolean.TRUE`, `Boolean.FALSE`, `Boolean.logicalAnd()`

Autoboxing y Unboxing Automático

Desde Java 5, el compilador realiza de forma totalmente automática la conversión entre el tipo primitivo y su correspondiente objeto envoltorio:

- **Autoboxing**: Conversión automática de un primitivo a su clase Wrapper (`int` → `Integer`).
- **Unboxing**: Conversión automática de un objeto Wrapper a su primitivo básico (`Integer` → `int`).

```
// Autoboxing: el compilador ejecuta internamente Integer.valueOf(50)
Integer objetoEntero = 50;

// Unboxing: el compilador extrae el valor primitivo con objetoEntero.intValue()
int primitivo = objetoEntero;

// Operaciones aritméticas transparentes entre wrappers y primitivos
objetoEntero = objetoEntero + 10;
```

La Trampa Mortal del Unboxing con valores `null`

Un primitivo (`int`) siempre tiene un valor numérico. Sin embargo, un objeto Wrapper (`Integer`) es una referencia y **puede valer** `null` . Si la JVM intenta realizar un unboxing sobre una referencia nula, provocará una catástrofe en tiempo de ejecución:

```
Integer puntos = null; // No tiene ningún objeto asociado
int resultado = puntos + 5; // ¡Lanza NullPointerException en tiempo de ejecución!
```

---

7. Operadores y Precedencia

Los operadores son símbolos especiales que indican a la CPU que realice cálculos matemáticos, comparaciones lógicas o manipulaciones de bits sobre uno o más operandos.

7.1 Operadores Aritméticos y Asignación Compuesta

Java incluye los 5 operadores aritméticos fundamentales:

```
int a = 15;
int b = 4;

int suma = a + b;           // 19
int resta = a - b;          // 11
int multiplicacion = a * b; // 60
int divisionEntera = a / b; // 3  (¡Atención: trunca la parte decimal!)
int modulo = a % b;         // 3  (Resto exacto de la división: 15 = 4 * 3 + 3)
```

División Entera vs División Real

Si ambos operandos son números enteros, el operador `/` efectúa una **división entera**, ignorando por completo cualquier residuo fraccionario. Para obtener un resultado con decimales, **al menos uno de los operandos debe ser de coma flotante** (`double` o `float`):

```
int x = 7;
int y = 2;
double r1 = x / y;          // Evalúa 7 / 2 = 3 → asigna 3.0 (¡Error común!)
double r2 = (double) x / y;  // Hace casting de x a 7.0 → 7.0 / 2 = 3.5 (¡Correcto!)
```

Aplicaciones Prácticas del Operador Módulo (`%`)

El operador `%` no es una simple curiosidad matemática; es una de las herramientas más utilizadas en la algoritmia diaria:

1. **Comprobar si un número es par o impar**: `if (n % 2 == 0) // Es par`
2. **Comprobar múltiplos exactos**: `if (anio % 4 == 0) // Posible bisiesto`
3. **Controlar ciclos y relojes circulares**: `hora = (hora + 1) % 24;`
4. **Descomposición de dígitos decimales**:
    - Última cifra de un número: `numero % 10`
    - Eliminar la última cifra: `numero / 10`

Operadores de Asignación Compuesta

Permiten abreviar operaciones donde una variable se modifica a sí misma:

Operador Compuesto	Expresión Equivalente	Ejemplo
`+=`	`a = a + b`	`saldo += ingreso;`
`-=`	`a = a - b`	`stock -= unidadesVendidas;`
`*=`	`a = a * b`	`precio *= 1.21;`
`/=`	`a = a / b`	`puntos /= 2;`
`%=`	`a = a % b`	`minutos %= 60;`

7.2 Operadores de Incremento y Decremento

Java proporciona los operadores unarios `++` (sumar 1) y `--` (restar 1). Pueden situarse antes de la variable (**prefijo**) o después (**postfijo**), y su comportamiento en expresiones combinadas es profundamente diferente:

```
int x = 5;
int y = 5;

// Postfijo (x++): Se utiliza primero el valor actual en la expresión, y luego se
incrementa
int resultadoPost = x++;
System.out.println(resultadoPost); // Imprime: 5
System.out.println(x);             // Imprime: 6

// Prefijo (++y): Se incrementa primero el valor, y luego se utiliza en la expresión
int resultadoPre = ++y;
System.out.println(resultadoPre);  // Imprime: 6
System.out.println(y);             // Imprime: 6
```

> **Consejo de Legibilidad**
>
> Salvo en la cabecera de bucles `for` (donde `i++` es estándar), evita incrustar operadores `++` y `--` dentro de expresiones aritméticas complejas o llamadas a métodos. Separar la operación en una línea independiente (`x++;`) hace que el código sea inmune a confusiones lógicas.

7.3 Operadores Relacionales y de Comparación

Se utilizan para comparar dos valores. El resultado de cualquier operación relacional es siempre un valor primitivo `boolean` (`true` o `false`):

Operador	Significado	Ejemplo (`a = 10` , `b = 20`)	Resultado
`==`	Igual a	`a == b`	`false`
`≠`	Distinto de	`a ≠ b`	`true`
`>`	Mayor que	`a > b`	`false`
`<`	Menor que	`a < b`	`true`
`≥`	Mayor o igual que	`a ≥ 10`	`true`
`≤`	Menor o igual que	`b ≤ 20`	`true`

7.4 Operadores Lógicos y Cortocircuito (*Short-Circuit Evaluation*)

Permiten combinar múltiples expresiones booleanas para formar condiciones complejas:

Operador	Nombre Lógico	Comportamiento
`&&`	**AND lógico condicional**	Devuelve `true` si y solo si **ambos operandos** son `true`. Con cortocircuito.
`||`	**OR lógico condicional**	Devuelve `true` si **al menos uno** de los operandos es `true`. Con cortocircuito.
`!`	**NOT (Negación)**	Invierte el valor booleano: `!true` da `false`, `!false` da `true`.
`^`	**XOR (O exclusivo)**	Devuelve `true` si uno de los operandos es `true` y el otro `false`.
`&`	**AND estricto / lógico**	Evalúa **siempre** ambos operandos (sin cortocircuito).
`|`	**OR estricto / lógico**	Evalúa **siempre** ambos operandos (sin cortocircuito).

¿Qué es la Evaluación en Cortocircuito (*Short-Circuit*)?

Los operadores `&&` y `||` son enormemente eficientes y seguros gracias a la evaluación en cortocircuito:

- En una expresión `A && B`, si `A` resulta ser `false`, el resultado global será inevitablemente `false`. Java **no se molesta en evaluar la expresión** `B`.
- En una expresión `A || B`, si `A` resulta ser `true`, el resultado global será indefectiblemente `true`. Java **omite la evaluación de** `B`.

Patrón Defensivo Fundamental en Java: El cortocircuito permite proteger nuestro programa contra fallos críticos en tiempo de ejecución:

```
String texto = null;

// Protección perfecta contra NullPointerException:
// Si texto == null es true, texto.length() JAMÁS llega a ejecutarse
if (texto ≠ null && texto.length() > 0) {
    System.out.println("El texto no está vacío");
}

// División segura contra división por cero:
int divisor = 0;
if (divisor ≠ 0 && (100 / divisor > 2)) {
    System.out.println("Operación válida");
}
```

Si hubieras utilizado el operador `&` en lugar de `&&`, Java habría evaluado `texto.length()` o `100 / divisor`, estrellando la aplicación con un error fatal.

7.5 Operador Condicional Ternario (`?:`)

Es el único operador ternario de Java (requiere tres operandos). Constituye una versión compacta de una sentencia `if-else` que devuelve directamente un valor evaluado:

```
variable = (condicion) ? valor_si_verdadero : valor_si_falso;
```

```
int edad = 19;
String estado = (edad ≥ 18) ? "Mayor de edad" : "Menor de edad";

// Cálculo de valor absoluto:
int valor = -45;
int absoluto = (valor < 0) ? -valor : valor;
```

7.6 Tabla Completa de Precedencia y Asociatividad de Operadores

Cuando una misma expresión contiene múltiples operadores, Java sigue un orden estricto de precedencia matemática y lógica para determinar qué operación se resuelve primero:

Precedencia	Tipo de Operador	Operadores	Asociatividad
1 (Máxima)	Acceso, llamadas, corchetes	`.` `[]` `()`	De izquierda a derecha
2	Unarios y sufijos	`expr++` `expr--`	De derecha a izquierda
3	Unarios y prefijos	`++expr` `--expr` `+` `-` `!` `~` `(tipo)`	De derecha a izquierda
4	Multiplicativos	`*` `/` `%`	De izquierda a derecha
5	Aditivos	`+` `-`	De izquierda a derecha
6	Desplazamiento de bits	`<<` `>>` `>>>`	De izquierda a derecha
7	Relacionales	`<` `>` `≤` `≥` `instanceof`	De izquierda a derecha
8	Igualdad	`==` `≠`	De izquierda a derecha
9	AND a nivel de bits	`&`	De izquierda a derecha
10	XOR a nivel de bits	`^`	De izquierda a derecha
11	OR a nivel de bits	`|`	De izquierda a derecha
12	AND lógico (cortocircuito)	`&&`	De izquierda a derecha
13	OR lógico (cortocircuito)	`||`	De izquierda a derecha
14	Ternario	`? :`	De derecha a izquierda
15 (Mínima)	Asignación	`=` `+=` `-=` `*=` `/=` `%=` etc.	De derecha a izquierda

> **Usa Paréntesis Sin Miedo**
>
> No intentes memorizar toda la tabla de precedencia ni obligues a otros desarrolladores a descifrar expresiones complejas como `x + y * z > w && a || b`. **Usa paréntesis deliberadamente**: clarifican la intención del algoritmo y garantizan que el orden de evaluación sea inequívoco.

---

8. Clases Estándar Fundamentales de la Biblioteca de Java

El Java Development Kit (JDK) incluye miles de clases listas para usar dentro del paquete `java.lang` (que se importa de forma automática e implícita en todos los ficheros `.java`) y otros paquetes estándar como `java.time` o `java.math`.

8.1 La Clase `Math`

Ubicada en `java.lang.Math`, ofrece una colección completa de métodos matemáticos y constantes estáticas:

```
// 1. Constantes matemáticas universales
double pi = Math.PI; // 3.141592653589793
double e = Math.E;   // 2.718281828459045

// 2. Funciones de cálculo y raíces
double potencia = Math.pow(2, 8);      // 2 elevado a 8 = 256.0
double raizCuadrada = Math.sqrt(49);  // 7.0
double raizCubica = Math.cbrt(27);    // 3.0
int absoluto = Math.abs(-15);         // 15

// 3. Métodos de redondeo
double valor = 4.7;
long redondeo = Math.round(valor);    // 5 (redondeo estándar al entero más próximo)
double suelo = Math.floor(valor);     // 4.0 (mayor entero menor o igual al valor)
double techo = Math.ceil(valor);      // 5.0 (menor entero mayor o igual al valor)

// 4. Comparaciones extremas
int maximo = Math.max(10, 25);        // 25
int minimo = Math.min(10, 25);        // 10
```

Generación de Números Aleatorios con `Math.random()`

El método `Math.random()` devuelve un número pseudoaleatorio de tipo `double` comprendido en el intervalo semiabierto `[0.0, 1.0)` (incluye el `0.0`, pero nunca alcanza el `1.0`).

Para generar un número entero aleatorio dentro de un rango inclusivo arbitrario `[min, max]`, aplicamos la siguiente fórmula estándar:

```
// Fórmula Universal: (int)(Math.random() * (max - min + 1)) + min
int min = 1;
int max = 6;
int dado = (int)(Math.random() * (max - min + 1)) + min;
System.out.println("Tirada de dado: " + dado);
```

8.2 Cadenas de Texto con la Clase `String`

En Java, un texto **no es un tipo primitivo**, sino un objeto de la clase `java.lang.String`.

La Gran Característica: Inmutabilidad de los Strings

Una vez que un objeto `String` es creado en la memoria Heap, **su contenido jamás puede ser modificado**. Cualquier método que supuestamente "modifique" un `String` (como `.toUpperCase()`, `.trim()` o `.replace()`) en realidad genera un **nuevo objeto** `String` en memoria con los cambios, dejando el original intacto:

```
String saludo = "hola";
saludo.toUpperCase(); // ¡No modifica 'saludo'! Crea un nuevo String "HOLA" en el aire
System.out.println(saludo); // Imprime: hola

// Para conservar los cambios, debemos reasignar la referencia:
saludo = saludo.toUpperCase();
System.out.println(saludo); // Imprime: HOLA
```

El String Pool y el Gran Error de Comparar Cadenas con `==`

La JVM optimiza el uso de memoria gestionando una zona especial en el Heap llamada **String Constant Pool**. Cuando creas un literal de texto como `String a = "Java"`, la JVM lo almacena en el pool. Si luego creas `String b`

`= "Java"`, ambas variables apuntan a la misma dirección física de memoria.

Sin embargo, si la cadena se crea en tiempo de ejecución (por ejemplo, leyéndola con `Scanner` o con `new String("Java")`), se aloja en un espacio de memoria distinto:

```
String s1 = "DAM";
String s2 = "DAM";
String s3 = new String("DAM");

System.out.println(s1 == s2); // true (¡Coincidencia fortuita! Apuntan al mismo String Pool)
System.out.println(s1 == s3); // FALSE (¡Error catastrófico! Son dos objetos distintos en Heap)

// REGLA UNIVERSAL E INQUEBRANTABLE EN JAVA:
// Para comparar el contenido de objetos y textos, usa SIEMPRE .equals()
System.out.println(s1.equals(s3)); // TRUE (compara los caracteres reales)
System.out.println(s1.equalsIgnoreCase("dam")); // TRUE (ignora mayúsculas/minúsculas)
```

> **Prohibido usar `==` con objetos en Java**
>
> El operador `==` compara **direcciones de memoria física** (si ambos punteros señalan al mismísimo objeto). Para comparar si dos cadenas tienen el mismo texto alfanumérico, debes usar **siempre** `cadena1.equals(cadena2)`.

Métodos Clave Indispensables de la Clase `String`:

```
String frase = "  Aprender Java en DAM es Fantástico!  ";

// 1. Longitud del texto
int totalCaracteres = frase.length(); // 39

// 2. Extracción de caracteres por índice (0-based)
char primerCaracter = frase.charAt(2); // 'A'

// 3. Subcadenas (índice inicio inclusivo, índice fin exclusivo)
String sub = frase.substring(2, 10); // "Aprender"

// 4. Limpieza de espacios en blanco
String limpia = frase.strip(); // "Aprender Java en DAM es Fantástico!" (Java 11+)

// 5. Búsqueda de patrones
boolean contieneJava = limpia.contains("Java"); // true
boolean empiezaPor = limpia.startsWith("Aprender"); // true
int posicionDam = limpia.indexOf("DAM"); // 17 (-1 si no existe)

// 6. Reemplazo de caracteres o palabras
String sustituida = limpia.replace("DAM", "1º DAM");

// 7. Comprobación de vacíos (Java 11+)
boolean vacia = "".isEmpty(); // true
boolean enBlanco = "   ".isBlank(); // true (solo contiene espacios en blanco)
```

Bloques de Texto (*Text Blocks* - Java 15+)

Para textos multilínea extensos (como consultas SQL, fragmentos HTML o JSON), Java 15 introdujo los bloques de texto delimitados por tres comillas dobles `"""`:

```
String consultaSql = """
    SELECT id, nombre, apellidos, saldo
    FROM usuarios
    WHERE activo = true
    ORDER BY apellidos ASC;
    """;
```

8.3 Gestión Moderna de Fechas y Tiempo (`java.time`)

Históricamente, Java utilizaba las clases `java.util.Date` y `java.util.Calendar`. Dichas clases fueron declaradas obsoletas debido a graves fallos de diseño: los meses empezaban en 0 (enero era el mes 0), los años empezaban en 1900, eran objetos mutables que provocaban errores de concurrencia y sus métodos tenían nombres confusos.

Desde Java 8, disponemos de la moderna **API** `java.time` (basada en el estándar internacional ISO-8601), que es **completamente inmutable y segura frente a hilos**:

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.temporal.TemporalAdjusters;

public class FechasDemo {
    public static void main(String[] args) {
        // 1. Obtener la fecha y hora actual del sistema
        LocalDate hoy = LocalDate.now();
        LocalTime horaActual = LocalTime.now();
        LocalDateTime momentoActual = LocalDateTime.now();

        // 2. Crear fechas concretas (los meses se indican del 1 al 12 o con el enum
Month)
        LocalDate inicioCurso = LocalDate.of(2026, 9, 15);

        // 3. Aritmética de fechas (al ser inmutable, devuelve una nueva fecha)
        LocalDate dentroDeDosSemanas = hoy.plusWeeks(2);
        LocalDate haceTresMeses = hoy.minusMonths(3);

        // 4. Cálculo de periodos transcurridos
        Period diferencia = Period.between(inicioCurso, hoy);
        System.out.printf("Tiempo transcurrido: %d meses y %d días%n",
            diferencia.getMonths(), diferencia.getDays());

        // 5. Ajustadores temporales avanzados (ej. último día del mes actual)
        LocalDate finDeMes = hoy.with(TemporalAdjusters.lastDayOfMonth());

        // 6. Formateo y parseo profesional
        DateTimeFormatter formatoEspanol = DateTimeFormatter.ofPattern("dd/MM/yyyy
HH:mm:ss");
        String fechaTexto = momentoActual.format(formatoEspanol);
        System.out.println("Fecha formateada: " + fechaTexto);

        // Parsear un texto a fecha
        LocalDate fechaParseada = LocalDate.parse("25/12/2026",
DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }
}
```

8.4 Precisión Financiera y Números Gigantes: `BigDecimal` y `BigInteger`

¿Cuánto es `0.1 + 0.2` en matemáticas? Es exactamente `0.3`.
Sin embargo, comprueba qué imprime Java al ejecutar este código:

```
double suma = 0.1 + 0.2;
System.out.println(suma); // Imprime: 0.30000000000000004
```

El Problema del Estándar IEEE 754 de Coma Flotante

Los ordenadores utilizan base binaria (ceros y unos). En binario, fracciones sencillas en base decimal como `0.1` (1/10) o `0.2` (1/5) se convierten en números periódicos infinitos (análogo a un tercio `1/3 = 0.33333...` en base decimal). Al tener un número finito de bits (64 bits en `double`), el procesador debe truncar la cifra, generando un diminuto error de redondeo.

En un videojuego o en un cálculo gráfico, ese error es irrelevante. Pero en **aplicaciones bancarias, nóminas, facturas comerciales o criptografía, este error acumulado es inaceptable**.

La Solución Oficial: `BigDecimal`

Para cálculos monetarios exactos, Java proporciona la clase `java.math.BigDecimal`:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

public class FinanzasDemo {
    public static void main(String[] args) {
        // REGLA FUNDAMENTAL: Instanciar siempre BigDecimal usando Strings, no doubles
        BigDecimal precio = new BigDecimal("0.1");
        BigDecimal iva = new BigDecimal("0.2");

        BigDecimal total = precio.add(iva);
        System.out.println("Total exacto: " + total); // Imprime: 0.3

        // Operaciones aritméticas: add, subtract, multiply, divide
        BigDecimal subtotal = new BigDecimal("100.00");
        BigDecimal tasa = new BigDecimal("0.21");
        BigDecimal impuesto = subtotal.multiply(tasa);

        // División con control estricto de redondeo a 2 decimales
        BigDecimal cuota = subtotal.divide(new BigDecimal("3"), 2,
RoundingMode.HALF_UP);
        System.out.println("Cuota mensual: " + cuota); // Imprime: 33.33
    }
}
```

Números Enteros de Precisión Arbitraria: `BigInteger`

Si necesitas calcular números astronómicos (por ejemplo, el factorial de 100 o claves criptográficas RSA de 2048 bits) que superan el límite de 64 bits de `long` (~9 trillones), utiliza `java.math.BigInteger`:

```
import java.math.BigInteger;

BigInteger n1 = new BigInteger("92233720368547758071234567890");
BigInteger n2 = new BigInteger("50000000000000000000000000000");
BigInteger producto = n1.multiply(n2);
```

Formateo de Monedas y Números con `DecimalFormat`

Para mostrar importes numéricos con separadores de miles y decimales acordes al formato español:

```
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.util.Locale;

double salario = 2450.60;
DecimalFormat df = new DecimalFormat("#,##0.00 €", new
DecimalFormatSymbols(Locale.of("es", "ES")));
System.out.println(df.format(salario)); // Imprime: 2.450,60 €
```

8.5 Tipos Enumerados (`enum`)

Cuando una variable solo puede tomar un conjunto cerrado, finito y conocido de valores posibles (por ejemplo, los días de la semana, los meses del año, los estados de un pedido o los niveles de dificultad de un juego), utilizar cadenas (`String`) o números enteros (`int`) es propenso a errores tipográficos.

Para resolverlo, Java cuenta con el tipo especial `enum`:

```
public class EnumDemo {
    // Definición del tipo enumerado
    public enum NivelAcceso {
        INVITADO,
        USUARIO,
        ADMINISTRADOR
    }

    public static void main(String[] args) {
        // Uso con seguridad estricta de tipos
        NivelAcceso rolActual = NivelAcceso.USUARIO;

        if (rolActual == NivelAcceso.ADMINISTRADOR) {
            System.out.println("Acceso concedido a la consola de control");
        } else {
            System.out.println("Acceso limitado: Privilegios insuficientes");
        }
    }
}
```

---

9. Entrada y Salida Estándar por Consola

Todo programa interactivo requiere un canal para recibir datos del usuario y un canal para proyectar sus resultados.

9.1 Salida por Consola: `print()`, `println()` y `printf()`

En Java, la consola se representa mediante el objeto estático `System.out`:

- `System.out.print( ... )`: Muestra el contenido sin salto de línea al final.
- `System.out.println( ... )`: Muestra el contenido y añade automáticamente un salto de línea (`\n`).
- `System.out.printf( ... )`: Permite dar formato avanzado a la salida combinando texto con especificadores de formato (similar a la función `printf` clásica de C).

Tabla Completa de Especificadores de Formato para `printf()`

Especificador	Tipo de Dato	Ejemplo de Entrada	Resultado Formateado
`%d`	Entero decimal (`int` , `long` , `byte` , `short`)	`printf("%d", 42)`	`42`
`%f`	Coma flotante (`float` , `double`)	`printf("%.2f", 3.14159)`	`3,14` (o `3.14` según Locale)
`%s`	Cadena de texto (`String`)	`printf("%s", "Java")`	`Java`
`%c`	Carácter primitivo (`char`)	`printf("%c", 'A')`	`A`
`%b`	Booleano (`boolean`)	`printf("%b", true)`	`true`
`%n`	Salto de línea independiente del SO	`printf("Hola%nMundo")`	Genera salto de línea universal
`%%`	Carácter de porcentaje literal	`printf("%d%%", 100)`	`100%`

Modificadores de Ancho, Alineación y Relleno

Podemos intercalar modificadores entre el `%` y la letra del especificador:

- `%10d` : Justifica a la derecha en un ancho mínimo de 10 columnas.
- `%-10s` : Justifica a la izquierda rellenando con espacios hasta 10 caracteres.
- `%05d` : Rellena con ceros a la izquierda hasta completar 5 dígitos (ej. `00007`).
- `%.2f` : Fija exactamente dos posiciones decimales tras la coma.

```
String producto = "Teclado Mecánico";
int unidades = 4;
double precioUnitario = 79.99;
double total = unidades * precioUnitario;

System.out.println("————————————————————————————————————————————————");
System.out.printf("%-20s %5s %10s %10s%n", "PRODUCTO", "CANT", "PRECIO", "TOTAL");
System.out.println("————————————————————————————————————————————————");
System.out.printf("%-20s %5d %10.2f %10.2f €%n", producto, unidades, precioUnitario,
total);
System.out.println("————————————————————————————————————————————————");
```

9.2 Entrada de Datos con la Clase `Scanner`

Para leer datos introducidos por el usuario a través del teclado, la forma estándar en Java es utilizar la clase `java.util.Scanner` , vinculándola al flujo de entrada estándar del sistema `System.in` :

```
import java.util.Scanner;
import java.util.Locale;

public class EntradaDatosDemo {
    public static void main(String[] args) {
        // Creamos una instancia de Scanner asociada al teclado
        Scanner teclado = new Scanner(System.in);

        // Configuramos el punto como separador decimal estándar internacional
        teclado.useLocale(Locale.US);

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.nextLine(); // Lee una línea completa de texto

        System.out.print("Introduce tu edad: ");
        int edad = teclado.nextInt(); // Lee un número entero

        System.out.print("Introduce tu estatura en metros (ej. 1.78): ");
        double estatura = teclado.nextDouble(); // Lee un número decimal

        System.out.printf("Hola %s, tienes %d años y mides %.2f metros.%n", nombre,
edad, estatura);

        // Buena práctica: cerrar el Scanner al finalizar el programa
        teclado.close();
    }
}
```

Métodos Principales de Lectura de `Scanner`:

- `nextLine()`: Lee todo el texto hasta que el usuario pulsa Enter (incluye espacios).
- `next()`: Lee únicamente la siguiente palabra (se detiene en el primer espacio en blanco).
- `nextInt()`: Lee el siguiente token interpretándolo como `int`.
- `nextDouble()`: Lee el siguiente token interpretándolo como `double`.
- `nextBoolean()`: Lee `true` o `false`.

9.3 ⭐ La Trampa del Salto de Línea Residual en `Scanner`

Este es, sin lugar a dudas, **el error número uno que confunde a todos los estudiantes de programación en Java**.

Imagina que ejecutas este código:

```
Scanner sc = new Scanner(System.in);

System.out.print("Introduce tu edad: ");
int edad = sc.nextInt();

System.out.print("Introduce tu nombre completo: ");
String nombre = sc.nextLine(); // ¡SE LO SALTA! No deja escribir nada

System.out.println("Nombre: " + nombre + ", Edad: " + edad);
```

¿Por qué ocurre esto?

Cuando el usuario teclea `25` y pulsa la tecla `Enter`, en el búfer de entrada de la consola se depositan tres caracteres: `'2'`, `'5'` y el carácter de salto de línea `'\n'`.

1. El método `sc.nextInt()` lee únicamente los caracteres numéricos `'2'` y `'5'`, pero **deja el `\n` abandonado en el búfer**.
2. A continuación, `sc.nextLine()` entra en acción buscando una línea hasta encontrar un `\n`.
3. Inmediatamente se encuentra con el `\n` que dejó abandonado `nextInt()`. Como ya ha encontrado un fin de línea, consume el `\n`, devuelve una cadena vacía `""` y el programa continúa sin esperar a que el usuario escriba su nombre.

```
Búfer tras teclear '25' + Enter:
[ '2' | '5' | '\n' ]
   ▲
   nextInt() lee '25' y se detiene aquí. El '\n' permanece en el búfer.

Cuando llega nextLine():
Encuentra inmediatamente el '\n' remanente y termina devolviendo cadena vacía.
```

La Solución: Purgar el Búfer

Siempre que vayas a leer un texto con `nextLine()` después de haber leído un número con `nextInt()`, `nextDouble()` o cualquier otro método que no sea `nextLine()`, debes realizar una **lectura de purga** intermedia para limpiar el salto de línea residual:

```
System.out.print("Introduce tu edad: ");
int edad = sc.nextInt();

sc.nextLine(); // ¡PURGA DEL BÚFER! Consume el '\n' residual abandonado

System.out.print("Introduce tu nombre completo: ");
String nombre = sc.nextLine(); // Ahora sí se detiene y espera la entrada del usuario
```

---

10. Sentencias de Decisión y Control de Flujo

Por defecto, la Máquina Virtual de Java ejecuta las instrucciones de forma estrictamente secuencial: una línea tras otra, desde la primera sentencia del método `main` hasta la última. Sin embargo, los algoritmos reales requieren tomar bifurcaciones (*branching*) y elegir caminos alternativos en función del estado de las variables y las decisiones del usuario.

10.1 Las Sentencias Condicionales: `if`, `if-else` y Escaleras `if-else if`

La estructura `if` evalúa una expresión lógica de tipo `boolean`. Si la condición resulta ser `true`, ejecuta el bloque de instrucciones delimitado por llaves; si es `false`, omite su ejecución:

```
int edad = 18;

// 1. Estructura if simple
if (edad ≥ 18) {
    System.out.println("Acceso autorizado: Usuario mayor de edad.");
}

// 2. Estructura if-else (Bifurcación completa)
double saldo = 45.50;
double precioArticulo = 60.00;

if (saldo ≥ precioArticulo) {
    saldo -= precioArticulo;
    System.out.println("Compra realizada con éxito. Saldo restante: " + saldo);
} else {
    double falta = precioArticulo - saldo;
    System.out.printf("Saldo insuficiente. Faltan %.2f € para completar la compra.%n",
falta);
}
```

Escalera de Condiciones Múltiples: `if - else if - else`

Cuando debemos clasificar una situación entre múltiples rangos excluyentes:

```
double nota = 7.8;

if (nota < 0.0 || nota > 10.0) {
    System.out.println("Error: Calificación fuera de rango válido (0-10).");
} else if (nota < 5.0) {
    System.out.println("Suspenso");
} else if (nota < 6.5) {
    System.out.println("Aprobado");
} else if (nota < 8.5) {
    System.out.println("Notable");
} else if (nota < 10.0) {
    System.out.println("Sobresaliente");
} else {
    System.out.println("Matrícula de Honor");
}
```

> **El Peligro Mortal de Omitir las Llaves `{}`**
>
> En Java, si una cláusula `if` o `else` contiene una única sentencia, las llaves `{}` son opcionales según la gramática del compilador. **Sin embargo, en el desarrollo de software profesional está terminantemente prohibido omitirlas**.
>
> Observa este clásico desastre:
>
> ```
> // Código engañoso sin llaves:
> if (usuarioEsAdmin)
>     registrarAcceso();
>     borrarBaseDeDatos(); // ¡PELIGRO! Esta línea se ejecuta SIEMPRE, sea admin o
> no
> ```
>
> La sangría visual engaña al ojo humano, pero el compilador solo vincula la primera sentencia (`registrarAcceso()`) al `if`. La segunda línea no forma parte del condicional. El infame fallo de seguridad SSL de Apple en 2014 (*goto fail;*) que costó millones de dólares se debió exactamente a omitir las llaves. **Pon siempre llaves `{}` en todos tus bloques de control**.

10.2 La Sentencia `switch` Clásica

Cuando una misma variable debe compararse frente a múltiples valores constantes discretos, encadenar decenas de `if-else if` resulta farragoso y poco eficiente. Para estos escenarios, Java proporciona la sentencia `switch`:

Tipos de Datos Admitidos por `switch`:

Java permite utilizar `switch` únicamente con los siguientes tipos:

- Primitivos enteros pequeños: `byte`, `short`, `char`, `int`.
- Sus correspondientes clases envoltorio: `Byte`, `Short`, `Character`, `Integer`.
- Cadenas de texto `String` (admitido desde Java 7).
- Tipos enumerados `enum` (admitido desde Java 5).

Tipos NO admitidos en switch

En Java **está prohibido** utilizar `switch` con variables de tipo `long`, `float`, `double` y `boolean`. Para números decimales o rangos continuos debes emplear `if-else`.

```
int diaSemana = 3;
String nombreDia;

switch (diaSemana) {
    case 1:
        nombreDia = "Lunes";
        break; // Detiene la ejecución y sale del bloque switch
    case 2:
        nombreDia = "Martes";
        break;
    case 3:
        nombreDia = "Miércoles";
        break;
    case 4:
        nombreDia = "Jueves";
        break;
    case 5:
        nombreDia = "Viernes";
        break;
    case 6:
        nombreDia = "Sábado";
        break;
    case 7:
        nombreDia = "Domingo";
        break;
    default: // Se ejecuta si ningún caso anterior coincide
        nombreDia = "Día inválido";
        break;
}

System.out.println("Hoy es: " + nombreDia);
```

El Fenómeno de Caída Secuencial (*Fall-Through*)

En el `switch` clásico heredado del lenguaje C, si olvidas colocar la sentencia `break` al final de un `case`, la JVM continuará ejecutando las instrucciones de los siguientes `case` de manera secuencial, sin importar si sus condiciones coinciden o no:

- **Caída Involuntaria (Bug común)**: Si olvidas el `break` en `case 1:`, al valer 1 ejecutará el código del caso 1 y también el código del caso 2.
- **Caída Intencionada (Técnica útil)**: Podemos agrupar deliberadamente múltiples casos para que compartan la misma lógica:

```
int mes = 2;
int anio = 2024;
int diasEnElMes;

switch (mes) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        diasEnElMes = 31;
        break;
    case 4: case 6: case 9: case 11:
        diasEnElMes = 30;
        break;
    case 2:
        // Comprobación de año bisiesto
        if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
            diasEnElMes = 29;
        } else {
            diasEnElMes = 28;
        }
        break;
    default:
        diasEnElMes = 0;
        System.out.println("Mes erróneo");
        break;
}
```

10.3 El `switch` Moderno: Expresiones `switch` y Flechas `→` (Java 14+)

Java 14 estandarizó una revolución en la sintaxis de control: las **expresiones** `switch` con operadores de flecha (`->`). Esta sintaxis moderna resuelve todos los problemas históricos del `switch` clásico:

1. **Elimina el peligro de *fall-through***: No se requiere escribir `break`. Solo se ejecuta la instrucción situada tras la flecha.
2. **Agrupación limpia por comas**: Múltiples etiquetas pueden separarse sencillamente con comas (`case 1, 2, 3 →`).
3. **Funciona como expresión**: Puede devolver un valor directamente para asignarlo a una variable.
4. **Comprobación exhaustiva**: El compilador verifica que todos los casos posibles estén cubiertos (o que exista una cláusula `default`).

```
int dia = 6;

// 1. Switch moderno como sentencia de acción (sin break, sin fall-through)
switch (dia) {
    case 1, 2, 3, 4, 5 → System.out.println("Día laborable: ¡A trabajar!");
    case 6, 7          → System.out.println("Fin de semana: ¡A descansar!");
    default            → System.out.println("Número de día inválido.");
}

// 2. Switch moderno como EXPRESIÓN (asigna el resultado directamente)
String tipoDeDia = switch (dia) {
    case 1, 2, 3, 4, 5 → "Laborable";
    case 6, 7          → "Festivo";
    default            → "Desconocido";
};

System.out.println("El día seleccionado es de tipo: " + tipoDeDia);
```

La Palabra Clave Contextual `yield`

Si un caso de una expresión `switch` requiere un bloque de varias líneas de código para realizar cálculos antes de devolver el valor, se utilizan llaves `{ ... }` y la palabra clave `yield` para emitir el resultado:

```
int nivelUsuario = 2;

double descuento = switch (nivelUsuario) {
    case 1 -> 0.05;
    case 2 -> {
        double bonusFidelidad = 0.03;
        double base = 0.10;
        yield base + bonusFidelidad; // Emite 0.13 como valor resultante
    }
    case 3 -> 0.20;
    default -> {
        System.out.println("Nivel no reconocido, asignando descuento cero.");
        yield 0.0;
    }
};
```

---

11. Tipología Práctica de Errores en Java y Depuración con IntelliJ IDEA

El desarrollo de software no consiste únicamente en escribir código; una gran parte de la labor diaria de un desarrollador es diagnosticar, aislar y corregir errores (*debugging*).

En Java, los errores se dividen en tres grandes categorías según la etapa en la que se manifiestan:

```
Clasificación de Errores en Java
├── 1. Errores de Compilación (Sintácticos / Semánticos) → Detectados por javac /
IntelliJ
├── 2. Errores en Tiempo de Ejecución (Excepciones)    → Detienen la JVM abruptamente
└── 3. Errores Lógicos (Bugs de Algoritmo)             → El programa corre, pero el
resultado es erróneo
```

11.1 Errores de Compilación (*Compile-time Errors*)

Ocurren cuando el código viola las reglas sintácticas o semánticas del lenguaje Java. El compilador `javac` no genera el archivo `.class` y el programa no puede ser ejecutado.

IntelliJ IDEA detecta estos errores en tiempo real conforme tecleas, subrayando el código con una línea ondulada roja y ofreciendo soluciones contextuales mediante el atajo universal `Alt + Enter`:

```
// Ejemplos típicos:
int numero = "Hola";       // Incompatibilidad de tipos: String no puede asignarse a
int
int total                  // Falta el punto y coma final
System.out.println(valor); // Variable 'valor' no declarada o fuera de ámbito
int x; int y = x + 1;      // Variable local 'x' no inicializada
```

11.2 Errores en Tiempo de Ejecución (*Runtime Errors / Excepciones*)

El programa compila limpiamente a Bytecode, pero durante la ejecución ocurre una situación anómala o imprevista que la Máquina Virtual no puede resolver por sí misma. Como consecuencia, el hilo principal (`Thread "main"`) se estrella, aborta el programa y escupe por consola una traza de error denominada **Stack Trace**.

Las 4 Excepciones Clásicas de la Unidad 2:

1. `java.lang.ArithmeticException: / by zero`:
   Ocurre al intentar dividir un número entero por cero (`10 / 0`).
   (Curiosidad: en números decimales, `10.0 / 0.0` no lanza excepción; devuelve `Infinity` según el estándar IEEE 754).

2. `java.lang.NullPointerException` **(NPE)**:
   El error más famoso del ecosistema Java. Ocurre cuando intentas invocar un método o acceder a un atributo a través de una variable de referencia que vale `null` (no apunta a ningún objeto real en la memoria Heap):

   ```
   String nombre = null;
   System.out.println(nombre.length()); // ¡Lanza NullPointerException!
   ```

3. `java.util.InputMismatchException`:
   Ocurre cuando la clase `Scanner` espera recibir un token de un tipo específico y el usuario teclea algo incompatible:

   ```
   System.out.print("Introduce tu edad: ");
   int edad = sc.nextInt(); // El usuario teclea "veinte" → ¡InputMismatchException!
   ```

4. `java.lang.NumberFormatException`:
   Ocurre al intentar parsear una cadena alfanumérica mediante `Integer.parseInt("abc")` o `Double.parseDouble("12,5")` (con coma en vez de punto en configuración US).

Cómo Leer e Interpretar un Stack Trace

Cuando veas un mensaje de error rojo en la consola de IntelliJ IDEA, no te asustes: es un mapa exacto que te guía directamente hacia el culpable:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at es.iesperenxisa.primerospasos.Calculadora.dividir(Calculadora.java:24)
    at es.iesperenxisa.primerospasos.Calculadora.main(Calculadora.java:12)
```

Pasos para interpretarlo:

1. **Primera línea**: Nombre de la excepción y causa (`ArithmeticException: / by zero`).
2. **Segunda línea**: Método donde ocurrió la catástrofe (`dividir`), archivo (`Calculadora.java`) y **número exacto de línea (`24`)**. En IntelliJ IDEA, ese texto es un hipervínculo azul: ¡haz clic sobre él y el editor te llevará directamente a la línea que falló!

11.3 Errores Lógicos (*Bugs*)

Son los más traicioneros y difíciles de detectar. El programa compila sin advertencias y se ejecuta sin lanzar ninguna excepción, pero el resultado que arroja es matemáticamente o lógicamente incorrecto.

```
// Ejemplo de Bug Lógico:
double nota1 = 8.0;
double nota2 = 6.0;

// Intención: calcular la media aritmética (8 + 6) / 2 = 7.0
// Error: por precedencia de operadores, primero divide 6.0 / 2 = 3.0, y luego suma 8.0
= 11.0
double media = nota1 + nota2 / 2; // ¡Error lógico! Faltan los paréntesis: (nota1 +
nota2) / 2
```

11.4 Depuración Profesional (*Debugging*) con IntelliJ IDEA

Para cazar errores lógicos, los principiantes suelen llenar el código de `System.out.println()`. Los ingenieros de software profesionales utilizan el **Depurador Integrado** (*Debugger*).

El depurador te permite "congelar el tiempo", detener la ejecución de tu programa en una línea exacta y avanzar paso a paso mientras inspeccionas el valor vivo de cada variable en la memoria RAM:

Guía Rápida de Depuración en IntelliJ IDEA:

1. **Poner un Punto de Interrupción (*Breakpoint*)**: Haz clic con el ratón en el margen gris izquierdo (*gutter*) justo al lado del número de línea donde deseas que el programa se pause. Aparecerá un círculo rojo sólido.
2. **Lanzar en Modo Debug**: En lugar de pulsar el botón verde de Play normal, pulsa el icono de la **cucaracha verde** (*Debug*) o utiliza el atajo `Shift + F9`.
3. **Inspección en Pausa**: Cuando la ejecución alcance tu breakpoint, la línea se resaltará en azul y el programa quedará congelado. En la parte inferior se abrirá el panel **Debug**:
    - Pestaña **Variables**: Muestra el valor en tiempo real de cada variable en el Stack y en el Heap.
4. **Controles de Avance Paso a Paso**:
    - **Step Over (`F8`)**: Ejecuta la línea actual y avanza a la siguiente línea del mismo método.
    - **Step Into (`F7`)**: Si la línea actual contiene una llamada a un método propio, se adentra en su código para ver qué hace por dentro.
    - **Step Out (`Shift + F8`)**: Sale del método actual y regresa al llamador.
    - **Resume Program (`F9`)**: Reanuda la ejecución continua hasta encontrar el siguiente breakpoint.
    - **Stop (`Ctrl + F2`)**: Detiene la sesión de depuración.
5. **Evaluar Expresiones al Vuelo (*Evaluate Expression* - `Alt + F8`)**: Puedes abrir una ventana flotante para escribir cualquier cálculo de Java y evaluarlo al instante utilizando los valores vivos de las variables congeladas.

---

12. ⭐ Be the Code: Desafíos de Traza Mental y Memoria

Para ser un programador excepcional debes ser capaz de "ser el código": proyectar mentalmente el estado de la memoria RAM y predecir con exactitud qué valor adoptará cada variable en cada ciclo de instrucción.

Desafío 1: El Laberinto de los Incrementos y la Precedencia

Analiza el siguiente fragmento de código sin ejecutarlo en la máquina. ¿Qué valor imprimirán las variables `a`, `b` y `c`?

```
int a = 10;
int b = 20;
int c = a++ + ++b * 2;
```

Traza de Ejecución Paso a Paso:

1. Inicialmente: `a = 10`, `b = 20`.
2. Evaluamos la expresión de derecha a izquierda respetando la precedencia:
   - El operador `++` prefijo sobre `b` (`++b`) tiene máxima prioridad: `b` se incrementa inmediatamente de 20 a **21**, y ese valor `21` se utiliza en la expresión.
   - Operador multiplicativo `*`: se multiplica `21 * 2 = 42`.
   - El operador `++` postfijo sobre `a` (`a++`): se utiliza en la suma su valor actual (**10**), y justo después de ser leído, `a` se incrementa en memoria a **11**.
   - Operador aditivo `+`: se suma `10 + 42 = 52`.
3. Asignación a `c`: `c` recibe el valor **52**.

Variable	Valor Final en Memoria
`a`	**11**
`b`	**21**
`c`	**52**

Desafío 2: Algoritmo de Descomposición de Cifras con `/` y `%`

Supongamos que un usuario introduce el número entero `7429`. Diseña mentalmente el algoritmo aritmético para aislar de forma independiente las unidades, decenas, centenas y millares:

```
int numero = 7429;

int unidades = numero % 10;         // 7429 % 10 = 9
int decenas  = (numero / 10) % 10;  // 7429 / 10 = 742 → 742 % 10 = 2
int centenas = (numero / 100) % 10; // 7429 / 100 = 74 → 74 % 10 = 4
int millares = numero / 1000;       // 7429 / 1000 = 7
```

Principio algorítmico universal:

- El operador `% 10` extrae siempre el último dígito menos significativo de un número decimal.
- La división entera `/ 10` desplaza el número hacia la derecha, descartando el último dígito.

Desafío 3: El String Pool y el Mapa de Memoria Stack vs Heap

Observa detenidamente este código y predice qué imprimirá cada una de las 4 comparaciones:

```
String texto1 = "Java";
String texto2 = "Java";
String texto3 = new String("Java");
String texto4 = "Ja" + "va"; // Expresión constante evaluada por el compilador

System.out.println(texto1 == texto2);      // ¿true o false?
System.out.println(texto1 == texto3);      // ¿true o false?
System.out.println(texto1 == texto4);      // ¿true o false?
System.out.println(texto1.equals(texto3)); // ¿true o false?
```

Solución Razonada con Arquitectura de Memoria:

1. `texto1 == texto2` devuelve `true`: Ambas variables apuntan a la misma entrada compartida en el *String Constant Pool*.
2. `texto1 == texto3` devuelve `false`: La palabra clave `new` fuerza la creación de un nuevo objeto independiente en el Heap general, con una dirección física de memoria distinta.
3. `texto1 == texto4` devuelve `true`: Como `"Ja"` y `"va"` son literales constantes conocidos en tiempo de compilación, el compilador `javac` optimiza la concatenación y la resuelve como `"Java"`, reutilizando la referencia del String Pool.
4. `texto1.equals(texto3)` devuelve `true`: El método `.equals()` no compara direcciones de memoria física, sino la secuencia exacta de caracteres alfanuméricos (`'J'`, `'a'`, `'v'`, `'a'`).

---

13. ¡No Hay Preguntas Tontas!

1. ¿Por qué Java tiene tantos tipos enteros (`byte`, `short`, `int`, `long`) si lenguajes modernos como JavaScript o Python usan uno solo?

Porque Java fue diseñado desde su origen para sistemas de misión crítica, servidores de alto rendimiento y dispositivos embebidos. Si tienes un array de 10 millones de números que solo van del 0 al 100:

- Con `int` (4 bytes por número) consumes **40 MB** de RAM.
- Con `byte` (1 byte por número) consumes solo **10 MB** de RAM (un 75% menos de memoria).
  En aplicaciones a gran escala o microservicios que procesan millones de peticiones por segundo, la gestión milimétrica de la memoria y la caché del procesador marca la diferencia entre un sistema fluido y uno colapsado.

2. ¿Por qué `Scanner` me lanza un error si escribo `1.75` con punto en lugar de coma?

Porque la clase `Scanner` por defecto utiliza la **configuración regional (*Locale*) del sistema operativo** de tu máquina. En los sistemas configurados en España u otros países europeos, el separador decimal estándar oficial es la coma (`,`), por lo que espera `1,75`. Si introduces `1.75` con punto, lanza `InputMismatchException`. Para forzar a que tu programa utilice siempre el punto decimal estándar internacional, añade la siguiente línea tras instanciar el Scanner:
`sc.useLocale(Locale.US);`

3. ¿Por qué `String` no es un tipo primitivo si lo usamos constantemente como si lo fuera?

Porque un texto no tiene un tamaño fijo en bytes en la memoria física: puede tener cero caracteres, una palabra o una enciclopedia completa de millones de caracteres. Los tipos primitivos tienen un tamaño estrictamente inmutable

en hardware (8, 16, 32 o 64 bits). Por ello, `String` es un objeto complejo alojado en el Heap que internamente encapsula un array de bytes y ofrece decenas de métodos utilitarios.

4. ¿Es `var` una señal de pereza o mala práctica de programación?

Rotundamente no, siempre que se use con criterio profesional. `var` fue incorporado en Java 10 para eliminar la redundancia innecesaria que ensuciaba el código clásico:
`Map<String, List<Pedido>> mapa = new HashMap<String, List<Pedido>>();`
Con `var`, la línea se reduce limpiamente a:
`var mapa = new HashMap<String, List<Pedido>>();`
No se pierde ninguna información de tipos (el compilador sabe exactamente qué es) y el código gana en claridad y limpieza visual.

5. ¿Por qué Java mantiene `float` y `double` si tienen errores de redondeo y en banca se usa `BigDecimal`?

Porque `double` y `float` están implementados directamente en la circuitería de hardware del procesador (la FPU, *Floating Point Unit*), lo que permite ejecutar **miles de millones de cálculos por segundo**. En videojuegos 3D, simulaciones físicas, renderizado gráfico e inteligencia artificial, la velocidad extrema es prioritaria y una diferencia en el decimosexto decimal es irrelevante. En cambio, `BigDecimal` es una clase de software: sus operaciones son miles de veces más lentas, pero garantizan precisión matemática absoluta. Cada herramienta tiene su propósito.

6. ¿Por qué un `char` puede sumarse con un número `int` sin que el compilador se queje?

Porque en la arquitectura de Java, un `char` es en realidad un número entero sin signo de 16 bits que almacena el código numérico Unicode del carácter. Por ejemplo, el carácter `'A'` equivale internamente al número `65`. Al hacer `'A' + 1`, Java promociona el `char` a `int` y efectúa la suma matemática `65 + 1 = 66`. Si luego haces un casting explícito `(char) 66`, obtendrás el carácter `'B'`.

7. ¿Qué diferencia real hay entre `&&` y `&`, o `||` y `|` si ambos dan el mismo resultado lógico?

La diferencia radica en la **evaluación en cortocircuito**:
`&&` y `||` se detienen tan pronto como conocen el resultado definitivo, ahorrando tiempo de CPU y permitiendo programar barreras de protección contra excepciones como `NullPointerException` o división por cero. En cambio, `&` y `|` evalúan obligatoriamente ambos lados de la expresión, sin importar lo que haya devuelto el primero. En lógica condicional ordinaria, **debes usar siempre `&&` y `||`**.

---

14. Resumen de la Unidad y Enlace con la Unidad 3

Lo que hemos conquistado en esta Unidad:

1. **Ecosistema**: La JVM, el compilador `javac`, el Bytecode independiente de plataforma y el IDE IntelliJ IDEA con soporte para las últimas versiones LTS (Java 21 / Java 25 LTS).
2. **Anatomía**: Estructura de clases, paquetes de dominio inverso, imports y el método canónico `public static void main(String[] args)`.
3. **Tipos y Memoria**: Los 8 tipos primitivos, variables vs constantes (`final`), inferencia con `var`, y el desbordamiento en complemento a dos.
4. **Conversiones**: Promoción implícita (*widening*), casting explícito (*narrowing*), parseo de texto y clases envoltorio (*wrappers*) con autoboxing.

5. **Operadores**: Aritmética entera vs real, módulo `%`, incrementos pre/post, operadores relacionales y operadores lógicos con evaluación en cortocircuito.
6. **Biblioteca Estándar**: Cálculos y aleatoriedad con `Math`, inmutabilidad y String Pool con `String`, precisión monetaria con `BigDecimal`, tipos cerrados con `enum` y fechas modernas con `java.time`.
7. **Entrada/Salida**: Formateo con `printf`, lectura por teclado con `Scanner` y la técnica de purga del salto de línea residual.
8. **Decisiones**: Ramificaciones lógicas con `if-else` y la potencia del `switch` moderno con sintaxis de flecha `->` y `yield`.
9. **Depuración**: Diagnóstico de excepciones y rastreo paso a paso con puntos de interrupción en IntelliJ IDEA.

¿Hacia dónde vamos en la Unidad 3?

Hasta ahora nuestros programas han tomado decisiones y ejecutado instrucciones de arriba hacia abajo una única vez. Pero el verdadero poder de la computación reside en la **automatización repetitiva masiva**:

- En la **Unidad 3: Estructuras de Control Iterativas, Bucles y Métodos**, aprenderemos a repetir bloques de código miles o millones de veces con bucles `while`, `do-while` y `for`.
- Descubriremos cómo empaquetar conjuntos de datos en **arrays** unidimensionales y matrices bidimensionales.
- Dividiremos programas complejos en bloques reutilizables mediante **métodos estáticos** modulares.
- Blindaremos nuestras aplicaciones ante fallos del usuario implementando una gestión profesional de excepciones con bloques `try-catch-finally`.

---

15. Relación Curricular Oficial (CFGS DAM - Módulo 0485)

Esta unidad cubre de manera directa y exhaustiva los siguientes **Resultados de Aprendizaje (RA)** y **Criterios de Evaluación (CE)** establecidos en el currículo oficial del ciclo formativo:

- **RA 1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
    - **CE 1.a**: Se ha identificado la estructura general de un programa y sus bloques constituyentes.
    - **CE 1.b**: Se han utilizado entornos integrados de desarrollo (IntelliJ IDEA) para la edición, compilación y ejecución de programas.
    - **CE 1.c**: Se han identificado los diferentes tipos de datos primitivos disponibles en el lenguaje.
    - **CE 1.d**: Se han declarado y utilizado variables, constantes y literales respetando las normas léxicas y de estilo.
    - **CE 1.e**: Se han utilizado operadores aritméticos, relacionales y lógicos respetando la precedencia de operadores.
    - **CE 1.f**: Se han clasificado y documentado los programas utilizando comentarios descriptivos de línea, bloque y Javadoc.
- **RA 4: Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.**
    - **CE 4.a**: Se ha reconocido la sintaxis, estructura y componentes de una clase en Java.
    - **CE 4.e**: Se han utilizado métodos estándar de la biblioteca del lenguaje (`Math`, `String`, `BigDecimal`, `LocalDate`).

- **RA 5: Realiza operaciones de entrada y salida de información utilizando la consola y clases estándar.**
  - **CE 5.a**: Se han utilizado los flujos estándar de entrada y salida (`System.out` , `System.in`).
  - **CE 5.b**: Se ha utilizado la clase `Scanner` para la lectura interactiva de datos por teclado, gestionando delimitadores y configuraciones regionales.
  - **CE 5.c**: Se ha aplicado formato avanzado a los datos de salida mediante cadenas de formato y `printf` .

---

16. Boletines de Ejercicios y Retos Prácticos

Para afianzar cada uno de los conceptos aprendidos, dispones de una batería progresiva de ejercicios prácticos especialmente diseñados para este nivel:

1. **Boletín 2.1 — Inicial**: 13 ejercicios guiados paso a paso sobre declaración de variables, cálculos aritméticos elementales, conversiones de tipos, entrada/salida formateada por consola y evaluación de condiciones lógicas.
   - Consulta también el **Boletín Inicial (Resuelto)** con explicaciones y código fuente completo en Java 21 LTS.
2. **Boletín 2.2 — Intermedio**: 13 ejercicios aplicados sobre sentencias condicionales `if-else` y `switch` , manipulación avanzada de cadenas con `String` , funciones de `Math` , tipos `enum` , cálculo temporal con `java.time` y geometría computacional.
   - Consulta también el **Boletín Intermedio (Resuelto)** con soluciones detalladas.
3. **Boletín 2.3 — Retos y Extras**: 6 grandes desafíos algorítmicos competitivos (verificación oficial de DNI, año bisiesto gregoriano, simulación probabilística de dados, factoriales astronómicos con `BigInteger` , algoritmo voraz de cambio monetario exacto y resolución de ecuaciones de segundo grado con análisis de discriminante).

DAM 0485 · Programación · Unidad Didáctica 2

📝 Boletín 02 Inicial — Primeros Pasos con Java

13 ejercicios prácticos fundamentales en Java 21 / 25 LTS con IntelliJ IDEA sobre tipos primitivos, variables, operadores aritméticos y lógicos, entrada/salida por consola y sentencias condicionales.

Este boletín reúne **13 ejercicios prácticos fundamentales** diseñados para consolidar tu soltura con el entorno de desarrollo **IntelliJ IDEA**, la sintaxis básica del lenguaje **Java** (versión moderna 21 LTS), el uso de la clase `Scanner`, la salida formateada con `printf` y las bifurcaciones condicionales `if-else` y `switch`.

Antes de consultar el Boletín Inicial Resuelto, implementa y ejecuta cada solución en tu propio proyecto de IntelliJ IDEA.

Ejercicio 1: Saludo Personalizado y Formato de Ficha de Usuario

Escribe un programa en Java que solicite al usuario a través del teclado:

1. Su nombre de pila.
2. Su primer apellido.
3. Su edad (número entero).
4. La inicial de su grupo de DAM (un solo carácter: `'A'`, `'B'`, etc.).
5. La cuota mensual del centro en euros (número con decimales).

El programa debe imprimir una ficha resumen limpia y tabulada empleando **únicamente una llamada a** `System.out.printf()` con los especificadores de formato `%s`, `%d`, `%c` y `%.2f`.

Ejercicio 2: Calculadora Geométrica de Rectángulo y Círculo

Escribe un programa que:

1. Pida al usuario la **base** y la **altura** de un rectángulo en metros (valores decimales) y calcule su perímetro y su superficie (área).
2. Pida el **radio** de una circunferencia y calcule la longitud de la circunferencia y el área del círculo utilizando la constante `Math.PI` y el método `Math.pow()`.
3. Si el usuario introduce cualquier dimensión menor o igual a cero (`≤ 0`), el programa debe emitir un mensaje de advertencia y abortar el cálculo.

Ejercicio 3: Conversor Bidireccional de Escalas Térmicas

Escribe un programa que implemente la conversión entre grados Celsius (°C) y grados Fahrenheit (°F):

- Fórmula de conversión a Fahrenheit: `F = (C × 9 / 5) + 32`
- Fórmula de conversión a Celsius: `C = (F − 32) × 5 / 9`

El programa debe:

1. Pedir una temperatura en Celsius y mostrar su equivalente en Fahrenheit con 2 decimales.
2. Pedir una temperatura en Fahrenheit y mostrar su equivalente en Celsius con 2 decimales.

> **¡Cuidado con la trampa de la división entera!**
> En Java, si escribes `9 / 5` o `5 / 9`, el compilador realiza una división entera truncada (`5 / 9 = 0`).
> Asegúrate de forzar literales de coma flotante (`9.0 / 5.0` o `5.0f / 9.0f`).

Ejercicio 4: Calificación Final Ponderada

En el módulo de Programación DAM, la nota final de una evaluación se desglosa según las siguientes ponderaciones:

- Exámenes teóricos: **30%**
- Prácticas y proyectos en IntelliJ IDEA: **50%**
- Actitud, asistencia y participación: **20%**

Diseña un programa que solicite las tres notas (valores decimales entre `0.0` y `10.0`), calcule la nota final ponderada e imprima la calificación cualitativa oficial:

- Menor que 5.0: **Suspenso**
- De 5.0 a 6.9: **Aprobado**
- De 7.0 a 8.9: **Notable**
- De 9.0 a 10.0: **Sobresaliente**

Si alguna de las notas introducidas no se encuentra en el rango `[0.0, 10.0]`, el programa debe avisar del error y no mostrar ninguna media.

Ejercicio 5: Facturación con Descuentos Escalonados e IVA

Una tienda de componentes informáticos aplica descuentos en función del importe total bruto de la compra:

- Si la compra es menor de 100 €, no hay descuento (0%).
- Si la compra es de 100 € o más pero menor de 300 €, se aplica un **5% de descuento**.
- Si la compra es de 300 € o más, se aplica un **12% de descuento**.

Escribe un programa que solicite el precio unitario del producto y el número de unidades compradas. A continuación, debe calcular y mostrar:

1. Importe bruto inicial.
2. Porcentaje y euros de descuento aplicado.
3. Base imponible con descuento.
4. Importe de IVA (al 21%).
5. Importe final a pagar por el cliente.

Todos los importes monetarios deben mostrarse alineados y con exactamente 2 cifras decimales.

Ejercicio 6: Descomposición Temporal de Segundos

Escribe un programa que solicite al usuario una cantidad entera de segundos (por ejemplo, `7534` segundos) y la descomponga en sus unidades equivalentes:

- **Horas completas**
- **Minutos restantes**

- **Segundos finales**

Ejemplo de salida: `7534 segundos equivalen a: 2 horas, 5 minutos y 34 segundos.`
Debes utilizar exclusivamente los operadores de división entera (`/`) y módulo (`%`).

Ejercicio 7: Detector de Paridad y Múltiplos

Escribe un programa que solicite un número entero por teclado e informe:

1. Si el número es **positivo**, **negativo** o **cero**.
2. Si el número es **par** o **impar** (utilizando el operador `%`).
3. Si el número es **múltiplo de 3**, **múltiplo de 5** o **múltiplo de ambos a la vez** (como el 15, 30, etc.).

Ejercicio 8: Días del Mes con `switch` Moderno

Escribe un programa que pida al usuario el número de un mes (un entero del 1 al 12) y devuelva cuántos días tiene ese mes (asumiendo un año no bisiesto de 28 días para febrero).

1. Resuelve el problema utilizando una **expresión** `switch` **moderna** con sintaxis de flecha (`→`) y agrupación de casos por comas (`case 1, 3, 5, 7, 8, 10, 12 → 31;`).
2. Si el número introducido no está entre 1 y 12, debe devolver `-1` y mostrar un mensaje de mes inválido.

Ejercicio 9: El Misterio del Salto de Línea en Scanner

Escribe un programa para registrar el número de matrícula de un vehículo y el nombre completo de su titular:

1. Solicita primero el número de bastidor (un número entero `int` leído con `nextInt()`).
2. A continuación, solicita el nombre completo del titular (una cadena leída con `nextLine()`).

Implementa el programa prestando atención a limpiar el carácter de salto de línea residual (`\n`) que queda en el búfer del teclado tras leer el número. Muestra por pantalla los datos leídos para verificar que el nombre no se ha saltado.

Ejercicio 10: Simulador de Lanzamiento de Dados

Escribe un programa que simule el lanzamiento de dos dados de juego:

- Dado 1: Un dado tradicional de 6 caras (valores del 1 al 6).
- Dado 2: Un dado de rol de 20 caras (D20, valores del 1 al 20).

Utiliza la función `Math.random()` con la fórmula general `(int)(Math.random() * (max - min + 1)) + min` .
El programa debe:

1. Generar la tirada de ambos dados.
2. Calcular y mostrar la suma total de ambos.
3. Si en el dado de 20 caras se obtiene un `20` , debe imprimir: *"¡Éxito Crítico!"*. Si se obtiene un `1` , debe imprimir: *"¡Pifia Crítica!"*.

Ejercicio 11: Traducción y Evaluación de Condiciones Lógicas Complejas

El modelado de reglas de negocio en software exige traducir especificaciones en lenguaje natural a expresiones booleanas rigurosas. Escribe un programa que solicite los datos necesarios y evalúe las siguientes tres condiciones del mundo real:

1. **Concesión de Beca de Estudios:** Un alumno obtiene beca si su nota media es mayor o igual a `8.5` O BIEN si sus ingresos familiares anuales son inferiores a `15.000 €` y su nota media es al menos de `6.0`.
2. **Autorización de Despegue de Aeronave:** Un avión puede despegar si la velocidad del viento es inferior a `20 m/s`, la pista disponible mide al menos `2.500 metros` y la visibilidad es superior a `1.000 metros`.
3. **Control de Acceso a Sala Exclusiva:** Se autoriza la entrada si la persona es mayor de edad (`edad ≥ 18`) y (posee invitación VIP O está acompañada por un miembro premium).

Muestra el resultado booleano (`true` o `false`) de cada evaluación y explica qué regla determinó el veredicto.

Ejercicio 12: Valor Absoluto y Mayor de Números sin Sentencias de Control

Escribe un programa que:

1. Pida por teclado un número decimal `x` (positivo o negativo) y calcule su valor absoluto `|x|`:
   - Primero, utilizando una estructura condicional `if-else`.
   - Segundo, utilizando el **operador ternario** `?:`.
   - Tercero, utilizando el método matemático oficial `Math.abs(x)`.
2. Pida dos números enteros `a` y `b` e imprima cuál es el mayor de los dos **sin utilizar ninguna sentencia** `if` **ni** `switch`, empleando exclusivamente el método `Math.max(a, b)` y el operador ternario.

Ejercicio 13: Resolución de la Ecuación de Primer Grado `ax + b = 0`

Escribe un programa que solicite por consola los coeficientes `a` y `b` (números reales con decimales) de una ecuación lineal de primer grado: `a·x + b = 0`

El programa debe calcular y clasificar rigurosamente la solución según los siguientes casos algebraicos:

1. Si `a ≠ 0`: Existe una solución única dada por `x = −b / a`. Muestra el valor de `x` con exactamente 4 decimales de precisión.
2. Si `a = 0` y `b ≠ 0`: La ecuación se reduce a `b = 0` (falso), por lo que **no existe ninguna solución**.
3. Si `a = 0` y `b = 0`: La igualdad `0 = 0` es siempre cierta, por lo que existen **infinitas soluciones**.

DAM 0485 · Programación · Unidad Didáctica 2

📝 Boletín 02 Inicial (Resuelto) — Primeros Pasos con Java

Soluciones completas, verificadas y comentadas línea a línea en Java 21 / 25 LTS con IntelliJ IDEA para los 13 ejercicios del Boletín Inicial de la Unidad 2.

A continuación se presentan las soluciones comentadas a los 13 ejercicios propuestos en el Boletín Inicial. Cada programa ha sido verificado con Java 21 / 25 LTS en IntelliJ IDEA.

Solución al Ejercicio 1: Saludo Personalizado y Formato de Ficha de Usuario

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class FichaUsuario {
    public static void main(String[] args) {
        // Configuramos Scanner con Locale.US para asegurar el punto decimal en la
cuota
        Scanner teclado = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.next();

        System.out.print("Introduce tu primer apellido: ");
        String apellido = teclado.next();

        System.out.print("Introduce tu edad: ");
        int edad = teclado.nextInt();

        System.out.print("Introduce tu grupo (A/B): ");
        char grupo = teclado.next().toUpperCase().charAt(0);

        System.out.print("Introduce la cuota mensual en euros: ");
        double cuota = teclado.nextDouble();

        System.out.println("\n" + "=".repeat(40));
        System.out.println("         FICHA DE MATRÍCULA DAM        ");
        System.out.println("=".repeat(40));

        // Formato tabulado con una única llamada a System.out.printf
        System.out.printf("Alumno:     %-15s, %-10s%n" +
                          "Edad:       %d años%n" +
                          "Grupo DAM:  %c%n" +
                          "Cuota mes:  %8.2f €%n",
                          apellido, nombre, edad, grupo, cuota);

        System.out.println("=".repeat(40));
        teclado.close();
    }
}
```

Solución al Ejercicio 2: Calculadora Geométrica de Rectángulo y Círculo

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class CalculadoraGeometrica {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce la base del rectángulo (m): ");
        double base = sc.nextDouble();

        System.out.print("Introduce la altura del rectángulo (m): ");
        double altura = sc.nextDouble();

        if (base ≤ 0 || altura ≤ 0) {
            System.out.println("Error: Las dimensiones del rectángulo deben ser
estrictamente positivas.");
            sc.close();
            return; // Detiene la ejecución del programa
        }

        double perimetro = 2 * (base + altura);
        double areaRectangulo = base * altura;

        System.out.printf("Rectángulo → Perímetro: %.2f m | Área: %.2f m²%n",
perimetro, areaRectangulo);

        System.out.print("\nIntroduce el radio de la circunferencia (m): ");
        double radio = sc.nextDouble();

        if (radio ≤ 0) {
            System.out.println("Error: El radio debe ser mayor que cero.");
            sc.close();
            return;
        }

        double longitud = 2 * Math.PI * radio;
        double areaCirculo = Math.PI * Math.pow(radio, 2);

        System.out.printf("Círculo    → Longitud: %.2f m | Área: %.2f m²%n", longitud,
areaCirculo);

        sc.close();
    }
}
```

Solución al Ejercicio 3: Conversor Bidireccional de Escalas Térmicas

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class ConversorTermico {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce la temperatura en grados Celsius (°C): ");
        double celsius = sc.nextDouble();

        // ¡IMPORTANTE!: Usar 9.0 / 5.0 en coma flotante para evitar división entera
truncada a 1
        double fahrenheitCalculado = (celsius * 9.0 / 5.0) + 32.0;
        System.out.printf("%.2f °C equivalen a %.2f °F%n", celsius,
fahrenheitCalculado);

        System.out.print("\nIntroduce la temperatura en grados Fahrenheit (°F): ");
        double fahrenheit = sc.nextDouble();

        double celsiusCalculado = (fahrenheit - 32.0) * 5.0 / 9.0;
        System.out.printf("%.2f °F equivalen a %.2f °C%n", fahrenheit,
celsiusCalculado);

        sc.close();
    }
}
```

Solución al Ejercicio 4: Calificación Final Ponderada

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class CalificacionPonderada {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce la nota de teoría (0-10): ");
        double teoria = sc.nextDouble();

        System.out.print("Introduce la nota de prácticas (0-10): ");
        double practicas = sc.nextDouble();

        System.out.print("Introduce la nota de actitud (0-10): ");
        double actitud = sc.nextDouble();

        // Validación defensiva del rango de notas
        if (teoria < 0 || teoria > 10 || practicas < 0 || practicas > 10 || actitud < 0
|| actitud > 10) {
            System.out.println("Error: Todas las calificaciones deben encontrarse en el
intervalo [0.0, 10.0].");
            sc.close();
            return;
        }

        // Cálculo ponderado: 30% teoría + 50% prácticas + 20% actitud
        double notaFinal = (teoria * 0.30) + (practicas * 0.50) + (actitud * 0.20);

        String mencion;
        if (notaFinal < 5.0) {
            mencion = "Suspenso";
        } else if (notaFinal < 7.0) {
            mencion = "Aprobado";
        } else if (notaFinal < 9.0) {
            mencion = "Notable";
        } else {
            mencion = "Sobresaliente";
        }

        System.out.printf("%nNota final calculada: %.2f (%s)%n", notaFinal, mencion);
        sc.close();
    }
}
```

Solución al Ejercicio 5: Facturación con Descuentos Escalonados e IVA

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class FacturacionComponentes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce el precio unitario del componente (€): ");
        double precioUnitario = sc.nextDouble();

        System.out.print("Introduce el número de unidades: ");
        int unidades = sc.nextInt();

        if (precioUnitario ≤ 0 || unidades ≤ 0) {
            System.out.println("Error: El precio y las unidades deben ser mayores que
cero.");
            sc.close();
            return;
        }

        double importeBruto = precioUnitario * unidades;
        double porcentajeDescuento;

        if (importeBruto < 100.0) {
            porcentajeDescuento = 0.0;
        } else if (importeBruto < 300.0) {
            porcentajeDescuento = 0.05; // 5%
        } else {
            porcentajeDescuento = 0.12; // 12%
        }

        double eurosDescuento = importeBruto * porcentajeDescuento;
        double baseImponible = importeBruto - eurosDescuento;
        double iva = baseImponible * 0.21; // IVA general al 21%
        double totalFactura = baseImponible + iva;

        System.out.println("\n" + "-".repeat(45));
        System.out.println("            DESGLOSE DE FACTURA            ");
        System.out.println("-".repeat(45));
        System.out.printf("Importe Bruto:                %8.2f €%n", importeBruto);
        System.out.printf("Descuento aplicado (%.0f%%):     -%8.2f €%n",
porcentajeDescuento * 100, eurosDescuento);
        System.out.printf("Base Imponible:               %8.2f €%n", baseImponible);
        System.out.printf("IVA (21%%):                    +%8.2f €%n", iva);
        System.out.println("-".repeat(45));
        System.out.printf("TOTAL A PAGAR:                %8.2f €%n", totalFactura);
        System.out.println("-".repeat(45));

        sc.close();
    }
}
```

Solución al Ejercicio 6: Descomposición Temporal de Segundos

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class DescomposicionTiempo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce el total de segundos: ");
        long segundosTotales = sc.nextLong();

        if (segundosTotales < 0) {
            System.out.println("Error: El tiempo no puede ser negativo.");
            sc.close();
            return;
        }

        // 1 hora = 3600 segundos
        long horas = segundosTotales / 3600;
        long segundosRestantes = segundosTotales % 3600;

        // 1 minuto = 60 segundos
        long minutos = segundosRestantes / 60;
        long segundos = segundosRestantes % 60;

        System.out.printf("%d segundos equivalen a: %d horas, %d minutos y %d
segundos.%n",
                          segundosTotales, horas, minutos, segundos);

        sc.close();
    }
}
```

Solución al Ejercicio 7: Detector de Paridad y Múltiplos

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class DetectorParidadMultiplos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce un número entero: ");
        int n = sc.nextInt();

        // 1. Signo
        if (n > 0) {
            System.out.println("→ Signo: Es un número POSITIVO.");
        } else if (n < 0) {
            System.out.println("→ Signo: Es un número NEGATIVO.");
        } else {
            System.out.println("→ Signo: Es el CERO neutro.");
        }

        // 2. Paridad mediante operador módulo
        if (n % 2 == 0) {
            System.out.println("→ Paridad: Es un número PAR.");
        } else {
            System.out.println("→ Paridad: Es un número IMPAR.");
        }

        // 3. Múltiplos de 3 y 5
        boolean mult3 = (n % 3 == 0);
        boolean mult5 = (n % 5 == 0);

        if (mult3 && mult5) {
            System.out.println("→ Divisibilidad: Es múltiplo simultáneo de 3 y de 5
(múltiplo de 15).");
        } else if (mult3) {
            System.out.println("→ Divisibilidad: Es múltiplo de 3.");
        } else if (mult5) {
            System.out.println("→ Divisibilidad: Es múltiplo de 5.");
        } else {
            System.out.println("→ Divisibilidad: No es múltiplo ni de 3 ni de 5.");
        }

        sc.close();
    }
}
```

Solución al Ejercicio 8: Días del Mes con `switch` Moderno

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class DiasDelMesSwitch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce el número del mes (1 al 12): ");
        int mes = sc.nextInt();

        // Expresión switch moderna de Java (Java 14+):
        // Sin break, asignación directa a variable y múltiples casos separados por
comas
        int dias = switch (mes) {
            case 1, 3, 5, 7, 8, 10, 12 → 31;
            case 4, 6, 9, 11           → 30;
            case 2                     → 28; // Asumiendo año no bisiesto
            default                    → -1; // Mes no válido
        };

        if (dias ≠ -1) {
            System.out.printf("El mes %d tiene %d días.%n", mes, dias);
        } else {
            System.out.println("Error: El mes introducido debe estar entre 1 y 12.");
        }

        sc.close();
    }
}
```

Solución al Ejercicio 9: El Misterio del Salto de Línea en Scanner

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class SolucionSaltoLineaScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce el número de bastidor del vehículo: ");
        int bastidor = sc.nextInt();

        // ¡ATENCIÓN CLAVE!
        // nextInt() lee únicamente los dígitos, pero deja el carácter '\n' (Enter) en
el búfer.
        // Si no purgamos el búfer, el siguiente nextLine() se tragará ese '\n'
residual
        // y devolverá una cadena vacía sin esperar a que el usuario escriba su nombre.
        sc.nextLine(); // Lectura de purga

        System.out.print("Introduce el nombre completo del titular: ");
        String titular = sc.nextLine();

        System.out.println("\n--- REGISTRO COMPLETADO CORRECTAMENTE ---");
        System.out.printf("Titular:           %s%n", titular);
        System.out.printf("N° de Bastidor:    %d%n", bastidor);

        sc.close();
    }
}
```

Solución al Ejercicio 10: Simulador de Lanzamiento de Dados

```
package es.iesperenxisa.primerospasos;

public class SimuladorDados {
    public static void main(String[] args) {
        // Fórmula general para entero aleatorio en [min, max]:
        // (int)(Math.random() * (max - min + 1)) + min

        int dado6 = (int)(Math.random() * 6) + 1;       // Rango [1, 6]
        int dado20 = (int)(Math.random() * 20) + 1;     // Rango [1, 20]
        int total = dado6 + dado20;

        System.out.println("🎲 Tirada de dados:");
        System.out.printf("→ Dado clásico (D6):   %d%n", dado6);
        System.out.printf("→ Dado de rol  (D20):  %d%n", dado20);
        System.out.printf("→ Suma total:          %d%n", total);

        // Comprobación de eventos especiales en el D20
        if (dado20 == 20) {
            System.out.println("🌟 ¡ÉXITO CRÍTICO! Impacto definitivo garantizado.");
        } else if (dado20 == 1) {
            System.out.println("💀 ¡PIFIA CRÍTICA! Fallo catastrófico en la acción.");
        }
    }
}
```

Solución al Ejercicio 11: Traducción y Evaluación de Condiciones Lógicas Complejas

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class EvaluadorCondicionesLogicas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ 1. CONCESIÓN DE BECA DE ESTUDIOS ═══");
        System.out.print("Nota media del alumno (0.0 - 10.0): ");
        double notaMedia = sc.nextDouble();
        System.out.print("Ingresos familiares anuales (€): ");
        double ingresos = sc.nextDouble();

        // Regla: nota ≥ 8.5 O (ingresos < 15000 Y nota ≥ 6.0)
        boolean becaConcedida = (notaMedia ≥ 8.5) || (ingresos < 15000.0 && notaMedia
≥ 6.0);
        System.out.printf("¿Beca concedida?: %b%n%n", becaConcedida);

        System.out.println("═══ 2. AUTORIZACIÓN DE DESPEGUE ═══");
        System.out.print("Velocidad del viento (m/s): ");
        double viento = sc.nextDouble();
        System.out.print("Longitud de pista disponible (m): ");
        double pista = sc.nextDouble();
        System.out.print("Visibilidad meteorológica (m): ");
        double visibilidad = sc.nextDouble();

        // Regla: viento < 20 Y pista ≥ 2500 Y visibilidad > 1000
        boolean despegueAutorizado = (viento < 20.0) && (pista ≥ 2500.0) &&
(visibilidad > 1000.0);
        System.out.printf("¿Despegue autorizado?: %b%n%n", despegueAutorizado);

        System.out.println("═══ 3. CONTROL DE ACCESO A SALA EXCLUSIVA ═══");
        System.out.print("Edad del visitante: ");
        int edad = sc.nextInt();
        System.out.print("¿Tiene invitación VIP? (true/false): ");
        boolean tieneVip = sc.nextBoolean();
        System.out.print("¿Acompañado por miembro premium? (true/false): ");
        boolean acompanadoPremium = sc.nextBoolean();

        // Regla: edad ≥ 18 Y (tieneVip O acompanadoPremium)
        boolean accesoPermitido = (edad ≥ 18) && (tieneVip || acompanadoPremium);
        System.out.printf("¿Acceso a la sala permitido?: %b%n", accesoPermitido);

        sc.close();
    }
}
```

Solución al Ejercicio 12: Valor Absoluto y Mayor de Números sin Sentencias de Control

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class ComparadorYValorAbsoluto {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce un número decimal (puede ser negativo): ");
        double x = sc.nextDouble();

        // 1. Cálculo con if-else
        double absIf;
        if (x < 0) {
            absIf = -x;
        } else {
            absIf = x;
        }

        // 2. Cálculo con operador ternario ?:
        double absTernario = (x < 0) ? -x : x;

        // 3. Cálculo con método de la biblioteca estándar
        double absMath = Math.abs(x);

        System.out.printf("Valor original: %.3f%n", x);
        System.out.printf("→ Valor absoluto (if-else):   %.3f%n", absIf);
        System.out.printf("→ Valor absoluto (ternario):  %.3f%n", absTernario);
        System.out.printf("→ Valor absoluto (Math.abs):  %.3f%n%n", absMath);

        System.out.print("Introduce el primer entero (a): ");
        int a = sc.nextInt();
        System.out.print("Introduce el segundo entero (b): ");
        int b = sc.nextInt();

        // Determinar el mayor sin if:
        int mayorMath = Math.max(a, b);
        String diagnostico = (a == b) ? "Ambos números son idénticos" :
                             (a > b)  ? "El primero (a) es mayor que el segundo (b)" :
                                        "El segundo (b) es mayor que el primero (a)";

        System.out.printf("Mayor determinado con Math.max: %d%n", mayorMath);
        System.out.printf("Diagnóstico con ternario:       %s%n", diagnostico);

        sc.close();
    }
}
```

Solución al Ejercicio 13: Resolución de la Ecuación de Primer Grado `ax + b = 0`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class EcuacionPrimerGrado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("Resolución de la ecuación de primer grado: a·x + b = 0");
        System.out.print("Introduce el coeficiente 'a': ");
        double a = sc.nextDouble();

        System.out.print("Introduce el término independiente 'b': ");
        double b = sc.nextDouble();

        System.out.println("\n————————————————————————————————————————————————");
        if (a ≠ 0.0) {
            // Caso 1: Solución única x = -b / a
            double x = -b / a;
            // Prevenimos el caso de -0.0 visual en coma flotante IEEE 754
            if (x == -0.0) {
                x = 0.0;
            }
            System.out.printf("Ecuación: %.2f·x + (%.2f) = 0%n", a, b);
            System.out.printf("→ Solución única: x = %.4f%n", x);
        } else if (b ≠ 0.0) {
            // Caso 2: 0·x + b = 0 → b = 0 (inconsistencia matemática)
            System.out.printf("Ecuación: 0·x + (%.2f) = 0%n", b);
            System.out.println("→ Sin solución: Inconsistencia matemática (la igualdad
es falsa).");
        } else {
            // Caso 3: 0·x + 0 = 0 (identidad matemática)
            System.out.println("Ecuación: 0·x + 0 = 0");
            System.out.println("→ Infinitas soluciones: Cualquier valor real de x
satisface la igualdad.");
        }
        System.out.println("————————————————————————————————————————————————");

        sc.close();
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 2

📝 Boletín 02 Intermedio — Primeros Pasos con Java

13 ejercicios de nivel intermedio en Java 21 / 25 LTS sobre métodos de String, funciones de Math, expresiones switch con yield, enum, fechas con java.time, desbordamiento de tipos y geometría computacional.

Este boletín contiene **13 ejercicios prácticos aplicados** para consolidar el manejo avanzado de tipos primitivos, la inmutabilidad y los métodos de búsqueda de la clase `String`, la precisión matemática con `Math`, los tipos enumerados `enum`, la manipulación de fechas con `java.time`, el control riguroso de memoria y la resolución de problemas de geometría computacional.

Antes de consultar el Boletín Intermedio Resuelto, programa y verifica cada solución de manera autónoma en **IntelliJ IDEA**.

Ejercicio 1: Desglose Óptimo de Billetes y Monedas (Cajero Automático)

Diseña un programa que simule la dispensación de efectivo de un cajero automático. El usuario introduce una cantidad entera en euros (por ejemplo, `488 €`).

El programa debe calcular y mostrar el **mínimo número de billetes y monedas** necesarios para satisfacer el importe, utilizando las siguientes denominaciones de curso legal:

- Billetes: `50 €`, `20 €`, `10 €` y `5 €`.
- Monedas: `2 €` y `1 €`.

Solo deben mostrarse aquellas denominaciones cuya cantidad sea estrictamente superior a cero (`> 0`).

Ejercicio 2: El Envasado de Huevos de la Granja Avícola

El encargado de una granja avícola necesita un programa para clasificar los huevos recolectados en el día por sus gallinas.

Sabiendo que dispone de:

- **Envases gigantes (gruesas):** Capacidad para `144` huevos (12 docenas).
- **Envases medianos (docenas):** Capacidad para `12` huevos.
- **Envases pequeños (medias docenas):** Capacidad para `6` huevos.

Escribe un programa que solicite el total de huevos puestos hoy (por ejemplo, `308`) y determine:

1. Cuántos envases grandes se llenan completamente.
2. Cuántos envases de una docena se llenan con los huevos restantes.
3. Cuántos envases de media docena se llenan con el residuo.
4. Cuántos huevos quedan sueltos sin envasar.

Ejercicio 3: Teorema de Pitágoras con Validación Dinámica

Escribe un programa que calcule la hipotenusa de un triángulo rectángulo dados los dos catetos `a` y `b`.

1. Solicita ambos catetos en metros por teclado.
2. Calcula la hipotenusa de dos formas distintas:
   - Mediante `Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2))`.
   - Mediante la función especializada de la biblioteca estándar: `Math.hypot(a, b)`.
3. Muestra el resultado formateado con **3 decimales de precisión** (`%.3f`).
4. Si alguno de los catetos introducidos es `≤ 0` , advierte al usuario de que las longitudes geométricas no pueden ser nulas ni negativas.

Ejercicio 4: Analizador y Parser de Nombres y Apellidos con `String`

Escribe un programa que solicite al usuario que teclee su nombre de pila y sus dos apellidos en una sola línea separados por espacios (sin partículas intermedias, por ejemplo: `Ada Lovelace Byron`).

El programa debe procesar la cadena utilizando exclusivamente métodos de la clase `String` (`indexOf` , `lastIndexOf` , `substring` , `length` , `toUpperCase`):

1. Aislar y mostrar el **nombre de pila** en mayúsculas y su número de letras.
2. Aislar y mostrar el **primer apellido** en mayúsculas y su número de letras.
3. Aislar y mostrar el **segundo apellido** en mayúsculas y su número de letras.
4. Generar y mostrar las **tres iniciales** unidas (ejemplo: `ALB`).

Ejercicio 5: Conversor de Potencia de Motores (CV a kW)

En la ficha técnica de los automóviles, la potencia del motor suele expresarse en **Caballos de Vapor (CV)** o en **Kilovatios (kW)**. La equivalencia física oficial es:

- `1 CV = 735.49875 W` (`0.73549875 kW`)
- `1 kW = 1.35962 CV`

Escribe un programa que ofrezca al usuario un menú de opciones:

1. Convertir de CV a kW.
2. Convertir de kW a CV.
3. Salir.

Muestra los resultados formateados con exactamente 2 decimales y advierte si la potencia introducida es menor o igual a cero.

Ejercicio 6: Tarificación Eléctrica con Tipos Enumerados (`enum`)

Crea una enumeración llamada `TramoHorario` con tres constantes: `VALLE` , `LLANO` y `PUNTA` . Las tarifas de la energía son:

- `VALLE` : 0.11 € / kWh
- `LLANO` : 0.18 € / kWh
- `PUNTA` : 0.28 € / kWh

Escribe un programa que solicite al usuario el número de kWh consumidos (valor decimal) y el tramo horario (como texto `VALLE` , `LLANO` o `PUNTA`).

1. Convierte el texto al valor correspondiente de la enumeración usando `TramoHorario.valueOf(texto.toUpperCase())`.
2. Calcula el coste de energía mediante un `switch`.
3. Aplica un **Impuesto sobre la Electricidad del 5.11%** y posteriormente el **IVA del 21%** sobre el total acumulado. Muestra la factura detallada.

Ejercicio 7: Clasificador de Caracteres Unicode y Métodos de `Character`

Escribe un programa que solicite al usuario pulsar una única tecla (leer un `char`). Utilizando los métodos estáticos de la clase `java.lang.Character` (`isDigit`, `isLetter`, `isUpperCase`, `isLowerCase`, `isWhitespace`), el programa debe determinar:

1. Si el carácter es un **dígito numérico** (del `'0'` al `'9'`) y mostrar su valor numérico entero.
2. Si es una **letra del alfabeto**, si está en mayúscula o minúscula, y si se trata de una **vocal** (`a, e, i, o, u`, incluidas mayúsculas y con tilde) o una **consonante**.
3. Si es un **espacio en blanco** o tabulador.
4. Si es un **símbolo o signo de puntuación**.
5. Muestra adicionalmente el **código Unicode / ASCII** numérico del carácter introducido.

Ejercicio 8: Evaluación de Expediente con Expresión `switch` y `yield`

Escribe un programa que reciba una nota numérica entera del 1 al 10 e implemente una expresión `switch` moderna que utilice la palabra clave `yield` para calcular un mensaje de mención académica personalizada:

- Notas 1 a 4: yield `"Insuficiente - Debe presentarse a la convocatoria extraordinaria"`.
- Notas 5 a 6: yield `"Suficiente / Bien - Supera los mínimos exigidos en el módulo"`.
- Notas 7 a 8: yield `"Notable - Demuestra un sólido dominio técnico de Java"`.
- Notas 9 a 10: yield `"Sobresaliente - Candidato a Matrícula de Honor"`.
- Cualquier otra nota: yield `"Nota no válida"`.

Ejercicio 9: Calculadora de Edad Exacta con `java.time`

Escribe un programa que solicite al usuario su fecha de nacimiento introduciendo por separado el **año**, el **mes** (1-12) y el **día**.

1. Construye un objeto `LocalDate` con la fecha indicada.
2. Obtén la fecha actual del sistema mediante `LocalDate.now()`.
3. Utiliza la clase `java.time.Period` (`Period.between(nacimiento, hoy)`) para calcular:
    - Años cumplidos.
    - Meses transcurridos desde el último cumpleaños.
    - Días transcurridos desde el último cumplemés.
4. Si hoy coincide en día y mes con la fecha de nacimiento, el programa debe felicitar al usuario con un mensaje especial: *"¡Felicidades, hoy es tu cumpleaños!"*.

Ejercicio 10: Laboratorio de Desbordamiento Numérico (*Overflow*) y Casting

Escribe un programa experimental en Java que demuestre de forma tangible los peligros del estrechamiento (*narrowing casting*) y el desbordamiento de bits:

1. Asigna el valor `130` a una variable `int` y fuérzala mediante cast a un `byte`. Imprime el resultado y explica por qué se convierte en un número negativo (`-126`).
2. Multiplica dos números enteros grandes: `int millon = 1_000_000; int resultado = millon * millon;`. Imprime el resultado y explica por qué no da un billón sino un número erróneo por desbordamiento de 32 bits. Muestra cómo solucionarlo realizando la operación con tipo `long`.

Ejercicio 11: Posición Relativa de Dos Circunferencias en el Plano

Escribe un programa en Java que solicite al usuario los parámetros geométricos de dos circunferencias en un plano cartesiano bidimensional:

- Circunferencia 1: Centro `(x1, y1)` y radio `r1`.
- Circunferencia 2: Centro `(x2, y2)` y radio `r2`.

El programa debe verificar que ambos radios sean estrictamente positivos (`r1 > 0` y `r2 > 0`), calcular la distancia euclídea entre sus centros: `d = √((x2 − x1)² + (y2 − y1)²)`
y determinar su posición geométrica relativa según los siguientes criterios:

1. **Concéntricas:** `d == 0` y `r1 == r2` (coincidentes) o `d == 0` con radios distintos.
2. **Exteriores:** `d > r1 + r2`.
3. **Tangentes exteriores:** `d == r1 + r2`.
4. **Secantes:** `|r1 − r2| < d < r1 + r2`.
5. **Tangentes interiores:** `d == |r1 − r2|` con `r1 ≠ r2`.
6. **Interiores no concéntricas:** `d < |r1 − r2|`.

Ejercicio 12: Validador Integral de Fechas con Año Bisiesto Gregoriano

Diseña un validador riguroso de fechas que solicite tres números enteros correspondientes al **día**, **mes** y **año**.

El programa debe determinar si la combinación introducida conforma una fecha real y válida según las siguientes reglas:

1. El año debe ser mayor que cero (`año > 0`).
2. El mes debe estar comprendido entre `1` y `12`.
3. Si el mes es `1, 3, 5, 7, 8, 10` o `12`, el día debe estar entre `1` y `31`.
4. Si el mes es `4, 6, 9` o `11`, el día debe estar entre `1` y `30`.
5. Si el mes es `2` (febrero):
    - Comprobar si el año es bisiesto: `(año % 4 == 0 && año % 100 ≠ 0) || (año % 400 == 0)`.
    - Si es bisiesto, febrero tiene hasta `29` días.
    - Si no es bisiesto, febrero tiene hasta `28` días.

Si la fecha no es válida, el programa debe informar con precisión del motivo exacto del rechazo (mes incorrecto, día fuera de rango para ese mes o intento de 29 de febrero en año no bisiesto).

Ejercicio 13: Ecuación de la Recta entre Dos Puntos y Tabla de Valores Tabulada

Escribe un programa que solicite las coordenadas de dos puntos en el plano cartesiano: `P1(x1, y1)` y `P2(x2, y2)` con valores decimales.

El programa debe:

1. Comprobar si `x1 == x2`. En tal caso, informar de que la recta es vertical (`x = x1`) y su pendiente es infinita, no pudiéndose expresar en la forma explícita habitual.
2. En caso contrario, calcular la **pendiente** `m` y la **ordenada en el origen** `n` de la ecuación explícita `y = m·x + n`:
   - `m = (y2 − y1) / (x2 − x1)`
   - `n = y1 − (m × x1)`
3. Mostrar por pantalla la ecuación resultante con 3 cifras decimales.
4. Generar una tabla de valores tabulada y alineada con `printf` evaluando la función en los siguientes 5 puntos: `x = −1.0`, `x = −0.5`, `x = 0.0`, `x = 0.5`, `x = 1.0`.

DAM 0485 · Programación · Unidad Didáctica 2

📝 Boletín 02 Intermedio (Resuelto) — Primeros Pasos con Java

Soluciones completas, verificadas y comentadas línea a línea en Java 21 / 25 LTS con IntelliJ IDEA para los 13 ejercicios del Boletín Intermedio de la Unidad 2.

A continuación se presentan las soluciones comentadas a los 13 ejercicios propuestos en el Boletín Intermedio. Cada solución ha sido optimizada y probada con Java 21 / 25 LTS en IntelliJ IDEA.

Solución al Ejercicio 1: Desglose Óptimo de Billetes y Monedas (Cajero Automático)

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class CajeroAutomatico {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce la cantidad en euros a retirar (entero): ");
        int cantidad = sc.nextInt();

        if (cantidad ≤ 0) {
            System.out.println("Error: La cantidad a retirar debe ser positiva.");
            sc.close();
            return;
        }

        System.out.printf("%nDispensando %d € con el mínimo número de piezas:%n",
cantidad);
        System.out.println("──────────────────────────────────────────────────");

        int restante = cantidad;

        // Billetes de 50 €
        int b50 = restante / 50;
        restante %= 50;
        if (b50 > 0) System.out.printf("→ Billetes de 50 €: %d%n", b50);

        // Billetes de 20 €
        int b20 = restante / 20;
        restante %= 20;
        if (b20 > 0) System.out.printf("→ Billetes de 20 €: %d%n", b20);

        // Billetes de 10 €
        int b10 = restante / 10;
        restante %= 10;
        if (b10 > 0) System.out.printf("→ Billetes de 10 €: %d%n", b10);

        // Billetes de 5 €
        int b5 = restante / 5;
        restante %= 5;
        if (b5 > 0) System.out.printf("→ Billetes de  5 €: %d%n", b5);

        // Monedas de 2 €
        int m2 = restante / 2;
        restante %= 2;
        if (m2 > 0) System.out.printf("→ Monedas  de  2 €: %d%n", m2);

        // Monedas de 1 €
        int m1 = restante;
        if (m1 > 0) System.out.printf("→ Monedas  de  1 €: %d%n", m1);

        System.out.println("──────────────────────────────────────────────────");
        sc.close();
    }
}
```

Solución al Ejercicio 2: El Envasado de Huevos de la Granja Avícola

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class EnvasadoGranja {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce el total de huevos recolectados hoy: ");
        int totalHuevos = sc.nextInt();

        if (totalHuevos < 0) {
            System.out.println("Error: La cantidad de huevos no puede ser negativa.");
            sc.close();
            return;
        }

        int restante = totalHuevos;

        // Envases gigantes: 144 huevos (1 gruesa)
        int envasesGrandes = restante / 144;
        restante %= 144;

        // Envases medianos: 12 huevos (1 docena)
        int docenas = restante / 12;
        restante %= 12;

        // Envases pequeños: 6 huevos (media docena)
        int mediasDocenas = restante / 6;
        restante %= 6;

        int sueltos = restante;

        System.out.println("\n" + "=".repeat(45));
        System.out.println("         INFORME DE CLASIFICACIÓN AVÍCOLA    ");
        System.out.println("=".repeat(45));
        System.out.printf("Total huevos procesados:        %d%n", totalHuevos);
        System.out.printf("Envases gigantes (144 ud):      %d%n", envasesGrandes);
        System.out.printf("Envases docenas   (12 ud):      %d%n", docenas);
        System.out.printf("Envases medias docenas (6 ud):  %d%n", mediasDocenas);
        System.out.printf("Huevos sueltos sin empaquetar:  %d%n", sueltos);
        System.out.println("=".repeat(45));

        sc.close();
    }
}
```

Solución al Ejercicio 3: Teorema de Pitágoras con Validación Dinámica

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class TeoremaPitagoras {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce la longitud del cateto 1 (m): ");
        double c1 = sc.nextDouble();

        System.out.print("Introduce la longitud del cateto 2 (m): ");
        double c2 = sc.nextDouble();

        if (c1 ≤ 0 || c2 ≤ 0) {
            System.out.println("Error: Los catetos de un triángulo deben ser
estrictamente mayores que cero.");
            sc.close();
            return;
        }

        // Método 1: Raíz cuadrada de la suma de cuadrados
        double hipotenusa1 = Math.sqrt(Math.pow(c1, 2) + Math.pow(c2, 2));

        // Método 2: Función nativa optimizada de la clase Math
        double hipotenusa2 = Math.hypot(c1, c2);

        System.out.println("\n————————————————————————————————————————————————");
        System.out.printf("Cateto A:                %.3f m%n", c1);
        System.out.printf("Cateto B:                %.3f m%n", c2);
        System.out.printf("Hipotenusa (Math.sqrt):  %.3f m%n", hipotenusa1);
        System.out.printf("Hipotenusa (Math.hypot): %.3f m%n", hipotenusa2);
        System.out.println("————————————————————————————————————————————————");

        sc.close();
    }
}
```

Solución al Ejercicio 4: Analizador y Parser de Nombres y Apellidos con `String`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ParserNombreCompleto {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce tu nombre y dos apellidos (separados por espacios):
");
        String linea = sc.nextLine().trim();

        // Localizamos los dos espacios delimitadores clave
        int primerEspacio = linea.indexOf(' ');
        int ultimoEspacio = linea.lastIndexOf(' ');

        if (primerEspacio == -1 || primerEspacio == ultimoEspacio) {
            System.out.println("Error: Debes introducir un nombre y dos apellidos
separados exactamente por espacios.");
            sc.close();
            return;
        }

        // Extracción de las 3 partes
        String nombre = linea.substring(0, primerEspacio);
        String apellido1 = linea.substring(primerEspacio + 1, ultimoEspacio);
        String apellido2 = linea.substring(ultimoEspacio + 1);

        // Iniciales
        char inNombre = Character.toUpperCase(nombre.charAt(0));
        char inApe1 = Character.toUpperCase(apellido1.charAt(0));
        char inApe2 = Character.toUpperCase(apellido2.charAt(0));
        String iniciales = "" + inNombre + inApe1 + inApe2;

        System.out.println("\n--- RESULTADOS DEL ANÁLISIS DE TEXTO ---");
        System.out.printf("Nombre:          %-15s (%d letras)%n", nombre.toUpperCase(),
nombre.length());
        System.out.printf("Primer apellido: %-15s (%d letras)%n",
apellido1.toUpperCase(), apellido1.length());
        System.out.printf("Segundo apellido:%-15s (%d letras)%n",
apellido2.toUpperCase(), apellido2.length());
        System.out.printf("Iniciales:       %s%n", iniciales);

        sc.close();
    }
}
```

Solución al Ejercicio 5: Conversor de Potencia de Motores (CV a kW)

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class ConversorPotenciaMotores {
    private static final double KW_POR_CV = 0.73549875;
    private static final double CV_POR_KW = 1.35962;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ CONVERSOR DE POTENCIA DE VEHÍCULOS ═══");
        System.out.println("1. Convertir de Caballos de Vapor (CV) a Kilovatios (kW)");
        System.out.println("2. Convertir de Kilovatios (kW) a Caballos de Vapor (CV)");
        System.out.print("Selecciona una opción (1-2): ");
        int opcion = sc.nextInt();

        if (opcion == 1) {
            System.out.print("Introduce la potencia en CV: ");
            double cv = sc.nextDouble();
            if (cv ≤ 0) {
                System.out.println("Error: La potencia debe ser positiva.");
            } else {
                double kw = cv * KW_POR_CV;
                System.out.printf("%.2f CV equivalen exactamente a %.2f kW%n", cv, kw);
            }
        } else if (opcion == 2) {
            System.out.print("Introduce la potencia en kW: ");
            double kw = sc.nextDouble();
            if (kw ≤ 0) {
                System.out.println("Error: La potencia debe ser positiva.");
            } else {
                double cv = kw * CV_POR_KW;
                System.out.printf("%.2f kW equivalen exactamente a %.2f CV%n", kw, cv);
            }
        } else {
            System.out.println("Opción no válida.");
        }

        sc.close();
    }
}
```

Solución al Ejercicio 6: Tarificación Eléctrica con Tipos Enumerados (`enum`)

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class FacturacionElectrica {
    public enum TramoHorario {
        VALLE(0.11),
        LLANO(0.18),
        PUNTA(0.28);

        private final double precioKwh;

        TramoHorario(double precioKwh) {
            this.precioKwh = precioKwh;
        }

        public double getPrecioKwh() {
            return precioKwh;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce los kWh consumidos: ");
        double consumo = sc.nextDouble();

        System.out.print("Introduce el tramo horario (VALLE / LLANO / PUNTA): ");
        String tramoTexto = sc.next().toUpperCase();

        TramoHorario tramo;
        try {
            tramo = TramoHorario.valueOf(tramoTexto);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: Tramo desconocido. Opciones válidas: VALLE,
LLANO o PUNTA.");
            sc.close();
            return;
        }

        double costeEnergia = consumo * tramo.getPrecioKwh();
        double impuestoElectrico = costeEnergia * 0.0511; // 5.11%
        double baseImponible = costeEnergia + impuestoElectrico;
        double iva = baseImponible * 0.21; // 21%
        double totalFactura = baseImponible + iva;

        System.out.println("\n" + "=".repeat(45));
        System.out.println("             FACTURA DE ELECTRICIDAD          ");
        System.out.println("=".repeat(45));
        System.out.printf("Tramo seleccionado:       %s (%.2f €/kWh)%n", tramo,
tramo.getPrecioKwh());
        System.out.printf("Consumo registrado:       %.2f kWh%n", consumo);
        System.out.printf("Término de energía:       %8.2f €%n", costeEnergia);
        System.out.printf("Impuesto eléctrico (5.11%%):%7.2f €%n", impuestoElectrico);
        System.out.printf("Base imponible:           %8.2f €%n", baseImponible);
        System.out.printf("IVA (21%%):                 %8.2f €%n", iva);
        System.out.println("-".repeat(45));
        System.out.printf("TOTAL A PAGAR:            %8.2f €%n", totalFactura);
        System.out.println("=".repeat(45));
```

```
        sc.close();
    }
}
```

Solución al Ejercicio 7: Clasificador de Caracteres Unicode y Métodos de `Character`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ClasificadorCaracteres {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Pulsa una tecla y presiona Enter: ");
        String entrada = sc.nextLine();

        if (entrada.isEmpty()) {
            System.out.println("No se ha introducido ningún carácter.");
            sc.close();
            return;
        }

        char c = entrada.charAt(0);
        int codigoAscii = (int) c;

        System.out.println("\n--- ANÁLISIS DE CARÁCTER ---");
        System.out.printf("Carácter analizado: '%c' (Código Unicode/ASCII: %d)%n", c,
codigoAscii);

        if (Character.isDigit(c)) {
            System.out.printf("→ Tipo: DÍGITO NUMÉRICO (Valor entero: %d)%n",
Character.getNumericValue(c));
        } else if (Character.isLetter(c)) {
            String caso = Character.isUpperCase(c) ? "MAYÚSCULA" : "MINÚSCULA";
            char normalizada = Character.toLowerCase(c);
            boolean esVocal = "aeiouáéíóúü".indexOf(normalizada) ≠ -1;
            String subtipo = esVocal ? "VOCAL" : "CONSONANTE";
            System.out.printf("→ Tipo: LETRA DEL ALFABETO (%s, %s)%n", caso, subtipo);
        } else if (Character.isWhitespace(c)) {
            System.out.println("→ Tipo: ESPACIO EN BLANCO / TABULADOR / SALTO DE
LÍNEA");
        } else {
            System.out.println("→ Tipo: SÍMBOLO O SIGNO DE PUNTUACIÓN");
        }

        sc.close();
    }
}
```

Solución al Ejercicio 8: Evaluación de Expediente con Expresión `switch` y `yield`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class EvaluacionExpedienteYield {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce la calificación entera del expediente (1-10): ");
        int nota = sc.nextInt();

        // Expresión switch que utiliza yield en bloques multilínea
        String informe = switch (nota) {
            case 1, 2, 3, 4 -> {
                System.out.println("[AUDITORÍA] Calificación por debajo del corte
legal.");
                yield "Insuficiente - Debe presentarse a la convocatoria
extraordinaria.";
            }
            case 5, 6 -> {
                System.out.println("[AUDITORÍA] Supera las competencias básicas.");
                yield "Suficiente / Bien - Supera los mínimos exigidos en el módulo.";
            }
            case 7, 8 -> {
                System.out.println("[AUDITORÍA] Rendimiento notable constatado.");
                yield "Notable - Demuestra un sólido dominio técnico de Java.";
            }
            case 9, 10 -> {
                System.out.println("[AUDITORÍA] Rendimiento excelente de alta
graduación.");
                yield "Sobresaliente - Candidato a Matrícula de Honor.";
            }
            default -> {
                System.out.println("[AUDITORÍA] Calificación anómala detectada.");
                yield "Nota no válida - El valor debe encontrarse en el rango 1 a 10.";
            }
        };

        System.out.printf("%nResultado del informe oficial:%n%s%n", informe);
        sc.close();
    }
}
```

Solución al Ejercicio 9: Calculadora de Edad Exacta con `java.time`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.time.LocalDate;
import java.time.Period;

public class CalculadoraEdadExacta {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("═══ CALCULADORA DE EDAD CRONOLÓGICA EXACTA ═══");
        System.out.print("Introduce tu año de nacimiento (ej. 2004): ");
        int anio = sc.nextInt();

        System.out.print("Introduce tu mes de nacimiento (1-12): ");
        int mes = sc.nextInt();

        System.out.print("Introduce tu día de nacimiento (1-31): ");
        int dia = sc.nextInt();

        LocalDate nacimiento;
        try {
            nacimiento = LocalDate.of(anio, mes, dia);
        } catch (Exception e) {
            System.out.println("Error: La fecha indicada no es válida en el
calendario.");
            sc.close();
            return;
        }

        LocalDate hoy = LocalDate.now();

        if (nacimiento.isAfter(hoy)) {
            System.out.println("Error: ¡Aún no has nacido! La fecha de nacimiento es
posterior a hoy.");
            sc.close();
            return;
        }

        // Period calcula la diferencia exacta en años, meses y días
        Period edad = Period.between(nacimiento, hoy);

        System.out.println("\n──────────────────────────────────────────────────");
        System.out.printf("Fecha de nacimiento: %s%n", nacimiento);
        System.out.printf("Fecha actual:        %s%n", hoy);
        System.out.printf("Tu edad exacta es:   %d años, %d meses y %d días.%n",
                          edad.getYears(), edad.getMonths(), edad.getDays());

        // Comprobación de cumpleaños hoy
        if (nacimiento.getMonthValue() == hoy.getMonthValue() &&
nacimiento.getDayOfMonth() == hoy.getDayOfMonth()) {
            System.out.println("🎂🎉 ¡FELICIDADES! ¡Hoy es el día de tu cumpleaños!");
        }
        System.out.println("──────────────────────────────────────────────────");

        sc.close();
    }
}
```

Solución al Ejercicio 10: Laboratorio de Desbordamiento Numérico (*Overflow*) y Casting

```
package es.iesperenxisa.primerospasos;

public class LaboratorioOverflow {
    public static void main(String[] args) {
        System.out.println("═══ LABORATORIO DE OVERFLOW Y CASTING ═══");

        // 1. Desbordamiento de 8 bits al convertir int a byte
        int entero = 130;
        byte byteForzado = (byte) entero;

        System.out.println("1. Casting de int (130) a byte:");
        System.out.printf("   Valor int original: %d%n", entero);
        System.out.printf("   Valor tras (byte):  %d%n", byteForzado);
        System.out.println("   → Explicación: Un byte tiene un rango de -128 a 127.");
        System.out.println("      El valor 130 supera el límite máximo en 3
unidades.");
        System.out.println("      Al dar la vuelta en complemento a dos: 127 → -128,
-127, -126.\n");

        // 2. Desbordamiento en multiplicación de enteros de 32 bits
        int millon = 1_000_000;
        int productoErroneo = millon * millon; // 10^12 supera 2.147.483.647
(Integer.MAX_VALUE)

        // Solución correcta forzando evaluación en 64 bits (long)
        long productoCorrecto = (long) millon * millon;

        System.out.println("2. Multiplicación de 1.000.000 x 1.000.000:");
        System.out.printf("   Resultado erróneo con 'int' (32 bits): %d%n",
productoErroneo);
        System.out.printf("   Resultado exacto con 'long' (64 bits): %d%n",
productoCorrecto);
        System.out.println("   → Explicación: Al multiplicar dos 'int', Java calcula
en 32 bits");
        System.out.println("      desbordando silenciosamente. Se debe promocionar al
menos un operando a 'long'.");
    }
}
```

Solución al Ejercicio 11: Posición Relativa de Dos Circunferencias en el Plano

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class PosicionRelativaCircunferencias {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ POSICIÓN RELATIVA DE DOS CIRCUNFERENCIAS ═══");
        System.out.print("Circunferencia 1 → Centro x1, y1 y Radio r1: ");
        double x1 = sc.nextDouble();
        double y1 = sc.nextDouble();
        double r1 = sc.nextDouble();

        System.out.print("Circunferencia 2 → Centro x2, y2 y Radio r2: ");
        double x2 = sc.nextDouble();
        double y2 = sc.nextDouble();
        double r2 = sc.nextDouble();

        if (r1 ≤ 0.0 || r2 ≤ 0.0) {
            System.out.println("Error: Los radios deben ser estrictamente mayores que
cero.");
            sc.close();
            return;
        }

        // Distancia euclídea entre centros: d = sqrt((x2 - x1)^2 + (y2 - y1)^2)
        double d = Math.hypot(x2 - x1, y2 - y1);
        double sumaRadios = r1 + r2;
        double difRadios = Math.abs(r1 - r2);

        System.out.println("\n──────────────────────────────────────────────────");
        System.out.printf("Distancia entre centros (d): %.4f%n", d);
        System.out.printf("Suma de radios (r1 + r2):     %.4f%n", sumaRadios);
        System.out.printf("Diferencia de radios |r1-r2|: %.4f%n", difRadios);

        String clasificacion;
        if (d == 0.0 && r1 == r2) {
            clasificacion = "COINCIDENTES (Mismo centro y mismo radio)";
        } else if (d == 0.0) {
            clasificacion = "CONCÉNTRICAS (Mismo centro, distinto radio)";
        } else if (d > sumaRadios) {
            clasificacion = "EXTERIORES (No se tocan, d > r1 + r2)";
        } else if (d == sumaRadios) {
            clasificacion = "TANGENTES EXTERIORES (Se tocan en un punto exterior, d ==
r1 + r2)";
        } else if (d > difRadios && d < sumaRadios) {
            clasificacion = "SECANTES (Se cortan en dos puntos, |r1 - r2| < d < r1 +
r2)";
        } else if (d == difRadios) {
            clasificacion = "TANGENTES INTERIORES (Se tocan en un punto interior, d ==
|r1 - r2|)";
        } else {
            clasificacion = "INTERIORES NO CONCÉNTRICAS (Una dentro de otra sin
tocarse, d < |r1 - r2|)";
        }

        System.out.printf("→ Clasificación Geométrica: %s%n", clasificacion);
        System.out.println("──────────────────────────────────────────────────");
```

```
        sc.close();
    }
}
```

Solución al Ejercicio 12: Validador Integral de Fechas con Año Bisiesto Gregoriano

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ValidadorFechaCompleta {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("═══ VALIDADOR INTEGRAL DE FECHAS ═══");
        System.out.print("Introduce el día: ");
        int dia = sc.nextInt();

        System.out.print("Introduce el mes (1-12): ");
        int mes = sc.nextInt();

        System.out.print("Introduce el año: ");
        int anio = sc.nextInt();

        // 1. Validar año positivo
        if (anio ≤ 0) {
            System.out.printf("Fecha INVÁLIDA: El año %d debe ser estrictamente mayor
que cero.%n", anio);
            sc.close();
            return;
        }

        // 2. Validar rango de mes
        if (mes < 1 || mes > 12) {
            System.out.printf("Fecha INVÁLIDA: El mes %d no existe (debe ser entre 1 y
12).%n", mes);
            sc.close();
            return;
        }

        // 3. Determinar año bisiesto gregoriano
        boolean esBisiesto = (anio % 4 == 0 && anio % 100 ≠ 0) || (anio % 400 == 0);

        // 4. Determinar días máximos del mes
        int diasMaximos = switch (mes) {
            case 1, 3, 5, 7, 8, 10, 12 → 31;
            case 4, 6, 9, 11           → 30;
            case 2                     → esBisiesto ? 29 : 28;
            default                    → 0;
        };

        // 5. Validar día
        if (dia < 1 || dia > diasMaximos) {
            if (mes == 2 && dia == 29 && !esBisiesto) {
                System.out.printf("Fecha INVÁLIDA: El año %d no es bisiesto, por lo que
febrero solo tiene 28 días.%n", anio);
            } else {
                System.out.printf("Fecha INVÁLIDA: El día %d no es válido para el mes
%d (máximo permitido: %d).%n", dia, mes, diasMaximos);
            }
        } else {
            System.out.printf("¡Fecha VÁLIDA!: %02d/%02d/%04d %s%n",
                dia, mes, anio, (esBisiesto ? "(Año Bisiesto)" : "(Año Ordinario)"));
        }

        sc.close();
    }
```

```
}
```

Solución al Ejercicio 13: Ecuación de la Recta entre Dos Puntos y Tabla de Valores Tabulada

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class RectaEntrePuntos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ ECUACIÓN DE LA RECTA QUE UNE DOS PUNTOS ═══");
        System.out.print("Introduce coordenadas de P1 (x1 y1): ");
        double x1 = sc.nextDouble();
        double y1 = sc.nextDouble();

        System.out.print("Introduce coordenadas de P2 (x2 y2): ");
        double x2 = sc.nextDouble();
        double y2 = sc.nextDouble();

        System.out.println("\n──────────────────────────────────────────────────");
        if (x1 == x2) {
            // Recta vertical: pendiente indefinida
            System.out.printf("Ambos puntos comparten la misma coordenada X = %.3f%n",
x1);
            System.out.printf("Ecuación de la recta vertical: x = %.3f%n", x1);
            System.out.println("La pendiente es infinita; no es posible tabular como
función y = f(x).");
        } else {
            // Pendiente: m = (y2 - y1) / (x2 - x1)
            double m = (y2 - y1) / (x2 - x1);
            // Ordenada en el origen: n = y1 - m * x1
            double n = y1 - (m * x1);

            System.out.printf("Pendiente (m):            %8.3f%n", m);
            System.out.printf("Ordenada en origen (n):   %8.3f%n", n);
            System.out.printf("Ecuación explícita:       y = %.3f·x + (%.3f)%n%n", m,
n);

            // Tabla de valores para x en [-1.0, -0.5, 0.0, 0.5, 1.0]
            System.out.println("       TABLA DE VALORES CALCULADA      ");
            System.out.println("──────────────────────────────────────");
            System.out.printf("    %-10s | %-15s%n", "x", "y = m·x + n");
            System.out.println("──────────────────────────────────────");

            double[] valoresX = {-1.0, -0.5, 0.0, 0.5, 1.0};
            for (double vx : valoresX) {
                double vy = (m * vx) + n;
                if (vy == -0.0) vy = 0.0; // Normalización visual
                System.out.printf("    %8.3f   |   %8.3f%n", vx, vy);
            }
            System.out.println("──────────────────────────────────────");
        }

        sc.close();
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 2

🧠 Boletín 02 Extras — Retos y Plataformas Competitivas

Retos de algorítmica matemática y lógica en Java 21 / 25 LTS sin bucles complejos (DNI oficial, año bisiesto gregoriano, simulación probabilística, BigInteger, cambio voraz exacto y ecuaciones cuadráticas completas).

En este boletín extra nos enfrentaremos a **problemas reales y retos de plataformas de programación competitiva** (CodeWars, Acepta el Reto, Project Euler) utilizando exclusivamente los conceptos aprendidos en la Unidad 2: aritmética modular, sentencias condicionales rigurosas, manipulación de `String`, funciones de `Math` y tipos de precisión extendida (`BigInteger`).

Reto 1: Algoritmo Oficial de Verificación de DNI Español

(Inspirado en el procedimiento oficial del Ministerio del Interior y el problema clásico de Acepta el Reto)

Enunciado

El Documento Nacional de Identidad (DNI) español consta de un número de 8 dígitos seguido de una letra de control calculada mediante un algoritmo matemático que detecta erratas al teclear.

La letra se obtiene dividiendo el número de 8 cifras entre `23` y utilizando el **resto de la división entera (módulo 23)** como índice (del 0 al 22) en la siguiente tabla alfanumérica fija:

```
Resto:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
Letra:  T  R  W  A  G  M  Y  F  P  D  X  B  N  J  Z  S  Q  V  H  L  C  K  E
Cadena: "TRWAGMYFPDXBNJZSQVHLCKE"
```

Escribe un programa en Java que:

1. Pida al usuario su número de DNI (como entero de 8 dígitos).
2. Calcule la letra oficial correspondiente utilizando el operador `%` y el método `charAt()` sobre la cadena de referencia.
3. Solicite que introduzca la letra que figura en su documento físico.
4. Compare si la letra coincide (ignorando mayúsculas y minúsculas).
5. Muestre un mensaje indicando si el DNI es **válido** o si existe una **incongruencia**.

> 💡 **Pistas para la resolución**
>
> - Cadena clave: `String LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";`
> - Índice: `int indice = numeroDni % 23;`
> - Letra esperada: `char letraCorrecta = LETRAS.charAt(indice);`

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;

public class ValidadorDniOficial {
    private static final String TABLA_LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("═══ VALIDADOR OFICIAL DE DNI ESPAÑOL ═══");
        System.out.print("Introduce el número de DNI (8 dígitos enteros): ");
        int numero = sc.nextInt();

        System.out.print("Introduce la letra de control que figura en el documento: ");
        char letraUsuario = sc.next().toUpperCase().charAt(0);

        if (numero < 0 || numero > 99_999_999) {
            System.out.println("Error: El número de DNI debe contener entre 1 y 8
dígitos.");
            sc.close();
            return;
        }

        // Algoritmo: Resto de la división entera entre 23
        int resto = numero % 23;
        char letraCalculada = TABLA_LETRAS.charAt(resto);

        System.out.println("\n──────────────────────────────────────────");
        System.out.printf("Número evaluado: %08d%n", numero);
        System.out.printf("Resto (módulo 23): %d%n", resto);
        System.out.printf("Letra calculada:   %c%n", letraCalculada);
        System.out.printf("Letra aportada:    %c%n", letraUsuario);
        System.out.println("──────────────────────────────────────────");

        if (letraCalculada == letraUsuario) {
            System.out.println("✅ RESULTADO: DNI VÁLIDO y verificado con éxito.");
        } else {
            System.out.printf("❌ RESULTADO: DNI INCORRECTO. La letra oficial debería
ser la '%c'.%n", letraCalculada);
        }

        sc.close();
    }
}
```

Reto 2: El Algoritmo Astronómico del Año Bisiesto Gregoriano

(Clásico de CodeWars y entrevistas técnicas de algoritmos)

Enunciado

El calendario gregoriano universal establece que un año de 365 días debe añadir un día bisiesto (29 de febrero) cada 4 años para compensar el desfase de la órbita de la Tierra alrededor del Sol. Sin embargo, para evitar un exceso acumulativo secular, la regla formal completa tiene tres condiciones:

1. Un año es bisiesto si es **divisible entre 4**.

2. **EXCEPTO** si es divisible entre **100**, en cuyo caso **NO** es bisiesto.
3. **A MENOS QUE** sea también divisible entre **400**, en cuyo caso **SÍ** vuelve a ser bisiesto.

Ejemplos de control:

- `2024`: Bisiesto (divisible por 4 y no por 100).
- `1900`: **No** bisiesto (divisible por 4 y por 100, pero no por 400).
- `2000`: Bisiesto (divisible por 400).
- `2100`: **No** bisiesto.

Escribe un programa que solicite un año cualquiera y determine si es bisiesto utilizando una **única expresión booleana** con operadores lógicos (`&&`, `||`, `!`).

> 💡 **Pistas para la resolución**
>
> La condición matemática canónica se resume en: `(anio % 4 == 0 && anio % 100 ≠ 0) || (anio % 400 == 0)`

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;

public class AnioBisiestoGregoriano {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce un año para verificar si es bisiesto: ");
        int anio = sc.nextInt();

        // Expresión lógica canónica en una sola línea
        boolean esBisiesto = (anio % 4 == 0 && anio % 100 ≠ 0) || (anio % 400 == 0);

        System.out.println("\n————————————————————————————————————————");
        System.out.printf("Año analizado: %d%n", anio);
        System.out.printf("¿Es bisiesto según la regla gregoriana?: %b%n", esBisiesto);

        if (esBisiesto) {
            System.out.printf("→ El año %d tiene 366 días (febrero tiene 29 días).%n",
anio);
        } else {
            System.out.printf("→ El año %d es un año común de 365 días (febrero tiene
28 días).%n", anio);
        }
        System.out.println("————————————————————————————————————————");

        sc.close();
    }
}
```

Reto 3: Distribución Probabilística de Dos Dados en Juegos de Tablero

(Fundamentos de simulación Montecarlo y combinatoria discreta)

Enunciado

En juegos como *Catán*, *Monopoly* o *Craps*, se lanzan simultáneamente dos dados de 6 caras y se suman sus puntuaciones. Las sumas van desde `2` (1+1) hasta `12` (6+6).

No todas las sumas son equiprobables:

- La suma `7` tiene **6 combinaciones favorables** de 36 posibles (1+6, 2+5, 3+4, 4+3, 5+2, 6+1), con una probabilidad teórica del `16.67%`.
- Las sumas `2` y `12` solo tienen **1 combinación favorable** (1+1 y 6+6), con una probabilidad de solo `2.78%`.

Escribe un programa que:

1. Simule el lanzamiento independiente de dos dados usando `Math.random()`.
2. Calcule la suma de ambos dados.
3. Mediante una expresión `switch` moderna, imprima la puntuación obtenida, las combinaciones favorables y su probabilidad teórica asociada en forma de porcentaje.

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

public class ProbabilidadDados {
    public static void main(String[] args) {
        // Generamos dos tiradas independientes [1, 6]
        int d1 = (int)(Math.random() * 6) + 1;
        int d2 = (int)(Math.random() * 6) + 1;
        int suma = d1 + d2;

        // Expresión switch para consultar la distribución de probabilidad teórica
        String probabilidadInfo = switch (suma) {
            case 2, 12 → "1/36 (~2.78%) - Combinaciones extremas (1+1 ó 6+6)";
            case 3, 11 → "2/36 (~5.56%) - Poco frecuente";
            case 4, 10 → "3/36 (~8.33%) - Frecuencia moderada";
            case 5, 9  → "4/36 (~11.11%) - Frecuencia media";
            case 6, 8  → "5/36 (~13.89%) - Muy frecuente (excelente en Catán)";
            case 7     → "6/36 (~16.67%) - Puntuación de máxima probabilidad (¡El
ladrón despierta!)";
            default    → "Error probabilístico";
        };

        System.out.println("═══ SIMULADOR PROBABILÍSTICO DE DADOS ═══");
        System.out.printf("Dado 1:                    %d%n", d1);
        System.out.printf("Dado 2:                    %d%n", d2);
        System.out.printf("Suma obtenida:             %d%n", suma);
        System.out.println("──────────────────────────────────────────────");
        System.out.printf("Probabilidad teórica:      %s%n", probabilidadInfo);
        System.out.println("──────────────────────────────────────────────");
    }
}
```

Reto 4: Factoriales Astronómicos con `BigInteger`

(Problema clásico de Project Euler y CodeWars 4-Kyu)

Enunciado

El factorial de un número entero positivo `n` (representado como `n!`) es el producto de todos los números enteros desde 1 hasta `n`: `n! = 1 × 2 × 3 × ... × n`

- `5! = 120`
- `10! = 3.628.800`
- `20! = 2.432.902.008.176.640.000` (ya casi satura el límite de un `long` de 64 bits)
- `21!` desborda totalmente la capacidad de cualquier tipo primitivo de Java (`Long.MAX_VALUE`).

Escribe un programa en Java que solicite al usuario calcular el factorial de un número como `50` o `100`.
Utiliza la clase `java.math.BigInteger` para realizar la multiplicación encadenada sin ningún límite de precisión numérica, mostrando la cifra gigantesca resultante en consola junto con el número total de dígitos que contiene su representación textual.

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.math.BigInteger;

public class FactorialGigante {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("═══ CÁLCULO DE FACTORIALES GIGANTES (BigInteger) ═══");
        System.out.print("Introduce un número entero positivo (ej. 50): ");
        int n = sc.nextInt();

        if (n < 0) {
            System.out.println("Error: El factorial no está definido para números
negativos.");
            sc.close();
            return;
        }

        BigInteger factorial = BigInteger.ONE;

        // Multiplicamos acumulativamente usando BigInteger
        for (int i = 2; i ≤ n; i++) {
            factorial = factorial.multiply(BigInteger.valueOf(i));
        }

        String representacion = factorial.toString();

        System.out.println("\n──────────────────────────────────────────────");
        System.out.printf("%d! = %n%s%n", n, representacion);
        System.out.println("──────────────────────────────────────────────");
        System.out.printf("Total de dígitos en el resultado: %d dígitos decimales.%n",
representacion.length());

        sc.close();
    }
}
```

Reto 5: Algoritmo Voraz para el Cambio Monetario con Céntimos Exactos

(Problema de optimización financiera sin errores de coma flotante)

Enunciado

Una de las peores trampas en programación financiera es operar con importes monetarios en variables `double` (debido a que `0.1 + 0.2` da `0.30000000000000004`).

Escribe un programa que pida el coste total de una compra en euros (por ejemplo `3.47 €`) y la cantidad entregada por el cliente (por ejemplo `10.00 €`).

1. Convierte inmediatamente ambos importes a **céntimos enteros** multiplicando por 100 y redondeando con `Math.round()` a tipo `int`.
2. Calcula la diferencia a devolver en céntimos enteros (en este caso, `653` céntimos).
3. Desglosa el cambio utilizando el menor número de monedas posible: `200` (2 €), `100` (1 €), `50`, `20`, `10`, `5`, `2` y `1` céntimo.

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.util.Locale;

public class CambioMonetarioExacto {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ DISPENSADOR VORAZ DE CAMBIO EN CÉNTIMOS ═══");
        System.out.print("Introduce el precio total de la compra (€): ");
        double precioEuros = sc.nextDouble();

        System.out.print("Introduce la cantidad pagada por el cliente (€): ");
        double pagoEuros = sc.nextDouble();

        // 1. Transformación obligatoria a céntimos enteros con Math.round para evitar
imprecisiones IEEE 754
        int precioCentimos = (int) Math.round(precioEuros * 100);
        int pagoCentimos = (int) Math.round(pagoEuros * 100);

        if (pagoCentimos < precioCentimos) {
            double falta = (precioCentimos - pagoCentimos) / 100.0;
            System.out.printf("Error: Pago insuficiente. Faltan %.2f €.%n", falta);
            sc.close();
            return;
        }

        int cambioRestante = pagoCentimos - precioCentimos;

        System.out.println("\n──────────────────────────────────────────");
        System.out.printf("Total compra:   %8.2f € (%d céntimos)%n", precioEuros,
precioCentimos);
        System.out.printf("Total pagado:   %8.2f € (%d céntimos)%n", pagoEuros,
pagoCentimos);
        System.out.printf("Cambio exacto:  %8.2f € (%d céntimos)%n", cambioRestante /
100.0, cambioRestante);
        System.out.println("──────────────────────────────────────────");
        System.out.println("Desglose en monedas de curso legal:");

        int[] monedas = {200, 100, 50, 20, 10, 5, 2, 1};

        for (int moneda : monedas) {
            int cantidad = cambioRestante / moneda;
            cambioRestante %= moneda;

            if (cantidad > 0) {
                if (moneda ≥ 100) {
                    System.out.printf("→ %d moneda(s) de %d €%n", cantidad, moneda /
100);
                } else {
                    System.out.printf("→ %d moneda(s) de %d céntimos%n", cantidad,
moneda);
                }
            }
        }
        System.out.println("──────────────────────────────────────────");

        sc.close();
    }
}
```

Reto 6: Resolución de la Ecuación de Segundo Grado con Análisis de Discriminante

(Problema fundamental de álgebra y computación científica)

Enunciado

La ecuación de segundo grado general responde a la forma: `a·x² + b·x + c = 0`

Escribe un programa en Java que solicite los coeficientes reales `a`, `b` y `c` y determine todas sus soluciones posibles según el discriminante: `Δ = b² − 4·a·c`

El programa debe implementar el árbol de decisión completo:

1. **Si** `a == 0`: La ecuación no es cuadrática, sino lineal (`b·x + c = 0`):
   - Si `b ≠ 0`: solución única `x = −c / b`.
   - Si `b == 0` y `c ≠ 0`: sin solución.
   - Si `b == 0` y `c == 0`: infinitas soluciones.
2. **Si** `a ≠ 0`: Se calcula el discriminante `Δ`:
   - **Caso** `Δ > 0`: Existen **dos raíces reales distintas**:
     `x1 = (−b + √Δ) / (2·a)`
     `x2 = (−b − √Δ) / (2·a)`
   - **Caso** `Δ == 0`: Existe **una única raíz real doble**:
     `x = −b / (2·a)`
   - **Caso** `Δ < 0`: No existen soluciones en la recta real; existen **dos raíces complejas conjugadas** en la forma `u ± v·i`:
     Parte real: `u = −b / (2·a)`
     Parte imaginaria: `v = √(−Δ) / (2·a)`

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.util.Locale;

public class EcuacionSegundoGrado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("═══ RESOLUCIÓN DE LA ECUACIÓN DE SEGUNDO GRADO: a·x² + b·x
+ c = 0 ═══");
        System.out.print("Introduce coeficiente a: ");
        double a = sc.nextDouble();

        System.out.print("Introduce coeficiente b: ");
        double b = sc.nextDouble();

        System.out.print("Introduce coeficiente c: ");
        double c = sc.nextDouble();

        System.out.println("\n──────────────────────────────────────────────────────────
");
        System.out.printf("Ecuación analizada: (%.2f)·x² + (%.2f)·x + (%.2f) = 0%n", a,
b, c);

        if (a == 0.0) {
            System.out.println("→ Degeneración a ecuación lineal de 1er grado (a =
0):");
            if (b ≠ 0.0) {
                double x = -c / b;
                if (x == -0.0) x = 0.0;
                System.out.printf("   Solución única lineal: x = %.4f%n", x);
            } else if (c ≠ 0.0) {
                System.out.println("   Sin solución (inconsistencia matemática).");
            } else {
                System.out.println("   Infinitas soluciones (identidad 0 = 0).");
            }
        } else {
            // Cálculo del discriminante: delta = b^2 - 4ac
            double delta = Math.pow(b, 2) - (4 * a * c);
            System.out.printf("Discriminante (Δ = b² - 4ac): %.4f%n", delta);

            if (delta > 0.0) {
                double x1 = (-b + Math.sqrt(delta)) / (2 * a);
                double x2 = (-b - Math.sqrt(delta)) / (2 * a);
                System.out.println("→ Caso Δ > 0: Existen DOS raíces reales
distintas:");
                System.out.printf("   x1 = %8.4f%n", x1);
                System.out.printf("   x2 = %8.4f%n", x2);
            } else if (delta == 0.0) {
                double x = -b / (2 * a);
                if (x == -0.0) x = 0.0;
                System.out.println("→ Caso Δ = 0: Existe UNA raíz real DOBLE:");
                System.out.printf("   x1 = x2 = %.4f%n", x);
            } else {
                // delta < 0: Raíces complejas conjugadas u ± v*i
                double parteReal = -b / (2 * a);
                double parteImaginaria = Math.sqrt(-delta) / (2 * a);
                if (parteReal == -0.0) parteReal = 0.0;
```

```
                System.out.println("→ Caso Δ < 0: Sin soluciones reales. Existen DOS
raíces COMPLEJAS conjugadas:");
                System.out.printf("   x1 = %.4f + %.4f·i%n", parteReal,
Math.abs(parteImaginaria));
                System.out.printf("   x2 = %.4f - %.4f·i%n", parteReal,
Math.abs(parteImaginaria));
            }
        }
        System.out.println("──────────────────────────────────────────────────────────
");

        sc.close();
    }
}
```

CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

🔀 Unidad 3. Excepciones, bucles, arrays y métodos

Control estructurado de errores con excepciones (try-catch-finally), estructuras iterativas (while, do-while, for, for-each), estructuras de datos homogéneas (arrays y matrices), procesamiento avanzado de texto con StringBuilder, y diseño modular de software con métodos, paso de parámetros y recursividad en Java 25 LTS.



MÓDULO PROFESIONAL
Programación (0485) — 1º DAM

RESULTADOS DE APRENDIZAJE
RA1, RA2, RA3, RA6 (Estructuras de control, arrays, métodos y excepciones)

DURACIÓN ESTIMADA
28 horas lectivas

ENTORNO OFICIAL
IntelliJ IDEA / Java 25 LTS

Hasta este punto de tu formación como desarrollador de software, tus programas han seguido una lógica eminentemente lineal y selectiva: el código se ejecutaba de arriba abajo y tomaba bifurcaciones mediante sentencias `if-else` o `switch`. Sin embargo, la auténtica potencia de la computación reside en su capacidad para **procesar millones de datos en fracciones de segundo mediante la repetición controlada**, **blindarse ante fallos imprevistos** y **estructurar algoritmos complejos en piezas reutilizables y modulares**.

En esta unidad nos adentramos en el núcleo de la programación estructurada con **Java 25 LTS** (con compatibilidad plena para Java 21 LTS) y nuestro entorno de trabajo oficial, **IntelliJ IDEA (Visual Studio Code)**:

1. **Gestión de Excepciones**: Cómo evitar que nuestras aplicaciones se cuelguen ante entradas incorrectas del usuario, recursos no disponibles o errores aritméticos mediante bloques `try-catch-finally` y `try-with-resources`.
2. **Sentencias de Repetición (Bucles)**: Automatización iterativa con `while`, `do-while` y `for`, junto con técnicas de control de flujo (`break`, `continue` y variables centinela).
3. **Arrays y Matrices**: Almacenamiento contiguo y masivo de información en memoria, recorridos optimizados, algoritmos clásicos de búsqueda y ordenación, y herramientas de la clase estándar `java.util.Arrays`.
4. **Procesamiento de Cadenas**: Técnicas de recorrido e inspección sobre textos y optimización de memoria mediante la clase `StringBuilder`.
5. **Programación Modular (Métodos)**: Descomposición de problemas según el paradigma de "divide y vencerás", paso de parámetros por valor, mutabilidad en memoria Stack y Heap, sobrecarga y algoritmos recursivos.
6. **Expresiones Regulares (Regex)**: Validación profesional de patrones sintácticos complejos (DNI, correos, formatos numéricos).

---

1. Introducción a la Programación Estructurada y Control Avanzado

En los orígenes de la informática moderna, el control de flujo en lenguajes primitivos dependía en gran medida de saltos incondicionales (el tristemente célebre `GOTO`), lo que provocaba el llamado "código espagueti": un laberinto indescifrable de saltos cruzados imposible de depurar y mantener.

En 1966, los matemáticos Corrado Böhm y Giuseppe Jacopini demostraron el **Teorema del Programa Estructurado**: cualquier algoritmo computable puede expresarse combinando únicamente **tres estructuras fundamentales**:

```
Pilares de la Programación Estructurada
├── 1. Secuencia  → Instrucción tras instrucción de forma consecutiva
├── 2. Selección  → Bifurcaciones lógicas (if-else, switch) [Visto en la Unidad 2]
└── 3. Iteración  → Repetición controlada de bloques (while, do-while, for) [Núcleo de esta Unidad]
```

A estos tres pilares clásicos, la ingeniería de software moderna ha añadido dos disciplinas de vital importancia:

- **La Modularidad (Métodos):** Aislar responsabilidades en subrutinas independientes, testables y autoexplicativas.
- **La Robustez y Manejo de Errores (Excepciones):** Mecanismos formales para separar la lógica de negocio habitual de los caminos de contingencia ante fallos en tiempo de ejecución.

---

2. Gestión Profesional de Excepciones y Robustez de Software

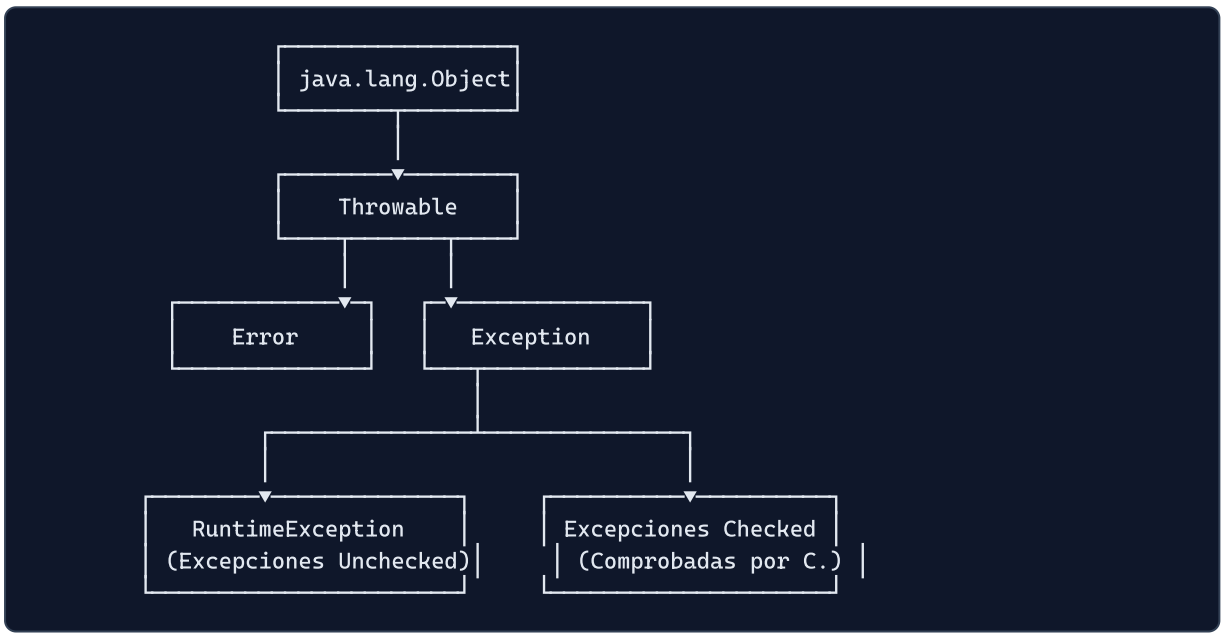
Todo programa informático opera en un entorno imperfecto: los usuarios introducen letras cuando se les piden números, los discos duros se llenan, las conexiones de red se interrumpen y los divisores pueden ser cero.

Un programa amateur se "estrella" arrojando un críptico error rojo en la consola. Un software profesional **anticipa las contingencias, intercepta los errores y reacciona de forma controlada**.

2.1 Qué es una Excepción y la Jerarquía `Throwable` de Java

En Java, un error en tiempo de ejecución no es una simple señal numérica del sistema operativo: **es un objeto de pleno derecho** que se instancia en la memoria Heap y se "dispara" (*throw*) hacia arriba en la pila de llamadas de la JVM.

Todos los objetos de error derivan de la superclase `java.lang.Throwable`, dividida en dos ramas cruciales:



1. `Error`: Fallos catastróficos a nivel de hardware o de la propia JVM (como `OutOfMemoryError` o `StackOverflowError`). El programador **no debe intentar capturarlos**, ya que la máquina se encuentra en un estado irrecuperable.
2. `Exception`: Situaciones anómalas de las que un programa bien diseñado sí puede recuperarse. Se subdividen en dos grandes familias:
    - **Excepciones no comprobadas (*Unchecked / RuntimeException*):** Suelen derivar de errores de programación lógica (como `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `InputMismatchException`). El compilador no te obliga a capturarlas explícitamente, pero debes prevenirlas con validaciones o capturarlas si proceden de la entrada del usuario.
    - **Excepciones comprobadas (*Checked Exceptions*):** Errores derivados de factores externos al código (fallos de ficheros `IOException`, bases de datos `SQLException`, redes). El compilador `javac` **obliga taxativamente** a capturarlas con `try-catch` o a declararlas en la cabecera del método con `throws`.

2.2 Captura Estructurada con Bloques `try-catch`

La estructura `try-catch` permite "probar" la ejecución de un bloque de código potencialmente peligroso. Si durante su ejecución se produce un error, la JVM detiene inmediatamente el bloque `try` y salta al bloque `catch`

correspondiente para ejecutar la rutina de rescate:

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class DivisionSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Introduce el numerador (entero): ");
            int num = sc.nextInt();

            System.out.print("Introduce el denominador (entero): ");
            int den = sc.nextInt();

            int cociente = num / den; // Peligro: ArithmeticException si den == 0
            System.out.printf("Resultado exacto: %d / %d = %d%n", num, den, cociente);

        } catch (ArithmeticException e) {
            System.out.println("❌ Error matemático: ¡No es posible dividir un número
entero por cero!");
        } catch (InputMismatchException e) {
            System.out.println("❌ Error de formato: Debes introducir dígitos enteros
válidos.");
        }

        System.out.println("→ El programa continúa su ejecución normal sin
estrellarse.");
        sc.close();
    }
}
```

Captura Múltiple Moderna (*Multi-catch* - Java 7+)

Si varios tipos de excepción deben recibir idéntico tratamiento, podemos agruparlas en un único bloque `catch` separadas por la barra vertical (`|`):

```
try {
    procesarDatos();
} catch (ArithmeticException | NumberFormatException e) {
    System.out.println("Error numérico detectado: " + e.getMessage());
}
```

> **El Orden de los bloques `catch` Importa**
>
> Las excepciones se evalúan de arriba abajo. Siempre debes colocar los `catch` de clases hijas más específicas primero (por ejemplo, `InputMismatchException`) y las clases padre más genéricas después (por ejemplo, `Exception`). Si colocas `catch (Exception e)` al principio, capturará cualquier error y los bloques inferiores se convertirán en código inalcanzable (*unreachable code*), provocando un error de compilación.

2.3 El Bloque `finally` y su Garantía de Ejecución

El bloque `finally` se sitúa tras los bloques `catch` . Su característica definitoria es tajante: **su código se ejecutará SIEMPRE**, tanto si el bloque `try` finalizó con éxito, como si se produjo una excepción que fue capturada, e incluso

si se produjo una excepción no capturada o se ejecutó una sentencia `return`:

```
Scanner sc = new Scanner(System.in);
try {
    System.out.print("Introduce un número: ");
    int n = sc.nextInt();
    System.out.println("Número al cuadrado: " + (n * n));
} catch (Exception e) {
    System.out.println("Ha ocurrido una incidencia: " + e.getMessage());
} finally {
    // Se ejecuta INCONDICIONALMENTE
    System.out.println("Limpiando y liberando recursos del sistema ... ");
}
```

2.4 La Sentencia Moderna `try-with-resources` (Java 7+)

Cerrar flujos de datos (`Scanner`, archivos, sockets de red o conexiones a bases de datos) manualmente en bloques `finally` suele requerir comprobaciones farragosas contra valores `null`.

Para resolverlo, Java introdujo la construcción `try-with-resources`. Cualquier objeto que implemente la interfaz `java.lang.AutoCloseable` puede declararse dentro de los paréntesis del `try ( ... )`, y la JVM garantizará su **cierre automático e inmediato** en cuanto el flujo abandone el bloque:

```
import java.util.Scanner;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        // El Scanner se cerrará automáticamente al salir del bloque try
        try (Scanner teclado = new Scanner(System.in)) {
            System.out.print("Escribe tu nombre: ");
            String nombre = teclado.nextLine();
            System.out.println("Hola, " + nombre);
        } catch (Exception e) {
            System.out.println("Incidencia durante la lectura: " + e.getMessage());
        }
        // Aquí 'teclado' ya ha sido cerrado de forma segura y liberado
    }
}
```

2.5 Lanzamiento y Propagación: `throw` y `throws`

A menudo, un método detecta que los datos que ha recibido violan las precondiciones necesarias para operar (por ejemplo, una edad negativa o una cuenta bancaria sin fondos suficientes). En lugar de retornar un valor mágico erróneo como `-1`, el método debe **lanzar una excepción** formal:

- `throw` **(en imperativo):** Dispara una instancia concreta de excepción dentro del cuerpo de un método.
- `throws` **(en indicativo):** Declara en la firma de un método las excepciones comprobadas que dicho método puede llegar a disparar hacia quien lo invoque.

```
public class BancoSeguro {
    // Declaramos que el método puede arrojar una excepción ante datos inválidos
    public static void retirarDinero(double saldoActual, double cantidad) throws
IllegalArgumentException {
        if (cantidad ≤ 0) {
            // Instanciamos y arrojamos la excepción
            throw new IllegalArgumentException("La cantidad a retirar debe ser
estrictamente positiva.");
        }
        if (cantidad > saldoActual) {
            throw new IllegalArgumentException("Saldo insuficiente. Saldo disponible: "
+ saldoActual);
        }

        System.out.println("Retirada completada. Nuevo saldo: " + (saldoActual -
cantidad));
    }

    public static void main(String[] args) {
        try {
            retirarDinero(100.0, 150.0);
        } catch (IllegalArgumentException e) {
            System.out.println("Transacción rechazada: " + e.getMessage());
        }
    }
}
```

El Anti-Patrón "Catch and Swallow" (Atrapar y Silenciar)

Uno de los hábitos más perniciosos de los desarrolladores noveles es dejar bloques `catch` completamente vacíos:

```
try {
    operacionCritica();
} catch (Exception e) {
    // Bloque vacío: ¡EL MAYOR ERROR DE LA INGENIERÍA DE SOFTWARE!
}
```

Hacer esto oculta los fallos bajo la alfombra: el programa fallará silenciosamente, corromperá datos en memoria y cuando la aplicación colapse minutos después, resultará imposible rastrear el origen del fallo. **Como mínimo, imprime siempre un mensaje o invoca `e.printStackTrace()`**.

---

3. Sentencias de Repetición: Bucles en Java

Un **bucle** es una estructura de control que repite la ejecución de un bloque de código mientras una determinada condición booleana se evalúe a `true`. Cada repetición del bloque recibe el nombre de **iteración**.

Java proporciona tres tipos fundamentales de bucles tradicionales más una variante optimizada para colecciones:

```
Bucles en Java
├── while     → Comprobación al inicio (de 0 a N iteraciones)
├── do-while  → Comprobación al final (de 1 a N iteraciones - ideal para menús)
├── for       → Control de iteraciones conocido con contador
└── for-each  → Recorrido simplificado de arrays y colecciones
```

3.1 El Bucle `while` (Comprobación Previa)

El bucle `while` evalúa la condición antes de cada iteración. Si la condición es `false` desde el principio, el cuerpo del bucle **no se ejecutará ni una sola vez**:

```
while (condicion) {
    // Instrucciones que se repiten mientras condicion sea true
    // DEBE existir algún mecanismo que modifique las variables de la condición
}
```

```
int contador = 1;
while (contador ≤ 5) {
    System.out.println("Iteración número: " + contador);
    contador++; // Modificación obligatoria del estado
}
```

El Patrón Bucle Centinela (*Sentinel Loop*)

Se utiliza cuando no sabemos cuántos datos introducirá el usuario y utilizamos un valor especial (el "centinela", por ejemplo el `0` o `-1`) para indicar el final de la serie:

```
import java.util.Scanner;

public class BucleCentinelaDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int conteo = 0;

        System.out.println("Introduce números enteros a sumar (teclea 0 para
finalizar):");
        int numero = sc.nextInt(); // Lectura previa

        while (numero ≠ 0) { // '0' actúa como valor centinela
            suma += numero;
            conteo++;
            System.out.print("Siguiente número (0 para salir): ");
            numero = sc.nextInt(); // Lectura dentro del bucle
        }

        System.out.printf("Has introducido %d números. Suma acumulada: %d%n", conteo,
suma);
        sc.close();
    }
}
```

Peligro de Bucle Infinito

Si olvidas actualizar la variable de control dentro del cuerpo del bucle (o si la condición jamás puede llegar a ser `false`), el programa entrará en un **bucle infinito**: la CPU se pondrá al 100% de uso y la aplicación quedará completamente congelada. En IntelliJ IDEA puedes detener un programa en bucle infinito pulsando el botón rojo cuadrado (*Stop*) o el atajo `Ctrl + F2`.

3.2 El Bucle `do-while` (Comprobación Posterior)

En el bucle `do-while`, el bloque de código se ejecuta **primero** y la condición se evalúa **después**. Esto garantiza que las instrucciones se ejecutarán **siempre al menos una vez**:

```
do {
    // Bloque de instrucciones (se ejecuta al menos una vez)
} while (condicion); // ¡Ojo al punto y coma final obligatorio!
```

La Aplicación Estrella: Menús Interactivos y Validación de Entrada

El escenario predilecto de `do-while` es mostrar un menú al usuario y forzarle a elegir una opción válida:

```
import java.util.Scanner;

public class MenuInteractivaDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n═══ PANEL DE CONTROL DAM ═══");
            System.out.println("1. Ver perfil de usuario");
            System.out.println("2. Consultar notas del módulo");
            System.out.println("3. Descargar boletín de ejercicios");
            System.out.println("0. Salir de la aplicación");
            System.out.print("Elige una opción (0-3): ");

            opcion = sc.nextInt();

            if (opcion < 0 || opcion > 3) {
                System.out.println("⚠️ Opción incorrecta. Por favor, selecciona un
número entre 0 y 3.");
            }

        } while (opcion ≠ 0); // Repite mientras el usuario no decida salir

        System.out.println("Sesión finalizada. ¡Hasta pronto!");
        sc.close();
    }
}
```

3.3 El Bucle `for` Clásico (Contador Determinado)

Cuando se conoce de antemano el número de repeticiones (o se desea recorrer un rango numérico determinado), el bucle `for` condensa toda la lógica de control en una sola línea canónica:

```
for (inicialización; condición; actualización) {
    // Instrucciones a iterar
}
```

1. **Inicialización:** Se ejecuta una única vez al arrancar el bucle (ej. `int i = 0;`). La variable declarada tiene ámbito (*scope*) local exclusivo al bucle.
2. **Condición:** Se evalúa antes de cada ciclo (ej. `i < 10;`). Si es `true`, entra al cuerpo; si es `false`, el bucle termina.
3. **Actualización:** Se ejecuta inmediatamente al final de cada iteración (ej. `i++`, `i += 2`, `i--`).

```
// Recorrido ascendente estándar de 1 a 10
for (int i = 1; i ≤ 10; i++) {
    System.out.print(i + " ");
}
System.out.println();

// Recorrido descendente (cuenta atrás)
for (int i = 10; i ≥ 0; i--) {
    System.out.print(i + "... ");
}
System.out.println("¡Despegue! 🚀");

// Incremento a saltos (números pares entre 0 y 20)
for (int i = 0; i ≤ 20; i += 2) {
    System.out.print(i + " ");
}
System.out.println();
```

3.4 Patrones Algorítmicos con Bucles: Contadores, Acumuladores y Banderas

Casi todos los algoritmos iterativos se construyen sobre la combinación de tres patrones fundamentales de variables:

1. **Contador:** Variable entera que se incrementa en una cantidad fija (habitualmente `+1`) cada vez que ocurre un evento de interés:

   ```
   int contadorPares = 0;
   if (n % 2 == 0) contadorPares++;
   ```

2. **Acumulador (o Totalizador):** Variable que suma o multiplica valores variables en cada ciclo para obtener un total agregado:

   ```
   double totalVentas = 0.0;
   totalVentas += precioArticulo;
   ```

3. **Bandera (*Flag* o Variable Booleana):** Variable de tipo `boolean` que comienza con un estado inicial (ej. `false`) y "se enciende" cuando se constata una condición especial:

   ```
   boolean encontrado = false;
   if (numero == 10) encontrado = true;
   ```

3.5 Sentencias de Ruptura de Flujo: `break` y `continue`

Java proporciona dos instrucciones especiales para alterar la secuencia normal de un bucle desde su interior:

- `break`: Aborta de forma fulminante la ejecución del bucle, saltando inmediatamente a la instrucción posterior al cierre de la llave `}`:

```
for (int i = 1; i ≤ 100; i++) {
    if (i == 42) {
        System.out.println("¡Elemento 42 localizado! Deteniendo búsqueda.");
        break; // Abandona el bucle inmediatamente
    }
}
```

- `continue`: Salta el resto de instrucciones de la iteración *actual* y avanza de inmediato al siguiente ciclo (en un `for`, ejecuta la cláusula de actualización `i++` y reevalúa la condición):

```
// Imprimir solo los números impares ignorando los pares
for (int i = 1; i ≤ 10; i++) {
    if (i % 2 == 0) {
        continue; // Pasa inmediatamente al siguiente número sin ejecutar el
println
    }
    System.out.println("Impar: " + i);
}
```

Bucles con Etiquetas (*Labeled Breaks*)

Cuando tenemos varios bucles anidados unos dentro de otros, un `break` simple solo rompe el bucle más interno en el que se encuentra. Si necesitamos abortar todos los bucles simultáneamente desde el nivel más profundo, utilizamos una **etiqueta (*label*)**:

```
buscarCoordenadas: // Etiqueta identificativa
for (int f = 0; f < 10; f++) {
    for (int c = 0; c < 10; c++) {
        if (matriz[f][c] == elementoBuscado) {
            System.out.printf("Encontrado en posición [%d, %d]%n", f, c);
            break buscarCoordenadas; // Rompe AMBOS bucles de golpe
        }
    }
}
```

3.6 Bucles Anidados (*Nested Loops*)

Un bucle anidado es sencillamente un bucle situado dentro del cuerpo de otro. El bucle interno completa todas sus iteraciones por cada ciclo individual del bucle externo.

Son esenciales para generar tablas de coordenadas, matrices bidimensionales y cuadrículas:

```
// Generación completa de las tablas de multiplicar del 1 al 9
for (int tabla = 1; tabla ≤ 9; tabla++) {
    System.out.println("\n--- TABLA DEL " + tabla + " ---");
    for (int mult = 1; mult ≤ 10; mult++) {
        System.out.printf("%d x %2d = %2d%n", tabla, mult, (tabla * mult));
    }
}
```

---

4. Arrays: Vectores y Matrices Multidimensionales

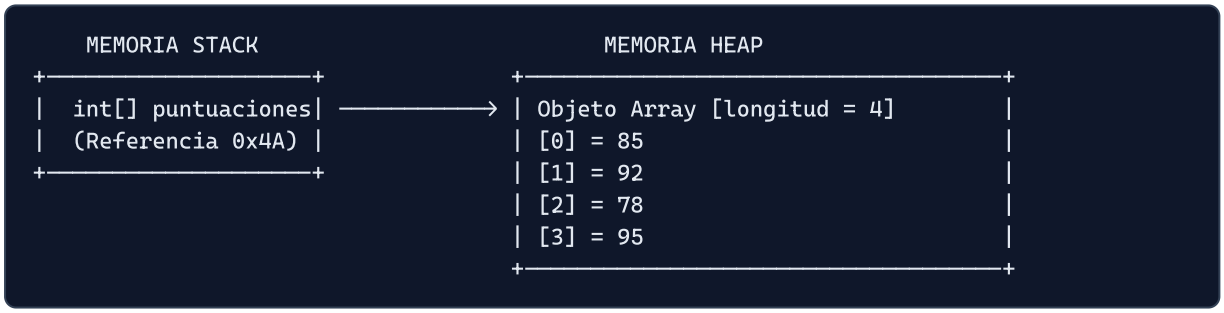
Hasta ahora, si un programa requería gestionar las notas de 50 alumnos, nos veíamos obligados a declarar 50 variables individuales (`nota1` , `nota2` , ..., `nota50`). Esta aproximación es inescalable e inmanejable: imposibilita la iteración mediante bucles y llena el código de redundancia.

Para solventar esta limitación, Java implementa los **arrays** (también denominados *vectores*, *arreglos* o *tablas*).

4.1. Concepto y Representación en Memoria (Stack vs. Heap)

Un **array** es una estructura de datos homogénea (todos sus elementos son del mismo tipo) y estática (su tamaño se define en el momento de la instanciación y permanece inmutable durante todo su ciclo de vida).

En Java, a diferencia de lenguajes como C/C++, **los arrays son objetos de pleno derecho**. Esto implica una diferenciación crucial en la memoria de la JVM:



1. **Memoria Stack:** Aloja la variable de referencia (`puntuaciones`), cuyo contenido no son los datos numéricos, sino la dirección de memoria donde reside el objeto array en el Heap.
2. **Memoria Heap:** Aloja el bloque contiguo de memoria donde se almacenan físicamente los elementos indexados consecutivamente.

> **Tamaño Fijo e Inmutable**
>
> Una vez reservado un array de tamaño `N` , **no es posible ampliarlo ni reducirlo**. Si en tiempo de ejecución necesitas albergar `N + 1` elementos, deberás instanciar un nuevo array de mayor capacidad y volcar los elementos del antiguo en el nuevo (o utilizar colecciones dinámicas como `ArrayList` , tratadas en la Unidad 6).

4.2. Declaración, Instanciación e Inicialización

Existen diversas formas sintácticas de declarar y poblar un array en Java:

1. Declaración e Instanciación con Tamaño Fijo (Valores por Defecto)

Al reservar memoria con el operador `new`, Java inicializa automáticamente cada posición del array con el valor neutro o por defecto de su tipo:

- Tipos numéricos enteros (`byte`, `short`, `int`, `long`): `0`.
- Tipos numéricos reales (`float`, `double`): `0.0`.
- Booleanos (`boolean`): `false`.
- Caracteres (`char`): `'\u0000'` (carácter nulo).
- Tipos por referencia (`String`, objetos, wrappers): `null`.

```
// Declaración canónica (corchetes junto al tipo, estándar oficial de Java)
int[] temperaturas = new int[7];

// Asignación manual por índice (los índices comienzan SIEMPRE en 0)
temperaturas[0] = 18;
temperaturas[1] = 21;
temperaturas[6] = 19; // Última posición válida (7 - 1)
```

2. Inicialización Directa mediante Literales

Cuando conocemos los valores iniciales de antemano, podemos declarar e inicializar en una única sentencia compacta:

```
// Inicialización literal inline (forma preferida)
String[] diasSemana = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"};

// Inicialización anónima explícita (útil para pasar arrays como argumentos a métodos)
int[] factoresPrimos = new int[]{2, 3, 5, 7, 11};
```

4.3. Acceso a Elementos, Atributo `.length` y la Excepción `ArrayIndexOutOfBoundsException`

El acceso y modificación de cualquier elemento se realiza en tiempo constante `O(1)` indicando el índice entre corchetes `[índice]`:

- **Primer elemento:** `array[0]`.
- **Último elemento:** `array[array.length - 1]`.
- **Propiedad `.length`:** Es una variable constante de solo lectura pública que almacena el número total de celdas reservadas. *(Nota: No lleva paréntesis; `length` es un atributo en arrays, mientras que en `String` es un método `length()`).*

El Riesgo del Error Fuera de Límites

Si intentamos acceder a un índice negativo o a un índice mayor o igual a `.length`, la JVM aborta de inmediato la ejecución lanzando una excepción no comprobada de tipo `ArrayIndexOutOfBoundsException`:

```
int[] codigos = {101, 102, 103};

System.out.println(codigos[3]);
// ¡ERROR EN TIEMPO DE EJECUCIÓN!
// Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of
bounds for length 3
```

4.4. Recorridos: Bucle `for` Clásico vs. Bucle Mejorado (`for-each`)

Para procesar o mostrar los elementos de un vector, disponemos de dos variantes principales:

```
double[] calificaciones = {7.5, 4.2, 9.0, 6.8, 8.3};

// Variante A: Bucle for indexado clásico
// Obligatorio si necesitamos conocer o utilizar el índice numérico,
// o si deseamos modificar el contenido almacenado en el array.
for (int i = 0; i < calificaciones.length; i++) {
    System.out.printf("Alumno %d: %.1f%n", (i + 1), calificaciones[i]);
}

// Variante B: Bucle for-each (Enhanced For Loop)
// Recomendado cuando solo deseamos LEER secuencialmente todos los elementos.
// Sintaxis limpia: 'por cada elemento de tipo double dentro de calificaciones'
double sumaTotal = 0;
for (double nota : calificaciones) {
    sumaTotal += nota;
}
double media = sumaTotal / calificaciones.length;
System.out.printf("Media del grupo: %.2f%n", media);
```

El for-each es de Solo Lectura para Primitivos

En un bucle `for-each`, la variable iteradora (`nota` en el ejemplo anterior) es una **copia local** del valor en cada celda. Si ejecutas `nota = 10.0;` dentro del bucle, solo modificarás la variable temporal local; el contenido original del array `calificaciones` permanecerá intacto.

4.5. Algoritmos Esenciales sobre Arrays

El dominio de la programación estructurada exige comprender los patrones algorítmicos fundamentales para buscar, calcular y ordenar información en vectores.

1. Búsqueda de Mínimo, Máximo y Cálculo de Estadísticas

```
public class AnalisisVectores {
    public static void main(String[] args) {
        int[] datos = {45, 12, 89, 34, 12, 99, 23, 7};

        if (datos.length == 0) {
            System.out.println("El array está vacío.");
            return;
        }

        int maximo = datos[0];
        int minimo = datos[0];
        int acumuladorSuma = 0;

        for (int valor : datos) {
            if (valor > maximo) {
                maximo = valor;
            }
            if (valor < minimo) {
                minimo = valor;
            }
            acumuladorSuma += valor;
        }

        double media = (double) acumuladorSuma / datos.length;

        System.out.println("Valor Máximo: " + maximo);
        System.out.println("Valor Mínimo: " + minimo);
        System.out.printf("Promedio: %.2f%n", media);
    }
}
```

2. Inversión In-Place de un Array

Invertir los elementos sin crear un segundo array, intercambiando los extremos hacia el centro (técnica de dos punteros):

```
int[] valores = {1, 2, 3, 4, 5, 6};

int izquierda = 0;
int derecha = valores.length - 1;

while (izquierda < derecha) {
    // Intercambio (swap) utilizando variable auxiliar temporal
    int temporal = valores[izquierda];
    valores[izquierda] = valores[derecha];
    valores[derecha] = temporal;

    izquierda++;
    derecha--;
}
// Ahora valores contiene: {6, 5, 4, 3, 2, 1}
```

3. Algoritmo de Búsqueda Lineal vs. Búsqueda Binaria

- **Búsqueda Lineal:** Recorre elemento a elemento hasta encontrar el objetivo o agotar el array. Complejidad en el peor caso: `O(n)`. Funciona en arrays desordenados.

- **Búsqueda Binaria (Dicotómica):** Divide el espacio de búsqueda a la mitad en cada paso. Complejidad en el peor caso: `O(log n)`. **Requisito obligatorio:** El array debe estar previamente ordenado de forma ascendente.

```
// Búsqueda binaria manual
public static int busquedaBinaria(int[] arrayOrdenado, int objetivo) {
    int inicio = 0;
    int fin = arrayOrdenado.length - 1;

    while (inicio ≤ fin) {
        // Evita el desbordamiento de enteros frente a (inicio + fin) / 2
        int medio = inicio + (fin - inicio) / 2;

        if (arrayOrdenado[medio] == objetivo) {
            return medio; // Encontrado, devuelve el índice
        } else if (arrayOrdenado[medio] < objetivo) {
            inicio = medio + 1; // Descartar mitad izquierda
        } else {
            fin = medio - 1;    // Descartar mitad derecha
        }
    }
    return -1; // No encontrado
}
```

4. Algoritmo de Ordenación por Burbuja (Bubble Sort)

Aunque en entornos profesionales se utilizan algoritmos optimizados como Quicksort o TimSort (`Arrays.sort()`), el algoritmo de la burbuja es fundamental en el aprendizaje académico para comprender la manipulación iterativa de índices y el intercambio (*swap*).

En cada pasada completa, el elemento más pesado (mayor) asciende hacia el final del array como una burbuja de aire en el agua:

```
public class OrdenacionBurbujaOptimizada {
    public static void main(String[] args) {
        int[] vector = {64, 34, 25, 12, 22, 11, 90};
        int n = vector.length;
        boolean huboIntercambio;

        for (int i = 0; i < n - 1; i++) {
            huboIntercambio = false;

            // En cada pasada i, los últimos i elementos ya están colocados
            for (int j = 0; j < n - 1 - i; j++) {
                if (vector[j] > vector[j + 1]) {
                    // Swap
                    int temporal = vector[j];
                    vector[j] = vector[j + 1];
                    vector[j + 1] = temporal;
                    huboIntercambio = true;
                }
            }

            // Si en una pasada completa no se hizo ningún intercambio,
            // el array ya está completamente ordenado; salimos antes.
            if (!huboIntercambio) {
                break;
            }
        }

        System.out.println(java.util.Arrays.toString(vector));
        // Salida: [11, 12, 22, 25, 34, 64, 90]
    }
}
```

4.6. La Navaja Suiza: `java.util.Arrays`

La biblioteca estándar de Java proporciona la clase de utilidad `java.util.Arrays`, equipada con métodos estáticos de alto rendimiento:

Método	Propósito	Complejidad / Comportamiento
`Arrays.toString(arr)`	Convierte el contenido del array a un String legible `"[elem1, elem2, ...]"`	Evita imprimir el identificador de referencia tipo `"[I@4f023edb"`
`Arrays.sort(arr)`	Ordena el array in-place en orden ascendente natural	Utiliza Dual-Pivot Quicksort para primitivos (`O(n log n)`)
`Arrays.binarySearch(arr, key)`	Busca un valor mediante búsqueda dicotómica	Requiere que `arr` esté ordenado. Si existe devuelve `index ≥ 0` ; si no, `-(puntoDeInsercion) - 1`
`Arrays.equals(arr1, arr2)`	Compara si dos arrays tienen la misma longitud y los mismos elementos en idéntico orden	Compara elemento a elemento en valor (no por identidad de memoria)
`Arrays.fill(arr, valor)`	Asigna el mismo valor a todas las posiciones del array	Útil para inicializar tableros con caracteres vacíos o `false`
`Arrays.copyOf(arr, nuevaLongitud)`	Clona el array con una nueva dimensión	Si es mayor, rellena con ceros/nulos. Si es menor, trunca
`Arrays.copyOfRange(arr, desde, hasta)`	Extrae un sub-array en el rango semiabierto `[desde, hasta)`	Crea una nueva instancia de array independiente
`Arrays.mismatch(arr1, arr2)`	Devuelve el primer índice donde difieren dos arrays	Devuelve `-1` si ambos son idénticos. Introducido en Java 9+

```
import java.util.Arrays;

public class UtilidadesArrays {
    public static void main(String[] args) {
        int[] base = {5, 2, 8, 1, 9};

        // 1. Representación textual
        System.out.println("Original: " + Arrays.toString(base)); // [5, 2, 8, 1, 9]

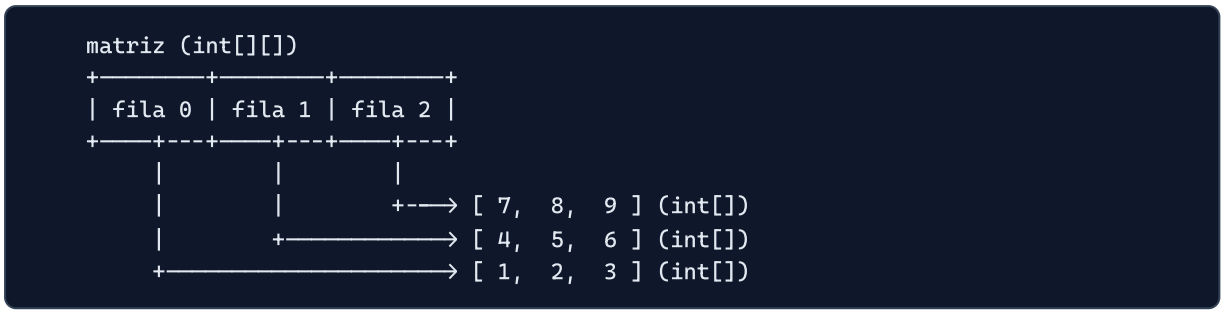
        // 2. Ordenación automática
        Arrays.sort(base);
        System.out.println("Ordenado: " + Arrays.toString(base)); // [1, 2, 5, 8, 9]

        // 3. Búsqueda binaria
        int pos = Arrays.binarySearch(base, 8);
        System.out.println("El número 8 está en el índice: " + pos); // 3

        // 4. Clonado y redimensionamiento
        int[] ampliado = Arrays.copyOf(base, 8);
        System.out.println("Ampliado: " + Arrays.toString(ampliado)); // [1, 2, 5, 8,
9, 0, 0, 0]
    }
}
```

4.7. Arrays Multidimensionales (Matrices)

Una **matriz** o array multidimensional en Java no es un bloque plano continuo de memoria en dos dimensiones, sino literalmente un **array cuyos elementos son referencias a otros arrays**.



Declaración e Inicialización de Matrices Bidimensionales

```
// Matriz de 3 filas x 4 columnas inicializada con ceros
int[][] tablero = new int[3][4];

tablero[0][0] = 5; // Primera fila, primera columna
tablero[2][3] = 9; // Última fila, última columna

// Inicialización directa mediante literales anidados
int[][] coordenadas = {
    {10, 20},
    {30, 40},
    {50, 60}
};
```

Dimensiones y Propiedad `.length`

- `tablero.length`: Devuelve el número de **filas** (arrays de primer nivel).
- `tablero[i].length`: Devuelve el número de **columnas** de la fila `i` (longitud del array interno).

Recorrido Completo con Bucles Anidados

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Recorrido indexado
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.printf("%3d", matriz[i][j]);
    }
    System.out.println(); // Salto de línea tras completar cada fila
}

// Recorrido limpio con for-each anidado
for (int[] fila : matriz) {
    for (int celda : fila) {
        System.out.print(celda + " ");
    }
    System.out.println();
}
```

Matrices Asimétricas o Dentadas (*Jagged Arrays*)

Dado que cada fila es un array independiente, no existe ninguna restricción de que todas las filas compartan la misma longitud. Una matriz con filas de distintas capacidades se denomina **array dentado**:

```
// Declarar una matriz de 3 filas sin especificar las columnas todavía
int[][] piramide = new int[3][];

// Instanciar cada fila con capacidades distintas
piramide[0] = new int[1]; // Fila 0 tiene 1 elemento
piramide[1] = new int[3]; // Fila 1 tiene 3 elementos
piramide[2] = new int[2]; // Fila 2 tiene 2 elementos

// Inicialización literal directa de matriz dentada:
String[][] menuSemanal = {
    {"Café", "Tostada"},
    {"Zumo", "Huevos", "Pancake", "Fruta"},
    {"Té"}
};
```

Métodos de Apoyo para Matrices: `Arrays.deepToString()` y `Arrays.deepEquals()`

Si aplicamos `Arrays.toString(matriz)` a una matriz bidimensional, imprimirá las referencias de memoria de cada fila (p. ej. `[[I@2a139a55, [I@15db9742]`). Para inspeccionar o comparar matrices multidimensionales completas, debemos invocar las variantes profundas (*deep*):

```
System.out.println(Arrays.deepToString(matriz));
// Imprime correctamente: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

---

5. Procesamiento Eficiente de Cadenas: Bucles y `StringBuilder`

El procesamiento textual iterativo es uno de los focos más habituales de cuellos de botella de rendimiento y consumo innecesario de memoria en aplicaciones Java.

5.1. El Peligro de Concatenar con `+` Dentro de Bucles

Recordemos que la clase `java.lang.String` en Java es **inmutable**: una vez creado un objeto `String`, sus caracteres nunca pueden modificarse. Cualquier operación de concatenación genera un objeto completamente nuevo en la memoria.

Analicemos el siguiente patrón nocivo:

```
// ¡ANTIPATRÓN DE RENDIMIENTO CRÍTICO!
String resultado = "";
for (int i = 0; i < 100_000; i++) {
    resultado += i; // En cada iteración: nuevo objeto String en el Heap + copia de
caracteres
}
```

¿Qué sucede en el Heap de la JVM?

1. En la iteración 1, se crea el String `"0"`.
2. En la iteración 2, se crea el String `"01"` copiando el contenido anterior.

3. En la iteración 10.000, se reserva memoria para 10.000 caracteres, se copian los 9.999 anteriores y se descarta el objeto previo, dejándolo como basura para el Garbage Collector.

Este bucle presenta una complejidad algorítmica cuadrática `O(n²)` en tiempo y memoria. Para 100.000 iteraciones, el programa puede tardar decenas de segundos y provocar pausas severas del recolector de basura.

5.2. `StringBuilder`: Mutabilidad y Eficiencia en Tiempo Lineal

Para manipular texto de forma dinámica y acumulativa dentro de estructuras repetitivas, Java proporciona la clase mutable `java.lang.StringBuilder`:

```
// SOLUCIÓN ÓPTIMA: O(n) lineal
StringBuilder sb = new StringBuilder(100_000); // Se recomienda preasignar capacidad
estimada
for (int i = 0; i < 100_000; i++) {
    sb.append(i); // Modifica el buffer interno in-place sin generar nuevos objetos
}
String resultadoFinal = sb.toString(); // Se convierte a String únicamente al finalizar
```

Nota: Mientras que el bucle con `+` tardaría más de 15 segundos, la solución con `StringBuilder` concluye en menos de 10 milisegundos.

Métodos Fundamentales de `StringBuilder`

Método	Descripción	Ejemplo
`append(valor)`	Concatena cualquier tipo de dato primitivo u objeto al final del buffer	`sb.append("Total: ").append(42);`
`insert(offset, valor)`	Inserta texto en una posición específica desplazando el resto a la derecha	`sb.insert(0, "INICIO: ");`
`delete(inicio, fin)`	Elimina los caracteres en el rango semiabierto `[inicio, fin)`	`sb.delete(2, 5);`
`reverse()`	Invierte la secuencia completa de caracteres in-place	`sb.reverse();`
`setCharAt(index, ch)`	Modifica el carácter en un índice concreto	`sb.setCharAt(0, 'A');`
`length()`	Devuelve la cantidad de caracteres contenidos en el buffer	`sb.length();`
`toString()`	Genera el objeto inmutable `String` final a partir del buffer acumulado	`String s = sb.toString();`

> **¿StringBuilder o StringBuffer?**
> `StringBuffer` es idéntico en API a `StringBuilder`, pero todos sus métodos están sincronizados mediante cerrojos de concurrencia (`synchronized`), haciéndolo seguro en entornos multihilo (*thread-safe*), a costa de una penalización de rendimiento. En programación modular y algoritmos estándar monotarea, **usa siempre `StringBuilder`**.

5.3. Recorrido de Cadenas Carácter a Carácter

Para examinar o transformar una cadena carácter a carácter disponemos de dos estrategias principales:

```
String texto = "Programacion DAM 2026";

// Estrategia 1: Indexación mediante charAt()
for (int i = 0; i < texto.length(); i++) {
    char c = texto.charAt(i);
    if (Character.isDigit(c)) {
        System.out.println("Dígito detectado en posición " + i + ": " + c);
    }
}

// Estrategia 2: Conversión a vector de caracteres toCharArray() para bucle for-each
int vocales = 0;
for (char c : texto.toLowerCase().toCharArray()) {
    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
        vocales++;
    }
}
System.out.println("Total de vocales: " + vocales);
```

---

6. Programación Modular: Métodos y Funciones en Java

A medida que los problemas crecen en envergadura, escribir todo el flujo dentro del método `main` genera código espagueti monolítico, imposible de mantener, depurar o testear.

La **programación modular** se basa en el principio clásico de *divide y vencerás*: descomponer un sistema complejo en subprogramas más pequeños e independientes denominados **métodos** (o funciones/procedimientos).

6.1. Anatomía y Firma de un Método

Un método en Java se declara con la siguiente estructura sintáctica:

[modificadores] tipoRetorno nombreMetodo([parámetros]) [throws Excepciones]

```
// [modificadores] tipoRetorno nombreMetodo([parámetros]) [throws Excepciones]
public static double calcularPrecioFinal(double baseImponible, double tipoIva) {
    if (baseImponible < 0 || tipoIva < 0) {
        throw new IllegalArgumentException("Los importes no pueden ser negativos.");
    }
    double cuotaIva = baseImponible * (tipoIva / 100.0);
    return baseImponible + cuotaIva;
}
```

Desglose de componentes:

1. **Modificadores:** Determinan la visibilidad y el comportamiento. En esta unidad utilizaremos `public static` (métodos de clase accesibles globalmente que pueden invocarse directamente desde el `main` sin instanciar objetos con `new`).
2. **Tipo de Retorno:** Especifica el tipo de dato que el método devuelve a quien lo invocó (`int`, `double`, `String`, `int[]`, etc.). Si el método no devuelve ningún valor, su tipo de retorno es `void`.

3. **Nombre del Método:** Debe seguir la convención `lowerCamelCase` y comenzar con un verbo imperativo o descriptivo de la acción (`calcularTotal` , `esNumeroPrimo` , `imprimirReporte`).
4. **Parámetros Formales:** Lista separada por comas de variables receptoras con su tipo correspondiente.
5. **Sentencia `return` :** Transfiere el control de vuelta a la llamada, entregando el valor computado. Si el tipo de retorno es `void` , la palabra reservada `return;` es opcional y sirve para salir prematuramente del método.
6. **Firma del Método (*Method Signature*):** En Java, la firma está compuesta **exclusivamente por el nombre del método y la lista ordenada de los tipos de sus parámetros**. *(El tipo de retorno NO forma parte de la firma)*.

6.2. El Gran Principio de Java: Paso de Parámetros Estrictamente POR VALOR

Uno de los errores conceptuales más extendidos entre programadores que inician en Java consiste en creer que los objetos o arrays se pasan «por referencia».

> **Regla de Oro Inquebrantable de la JVM**
>
> **Java es estrictamente PASO POR VALOR (*Pass by Value*) en todos y cada uno de los casos, sin ninguna excepción.**

La confusión radica en comprender qué es exactamente el «valor» que se copia en la llamada:

1. Paso de Tipos Primitivos (`byte` , `short` , `int` , `long` , `float` , `double` , `boolean` , `char`)

El valor copiado es el **dato binario numérico literal**. El método receptor recibe un duplicado en su propio marco de pila (*stack frame*). Cualquier modificación sobre el parámetro local no tiene ningún efecto sobre la variable original del invocador:

```
public static void intentarIncrementar(int numero) {
    numero = numero + 10; // Modifica la copia local de la pila
}

public static void main(String[] args) {
    int x = 5;
    intentarIncrementar(x);
    System.out.println(x); // Imprime 5 (x no se vio afectada)
}
```

2. Paso de Tipos por Referencia (Arrays y Objetos)

El valor copiado es **la dirección de memoria (el puntero de referencia) que apunta al objeto en el Heap**.

- Como ambas variables (la del método invocador y la del método receptor) poseen una copia de la misma dirección de memoria, **las modificaciones en el estado interno del objeto o en las celdas del array SÍ alteran el objeto original en el Heap**.
- Sin embargo, si dentro del método reasignas la variable parámetro haciéndola apuntar a una nueva dirección (`parametro = new int[ ... ]`), **esa reasignación NO afecta a la referencia original del invocador**.

```
public class DemostracionPasoPorValor {
    public static void modificarContenidoArray(int[] vector) {
        vector[0] = 999; // ¡Muta el contenido del array en el Heap!
    }

    public static void intentarReasignarArray(int[] vector) {
        vector = new int[]{100, 200, 300}; // Reasigna la copia local de la referencia
    }

    public static void main(String[] args) {
        int[] misNumeros = {1, 2, 3};

        modificarContenidoArray(misNumeros);
        System.out.println(misNumeros[0]); // Imprime 999 (el contenido cambió)

        intentarReasignarArray(misNumeros);
        System.out.println(misNumeros[0]); // Sigue imprimiendo 999 (la referencia NO
cambió)
    }
}
```

6.3. Sobrecarga de Métodos (*Overloading*)

La **sobrecarga** consiste en definir dos o más métodos en la misma clase con el **mismo nombre**, pero con **distinta firma** (diferente número de parámetros o diferentes tipos de parámetros).

Permite que una operación conceptualmente única se adapte a distintos conjuntos de datos sin inventar nombres forzados (`sumar2Enteros` , `sumar3Enteros` , `sumarDoubles`):

```
public class CalculadoraMatematica {

    // Sobrecarga 1: dos enteros
    public static int sumar(int a, int b) {
        return a + b;
    }

    // Sobrecarga 2: tres enteros
    public static int sumar(int a, int b, int c) {
        return a + b + c;
    }

    // Sobrecarga 3: dos números reales
    public static double sumar(double a, double b) {
        return a + b;
    }
}
```

> **El Tipo de Retorno No Distingue Sobrecargas**
> No es posible sobrecargar un método cambiando únicamente el tipo de retorno. Si intentas declarar `int procesar()` y `double procesar()` en la misma clase, el compilador emitirá un error indicando que el método ya está definido (`method already defined in class`).

6.4. Parámetros de Longitud Variable (*Varargs*)

Cuando un método necesita recibir una cantidad indeterminada de argumentos del mismo tipo (por ejemplo, calcular la media de 2, 5 o 20 números), Java dispone de los **varargs** (*variable arguments*), utilizando la sintaxis de puntos suspensivos `...`:

```
public class Estadisticas {
    // Internamente, Java empaqueta 'valores' en un array unidimensional int[]
    public static int calcularMaximo(int primerValor, int... restoValores) {
        int max = primerValor;
        for (int v : restoValores) {
            if (v > max) {
                max = v;
            }
        }
        return max;
    }

    public static void main(String[] args) {
        // Se puede invocar con cualquier número de argumentos:
        System.out.println(calcularMaximo(10));             // max: 10
        System.out.println(calcularMaximo(10, 45, 23, 89)); // max: 89
    }
}
```

Reglas sintácticas de Varargs:

1. Solo puede haber **un único** parámetro varargs en la lista de parámetros.
2. El parámetro varargs debe ser obligatoriamente el **último parámetro** de la declaración del método.

6.5. Ámbito (*Scope*) y Tiempo de Vida de las Variables

El ciclo de vida y la visibilidad de una variable están delimitados por el bloque de llaves `{ }` en el que ha sido declarada:

1. **Variables Locales:** Se crean en el Stack al entrar en el bloque donde residen y se destruyen instantáneamente al salir de él. No se inicializan automáticamente: intentar leer una variable local no inicializada produce un error de compilación.
2. **Parámetros del Método:** Variables locales inicializadas en el momento de la llamada con los valores suministrados por el invocador.
3. **Variables de Clase / Estáticas:** Declaradas a nivel de clase con `static`; permanecen en memoria durante toda la ejecución de la aplicación.
4. **Sombreado (*Shadowing*):** Si declaras una variable dentro de un bucle o bloque con el mismo nombre que una variable del método contenedor, Java emitirá un error de compilación por variable duplicada (a diferencia de lenguajes como C++ o JavaScript).

6.6. Recursividad: Concepto, Call Stack y Caso Base

La **recursividad** es una técnica de diseño algorítmico en la cual un método **se invoca a sí mismo** para resolver una instancia más reducida del mismo problema, hasta alcanzar un estado elemental no recursivo.

Todo método recursivo bien diseñado debe contener obligatoriamente dos partes:

1. **Caso Base (Condición de Parada):** Es el escenario trivial cuya solución se conoce de antemano sin realizar nuevas llamadas recursivas. Sin caso base, el método entraría en recursión infinita.
2. **Caso Recursivo:** Descompone el problema en un subproblema estrictamente más pequeño, garantizando el avance convergente hacia el caso base.

Anatomía de la Pila de Llamadas (*Call Stack*) con el Factorial

El factorial de un número entero positivo `n!` se define formalmente como:

- `0! = 1` y `1! = 1` (Casos Base).
- `n! = n × (n − 1)!` para `n > 1` (Caso Recursivo).

```
public class FactorialRecursivo {
    public static long factorial(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("El factorial no está definido para
negativos.");
        }
        // 1. CASO BASE
        if (n ≤ 1) {
            return 1;
        }
        // 2. CASO RECURSIVO
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("Factorial de 4: " + factorial(4)); // 24
    }
}
```

Evolución del Call Stack para `factorial(4)`:

```
FASE DE DESCENSO (Apilado de marcos en el Call Stack):
[ factorial(1) ] → Retorna 1 (Alcanza el Caso Base)
[ factorial(2) ] → Espera: 2 * factorial(1)
[ factorial(3) ] → Espera: 3 * factorial(2)
[ factorial(4) ] → Espera: 4 * factorial(3)
[ main()       ] → Invocador inicial

FASE DE ASCENSO (Desapilado y resolución de multiplicaciones):
factorial(1) retorna 1
factorial(2) calcula 2 * 1 = 2 y retorna 2
factorial(3) calcula 3 * 2 = 6 y retorna 6
factorial(4) calcula 4 * 6 = 24 y retorna 24 a main()
```

El Desbordamiento de Pila (`StackOverflowError`)

La memoria Stack de la JVM es limitada (típicamente entre 1 MB y 2 MB por hilo). Si un método recursivo omite el caso base o la profundidad de las llamadas es excesiva, la JVM agota el espacio de la pila y lanza un error fatal irrecuperable: `java.lang.StackOverflowError`.

> **Recursividad vs. Iteración**
> Todo algoritmo recursivo puede transformarse en una versión iterativa equivalente utilizando bucles y estructuras de datos explícitas. La recursividad proporciona soluciones elegantes y directas para estructuras arborescentes, fractales y algoritmos de retroceso (*backtracking*), pero la iteración suele ser más eficiente en memoria y velocidad.

6.7. Robustez Modular: Precondiciones y Aserciones (`assert`)

Un método modular debe protegerse frente a entradas corruptas antes de ejecutar su lógica de negocio:

1. Validación de Argumentos Defensiva

Utiliza excepciones estándar de la biblioteca Java:

- `IllegalArgumentException`: Si un argumento contiene un valor inapropiado (números negativos, cadenas vacías).
- `NullPointerException` u `Objects.requireNonNull(obj, "El parámetro no puede ser nulo")`: Si se pasa una referencia nula.

```
import java.util.Objects;

public static double calcularRaizCuadrada(Double valor) {
    Objects.requireNonNull(valor, "El número a calcular no puede ser nulo.");
    if (valor < 0) {
        throw new IllegalArgumentException("No es posible calcular la raíz de un número
negativo: " + valor);
    }
    return Math.sqrt(valor);
}
```

2. Aserciones en Java (`assert`)

Las aserciones se utilizan durante el desarrollo y depuración para comprobar invariantes internas del código que el programador asume como verdaderas en todo momento:

```
// Sintaxis: assert <condicionBooleana> : "Mensaje descriptivo";
assert saldoActual ≥ 0 : "Invariante rota: el saldo nunca puede ser negativo tras la
operación";
```

Por defecto en producción las aserciones están **desactivadas** por razones de rendimiento. Para habilitarlas en tiempo de ejecución se debe añadir el parámetro `-ea` (*enable assertions*) en las opciones de la máquina virtual (en IntelliJ IDEA: *Run → Edit Configurations → VM Options* → escribir `-ea`).

7. Expresiones Regulares (Regex) en Java

Una **expresión regular** (*regex*) es una secuencia de caracteres que conforma un patrón de búsqueda formal. En el desarrollo de software profesional, las expresiones regulares son indispensables para dos tareas críticas:

1. **Validación de entradas:** Comprobar que un dato introducido por el usuario cumple un formato estricto (DNI, correo electrónico, código postal, contraseña segura).
2. **Extracción y limpieza de texto:** Aislar subcadenas específicas o formatear logs y ficheros de texto.

7.1. Metacaracteres y la Regla del Doble Escape en Java

En Java, las cadenas literales interpretan la barra invertida `\` como carácter de escape (`\n` es salto de línea, `\t` es tabulador). Por tanto, para enviar un metacarácter de expresión regular al motor de Regex, **debemos escapar la barra invertida dos veces (`\\`)**:

Metacarácter	Significado	En Java (`String`)
`.`	Cualquier carácter individual (excepto salto de línea).	`"."`
`\d`	Cualquier dígito numérico `[0-9]`.	`"\\d"`
`\D`	Cualquier carácter que **NO** sea dígito `[^0-9]`.	`"\\D"`
`\w`	Carácter alfanumérico (letras, dígitos y guion bajo `[a-zA-Z_0-9]`).	`"\\w"`
`\W`	Cualquier carácter que **NO** sea alfanumérico.	`"\\W"`
`\s`	Espacio en blanco (espacio, tabulador, retorno de carro).	`"\\s"`
`\S`	Cualquier carácter que **NO** sea espacio en blanco.	`"\\S"`
`^`	Coincidencia al inicio exacto de la cadena o línea.	`"^"`
`$`	Coincidencia al final exacto de la cadena o línea.	`"$"`

7.2. Cuantificadores, Conjuntos y Grupos

Los **cuantificadores** especifican cuántas veces consecutivas debe repetirse el elemento precedente:

Cuantificador	Significado	Ejemplo
`?`	Cero o una vez (opcional).	`"https?"` coincide con `http` y `https`.
`*`	Cero o más veces.	`"\\d*"` coincide con cadena vacía, `"5"`, `"123"`.
`+`	Una o más veces (al menos una aparición obligatoria).	`"\\d+"` requiere al menos un número.
`{n}`	Exactamente `n` veces consecutivas.	`"\\d{4}"` exactamente 4 dígitos (p. ej. un año).
`{n,}`	Al menos `n` veces.	`"\\w{8,}"` al menos 8 caracteres alfanuméricos.
`{n,m}`	Entre `n` y `m` veces (ambos inclusive).	`"\\d{1,3}"` entre 1 y 3 dígitos.

Conjuntos y Rangos:

- `[aeiou]`: Coincide con cualquiera de las vocales minúsculas.
- `[a-zA-Z]`: Coincide con cualquier letra del abecedario en mayúscula o minúscula.
- `[^0-9]`: Negación (cualquier carácter excepto dígitos).
- `(rojo|verde|azul)`: Alternancia (coincide con cualquiera de las opciones completas).

7.3. Operaciones Nativas en `String`: `matches()`, `replaceAll()` y `split()`

La clase `String` incorpora métodos de conveniencia para interactuar con expresiones regulares de forma inmediata:

```
// 1. matches(regex): Comprueba si TODA la cadena encaja exactamente con el patrón
boolean esCodigoPostal = "46001".matches("\\d{5}"); // true

// 2. replaceAll(regex, sustituto): Reemplaza todas las coincidencias del patrón
String textoLimpio = "Usuario: 1234, Clave: 5678".replaceAll("\\d+", "XXXX");
// textoLimpio → "Usuario: XXXX, Clave: XXXX"

// 3. split(regex): Trocea la cadena según el delimitador especificado por el patrón
String lineaCSV = "Juan;DAM,Valencia;25";
String[] campos = lineaCSV.split("[;,]"); // Divide por punto y coma O por coma
// campos → ["Juan", "DAM", "Valencia", "25"]
```

7.4. La API de Alto Rendimiento: `Pattern` y `Matcher`

Cuando se ejecutan comprobaciones de expresiones regulares en bucles masivos, invocar repetidamente `string.matches(regex)` es ineficiente, pues Java recompila internamente el patrón de texto en cada iteración.

Para optimizar el rendimiento y extraer datos complejos, utilizamos las clases de `java.util.regex`:

1. `Pattern`: Representa el patrón precompilado e inmutable en memoria.
2. `Matcher`: El motor de coincidencia que escanea un texto contra el `Pattern`.

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class ExtraccionPatrones {
    // Patrón precompilado como constante estática (se compila una sola vez en toda la
aplicación)
    private static final Pattern PATRON_PRECIO = Pattern.compile("(\\d+)€");

    public static void main(String[] args) {
        String catalogo = "Teclado: 25€, Ratón: 15€, Monitor: 180€";
        Matcher matcher = PATRON_PRECIO.matcher(catalogo);

        int sumaPrecios = 0;
        // matcher.find() avanza buscando la siguiente coincidencia en el texto
        while (matcher.find()) {
            // group(1) extrae el contenido del primer grupo entre paréntesis (\\d+)
            int precio = Integer.parseInt(matcher.group(1));
            sumaPrecios += precio;
            System.out.println("Artículo detectado con precio: " + precio + "€");
        }
        System.out.println("Coste total del catálogo: " + sumaPrecios + "€");
    }
}
```

7.5. Casos Prácticos Esenciales en el Entorno DAM

A continuación se detallan los patrones de validación más frecuentes en aplicaciones empresariales y académicas:

1. Validación de DNI Español (8 dígitos y letra de control)

```
public static boolean validarFormatoDNI(String dni) {
    if (dni == null) return false;
    // Expresión: 8 dígitos exactos seguidos de una letra (mayúscula o minúscula)
    String regexDNI = "^\\d{8}[A-Za-z]$";
    if (!dni.matches(regexDNI)) {
        return false;
    }

    // Comprobación algorítmica de la letra oficial
    String letrasValidas = "TRWAGMYFPDXBNJZSQVHLCKE";
    int numero = Integer.parseInt(dni.substring(0, 8));
    char letraEsperada = letrasValidas.charAt(numero % 23);
    char letraReal = Character.toUpperCase(dni.charAt(8));

    return letraReal == letraEsperada;
}
```

2. Validación de Correo Electrónico Profesional

```
public static boolean validarEmail(String email) {
    if (email == null) return false;
    // Caracteres alfanuméricos y puntos/guiones antes del @, dominio y extensión de 2
a 6 letras
    String regexEmail = "^[\\w·−]+@[\\w·−]+\\.[a-zA-Z]{2,6}$";
    return email.matches(regexEmail);
}
```

3. Validación de Matrícula Española Moderna (4 números y 3 letras sin vocales)

```
public static boolean validarMatricula(String matricula) {
    if (matricula == null) return false;
    // En España las matrículas modernas no usan vocales ni las consonantes Ñ y Q para
evitar confusiones
    String regexMatricula = "^\\d{4}[BCDFGHJKLMNPRSTVWXYZ]{3}$";
    return matricula.toUpperCase().matches(regexMatricula);
}
```

---

8. Depuración Avanzada con IntelliJ IDEA

El desarrollo de software profesional no consiste en adivinar qué ocurre mediante impresiones constantes de `System.out.println()`. IntelliJ IDEA incorpora un depurador gráfico de última generación que permite congelar el tiempo de ejecución, inspeccionar la memoria y descubrir defectos lógicos al instante.

8.1. Breakpoints Condicionales: Detener Bucles en el Momento Exacto

Imagina un bucle que procesa 10.000 iteraciones y falla únicamente en la número 8.432. Si colocas un punto de interrupción normal, tendrías que pulsar «Continuar» más de 8.000 veces.

Cómo activar un Breakpoint Condicional en IntelliJ:

1. Haz clic con el botón izquierdo en el margen izquierdo (junto al número de línea) para colocar un punto rojo (*breakpoint*).

2. Haz **clic derecho** sobre el punto rojo.
3. En la casilla **Condition**, escribe la expresión booleana en Java que debe cumplirse para que el programa se congele (por ejemplo, `i == 8432` o `calificaciones[i] < 0`).
4. Pulsa *Done*. Inicia la ejecución con el botón de depuración (el icono del insecto verde o pulsando `Shift + F9`). El programa correrá a máxima velocidad y solo se detendrá cuando la condición sea verdadera.

```
      15 |   for (int i = 0; i < datos.length; i++) {
   ● → 16 |       procesarDato(datos[i]);  ⟵ [Condición: i == 8432]
      17 |   }
```

8.2. El Panel de Depuración: Pila de Llamadas, Variables y Expresiones en Caliente

Cuando el depurador detiene el programa, se abre la consola inferior dividida en áreas clave:

1. **Panel Frames (Call Stack):** Muestra la cadena de llamadas activa. En métodos recursivos, verás apilados todos los marcos de llamada pendientes (`factorial(1)`, `factorial(2)`, etc.). Al hacer clic sobre cualquier marco anterior, el editor salta al estado exacto del código en ese punto.
2. **Panel Variables:** Muestra el valor en vivo de cada variable local, parámetros y referencias a arrays en el Heap.
   - En arrays y matrices, al desplegar la flecha se visualiza cada celda indexada `[0], [1], [2]...` con sus valores actuales.
3. **Evaluate Expression (`Alt + F8`):** Abre un diálogo interactivo donde puedes escribir cualquier expresión en Java, consultar el estado de objetos o invocar métodos sobre las variables en vivo sin alterar el código fuente.

8.3. Atajos de Teclado Universales para la Navegación en IntelliJ

Acción	Atajo de Teclado	Descripción Funcional
Debug Program	`Shift + F9`	Compila y lanza la aplicación conectada al motor de depuración.
Step Over	`F8`	Ejecuta la línea actual por completo y avanza a la siguiente sin entrar al interior de los métodos invocados.
Step Into	`F7`	Si la línea contiene una llamada a un método propio, se adentra en su primera línea para depurarlo paso a paso.
Step Out	`Shift + F8`	Ejecuta de golpe el resto del método actual y se detiene en la línea donde fue invocado.
Resume Program	`F9`	Reanuda la ejecución continua hasta toparse con el siguiente breakpoint.
Stop Debugger	`Ctrl + F2`	Aborta y finaliza de inmediato la sesión de ejecución.

---

9. ⭐ Be the Code: Desafíos de Traza Mental

Un programador competente es capaz de ejecutar mentalmente el código como si fuera la propia Máquina Virtual de Java. Analiza minuciosamente los siguientes tres retos sin utilizar el ordenador y determina la salida exacta.

Desafío 1: Bucles Anidados con Etiquetas y Saltos de Flujo

¿Cuál es la salida exacta producida por el siguiente código en la consola?

```
public class RetoTraza1 {
    public static void main(String[] args) {
        int contador = 0;

        bucleExterno:
        for (int i = 1; i ≤ 3; i++) {
            for (int j = 1; j ≤ 3; j++) {
                if (i == 2 && j == 2) {
                    continue bucleExterno;
                }
                if (i == 3 && j == 2) {
                    break bucleExterno;
                }
                contador += (i * j);
                System.out.printf("[%d,%d] ", i, j);
            }
        }
        System.out.println("→ Total: " + contador);
    }
}
```

▶ 🔍 Ver Resolución y Traza Paso a Paso

Desafío 2: Mutación de Arrays vs. Reasignación de Punteros

¿Qué valores imprimirá el programa en cada una de las líneas comentadas?

```
public class RetoTraza2 {
    public static void alterar(int[] datos, int valorSimple) {
        valorSimple = 500;
        datos[0] = 88;
        datos = new int[]{10, 20, 30};
        datos[0] = 99;
    }

    public static void main(String[] args) {
        int x = 100;
        int[] lista = {1, 2, 3};

        alterar(lista, x);

        System.out.println("x = " + x);                    // Línea 1
        System.out.println("lista[0] = " + lista[0]);      // Línea 2
        System.out.println("longitud = " + lista.length); // Línea 3
    }
}
```

▶ 🔍 Ver Resolución y Traza de Memoria

Desafío 3: Traza del Call Stack en Recursividad No Lineal

Analiza el siguiente método recursivo y deduce qué imprime exactamente en la consola:

```
public class RetoTraza3 {
    public static void misterio(int n) {
        if (n ≤ 0) {
            return; // Caso Base
        }
        System.out.print(n + " ");
        misterio(n - 1);
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        misterio(3);
        System.out.println();
    }
}
```

▶ 🔍 Ver Resolución y Traza de la Pila

---

10. ¡No Hay Preguntas Tontas!

---

1. ¿Por qué en las expresiones regulares de Java tengo que escribir `\d` con doble barra en vez de `\d` como en otros lenguajes?

En Java, las cadenas entre comillas dobles `"..."` procesan la barra invertida `\` como carácter de escape propio del compilador (por ejemplo `\n` para nueva línea o `\"` para comillas). Si escribes `"\d"`, el compilador de Java cree que intentas usar una secuencia de escape especial de cadena y dará error de compilación porque `\d` no existe en la especificación sintáctica del lenguaje Java. Para que el carácter `\` llegue literalmente al motor de expresiones regulares, debes escaparlo anteponiendo otra barra: `"\\d"`.

---

2. Si comparo dos arrays con `array1 == array2`, ¿por qué da `false` aunque tengan exactamente los mismos números?

Porque en Java el operador `==` sobre tipos de referencia compara **identidades de memoria (direcciones físicas en el Heap)**, no el contenido de las celdas. Si creas dos arrays independientes con `new int[]{1, 2}` y `new int[]{1, 2}`, residen en dos posiciones de memoria distintas; por tanto `array1 == array2` es `false`. Para comparar el contenido posición a posición debes usar `Arrays.equals(array1, array2)` (o `Arrays.deepEquals()` si son matrices multidimensionales).

---

3. Si Java pasa siempre todo por valor, ¿por qué cuando modifico una celda de un array dentro de un método se altera el array original fuera?

Porque el **«valor»** que se copia en la llamada cuando trabajas con arrays u objetos es **la dirección de memoria de la referencia**. Imagina que tienes una casa (el array en el Heap) y una llave con su dirección (la variable en el Stack). Al invocar al método, Java le entrega al método un duplicado exacto de esa llave. Ambas llaves abren la misma casa. Si el método entra y pinta las paredes (modifica `array[0]`), la casa queda pintada para todos. Pero si el método tira su duplicado y compra una casa nueva (`array = new int[...]`), tu llave original sigue abriendo la casa de siempre.

---

4. ¿Por qué en los arrays la longitud es `.length` sin paréntesis, pero en las cadenas de texto es `.length()` con paréntesis?

En un array, `length` es un **campo o atributo público constante** propio de la estructura interna del objeto array en la JVM. En cambio, en la clase `java.lang.String`, `length()` es un **método público** que consulta internamente la cantidad de caracteres almacenados en su buffer interno de bytes.

---

5. ¿Qué diferencia práctica y conceptual existe entre las palabras reservadas `throw` y `throws`?

- `throw` **(conjunción imperativa en singular):** Es una sentencia de acción directa que **lanza** una instancia concreta de excepción en tiempo de ejecución (`throw new IllegalArgumentException("Dato inválido");`).
- `throws` **(declaración en plural):** Es una cláusula en la **cabecera o firma** de un método que advierte al compilador y a otros programadores de qué excepciones comprobadas (*checked exceptions*) es susceptible de provocar ese método (`public void cargarArchivo() throws IOException`).

---

6. Si `StringBuilder` es tan rápido, ¿por qué no lo usamos siempre en lugar del operador `+`?

Para operaciones simples como concatenar dos o tres variables en una única línea (`String saludo = "Hola " + nombre + "!";`), el compilador moderno de Java (desde Java 9+) optimiza automáticamente la expresión transformándola internamente en una llamada a `StringConcatFactory` de alto rendimiento. Usar `StringBuilder` es crítico y obligatorio **cuando concatenas texto dentro de bucles o de forma acumulativa e iterativa**, donde el compilador no puede predecir el número de vueltas.

---

7. ¿Por qué un método recursivo sin caso base termina con un error fatal `StackOverflowError` en lugar de congelarse en un bucle infinito?

Un bucle iterativo infinito (`while(true)`) reutiliza siempre el mismo marco de pila y simplemente consume ciclos de CPU al 100%. Sin embargo, cada invocación a un método (incluido el recursivo) reserva un nuevo bloque de memoria física en el **Call Stack** para almacenar sus variables locales y la dirección de retorno. Como la memoria Stack es finita (habitualmente 1 MB), tras unos miles de llamadas sucesivas la pila se desborda físicamente, provocando la terminación inmediata del programa por agotamiento de memoria.

---

8. ¿Por qué el bloque `finally` se ejecuta incluso si antes coloqué una sentencia `return` dentro del `try`?

La JVM está diseñada para garantizar que las tareas de limpieza crítica (cerrar conexiones a bases de datos, liberar descriptores de ficheros o desconectar sockets de red) se efectúen sin importar cómo finalice el bloque protegido. Si la JVM encuentra un `return` en el `try`, primero evalúa la expresión de retorno, suspende momentáneamente la salida del método, **ejecuta íntegramente el bloque** `finally`, y únicamente después concluye transfiriendo el control al invocador.

---

11. Resumen de la Unidad y Conexión con la Unidad 4

En esta Unidad 3 hemos consolidado las cuatro columnas maestras del paradigma estructurado y modular en Java:

```
+-----------------------------------------------------------------+
|                PILAS DE LA PROGRAMACIÓN ESTRUCTURADA EN JAVA    |
+-----------------+-----------------+-----------------+-----------+
|   EXCEPCIONES   |     BUCLES      |     ARRAYS      |  MÉTODOS  |
| Control robusto | while, do-while,| Vectores, tablas| Modularidad,|
| de fallos con   | for, for-each,  | multidimensional,| paso por  |
| try-catch-finally| centinelas, flags,| búsqueda y    | valor puro, |
| y recursos.     | break y continue.| Arrays utility.| recursión.  |
+-----------------+-----------------+-----------------+-----------+
```

El Salto Hacia la Programación Orientada a Objetos (Unidad 4)

Hasta este momento, nuestros métodos (`public static`) y nuestros datos (`int[]` , variables independientes) vivían separados: los métodos procesaban datos que les pasábamos como parámetros externos.

En la **Unidad 4: Clases y Objetos**, daremos el gran salto conceptual hacia la **Programación Orientada a Objetos (POO)**:

- Empaquetaremos los datos (atributos) y los métodos que operan sobre ellos en entidades únicas y coherentes llamadas **Clases**.
- Aprenderemos a crear nuestros propios tipos de datos complejos mediante constructores, encapsulación (`private` / `public`) y referencias a objetos en el Heap.
- Diremos adiós al modificador `static` en los métodos de negocio para abrazar el estado y comportamiento de las instancias reales.

---

12. Relación Curricular Oficial

Esta unidad cubre exhaustivamente los contenidos y criterios de evaluación prescritos en el currículo oficial del ciclo formativo **CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)** para el módulo **0485: Programación**:

- **RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
  - *CE a, b, c, e, f, g:* Identificación de bloques de código estructurado, declaración de arrays homogéneos y multidimensionales, y gestión de excepciones estándar de la plataforma Java.
- **RA2. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.**
  - *CE a, b, c, d, e, f:* Implementación correcta de bucles `while` , `do-while` y `for` , optimización con banderas y centinelas, uso de sentencias de ruptura controlada y depuración avanzada paso a paso con IntelliJ IDEA.
- **RA3. Escribe y optimiza código estructurado aplicando los principios de la programación modular.**
  - *CE a, b, c, d, e, f:* Descomposición de problemas en métodos reutilizables, paso riguroso de parámetros por valor en Stack y Heap, sobrecarga de métodos, varargs y análisis de la pila en algoritmos recursivos.
- **RA6. Escribe programas que apliquen estructuras de almacenamiento de información en memoria.**
  - *CE a, b, c, d, e, f:* Creación y manipulación de vectores y matrices, algoritmos de ordenación y búsqueda, procesamiento de cadenas con `StringBuilder` y validación robusta con expresiones regulares.

---

13. Boletines de Ejercicios y Retos Prácticos

Para afianzar y llevar a la práctica los conceptos teóricos y técnicos desarrollados en esta unidad, dispones de una batería de 5 boletines graduados por dificultad:

Recurso	Nivel / Enfoque	Descripción
📝 **Boletín 03: Nivel Inicial**	Básico / Consolidación	12 ejercicios prácticos para asentar bucles, arrays elementales y métodos sencillos.
✅ **Boletín 03: Inicial Resuelto**	Soluciones Comentadas	Soluciones completas con código Java 25 LTS y explicaciones paso a paso.
🚀 **Boletín 03: Nivel Intermedio**	Intermedio / Aplicado	12 ejercicios avanzados con matrices, algoritmos de ordenación, regex y recursividad.
✅ **Boletín 03: Intermedio Resuelto**	Soluciones Comentadas	Soluciones completas con trazas, capturas de excepciones y buenas prácticas.
🏆 **Boletín 03: Retos Algorítmicos (Extras)**	Avanzado / Competitivo	6 problemas de alta exigencia algorítmica (CodeWars / Acepta el Reto) con pistas y análisis.

DAM 0485 · Programación · Unidad Didáctica 3

📝 Boletín 03 Inicial — Ejercicios Prácticos

Ejercicios fundamentales de bucles, excepciones básicas, arrays unidimensionales y modularidad con métodos en Java.

Bienvenido al boletín de nivel inicial de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Estos 12 ejercicios están diseñados para asentar la lógica estructurada, el uso de las estructuras iterativas (`while` , `do-while` , `for`), la gestión elemental de excepciones de entrada con `Scanner` , la manipulación de vectores y la definición de métodos estáticos.

> **Consejo de Estudio**
> Intenta resolver cada ejercicio por tu cuenta en IntelliJ IDEA antes de consultar el boletín resuelto. Analiza las posibles entradas erróneas del usuario para proteger tu programa con bloques `try-catch` .

Ejercicio 1: Tabla de Multiplicar Segura

Enunciado:
Solicita al usuario por teclado un número entero entre 1 y 10. Muestra su tabla de multiplicar completa del 1 al 10 con formato tabulado. Si el usuario introduce letras o un número fuera del rango `[1, 10]` , captura la excepción correspondiente o repite la petición hasta que el dato sea válido.
Pistas: Utiliza un bucle `while` o `do-while` , un bloque `try-catch` capturando `InputMismatchException` y limpia el buffer con `scanner.nextLine()` .

Ejercicio 2: Contador de Dígitos y Suma de Cifras

Enunciado:
Pide al usuario un número entero positivo (de tipo `long`). Empleando exclusivamente divisiones sucesivas (`/ 10`) y el operador módulo (`% 10`) dentro de un bucle `while` , calcula cuántos dígitos tiene el número y cuánto suman todas sus cifras.
Ejemplo: Para `4523` , el programa debe indicar: 4 dígitos y suma total 14.

Ejercicio 3: Menú Interactivo de Gestión de Saldo

Enunciado:
Diseña un simulador de terminal de punto de venta. El programa parte de un saldo inicial de 100.0€. Presenta un menú en consola con las siguientes opciones:

1. Consultar saldo disponible.
2. Ingresar dinero (no se admiten cantidades negativas ni nulas).
3. Retirar dinero (no se puede retirar más del saldo disponible).
4. Salir.
   Requisitos: Usa un bucle `do-while` para mantener el menú activo hasta seleccionar la opción 4.

Ejercicio 4: Dibujando Patrones con Bucles Anidados

Enunciado:
Pide al usuario un número entero `N` (altura entre 2 y 15). Empleando dos bucles `for` anidados, dibuja en consola un triángulo rectángulo de asteriscos alineado a la izquierda:

```
*
**
***
****
*****
```

Ampliación voluntaria: Dibuja el triángulo invertido hacia abajo.

Ejercicio 5: La Conjetura de Collatz

Enunciado:
Pide un número entero `n > 0`. Si `n` es par, se divide entre 2 (`n / 2`). Si `n` es impar, se multiplica por 3 y se le suma 1 (`3n + 1`). Repite la operación hasta que `n` llegue a 1. El programa debe imprimir toda la secuencia en una sola línea separada por flechas `→` y mostrar al final el número total de pasos requeridos.

Ejercicio 6: Análisis de Temperaturas en un Vector

Enunciado:
Declara un array de `double` de tamaño 7 que represente las temperaturas medias de una semana. Solicita por teclado las 7 temperaturas protegiendo la lectura de fallos. A continuación, calcula e imprime:

- La temperatura máxima registrada y el día en que ocurrió (índice 0 = Lunes, etc.).
- La temperatura mínima registrada.
- La temperatura media de la semana formateada con 2 decimales.

Ejercicio 7: Registro de Calificaciones y Estadística de Grupo

Enunciado:
Crea un programa que almacene las calificaciones (entre 0.0 y 10.0) de un grupo de 10 alumnos en un array `double[]`. Recorre el array mediante un bucle mejorado `for-each` y determina:

- Cuántos alumnos han obtenido sobresaliente (nota ≥ 9.0).
- Cuántos han aprobado (nota ≥ 5.0).
- Cuántos han suspendido (nota < 5.0).

Ejercicio 8: Inversión In-Place de un Vector

Enunciado:
Escribe un programa que cree un array con los números `{10, 20, 30, 40, 50, 60, 70}`. Sin utilizar un segundo array auxiliar, invierte los elementos directamente en memoria intercambiando los extremos con la técnica de dos punteros (`izquierda` y `derecha`). Imprime el array resultante con `Arrays.toString()`.

Ejercicio 9: Contabilizador de Vocales y Consonantes

Enunciado:
Pide una frase por consola. Conviértela a minúsculas y, recorriéndola carácter a carácter con `charAt()`, cuenta cuántas vocales (`a, e, i, o, u`), cuántas consonantes y cuántos caracteres no alfabéticos (espacios, números, puntuación) contiene la cadena.

Ejercicio 10: Métodos Sobrecargados para el Cálculo de Áreas

Enunciado:
En una clase con métodos modulares, define tres métodos sobrecargados llamados `calcularArea`:

1. `public static double calcularArea(double lado)` -> Para el área de un cuadrado.
2. `public static double calcularArea(double base, double altura)` -> Para un rectángulo.
3. `public static double calcularArea(double radio, boolean esCirculo)` -> Para el área de un círculo (`π * radio²`).
   Valida que ninguna dimensión pueda ser menor o igual a cero lanzando `IllegalArgumentException`.

Ejercicio 11: Potencia Recursiva

Enunciado:
Implementa un método recursivo con la firma `public static long calcularPotencia(int base, int exponente)`.

- Caso base: Cualquier número elevado a 0 es 1 (`exponente == 0 → 1`).
- Caso recursivo: `base * calcularPotencia(base, exponente - 1)`.
- Control de precondición: Si el exponente es negativo, lanza una `IllegalArgumentException`.

Ejercicio 12: Validador de Código Postal Español con Regex

Enunciado:
Escribe un método `public static boolean esCodigoPostalValido(String cp)` que verifique mediante una expresión regular si una cadena cumple con el formato oficial de un código postal en España: exactamente 5 dígitos numéricos, donde los dos primeros dígitos deben estar comprendidos entre 01 y 52 (las 52 provincias españolas).

> **¿Necesitas comprobar tus soluciones?**
> Puedes ver todas las soluciones completas y explicadas en el Boletín 3.1 Resuelto.

DAM 0485 · Programación · Unidad Didáctica 3

📝 Boletín 03 Inicial (Resuelto) — Soluciones Comentadas

Soluciones completas y comentadas de los 12 ejercicios iniciales de la Unidad 3 en Java 25 LTS.

A continuación se presentan las soluciones comentadas de los 12 ejercicios del boletín inicial, desarrolladas siguiendo las directrices de robustez, legibilidad y buenas prácticas en Java 25 LTS.

Solución Ejercicio 1: Tabla de Multiplicar Segura

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejercicio01TablaSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero = 0;
        boolean entradaValida = false;

        while (!entradaValida) {
            try {
                System.out.print("Introduce un número entero (1 - 10): ");
                numero = sc.nextInt();

                if (numero ≥ 1 && numero ≤ 10) {
                    entradaValida = true;
                } else {
                    System.out.println("Error: El número debe estar entre 1 y 10.");
                }
            } catch (InputMismatchException e) {
                System.out.println("Error: Debes introducir un número entero válido.");
                sc.nextLine(); // Limpiar el token no numérico del buffer
            }
        }

        System.out.println("═══ TABLA DE MULTIPLICAR DEL " + numero + " ═══");
        for (int i = 1; i ≤ 10; i++) {
            System.out.printf("%2d × %2d = %3d%n", numero, i, (numero * i));
        }
    }
}
```

Solución Ejercicio 2: Contador de Dígitos y Suma de Cifras

```
import java.util.Scanner;

public class Ejercicio02DigitosSuma {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce un número entero positivo: ");
        long numero = sc.nextLong();

        if (numero < 0) {
            numero = Math.abs(numero);
        }

        if (numero == 0) {
            System.out.println("Tiene 1 dígito y la suma de sus cifras es 0.");
            return;
        }

        long copia = numero;
        int cantidadDigitos = 0;
        long sumaCifras = 0;

        while (copia > 0) {
            long ultimaCifra = copia % 10;
            sumaCifras += ultimaCifra;
            cantidadDigitos++;
            copia /= 10; // Desplaza una cifra a la derecha
        }

        System.out.println("Número analizado: " + numero);
        System.out.println("Total de dígitos: " + cantidadDigitos);
        System.out.println("Suma de sus cifras: " + sumaCifras);
    }
}
```

Solución Ejercicio 3: Menú Interactivo de Gestión de Saldo

```
import java.util.Scanner;

public class Ejercicio03MenuSaldo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double saldo = 100.0;
        int opcion = 0;

        do {
            System.out.println("\n--- CAJERO AUTOMÁTICO ---");
            System.out.println("1. Consultar saldo");
            System.out.println("2. Ingresar dinero");
            System.out.println("3. Retirar dinero");
            System.out.println("4. Salir");
            System.out.print("Seleccione una opción: ");

            if (!sc.hasNextInt()) {
                System.out.println("Opción no válida. Introduzca un número de 1 a 4.");
                sc.nextLine();
                continue;
            }

            opcion = sc.nextInt();

            switch (opcion) {
                case 1 -> System.out.printf("Saldo actual: %.2f€%n", saldo);
                case 2 -> {
                    System.out.print("Importe a ingresar: ");
                    double ingreso = sc.nextDouble();
                    if (ingreso > 0) {
                        saldo += ingreso;
                        System.out.printf("Ingreso realizado. Nuevo saldo: %.2f€%n",
saldo);
                    } else {
                        System.out.println("Error: El importe a ingresar debe ser
estrictamente positivo.");
                    }
                }
                case 3 -> {
                    System.out.print("Importe a retirar: ");
                    double retirada = sc.nextDouble();
                    if (retirada ≤ 0) {
                        System.out.println("Error: El importe a retirar debe ser mayor
que 0.");
                    } else if (retirada > saldo) {
                        System.out.println("Error: Saldo insuficiente. Saldo
disponible: " + saldo + "€");
                    } else {
                        saldo -= retirada;
                        System.out.printf("Retirada completada. Saldo restante: %.2f€
%n", saldo);
                    }
                }
                case 4 -> System.out.println("Gracias por utilizar nuestros servicios.
¡Hasta pronto!");
                default -> System.out.println("Opción desconocida. Elija entre 1 y
4.");
            }
        } while (opcion ≠ 4);
    }
}
```

Solución Ejercicio 4: Dibujando Patrones con Bucles Anidados

```
import java.util.Scanner;

public class Ejercicio04TrianguloAsteriscos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce la altura del triángulo (2 - 15): ");
        int altura = sc.nextInt();

        if (altura < 2 || altura > 15) {
            System.out.println("Altura fuera de límites. Ajustando a 5 por defecto.");
            altura = 5;
        }

        System.out.println("\nTriángulo normal:");
        for (int fila = 1; fila ≤ altura; fila++) {
            for (int col = 1; col ≤ fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nTriángulo invertido:");
        for (int fila = altura; fila ≥ 1; fila--) {
            for (int col = 1; col ≤ fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Solución Ejercicio 5: La Conjetura de Collatz

```
import java.util.Scanner;

public class Ejercicio05Collatz {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce un número entero positivo: ");
        long n = sc.nextLong();

        if (n ≤ 0) {
            System.out.println("El número debe ser estrictamente positivo.");
            return;
        }

        int pasos = 0;
        System.out.print("Secuencia: " + n);

        while (n ≠ 1) {
            if (n % 2 == 0) {
                n = n / 2;
            } else {
                n = (3 * n) + 1;
            }
            System.out.print(" → " + n);
            pasos++;
        }

        System.out.println();
        System.out.println("Proceso concluido en " + pasos + " pasos.");
    }
}
```

Solución Ejercicio 6: Análisis de Temperaturas en un Vector

```
import java.util.Scanner;

public class Ejercicio06AnalisisTemperaturas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"};
        double[] temp = new double[7];

        for (int i = 0; i < temp.length; i++) {
            System.out.print("Temperatura del " + dias[i] + ": ");
            temp[i] = sc.nextDouble();
        }

        double max = temp[0];
        double min = temp[0];
        int diaMax = 0;
        double suma = 0;

        for (int i = 0; i < temp.length; i++) {
            if (temp[i] > max) {
                max = temp[i];
                diaMax = i;
            }
            if (temp[i] < min) {
                min = temp[i];
            }
            suma += temp[i];
        }

        double media = suma / temp.length;

        System.out.println("═══ RESUMEN CLIMATOLÓGICO ═══");
        System.out.printf("Temperatura Máxima: %.1f°C (registrada el %s)%n", max,
dias[diaMax]);
        System.out.printf("Temperatura Mínima: %.1f°C%n", min);
        System.out.printf("Temperatura Media:  %.2f°C%n", media);
    }
}
```

Solución Ejercicio 7: Registro de Calificaciones con `for-each`

```
public class Ejercicio07Calificaciones {
    public static void main(String[] args) {
        double[] notas = {4.5, 7.8, 9.2, 3.0, 5.0, 8.4, 6.2, 9.6, 4.9, 10.0};

        int sobresalientes = 0;
        int aprobados = 0;
        int suspensos = 0;

        for (double nota : notas) {
            if (nota ≥ 9.0) {
                sobresalientes++;
            }
            if (nota ≥ 5.0) {
                aprobados++;
            } else {
                suspensos++;
            }
        }

        System.out.println("Total alumnos analizados: " + notas.length);
        System.out.println("Aprobados: " + aprobados);
        System.out.println("Suspensos: " + suspensos);
        System.out.println("Sobresalientes (≥ 9.0): " + sobresalientes);
    }
}
```

Solución Ejercicio 8: Inversión In-Place de un Vector

```
import java.util.Arrays;

public class Ejercicio08InversionVector {
    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50, 60, 70};

        System.out.println("Vector original: " + Arrays.toString(datos));

        int izquierda = 0;
        int derecha = datos.length - 1;

        while (izquierda < derecha) {
            // Intercambio sin crear un segundo vector
            int aux = datos[izquierda];
            datos[izquierda] = datos[derecha];
            datos[derecha] = aux;

            izquierda++;
            derecha--;
        }

        System.out.println("Vector invertido in-place: " + Arrays.toString(datos));
    }
}
```

Solución Ejercicio 9: Contabilizador de Vocales y Consonantes

```
import java.util.Scanner;

public class Ejercicio09VocalesConsonantes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce una frase: ");
        String frase = sc.nextLine().toLowerCase();

        int vocales = 0;
        int consonantes = 0;
        int otros = 0;

        for (int i = 0; i < frase.length(); i++) {
            char c = frase.charAt(i);

            if (c ≥ 'a' && c ≤ 'z') {
                if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                    vocales++;
                } else {
                    consonantes++;
                }
            } else {
                otros++;
            }
        }

        System.out.println("Vocales: " + vocales);
        System.out.println("Consonantes: " + consonantes);
        System.out.println("Espacios y otros símbolos: " + otros);
    }
}
```

Solución Ejercicio 10: Métodos Sobrecargados para Áreas

```
public class Ejercicio10AreasSobrecarga {

    // Cuadrado
    public static double calcularArea(double lado) {
        if (lado ≤ 0) {
            throw new IllegalArgumentException("El lado debe ser estrictamente
positivo.");
        }
        return lado * lado;
    }

    // Rectángulo
    public static double calcularArea(double base, double altura) {
        if (base ≤ 0 || altura ≤ 0) {
            throw new IllegalArgumentException("La base y la altura deben ser
positivas.");
        }
        return base * altura;
    }

    // Círculo
    public static double calcularArea(double radio, boolean esCirculo) {
        if (radio ≤ 0) {
            throw new IllegalArgumentException("El radio debe ser positivo.");
        }
        return Math.PI * Math.pow(radio, 2);
    }

    public static void main(String[] args) {
        System.out.printf("Área de cuadrado (lado 4): %.2f%n", calcularArea(4.0));
        System.out.printf("Área de rectángulo (base 5, altura 3): %.2f%n",
calcularArea(5.0, 3.0));
        System.out.printf("Área de círculo (radio 2.5): %.2f%n", calcularArea(2.5,
true));
    }
}
```

Solución Ejercicio 11: Potencia Recursiva

```
public class Ejercicio11PotenciaRecursiva {

    public static long calcularPotencia(int base, int exponente) {
        if (exponente < 0) {
            throw new IllegalArgumentException("Exponente negativo no soportado para
enteros.");
        }
        // Caso Base
        if (exponente == 0) {
            return 1;
        }
        // Caso Recursivo
        return base * calcularPotencia(base, exponente - 1);
    }

    public static void main(String[] args) {
        System.out.println("2^0 = " + calcularPotencia(2, 0));
        System.out.println("2^5 = " + calcularPotencia(2, 5));
        System.out.println("3^4 = " + calcularPotencia(3, 4));
        System.out.println("5^3 = " + calcularPotencia(5, 3));
    }
}
```

Solución Ejercicio 12: Validador de Código Postal Español con Regex

```
public class Ejercicio12ValidadorCP {

    public static boolean esCodigoPostalValido(String cp) {
        if (cp == null) {
            return false;
        }

        // Exactamente 5 dígitos
        if (!cp.matches("^\\d{5}$")) {
            return false;
        }

        // Las dos primeras cifras corresponden a la provincia (01 a 52)
        int provincia = Integer.parseInt(cp.substring(0, 2));
        return provincia ≥ 1 && provincia ≤ 52;
    }

    public static void main(String[] args) {
        String[] pruebas = {"46001", "28013", "08001", "53001", "1234", "ABCDE",
"00123"};

        for (String test : pruebas) {
            System.out.printf("Código '%s' → ¿Válido? %b%n", test,
esCodigoPostalValido(test));
        }
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 3

📝 Boletín 03 Intermedio — Ejercicios Prácticos

Ejercicios avanzados de matrices 2D, algoritmos de ordenación y búsqueda, manipulación con StringBuilder, recursividad y expresiones regulares en Java.

Bienvenido al boletín de nivel intermedio de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Este bloque de 12 ejercicios está orientado a afianzar el pensamiento algorítmico, el trabajo con matrices multidimensionales (incluyendo matrices dentadas), la optimización de texto con `StringBuilder`, el diseño de métodos recursivos y la aplicación de expresiones regulares para validación y saneamiento de datos.

> **Enfoque Profesional**
> Modulariza tu código: descompón cada ejercicio en métodos estáticos bien nombrados, valida las precondiciones con excepciones defensivas y apóyate en el depurador de IntelliJ IDEA para inspeccionar matrices y pilas recursivas.

Ejercicio 1: Matriz Transpuesta

Enunciado:
Escribe un método `public static int[][] transponer(int[][] matriz)` que reciba una matriz rectangular de dimensiones `F × C` y devuelva una nueva matriz transpuesta de dimensiones `C × F`, donde la fila `i` pasa a ser la columna `i`.
Precondición: Verifica que la matriz no sea nula ni vacía y que no sea dentada.

Ejercicio 2: Suma por Filas y Columnas en Tablero Bidimensional

Enunciado:
Dada una matriz de enteros de 4 filas y 5 columnas generada aleatoriamente con valores entre 10 y 99:

- Muestra la matriz en forma de tabla perfectamente alineada.
- Muestra al final de cada fila la suma de sus elementos.
- Muestra al pie de cada columna la suma de sus elementos.
- En la esquina inferior derecha muestra la suma total de toda la matriz.

Ejercicio 3: Verificador de Matriz Simétrica

Enunciado:
Una matriz cuadrada es simétrica si es idéntica a su matriz transpuesta (es decir, `matriz[i][j] == matriz[j][i]` para todo `i, j`). Implementa un método `public static boolean esSimetrica(int[][] matriz)` que realice esta comprobación en el menor número de comparaciones posible (recorriendo únicamente los elementos por encima de la diagonal principal).

Ejercicio 4: Algoritmo de Ordenación por Selección (*Selection Sort*)

Enunciado:
Implementa el algoritmo clásico de selección: en cada pasada `i` del array, localiza el índice del elemento mínimo en el subarray restante `[i, n - 1]` y realiza un intercambio (*swap*) con la posición `i`. Muestra el estado del array tras cada pasada para evidenciar el proceso.

Ejercicio 5: Búsqueda Binaria Recursiva

Enunciado:
Implementa la búsqueda binaria de forma recursiva con la firma:
`public static int busquedaBinariaRecursiva(int[] arr, int objetivo, int inicio, int fin)`.
Devuelve el índice si se encuentra el elemento, o `-1` si no existe. Explica con comentarios cuál es el caso base y cuál es el caso recursivo.

Ejercicio 6: Cifrado César con `StringBuilder`

Enunciado:
El cifrado César desplaza cada letra del alfabeto un número fijo de posiciones `K`. Diseña un método `public static String cifrarCesar(String texto, int desplazamiento)` que:

- Utilice `StringBuilder` para la construcción eficiente de la cadena resultante.
- Conserve mayúsculas y minúsculas respetando el ciclo del abecedario (`'Z'` pasa a `'A'`).
- Deje inalterados los espacios, números y caracteres de puntuación.

Ejercicio 7: Detector de Palíndromos Robusto

Enunciado:
Una frase es un palíndromo si se lee igual de izquierda a derecha que de derecha a izquierda, ignorando mayúsculas, tildes, signos de puntuación y espacios (por ejemplo: *"Dábale arroz a la zorra el abad"*).
Escribe un método `public static boolean esPalindromo(String frase)` que limpie la cadena utilizando expresiones regulares y compruebe la simetría mediante la técnica de dos punteros.

Ejercicio 8: Fusión Ordenada de Arrays (*Merge*)

Enunciado:
Implementa un método `public static int[] fusionar(int[] a, int[] b)` que reciba dos arrays previamente ordenados en orden ascendente (de tamaños `N` y `M`) y devuelva un nuevo array ordenado de tamaño `N + M`.
Requisito crítico: El algoritmo debe resolverse en tiempo lineal `O(N + M)` recorriendo ambos arrays simultáneamente con dos punteros, sin recurrir a `Arrays.sort()`.

Ejercicio 9: Validador Completo de DNI Español

Enunciado:
Diseña un método `public static boolean validarDniCompleto(String dni)` que:

1. Verifique con Regex que el formato conste de 8 dígitos seguidos de una letra.
2. Calcule matemáticamente la letra oficial de control correspondiente a los 8 números mediante la fórmula `letra = "TRWAGMYFPDXBNJZSQVHLCKE".charAt(numero % 23)`.

3. Compare la letra calculada con la letra facilitada por el usuario (sin distinguir mayúsculas de minúsculas).

Ejercicio 10: Triángulo de Pascal con Matriz Dentada

Enunciado:
El Triángulo de Pascal es una estructura numérica donde cada celda es la suma de los dos valores directamente situados sobre ella. Diseña un método `public static int[][] generarTrianguloPascal(int filas)` que devuelva un array bidimensional dentado (*jagged array*) donde cada fila `i` tenga exactamente `i + 1` columnas y los extremos valgan 1. Muestra el triángulo en consola.

Ejercicio 11: Enmascarador de Tarjetas de Crédito con Expresiones Regulares

Enunciado:
En aplicaciones de pasarelas de pago, los números de tarjeta nunca deben guardarse ni mostrarse en claro en los logs. Diseña un método `public static String enmascararTarjetas(String texto)` que detecte secuencias de 16 dígitos (escritos juntos o separados por guiones o espacios de cuatro en cuatro) y las sustituya por `****-****-****-` seguido únicamente de los últimos 4 dígitos reales.

Ejercicio 12: Conteo Recursivo de Caminos en una Cuadrícula

Enunciado:
Imagina un robot situado en la celda superior izquierda `(0, 0)` de una cuadrícula de `N × M`. El robot solo puede moverse hacia la derecha o hacia abajo. Diseña un método recursivo `public static int contarCaminos(int n, int m)` que calcule cuántos caminos posibles existen para alcanzar la esquina inferior derecha.

- ¿Cuál es el caso base cuando `n == 1` o `m == 1`?
- ¿Cuál es la llamada recursiva que divide el problema?

> **¿Quieres revisar el código completo?**
> Consulta el Boletín 3.2 Resuelto para examinar las soluciones completas con Java 25 LTS.

DAM 0485 · Programación · Unidad Didáctica 3

📝 Boletín 03 Intermedio (Resuelto) — Soluciones Comentadas

Soluciones completas, comentadas y optimizadas de los 12 ejercicios de nivel intermedio de la Unidad 3 en Java 25 LTS.

A continuación se detallan las soluciones completas a los 12 ejercicios de nivel intermedio, haciendo hincapié en la modularidad limpia, la optimización algorítmica y el manejo de excepciones.

Solución Ejercicio 1: Matriz Transpuesta

```
import java.util.Arrays;

public class Ejercicio01Transpuesta {

    public static int[][] transponer(int[][] matriz) {
        if (matriz == null || matriz.length == 0 || matriz[0] == null) {
            throw new IllegalArgumentException("La matriz no puede ser nula ni
vacía.");
        }

        int filasOriginales = matriz.length;
        int columnasOriginales = matriz[0].length;

        // La matriz transpuesta invierte las dimensiones: C x F
        int[][] transpuesta = new int[columnasOriginales][filasOriginales];

        for (int i = 0; i < filasOriginales; i++) {
            for (int j = 0; j < columnasOriginales; j++) {
                transpuesta[j][i] = matriz[i][j];
            }
        }

        return transpuesta;
    }

    public static void main(String[] args) {
        int[][] original = {
            {1, 2, 3},
            {4, 5, 6}
        };

        int[][] resultado = transponer(original);

        System.out.println("Original: " + Arrays.deepToString(original));
        System.out.println("Transpuesta: " + Arrays.deepToString(resultado));
    }
}
```

Solución Ejercicio 2: Suma por Filas y Columnas en Tablero 2D

```
import java.util.concurrent.ThreadLocalRandom;

public class Ejercicio02SumaMatriz {
    public static void main(String[] args) {
        int filas = 4;
        int cols = 5;
        int[][] tabla = new int[filas][cols];

        // Rellenar con números aleatorios [10, 99]
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < cols; j++) {
                tabla[i][j] = ThreadLocalRandom.current().nextInt(10, 100);
            }
        }

        int[] sumaColumnas = new int[cols];
        int sumaTotal = 0;

        System.out.println("═══════════════ TABLERO ═══════════════");
        for (int i = 0; i < filas; i++) {
            int sumaFila = 0;
            for (int j = 0; j < cols; j++) {
                System.out.printf("%4d ", tabla[i][j]);
                sumaFila += tabla[i][j];
                sumaColumnas[j] += tabla[i][j];
            }
            sumaTotal += sumaFila;
            System.out.printf("| %5d%n", sumaFila);
        }

        System.out.println("─────────────────────────────────────");
        for (int j = 0; j < cols; j++) {
            System.out.printf("%4d ", sumaColumnas[j]);
        }
        System.out.printf("| %5d%n", sumaTotal);
    }
}
```

Solución Ejercicio 3: Verificador de Matriz Simétrica

```
public class Ejercicio03MatrizSimetrica {

    public static boolean esSimetrica(int[][] m) {
        if (m == null || m.length == 0) return false;

        int n = m.length;
        // Debe ser estrictamente cuadrada
        for (int[] fila : m) {
            if (fila.length ≠ n) return false;
        }

        // Solo es necesario comparar los elementos estrictamente por encima de la
diagonal
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (m[i][j] ≠ m[j][i]) {
                    return false; // Diferencia detectada: finalización anticipada
                }
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[][] simetrica = {
            {1, 7, 3},
            {7, 4, -5},
            {3, -5, 6}
        };

        int[][] asimetrica = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        System.out.println("¿Es simétrica la primera? " + esSimetrica(simetrica)); //
true
        System.out.println("¿Es simétrica la segunda? " + esSimetrica(asimetrica)); //
false
    }
}
```

Solución Ejercicio 4: Algoritmo de Ordenación por Selección

```
import java.util.Arrays;

public class Ejercicio04SelectionSort {

    public static void ordenarPorSeleccion(int[] arr) {
        if (arr == null || arr.length ≤ 1) return;

        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int indiceMinimo = i;

            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[indiceMinimo]) {
                    indiceMinimo = j;
                }
            }

            // Si el mínimo no está en la posición i actual, intercambiar
            if (indiceMinimo ≠ i) {
                int aux = arr[i];
                arr[i] = arr[indiceMinimo];
                arr[indiceMinimo] = aux;
            }

            System.out.printf("Paso %d: %s%n", (i + 1), Arrays.toString(arr));
        }
    }

    public static void main(String[] args) {
        int[] datos = {29, 10, 14, 37, 13};
        System.out.println("Array original: " + Arrays.toString(datos));
        ordenarPorSeleccion(datos);
        System.out.println("Array ordenado: " + Arrays.toString(datos));
    }
}
```

Solución Ejercicio 5: Búsqueda Binaria Recursiva

```
public class Ejercicio05BusquedaBinariaRecursiva {

    public static int busquedaBinaria(int[] arr, int objetivo, int inicio, int fin) {
        // Caso Base 1: El rango de búsqueda se ha agotado (no encontrado)
        if (inicio > fin) {
            return -1;
        }

        // Cálculo seguro de la posición media
        int medio = inicio + (fin - inicio) / 2;

        // Caso Base 2: Elemento localizado
        if (arr[medio] == objetivo) {
            return medio;
        }

        // Casos Recursivos:
        if (arr[medio] > objetivo) {
            // Buscar en la mitad izquierda
            return busquedaBinaria(arr, objetivo, inicio, medio - 1);
        } else {
            // Buscar en la mitad derecha
            return busquedaBinaria(arr, objetivo, medio + 1, fin);
        }
    }

    public static void main(String[] args) {
        int[] ordenado = {3, 8, 12, 19, 27, 34, 45, 60, 72};

        int idx1 = busquedaBinaria(ordenado, 27, 0, ordenado.length - 1);
        int idx2 = busquedaBinaria(ordenado, 99, 0, ordenado.length - 1);

        System.out.println("Posición de 27: " + idx1); // 4
        System.out.println("Posición de 99: " + idx2); // -1
    }
}
```

Solución Ejercicio 6: Cifrado César con `StringBuilder`

```
public class Ejercicio06CesarStringBuilder {

    public static String cifrarCesar(String texto, int desplazamiento) {
        if (texto == null) return null;

        StringBuilder sb = new StringBuilder(texto.length());
        int shift = desplazamiento % 26; // Normalizar a las 26 letras del alfabeto
inglés

        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);

            if (c ≥ 'A' && c ≤ 'Z') {
                char nuevo = (char) ('A' + (c - 'A' + shift + 26) % 26);
                sb.append(nuevo);
            } else if (c ≥ 'a' && c ≤ 'z') {
                char nuevo = (char) ('a' + (c - 'a' + shift + 26) % 26);
                sb.append(nuevo);
            } else {
                sb.append(c); // Símbolos, espacios y números no se alteran
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String mensaje = "¡Ataque Inminente a las 05:00!";
        String cifrado = cifrarCesar(mensaje, 3);
        String descifrado = cifrarCesar(cifrado, -3);

        System.out.println("Original:   " + mensaje);
        System.out.println("Cifrado:    " + cifrado);
        System.out.println("Descifrado: " + descifrado);
    }
}
```

Solución Ejercicio 7: Detector de Palíndromos Robusto

```
import java.text.Normalizer;

public class Ejercicio07PalindromoRobusto {

    public static boolean esPalindromo(String frase) {
        if (frase == null) return false;

        // 1. Eliminar tildes diacríticas mediante Normalizer NFD
        String normalizada = Normalizer.normalize(frase, Normalizer.Form.NFD);
        normalizada = normalizada.replaceAll("[\\p{InCombiningDiacriticalMarks}]", "");

        // 2. Pasar a minúsculas y eliminar todo lo que no sea letra alfabética
        String soloLetras = normalizada.toLowerCase().replaceAll("[^a-z]", "");

        // 3. Comprobación con dos punteros en extremos opuestos
        int izquierda = 0;
        int derecha = soloLetras.length() - 1;

        while (izquierda < derecha) {
            if (soloLetras.charAt(izquierda) ≠ soloLetras.charAt(derecha)) {
                return false;
            }
            izquierda++;
            derecha--;
        }
        return true;
    }

    public static void main(String[] args) {
        String[] pruebas = {
            "Dábale arroz a la zorra el abad",
            "Anita lava la tina",
            "A man, a plan, a canal: Panama",
            "Java no es palindromo"
        };

        for (String p : pruebas) {
            System.out.printf("'%s' → ¿Palíndromo? %b%n", p, esPalindromo(p));
        }
    }
}
```

Solución Ejercicio 8: Fusión Ordenada de Arrays (*Merge*)

```
import java.util.Arrays;

public class Ejercicio08MergeArrays {

    public static int[] fusionar(int[] a, int[] b) {
        if (a == null) a = new int[0];
        if (b == null) b = new int[0];

        int[] fusion = new int[a.length + b.length];
        int i = 0; // Puntero para a
        int j = 0; // Puntero para b
        int k = 0; // Puntero para fusion

        // Avanzar comparando el elemento menor de ambos arrays
        while (i < a.length && j < b.length) {
            if (a[i] ≤ b[j]) {
                fusion[k++] = a[i++];
            } else {
                fusion[k++] = b[j++];
            }
        }

        // Volcar elementos restantes de a (si quedan)
        while (i < a.length) {
            fusion[k++] = a[i++];
        }

        // Volcar elementos restantes de b (si quedan)
        while (j < b.length) {
            fusion[k++] = b[j++];
        }

        return fusion;
    }

    public static void main(String[] args) {
        int[] vector1 = {2, 5, 8, 12, 20};
        int[] vector2 = {1, 3, 7, 9, 15, 25, 30};

        int[] resultado = fusionar(vector1, vector2);
        System.out.println("Array 1: " + Arrays.toString(vector1));
        System.out.println("Array 2: " + Arrays.toString(vector2));
        System.out.println("Fusión Ordenada: " + Arrays.toString(resultado));
    }
}
```

Solución Ejercicio 9: Validador Completo de DNI Español

```
public class Ejercicio09ValidadorDNI {

    private static final String LETRAS_ASOCIADAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    public static boolean validarDniCompleto(String dni) {
        if (dni == null) return false;

        // Limpieza de posibles espacios en blanco en extremos
        String texto = dni.trim().toUpperCase();

        // Regex: exactamente 8 dígitos y una letra mayúscula
        if (!texto.matches("^\\d{8}[A-Z]$")) {
            return false;
        }

        try {
            int numero = Integer.parseInt(texto.substring(0, 8));
            char letraEsperada = LETRAS_ASOCIADAS.charAt(numero % 23);
            char letraReal = texto.charAt(8);

            return letraReal == letraEsperada;
        } catch (NumberFormatException e) {
            return false;
        }
    }

    public static void main(String[] args) {
        String[] casos = {"12345678Z", "00000000T", "48712394A", "12345", "ABCDEFGHZ"};

        for (String c : casos) {
            System.out.printf("DNI '%s' → ¿Válido? %b%n", c, validarDniCompleto(c));
        }
    }
}
```

Solución Ejercicio 10: Triángulo de Pascal con Matriz Dentada

```
public class Ejercicio10PascalJagged {

    public static int[][] generarTrianguloPascal(int filas) {
        if (filas ≤ 0) {
            throw new IllegalArgumentException("El número de filas debe ser
positivo.");
        }

        int[][] pascal = new int[filas][];

        for (int i = 0; i < filas; i++) {
            pascal[i] = new int[i + 1]; // Cada fila tiene i + 1 columnas
            pascal[i][0] = 1;           // Primer elemento siempre 1
            pascal[i][i] = 1;           // Último elemento siempre 1

            for (int j = 1; j < i; j++) {
                // Suma de los dos elementos 바로 superiores
                pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
            }
        }
        return pascal;
    }

    public static void main(String[] args) {
        int filas = 6;
        int[][] triangulo = generarTrianguloPascal(filas);

        System.out.println("Triángulo de Pascal (" + filas + " niveles):");
        for (int i = 0; i < triangulo.length; i++) {
            // Sangría decorativa para forma piramidal
            System.out.print(" ".repeat((filas - i - 1) * 2));
            for (int valor : triangulo[i]) {
                System.out.printf("%4d", valor);
            }
            System.out.println();
        }
    }
}
```

Solución Ejercicio 11: Enmascarador de Tarjetas con Regex

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Ejercicio11EnmascaradorTarjetas {

    // Coincide con 16 dígitos separados o no por espacios o guiones
    private static final Pattern PATRON_TARJETA =
        Pattern.compile("\\b(\\d{4})[- ]?(\\d{4})[- ]?(\\d{4})[- ]?(\\d{4})\\b");

    public static String enmascararTarjetas(String texto) {
        if (texto == null) return null;

        Matcher matcher = PATRON_TARJETA.matcher(texto);
        // Sustituir los primeros 12 dígitos por asteriscos conservando el último grupo
($4)
        return matcher.replaceAll("****-****-****-$4");
    }

    public static void main(String[] args) {
        String log = "Pago aceptado con tarjeta 4545-1234-5678-9999 por importe de
45.50€. " +
                     "Tarjeta de reserva: 1111222233334444 verificada.";

        String seguro = enmascararTarjetas(log);
        System.out.println(seguro);
    }
}
```

Solución Ejercicio 12: Conteo Recursivo de Caminos en una Cuadrícula

```
public class Ejercicio12CaminosCuadricula {

    public static int contarCaminos(int n, int m) {
        // Precondición defensiva
        if (n ≤ 0 || m ≤ 0) {
            return 0;
        }

        // Caso Base: En una cuadrícula de 1 fila o 1 columna solo existe 1 camino
recto
        if (n == 1 || m == 1) {
            return 1;
        }

        // Caso Recursivo: La suma de caminos moviéndose hacia abajo + moviéndose a la
derecha
        return contarCaminos(n - 1, m) + contarCaminos(n, m - 1);
    }

    public static void main(String[] args) {
        System.out.println("Caminos en 1x1: " + contarCaminos(1, 1)); // 1
        System.out.println("Caminos en 2x2: " + contarCaminos(2, 2)); // 2
        System.out.println("Caminos en 3x3: " + contarCaminos(3, 3)); // 6
        System.out.println("Caminos en 3x7: " + contarCaminos(3, 7)); // 28
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 3

🧠 Boletín 03 Extras — Retos Algorítmicos y Competitivos

Retos de alta intensidad algorítmica inspirados en plataformas de programación competitiva (CodeWars, Acepta el Reto, LeetCode) para la Unidad 3 en Java 25 LTS.

Bienvenido al boletín de retos competitivos de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Los siguientes 6 desafíos plantean problemas de optimización computacional, complejidad temporal y espacial, manipulación avanzada de memoria y algoritmos clásicos de la informática. Cada reto incluye su planteamiento formal, análisis de complejidad, pistas estratégicas y la solución íntegra comentada en Java 25 LTS.

Reto 1: La Criba de Eratóstenes (Generador de Números Primos)

Contexto y Planteamiento

Encontrar todos los números primos hasta un límite `N` comprobando la primalidad de cada número uno por uno mediante divisiones sucesivas tiene una complejidad temporal inasumible `O(N × √N)`. El sabio griego Eratóstenes diseñó un método de filtrado que tacha sistemáticamente los múltiplos de cada primo encontrado.

Implementa un método `public static int[] cribaEratostenes(int n)` que devuelva un array con todos los números primos menores o iguales a `n`.

- **Complejidad requerida:** `O(N log log N)` en tiempo.
- **Pista algorítmica:** Usa un array booleano `esPrimo` de tamaño `n + 1` inicializado a `true`. Empieza en `p = 2`; tacha todos sus múltiplos comenzando directamente desde `p × p`. El bucle exterior solo necesita iterar hasta `√n`.

▶ 💡 Ver Solución Comentada

Reto 2: Compresión de Cadenas por Racha (Run-Length Encoding)

Contexto y Planteamiento

El algoritmo **RLE** es una técnica clásica de compresión de datos sin pérdida. Consiste en sustituir secuencias repetidas consecutivas del mismo carácter por el número de repeticiones seguido del carácter en cuestión. *Ejemplo:* La cadena `"AAAABBBCCDAA"` se comprime como `"4A3B2C1D2A"`.

Diseña dos métodos complementarios:

1. `public static String comprimirRLE(String texto)`
2. `public static String descomprimirRLE(String textoComprimido)`

- **Requisito:** Utiliza `StringBuilder` para garantizar una complejidad lineal `O(N)` en tiempo y evitar la sobrecarga del Garbage Collector.

▶ 💡 Ver Solución Comentada

Reto 3: El Número Perdido en Tiempo Lineal y Memoria O(1)

Contexto y Planteamiento

Se te proporciona un array desordenado de tamaño `N - 1` que contiene números enteros del `1` al `N` sin ningún duplicado, excepto **exactamente un número que ha desaparecido**.

Diseña un método `public static int encontrarFaltante(int[] arr, int n)` que localice el número omitido cumpliendo:

- Complejidad Temporal: `O(N)`.
- Espacio Adicional: `O(1)` (está prohibido crear nuevos arrays o estructuras intermedias).
- **Pista matemática:** La suma de los primeros `N` números naturales viene dada por la fórmula de Gauss `Suma = N × (N + 1) / 2`. La diferencia entre la suma teórica y la suma real de los elementos del array es el número que falta.

▶ 💡 Ver Solución Comentada

Reto 4: Rotación Cíclica de un Vector In-Place (Algoritmo de las Tres Inversiones)

Contexto y Planteamiento

Dado un array de enteros y un valor `k`, rota los elementos del array hacia la derecha `k` posiciones.
Ejemplo: Si `arr = {1, 2, 3, 4, 5, 6, 7}` y `k = 3`, el array resultante debe ser `{5, 6, 7, 1, 2, 3, 4}`.

- **Requisito estricto:** El algoritmo debe modificar el array in-place, con memoria adicional `O(1)` y tiempo `O(N)`.
- **Estrategia (Las 3 Inversiones):**
  1. Invierte el array completo: `{7, 6, 5, 4, 3, 2, 1}`.
  2. Invierte los primeros `k` elementos: `{5, 6, 7, 4, 3, 2, 1}`.
  3. Invierte los elementos restantes desde `k` hasta el final: `{5, 6, 7, 1, 2, 3, 4}`.

▶ 💡 Ver Solución Comentada

Reto 5: Las Torres de Hanói (Recursividad Pura)

Contexto y Planteamiento

Las Torres de Hanói es un rompecabezas clásico donde se deben trasladar `N` discos de radios decrecientes desde la varilla `Origen` hasta la varilla `Destino`, utilizando una varilla `Auxiliar` intermedia.

- Solo se puede mover un disco por turno.
- Ningún disco mayor puede descansar sobre uno menor en ningún momento.

Diseña un método recursivo `public static void resolverHanoi(int n, char origen, char auxiliar, char destino)` que imprima cada movimiento elemental y devuelva la cantidad total de movimientos (`2^N - 1`).

▶ 💡 Ver Solución Comentada

Reto 6: Validador de Delimitadores Equilibrados con Simulación de Pila

Contexto y Planteamiento

En compiladores e intérpretes, es crucial verificar que los delimitadores de apertura `(`, `[`, `{` se cierran en el orden correcto con sus respectivos cierres `)`, `]`, `}`.
Ejemplos:

- `"{ [ ( ) ] }"` -> **Válido** (anidamiento perfecto).
- `"( [ ) ]"` -> **Inválido** (cruce de delimitadores).
- `"(((("` -> **Inválido** (delimitadores sin cerrar).

Implementa un método `public static boolean sonDelimitadoresValidos(String codigo)` empleando un array de caracteres `char[]` para simular internamente la estructura de una pila (LIFO) y un puntero de tope `top`.

▶ 💡 Ver Solución Comentada

CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Unidad 4. Clases y objetos

Fundamentos de la programación orientada a objetos (POO), abstracción, encapsulamiento, instanciación en el Heap, constructores, atributos y métodos de instancia, la referencia this, modificadores de acceso, miembros static, la clase Object (toString, equals, hashCode), records inmutables, enums y depuración con IntelliJ IDEA.



MÓDULO PROFESIONAL
Programación (0485) — 1º DAM

RESULTADOS DE APRENDIZAJE
RA1, RA2, RA3, RA4 (Fundamentos de POO, clases, objetos y encapsulamiento)

DURACIÓN ESTIMADA
28 horas lectivas

ENTORNO OFICIAL
IntelliJ IDEA / Java 25 LTS

Hasta este punto, hemos aprendido a estructurar el flujo de ejecución con sentencias condicionales, bucles iterativos, captura de excepciones y funciones modulares (`public static`). Sin embargo, en el paradigma estructurado tradicional los datos (variables y arrays) y las funciones que los manipulan viven completamente separados. En proyectos de software medianos o grandes, esta separación provoca que el código se vuelva frágil, propenso a inconsistencias y difícil de mantener.

La **Programación Orientada a Objetos (POO)** supone una auténtica revolución conceptual en la ingeniería del software: propone modelar los programas informáticos no como secuencias lineales de instrucciones, sino como **sistemas colaborativos de entidades inteligentes denominadas objetos**, que encapsulan tanto sus **datos internos (estado)** como las **operaciones que pueden realizar (comportamiento)**.

En esta unidad nos adentraremos en el corazón de la POO utilizando **Java 25 LTS** (con compatibilidad plena con Java 21 LTS) y **IntelliJ IDEA** como nuestro entorno de desarrollo profesional.

---

1. El Paradigma de la Programación Orientada a Objetos (POO)

1.1 De la Programación Estructurada a los Objetos

Imagina que desarrollamos una aplicación bancaria con el enfoque estructurado de la Unidad 3. Declararíamos variables independientes:

```
// Enfoque estructurado (separación artificial entre datos y lógica)
String[] titulares = {"Laura Gómez", "Carlos Ruíz"};
String[] iban = {"ES211234...", "ES984321..."};
double[] saldos = {1450.50, 320.00};
boolean[] bloqueadas = {false, true};

// Método externo que debe recibir y coordinar todos los arrays sueltos
public static void transferir(int origenIdx, int destinoIdx, double cantidad) {
    if (!bloqueadas[origenIdx] && saldos[origenIdx] >= cantidad) {
        saldos[origenIdx] -= cantidad;
        saldos[destinoIdx] += cantidad;
    }
}
```

Este enfoque presenta debilidades críticas:

1. **Desincronización de datos:** Si se elimina o reordena un índice en `titulares` , todos los arrays paralelos pueden quedar desalineados.
2. **Falta de integridad:** Cualquier parte del código puede acceder directamente a `saldos[origenIdx] = -50000;` corrompiendo las reglas del negocio financiero sin ningún tipo de control o validación.
3. **Escalabilidad nula:** Añadir una nueva propiedad (como el límite de crédito o el tipo de cuenta) exige modificar decenas de métodos y parámetros en toda la aplicación.

La **POO** resuelve este problema agrupando los datos y las funciones que operan sobre ellos en una única unidad lógica coherente: **la clase** `CuentaBancaria` .

1.2 Los Cuatro Pilares Fundamentales de la POO

El paradigma orientado a objetos se sustenta sobre cuatro principios universales de diseño:

```
+-------------------------------------------------------------------------+
|                 LOS CUATRO PILARES DE LA ORIENTACIÓN A OBJETOS           |
+------------------+--------------------+------------------+-------------+
|    ABSTRACCIÓN   |   ENCAPSULAMIENTO  |     HERENCIA     | POLIMORFISMO|
| Aislar lo esencial| Ocultar el estado  | Reutilizar y     | Tratar a    |
| ignorando detalles| interno y exponer  | extender clases  | diferentes  |
| irrelevantes para | una interfaz       | base jerárquicas | objetos bajo|
| el problema.      | pública segura.    | (se ve en UD5).  | una interfaz|
+------------------+--------------------+------------------+-------------+
```

1. **Abstracción:** Capacidad de identificar las características y comportamientos esenciales de una entidad del mundo real que resultan pertinentes para nuestro sistema, descartando los detalles superfluos. Para una clínica veterinaria, un perro tiene `nombre`, `peso`, `raza` y `fechaVacunacion`; el color de sus ojos o su juguete favorito suelen ser irrelevantes.
2. **Encapsulamiento (*Information Hiding*):** Mecanismo que empaqueta los atributos y métodos dentro de una clase y restringe el acceso directo al estado interno desde el exterior. El objeto actúa como una cápsula protectora: solo se puede interactuar con él a través de métodos públicos controlados.
3. **Herencia (introducción):** Mecanismo de extensión que permite a una clase nueva (subclase) heredar atributos y métodos de una clase preexistente (superclase), promoviendo la reutilización del código (se profundizará en la Unidad 5).
4. **Polimorfismo (introducción):** Capacidad que permite enviar un mismo mensaje (invocar un método) a objetos de diferentes clases y que cada uno responda de forma específica a su propia naturaleza (se profundizará en la Unidad 5).

1.3 Diferencia Fundamental: Clase vs. Objeto

La distinción entre clase y objeto es el concepto más trascendental que debes dominar en esta etapa:

Concepto	Definición Analógica	Naturaleza en Java
Clase (*Class*)	Es el **plano arquitectónico**, la **receta** o el **molde**. Define qué atributos y qué métodos tendrán los elementos creados con ella, pero no ocupa memoria para datos reales.	Código fuente (`.java`), compilado en `.class` y cargado en el *Metaspace*.
Objeto (*Object / Instance*)	Es la **casa construida**, el **pastel horneado** a partir de la receta. Es una entidad concreta y viva con valores específicos para sus atributos.	Instancia física creada con el operador `new` alojada en la memoria **Heap**.

La Tríada del Objeto: Estado, Comportamiento e Identidad

Todo objeto en tiempo de ejecución se caracteriza por tres elementos:

1. **Estado:** Conjunto de valores almacenados en sus variables o atributos de instancia en un instante determinado.
2. **Comportamiento:** Conjunto de métodos y operaciones que el objeto puede ejecutar, capaces de consultar o alterar su estado interno.
3. **Identidad:** Propiedad intrínseca que distingue a un objeto de todos los demás en la memoria, incluso si dos objetos contienen exactamente los mismos valores en todos sus atributos (su dirección física de memoria o referencia en el Heap es única).

1.4 Nociones Básicas de Modelado UML: El Diagrama de Clases

El **Lenguaje Unificado de Modelado (UML - *Unified Modeling Language*)** es el estándar internacional de la industria para representar gráficamente la estructura y relaciones de un sistema software antes de codificarlo.

En UML, una clase se representa mediante un rectángulo dividido en tres compartimentos horizontales:

```
+------------------------------------------------------------+
|                      CuentaBancaria                        | <-- 1. Nombre de la clase
+------------------------------------------------------------+
| - titular: String                                          | <-- 2. Atributos
| - iban: String                                             |    (visibilidad, nombre, tip
o)
| - saldo: double                                            |
| - bloqueada: boolean                                       |
+------------------------------------------------------------+
| + ingresar(cantidad: double): void                         | <-- 3. Métodos
| + retirar(cantidad: double): boolean                       |    (visibilidad, nombre, par
ámetros, retorno)
| + getSaldo(): double                                       |
| + estaBloqueada(): boolean                                 |
+------------------------------------------------------------+
```

Símbolos Universales de Visibilidad en UML:

- `-` **(Privado /** `private` **):** Accesible exclusivamente desde dentro del código de la propia clase.
- `+` **(Público /** `public` **):** Accesible libremente desde cualquier clase del proyecto.
- `#` **(Protegido /** `protected` **):** Accesible dentro del mismo paquete y por subclases hereditarias.
- `~` **(Paquete /** ***package-private*** **):** Accesible únicamente por clases situadas dentro del mismo paquete (sin modificador en Java).

---

2. Definición de Clases en Java

2.1 Anatomía Canónica de una Clase

En Java, cada clase pública debe declararse en su propio fichero con el mismo nombre y extensión `.java` (por ejemplo, `CuentaBancaria.java`):

```
package es.ies.dam.banco; // Declaración del paquete de pertenencia

public class CuentaBancaria {

    // 1. ATRIBUTOS DE INSTANCIA (Estado interno)
    private String titular;
    private String iban;
    private double saldo;
    private boolean bloqueada;

    // 2. MÉTODOS CONSTRUCTORES (Inicialización)
    public CuentaBancaria(String titular, String iban) {
        this.titular = titular;
        this.iban = iban;
        this.saldo = 0.0;
        this.bloqueada = false;
    }

    // 3. MÉTODOS DE INSTANCIA (Comportamiento)
    public void ingresar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a ingresar debe ser positiva: "
+ cantidad);
        }
        if (this.bloqueada) {
            throw new IllegalStateException("Operación denegada: la cuenta está bloquead
a.");
        }
        this.saldo += cantidad;
    }

    public boolean retirar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a retirar debe ser positiva: " +
cantidad);
        }
        if (this.bloqueada || this.saldo < cantidad) {
            return false; // Retirada rechazada
        }
        this.saldo -= cantidad;
        return true; // Retirada completada
    }

    // 4. MÉTODOS ACCESORES (Getters)
    public double getSaldo() {
        return this.saldo;
    }

    public String getTitular() {
        return this.titular;
    }

    public boolean isBloqueada() {
        return this.bloqueada;
    }
}
```

2.2 Atributos de Instancia y Valores por Defecto en el Heap

Cuando se declara un atributo a nivel de clase (sin el modificador `static`), cada objeto instanciado tendrá su propia copia independiente de esa variable en el Heap.

A diferencia de las variables locales dentro de un método (que deben inicializarse explícitamente antes de leerse o el compilador arrojará un error), **los atributos de instancia se inicializan automáticamente por la JVM** al reservar memoria con `new`:

Tipo de Atributo	Valor Inicial por Defecto
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`boolean`	`false`
`char`	`'\u0000'` (carácter nulo)
Referencias a Objetos (`String`, arrays, envoltorios)	`null`

2.3 Métodos de Instancia y la Palabra Reservada `this`

Un **método de instancia** es una función que opera en el contexto de un objeto concreto. Dentro de cualquier método no estático de una clase, Java proporciona automáticamente una variable implícita especial: **`this`**.

`this` es una referencia que **apunta directamente a la instancia actual del objeto que está ejecutando el método**.

Usos Principales de `this`:

1. **Desambiguación de nombres (Sombreamiento de variables o *Shadowing*):**
   Cuando un parámetro formal o una variable local tiene exactamente el mismo nombre que un atributo de instancia, el identificador local oculta al atributo. Se utiliza `this.` para indicar inequívocamente al compilador que nos referimos al campo del objeto:

   ```
   public void setTitular(String titular) {
       this.titular = titular; // this.titular = Atributo; titular = Parámetro recibido
   }
   ```

2. **Invocación encadenada de constructores (`this(...)`):**
   Permite a un constructor llamar a otro constructor de la misma clase para evitar duplicación de código de inicialización (analizado en profundidad en la Sección 4).

3. **Paso del propio objeto como argumento:**
   Si un objeto necesita pasarse a sí mismo como argumento a otro método o registrarse en un servicio:

   ```
   gestorNotificaciones.registrarCuenta(this);
   ```

4. **Retorno de la propia instancia para interfaces fluidas (*Method Chaining*):**
   Al hacer que un método devuelva `this`, es posible encadenar múltiples llamadas seguidas en una única sentencia:

```
public CuentaBancaria asignarLimite(double limite) {
    this.limiteCredito = limite;
    return this; // Permite: cuenta.asignarLimite(500).ingresar(100);
}
```

2.4 Las Variables Objeto son Referencias (Continente vs. Contenido)

Cuando declaras una variable de tipo primitivo (`int edad = 8;`), la variable almacena directamente los bits del valor numérico en la pila (*Stack*). Sin embargo, en el caso de los objetos, **declarar una variable no crea el objeto ni lo almacena dentro de ella**: lo que guarda la variable es una **referencia** (un puntero o dirección de memoria) que señala el lugar del **Heap** donde vive realmente el objeto.

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void presentar() {
        System.out.println("Hola, me llamo " + this.nombre + " y tengo " + this.edad + " año
s.");
    }

    public String getNombre() { return this.nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}
```

¿Qué ocurre cuando asignamos una variable objeto a otra (`p2 = p1`)?

Observa con mucha atención el siguiente fragmento de código:

```
Persona p1 = new Persona("Ada", 36);
Persona p2 = p1; // ¡NO se duplica el objeto! Se copia únicamente la referencia (dirección)

p2.setNombre("Grace");
System.out.println(p1.getNombre()); // Imprime: "Grace"
```

Asignar Referencias NO Clona Objetos

Cuando asignas una variable objeto a otra (`Persona p2 = p1;`), **no se crea una copia del objeto en el Heap**, sino que se copia la dirección de memoria. Ambas variables (`p1` y `p2`) acaban apuntando exactamente **al mismo objeto físico**. Por tanto, cualquier modificación realizada a través de `p2` será inmediatamente visible al consultar `p1` .

```
       MEMORIA STACK                          MEMORIA HEAP
+------------------------+           +----------------------------------+
| Variable: p1 (0x4a10)  | --------> | Objeto Persona (0x4a10)          |
+------------------------+     ▲     | - nombre: "Grace" (antes "Ada")  |
                               |     | - edad: 36                       |
+------------------------+     |     +----------------------------------+
| Variable: p2 (0x4a10)  | ----+
+------------------------+
```

La Metáfora del Continente y el Contenido (`==` frente al estado interno)

Imagina dos recipientes físicos distintos: una **copa** y una **taza**, ambos llenos con el mismo líquido (`0xCAFE`):

```
public class Recipiente {
    private int contenido = 0xCAFE;

    public static void main(String[] args) {
        Recipiente copa = new Recipiente();
        Recipiente taza = new Recipiente();

        System.out.println("¿Mismo continente (copa == taza)? " + (copa == taza)); // false
        System.out.println("Contenido de la copa: " + copa.contenido);             // 51966
(0xCAFE)
        System.out.println("Contenido de la taza: " + taza.contenido);             // 51966
(0xCAFE)
    }
}
```

- El operador `==` aplicado a referencias pregunta: **«¿Son exactamente el mismo recipiente físico?»** (`false`, porque se crearon con dos `new` distintos).
- Para saber si dos recipientes distintos albergan **el mismo contenido**, nunca debemos usar `==`, sino el método `.equals()` (que estudiaremos en profundidad en la Sección 6).

La Referencia Nula (`null`) y `NullPointerException`

Cuando una variable de tipo objeto no apunta a ninguna instancia en la memoria Heap, decimos que contiene una **referencia nula (`null`)**:

```
Persona p3 = null; // p3 no apunta a ningún objeto vivo

if (p3 != null) {
    p3.presentar(); // Comprobación defensiva segura
}
```

Si intentas acceder a un atributo o invocar un método sobre una variable cuyo valor es `null` (por ejemplo, `p3.presentar()`), la JVM abortará la ejecución de inmediato lanzando la célebre excepción **`java.lang.NullPointerException`**.

---

3. Control de Visibilidad, Encapsulamiento y Ampliación de Métodos

El **encapsulamiento** es el escudo protector que garantiza la integridad del software. Consiste en declarar los atributos como **`private`** para que ningún código exterior pueda manipularlos directamente de forma arbitraria, y

proporcionar métodos públicos (`getters` y `setters`) con lógica de validación.

3.1 La Tabla Maestra de Modificadores de Acceso en Java

Java ofrece cuatro niveles de visibilidad para clases, atributos y métodos:

Modificador	Misma Clase	Mismo Paquete	Subclases (otro paquete)	Todo el Proyecto
`private`	**SÍ**	No	No	No
(default / package-private)	**SÍ**	**SÍ**	No	No
`protected`	**SÍ**	**SÍ**	**SÍ** (vía herencia)	No
`public`	**SÍ**	**SÍ**	**SÍ**	**SÍ**

> **El Modificador por Defecto (Package-Private)**
> Si no antepones ninguna palabra clave de visibilidad antes de un atributo o método (por ejemplo `double saldo;`), Java no lo hace privado ni público: aplica el nivel **por defecto o package-private**, permitiendo que cualquier otra clase situada dentro de la misma carpeta de paquete pueda leerlo y modificarlo directamente. En POO profesional, **declara siempre explícitamente `private` tus atributos**.

3.2 Métodos Accesores (*Getters*) y Mutadores (*Setters*)

Para permitir la lectura o actualización segura de atributos privados, la convención estándar de Java (*JavaBeans standard*) define:

- **Getters (Lectura):** Prefijo `get` seguido del nombre del atributo con la primera letra en mayúscula (ej. `getTitular()`). En variables booleanas es convención estándar usar el prefijo `is` (ej. `isActivo()` , `isAgotado()` , `isBloqueada()`).
- **Setters (Escritura controlada):** Prefijo `set` seguido del nombre del atributo. Deben incorporar validaciones de invariantes para proteger al objeto frente a estados corruptos.
- **Getters y Setters para Arrays:** Cuando una clase encapsula un array (como `private String[] autores`), además del getter/setter de la colección completa es muy buena práctica ofrecer métodos indexados para consultar o modificar un elemento individual (`getAutor(int i)` y `setAutor(int i, String autor)`).

```
public class Alumno {
    private String nombre;
    private double nota; // Invariante: debe estar entre 0.0 y 10.0

    public String getNombre() {
        return this.nombre;
    }

    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del alumno no puede ser nulo ni va
cío.");
        }
        this.nombre = nombre.trim();
    }

    public double getNota() {
        return this.nota;
    }

    public void setNota(double nota) {
        if (nota < 0.0 || nota > 10.0) {
            throw new IllegalArgumentException("Nota no válida (" + nota + "). Debe estar en
el rango [0.0, 10.0].");
        }
        this.nota = nota;
    }
}
```

Generación Automática en IntelliJ IDEA

Aunque ahora estamos aprendiendo y hay que crearlos a mano, muchos IDEs nos permiten automatizar esta tarea. En IntelliJ IDEA, coloca el cursor dentro de la clase y presiona `Alt + Insert` (o haz clic derecho y selecciona *Generate...*). Elige **Getter and Setter**, selecciona los atributos que deseas encapsular y el IDE generará el código de forma instantánea.

3.3 Atributos Inmutables con la Palabra Reservada `final`

Cuando un atributo de instancia se declara con la palabra clave `final`, solo se le puede asignar valor una única vez (ya sea en su propia declaración o en el constructor de la clase). Tras esa asignación inicial, su valor permanece inalterable durante toda la vida útil del objeto:

```
import java.util.Objects;

public class Empleado {
    private final String dni; // Inmutable: el DNI nunca cambia tras instanciarse
    private String departamento;
    private double sueldoBase;

    public Empleado(String dni, String departamento, double sueldoBase) {
        this.dni = Objects.requireNonNull(dni, "El DNI no puede ser nulo");
        this.departamento = departamento;
        this.sueldoBase = sueldoBase;
    }

    public String getDni() {
        return this.dni; // Proporciona getter pero NO setter
    }

    public String getDepartamento() {
        return this.departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }
}
```

3.4 La Interfaz Pública de una Clase y la Reducción del Acoplamiento

El conjunto de todos los métodos `public` de una clase constituye su **interfaz pública**: el contrato visible a través del cual el resto del programa puede comunicarse con los objetos de esa clase sin necesidad de saber cómo están programados por dentro.

```
   ❌ SIN ENCAPSULAMIENTO (Alto Acoplamiento)
   [Código Externo] -------- acceso directo --------> [Atributos internos: saldo, eval1...]
   (Si cambia el tipo o nombre del atributo interno, se rompe todo el código externo)

   ✅ CON INTERFAZ PÚBLICA (Bajo Acoplamiento)
   [Código Externo] ---> [Interfaz Pública: getMedia(), setNota()] ---> [Estado Interno Priv
ado]
   (La implementación interna puede cambiar libremente sin romper ningún programa externo)
```

Ocultar los detalles internos tras una interfaz pública de getters y setters aporta tres ventajas de ingeniería decisivas:

1. **Propiedades calculadas al vuelo:** Un getter no tiene por qué corresponderse con una variable física guardada en memoria. Por ejemplo, `getMedia()` puede calcular en tiempo real `(eval1 + eval2 + eval3) / 3.0`, o `getPrecioConIva()` devolver `this.precio * 1.21`.
2. **Validación y seguridad centralizada:** Cualquier intento de lectura o escritura pasa por un único punto de control donde podemos verificar permisos o lanzar `IllegalArgumentException` si el dato es inválido.
3. **Desacoplamiento y mantenibilidad:** Si dos años después decides cambiar la estructura interna de la clase (por ejemplo, sustituir tres variables `eval1, eval2, eval3` por un array `double[] evaluaciones`), ningún programa externo se romperá mientras mantengas intacta la firma de los métodos públicos `getEval1()`, `getMedia()`, etc.

3.5 Diseño de Métodos en Clases: Responsabilidad, Métodos Auxiliares Privados y Sobrecarga en POO

En la Unidad 3 aprendimos a estructurar métodos y funciones modulares (`public static`), el paso de parámetros por valor y los conceptos básicos de sobrecarga y recursividad. En la Programación Orientada a Objetos, los métodos dejan de ser funciones procedimentales aisladas para convertirse en las **capacidades operativas que dan vida al objeto** y gobiernan su estado interno.

1. El Principio de Responsabilidad Única y Métodos de Instancia

Un método de instancia debe cumplir un único propósito claro dentro del dominio de la clase:

- **Consultar estado:** Métodos que interrogan los atributos del objeto y devuelven una respuesta calculada o directa (como `getSaldo()` , `esMoroso()` o `calcularArea()`).
- **Mutar estado con validación:** Métodos que alteran los atributos asegurando que nunca se vulneren las reglas de integridad del objeto (como `ingresar(cantidad)` o `frenar(decremento)`).
- **Notificar o coordinar:** Métodos que permiten a la entidad interactuar con otros objetos colaboradores.

2. Métodos Auxiliares Privados (*Private Helper Methods*)

Una de las mejores prácticas de la ingeniería del software es mantener la **interfaz pública lo más reducida y limpia posible** (*Principio de Mínima Exposición*). Si una operación pública requiere cálculos matemáticos complejos, algoritmos de verificación o pasos intermedios, estos no deben exponerse al exterior: deben encapsularse en **métodos privados auxiliares**:

```
public class CuentaBancaria {
    private String titular;
    private String iban;
    private double saldo;

    // MÉTODO PÚBLICO: Interfaz visible y clara para el cliente de la clase
    public void ingresar(double cantidad) {
        validarCantidadPositiva(cantidad); // Delega la validación en un método privado
        this.saldo += cantidad;
    }

    public void retirar(double cantidad) {
        validarCantidadPositiva(cantidad);
        validarSaldoDisponible(cantidad);
        this.saldo -= cantidad;
    }

    // MÉTODOS PRIVADOS AUXILIARES: Ocultos tras la cápsula de la clase
    private void validarCantidadPositiva(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad debe ser estrictamente positiva:
" + cantidad);
        }
    }

    private void validarSaldoDisponible(double cantidad) {
        if (cantidad > this.saldo) {
            throw new IllegalArgumentException("Saldo insuficiente. Disponible: " + this.sal
do + ", Solicitado: " + cantidad);
        }
    }
}
```

Ventajas del método auxiliar privado:

- Evita la duplicación de código (*DRY - Don't Repeat Yourself*): varias operaciones públicas pueden reutilizar la misma comprobación interna.
- Si la regla de validación cambia en el futuro, solo se modifica ese método privado sin alterar ningún programa externo que use la clase.

3. Sobrecarga de Métodos de Objeto y Delegación

Como vimos en la Unidad 3, la **sobrecarga (*Overloading*)** consiste en definir dos o más métodos con el mismo nombre pero distinta firma (distinto número o tipos de parámetros) dentro de la misma clase, resolviéndose mediante **ligadura estática** en tiempo de compilación.

En el diseño de clases orientadas a objetos, el patrón canónico de sobrecarga consiste en hacer que los métodos más sencillos **deleguen en el método más completo**, proporcionando valores predeterminados:

```
public class Notificador {
    // Método sobrecargado básico: proporciona un asunto por defecto
    public void enviarMensaje(String destinatario, String cuerpo) {
        enviarMensaje(destinatario, "Notificación del Sistema", cuerpo); // Delega en el mét
odo completo
    }

    // Método completo: concentra toda la lógica real de envío
    public void enviarMensaje(String destinatario, String asunto, String cuerpo) {
        if (destinatario == null || destinatario.isBlank()) {
            throw new IllegalArgumentException("El destinatario no puede estar vacío");
        }
        System.out.printf("Enviando a [%s] con asunto '%s': %s%n", destinatario, asunto, cue
rpo);
    }
}
```

4. Colaboración entre Objetos: Métodos que Operan con Otras Instancias

Los objetos rara vez trabajan solos; su verdadero poder radica en la colaboración. Un método de instancia puede recibir como argumento otra instancia de su misma clase o de otra clase:

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    // Colaboración entre dos cuentas bancarias
    public void transferir(CuentaBancaria cuentaDestino, double cantidad) {
        if (cuentaDestino == null) {
            throw new IllegalArgumentException("La cuenta de destino no puede ser nula");
        }
        if (cuentaDestino == this) {
            throw new IllegalArgumentException("No puedes realizar una transferencia a la mi
sma cuenta");
        }

        // Reutiliza los métodos validados de la propia clase
        this.retirar(cantidad);
        cuentaDestino.ingresar(cantidad);
    }
}
```

> **Paso de Referencias por Valor**
>
> Recuerda el principio fundamental consolidado en la Unidad 3: en Java **todo se pasa por valor**. Al pasar `cuentaDestino` al método, se recibe una **copia de la dirección de memoria**. A través de esa referencia compartida podemos alterar el estado del objeto en el Heap (`cuentaDestino.ingresar(cantidad)`), pero si dentro del método reasignamos `cuentaDestino = null`, solo desconectamos la variable local del método, dejando intacta la variable original del exterior.

5. Recursividad Encapsulada como Comportamiento de Objeto

En la Unidad 3 estudiamos los fundamentos teóricos de la recursividad (el caso base y la pila de llamadas o *Call Stack*). En Programación Orientada a Objetos, los algoritmos recursivos a menudo se encapsulan como

comportamientos internos de una clase, permitiendo que el objeto mantenga protegidas sus métricas internas de ejecución (como contadores de pasos o historiales):

```
public class SolucionadorHanoi {
    private int totalMovimientos; // Estado interno protegido del objeto

    public SolucionadorHanoi() {
        this.totalMovimientos = 0;
    }

    public int getTotalMovimientos() {
        return totalMovimientos;
    }

    // Método público de entrada para iniciar la resolución
    public void resolver(int numDiscos, char origen, char auxiliar, char destino) {
        this.totalMovimientos = 0; // Reinicia el estado para cada ejecución
        ejecutarHanoi(numDiscos, origen, auxiliar, destino);
    }

    // Método recursivo privado que realiza el trabajo algorítmico interno
    private void ejecutarHanoi(int n, char desde, char aux, char hasta) {
        if (n == 1) {
            this.totalMovimientos++;
            System.out.printf("Paso %d: Mover disco 1 de %c a %c%n", totalMovimientos, desd
e, hasta);
            return; // Caso base
        }
        ejecutarHanoi(n - 1, desde, hasta, aux);
        this.totalMovimientos++;
        System.out.printf("Paso %d: Mover disco %d de %c a %c%n", totalMovimientos, n, desd
e, hasta);
        ejecutarHanoi(n - 1, aux, desde, hasta);
    }
}
```

---

4. Ciclo de Vida de los Objetos: Instanciación, Constructores y Destrucción

Un objeto en Java no nace por generación espontánea ni permanece para siempre en la memoria. Atraviesa un ciclo de vida bien definido: **nacimiento (instanciación con constructores)**, **vida activa (invocación de métodos y modificación de estado)** y **muerte (pérdida de referencias y reciclaje por el Garbage Collector)**.

4.1 El Operador `new` y la Reserva de Memoria en el Heap

Cuando el compilador y la JVM se encuentran con una sentencia como:

```
CuentaBancaria miCuenta = new CuentaBancaria("Sofía Morales", "ES9121000418450200051332");
```

La JVM ejecuta internamente una secuencia de cuatro pasos atómicos:

1. **Reserva física de espacio:** Se solicita un bloque contiguo de bytes en la memoria **Heap** con capacidad suficiente para albergar todos los atributos de instancia declarados en la clase (y los heredados de sus superclases).

2. **Inicialización por defecto:** Todas las variables del objeto recién reservado se inicializan a sus valores neutros o por defecto (`0` , `0.0` , `false` , `null`).
3. **Ejecución del constructor:** Se invoca el método constructor correspondiente, pasando los argumentos facilitados. En este paso se ejecutan las asignaciones iniciales y las validaciones de negocio.
4. **Devolución de la referencia:** El operador `new` devuelve la dirección física de memoria donde reside el objeto en el Heap (por ejemplo, `0x7b4a2f`), la cual queda almacenada en la variable local de la pila **Stack** (`miCuenta`).

```
      MEMORIA STACK                          MEMORIA HEAP
+------------------------+            +-----------------------------------+
| Variable: miCuenta     | ---------> | Objeto CuentaBancaria (0x7b4a2f)  |
| (Dirección 0x7b4a2f)   |            | - titular: "Sofía Morales"        |
+------------------------+            | - iban: "ES91210004..."           |
                                      | - saldo: 0.0                      |
                                      | - bloqueada: false                |
                                      +-----------------------------------+
```

4.2 Anatomía y Propósito de los Constructores

Un **constructor** es un método especial cuya única misión es inicializar un objeto en un estado coherente, válido y listo para operar desde el mismo instante en que se reserva su memoria.

Reglas Sintácticas Inquebrantables de los Constructores:

1. Su nombre debe coincidir **exactamente con el nombre de la clase** (respetando estrictamente mayúsculas y minúsculas).
2. **No tiene ningún tipo de retorno**, ni siquiera `void` . *(Si colocas `public void CuentaBancaria()` , Java lo interpretará como un método ordinario con un nombre desafortunado, no como un constructor).*
3. Se invoca única y exclusivamente mediante la palabra reservada `new` en el momento de crear el objeto.

4.3 El Constructor por Defecto Implícito (*Default Constructor*)

Si escribes una clase y **no declaras ningún constructor explícito**, el compilador de Java inserta automáticamente en el bytecode un constructor público sin parámetros y con cuerpo vacío:

```
public class Sensor {
    private double temperatura;
    // El compilador añade automáticamente: public Sensor() {}
}
```

> **La Desaparición del Constructor por Defecto**
>
> En el momento exacto en que declaras **cualquier constructor propio con parámetros**, el compilador de Java **retira de inmediato el constructor por defecto sin parámetros**.
>
> Si tras definir `public Sensor(double tempInicial)` intentas hacer `Sensor s = new Sensor();` , el código fallará en compilación con el error: `constructor Sensor in class Sensor cannot be applied to given types` . Si necesitas ambos, debes declarar explícitamente el constructor sin argumentos.

4.4 Sobrecarga de Constructores y Delegación con `this(...)`

Al igual que los métodos ordinarios, los constructores admiten **sobrecarga**: podemos proporcionar múltiples constructores con diferente número o tipo de parámetros para permitir distintas formas de inicializar un objeto.

Para evitar la duplicación de código de inicialización y validación entre constructores, Java permite que un constructor invoque a otro de su misma clase utilizando la sintaxis `this(argumentos)`:

```
public class Producto {
    private final String codigo;
    private String descripcion;
    private double precio;
    private int stock;

    // 1. Constructor Principal (Canónico): recibe todos los datos y valida
    public Producto(String codigo, String descripcion, double precio, int stock) {
        if (codigo == null || codigo.trim().isEmpty()) {
            throw new IllegalArgumentException("El código de producto es obligatorio.");
        }
        if (precio < 0 || stock < 0) {
            throw new IllegalArgumentException("El precio y el stock no pueden ser negativo
s.");
        }
        this.codigo = codigo.trim().toUpperCase();
        this.descripcion = (descripcion != null) ? descripcion.trim() : "Sin descripción";
        this.precio = precio;
        this.stock = stock;
    }

    // 2. Constructor Secundario: precio dado, stock inicial por defecto a 0
    public Producto(String codigo, String descripcion, double precio) {
        this(codigo, descripcion, precio, 0); // Delega en el constructor principal
    }

    // 3. Constructor Mínimo: producto recién catalogado con precio 0.0 y stock 0
    public Producto(String codigo, String descripcion) {
        this(codigo, descripcion, 0.0, 0); // Delega en el constructor principal
    }
}
```

> **Regla de Oro de `this(...)`**
>
> La llamada delegada a otro constructor mediante `this(...)` debe ser **estrictamente la primera sentencia ejecutable** del cuerpo del constructor. Colocar cualquier otra instrucción antes de `this(...)` provocará un error de compilación inmediato (`call to this must be first statement in constructor`).

4.5 Bloques de Inicialización y Orden Cronológico Exacto

Además de los constructores, Java permite definir **bloques de inicialización de instancia** (`{ ... }`) y **bloques de inicialización estáticos** (`static { ... }`) directamente en el cuerpo de la clase.

Cuando se utiliza una clase por primera vez y se instancia un objeto con `new`, la JVM respeta un orden cronológico estricto:

1. **Carga de la clase (una sola vez):** Se inicializan las variables `static` (con su valor por defecto o en línea) y se ejecutan los bloques `static { ... }` en orden de aparición.
2. **Reserva en el Heap:** Al ejecutar `new`, se reserva memoria para el objeto y todos sus atributos de instancia toman primero su valor por defecto (`0`, `false`, `null`).
3. **Inicialización en línea y bloques `{ ... }` de instancia:** Se asignan los valores declarados en la propia línea del atributo y se ejecutan los bloques `{ ... }` de arriba abajo.
4. **Cuerpo del constructor:** Finalmente se ejecuta el código dentro del constructor invocado.

```
public class AlumnoCFGS {
    private static int edadMinima = 18;
    private int nia;
    private String nombre;
    private int[] notas = new int[3];
    private boolean repetidor;

    // 1. Bloque estático: se ejecuta una única vez al cargar la clase en la JVM
    static {
        edadMinima--; // Pasa de 18 a 17
    }

    // 2. Bloque de instancia: se ejecuta cada vez que nace un objeto, ANTES del constructor
    {
        this.nia = (int) (Math.random() * 900000) + 100000;
    }

    // 3. Constructor: último paso de inicialización
    public AlumnoCFGS(String nombre) {
        this.nombre = nombre;
    }
}
```

Tras ejecutar `AlumnoCFGS a1 = new AlumnoCFGS("Pepe");`, el estado resultante es: `edadMinima = 17`, `nia` aleatorio de 6 cifras, `nombre = "Pepe"`, `notas = {0, 0, 0}` y `repetidor = false`.

4.6 Destrucción de Objetos y el Recolector de Basura (*Garbage Collector*)

En lenguajes de bajo nivel como C o C++, el programador debe liberar manualmente la memoria con `free()` o `delete`. Un descuido produce fugas de memoria (*memory leaks*) que saturan el sistema.

Java utiliza un modelo de gestión automática de memoria mediante el **Garbage Collector (GC)**, un hilo de fondo de la JVM que monitoriza constantemente el Heap.

¿Cuándo es un Objeto Elegible para Destrucción?

Un objeto se convierte en candidato a ser eliminado cuando **deja de ser alcanzable por cualquier hilo de ejecución activo** (es decir, ya no existe ninguna variable de referencia en el Stack que apunte a él, directa o indirectamente):

```
public static void demostracionMemoria() {
    CuentaBancaria c1 = new CuentaBancaria("Juan", "ES111");
    CuentaBancaria c2 = new CuentaBancaria("Ana", "ES222");

    c1 = c2;
    // Ahora c1 apunta al objeto de Ana.
    // El objeto original de Juan ("ES111") ha quedado HUÉRFANO en el Heap:
    // Ya ninguna variable apunta a él. Es candidato inmediato para el Garbage Collector.

    c2 = null;
    // El objeto de Ana sigue vivo porque c1 aún lo referencia.
}
// Al salir del método demostracionMemoria(), c1 desaparece del Stack.
// Ahora el objeto de Ana también queda huérfano y será reclamado por el GC.
```

El Método finalize() está Prohibido en Java Moderno

Históricamente, la clase `Object` incluía un método `finalize()` que se ejecutaba supuestamente antes de que el GC destruyera el objeto. Sin embargo, su comportamiento impredecible, bloqueos de hilos y graves problemas de seguridad hicieron que Oracle lo declarase **obsoleto desde Java 9 y marcado para eliminación definitiva**.

Para liberar recursos del sistema (archivos abiertos, conexiones a bases de datos, sockets de red), la práctica oficial y obligatoria es implementar la interfaz `java.lang.AutoCloseable` y utilizar la estructura `try-with-resources` vista en la Unidad 3.

---

5. Miembros Estáticos: La Palabra Reservada `static` y Combinaciones con `final`

Hasta ahora, todos los atributos y métodos estudiados pertenecían a **instancias individuales**: cada objeto tenía su propio `saldo` y su propio `titular`.

Sin embargo, en multitud de escenarios necesitamos elementos que no pertenezcan a ningún objeto concreto, sino a **la clase en su conjunto**. Para este propósito, Java proporciona el modificador `static`.

5.1 Variables Estáticas (Atributos de Clase)

Una **variable estática** es un campo declarado con `static`. Existe **una única copia física** de esa variable en toda la JVM (alojada en la zona de memoria *Metaspace*), compartida solidariamente por todos los objetos de esa clase:

```
public class Empleado {
    // 1. Atributo estático: compartido por todos los empleados
    private static int contadorTotalEmpleados = 0;

    // 2. Atributos de instancia: propios de cada empleado individual
    private final int idEmpleado;
    private String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
        // Incrementamos el contador global y asignamos un ID secuencial automático
        contadorTotalEmpleados++;
        this.idEmpleado = contadorTotalEmpleados;
    }

    public static int getContadorTotalEmpleados() {
        return contadorTotalEmpleados;
    }

    public int getIdEmpleado() {
        return this.idEmpleado;
    }
}
```

Si instanciamos tres objetos:

```
Empleado e1 = new Empleado("Marina"); // id: 1
Empleado e2 = new Empleado("Roberto"); // id: 2
Empleado e3 = new Empleado("Elena");   // id: 3

System.out.println(Empleado.getContadorTotalEmpleados()); // Imprime: 3
```

5.2 Métodos Estáticos y Reglas de Acceso entre Contextos

Un **método estático** es una función asociada a la clase, no a ningún objeto. Se invoca directamente utilizando el nombre de la clase (`Clase.metodo()`), sin necesidad de crear ninguna instancia con `new` :

```
double hipotenusa = Math.sqrt(Math.pow(3, 2) + Math.pow(4, 2)); // Métodos estáticos de jav
a.lang.Math
```

La Regla de Oro entre Contexto Estático y Contexto de Instancia:

Puesto que un método estático se ejecuta a nivel de clase (incluso cuando se han creado 0 objetos en el Heap):

1. **Dentro de un método `static` NO existe la referencia `this`** .
2. **Un método `static` NO puede acceder directamente a atributos ni métodos de instancia** (salvo que instancie primero un objeto con `new` o lo reciba por parámetro).
3. **Un método de instancia SÍ puede acceder libremente tanto a miembros de instancia (`this`) como a miembros `static` de su clase**.

```
public class EjemploContextos {
    private static int variableDeClase = 1;
    private int variableDeObjeto = 2;

    public static void metodoDeClase() {
        variableDeClase = 3;      // ✅ OK: acceso de estático a estático
        // variableDeObjeto = 2;  // ❌ ERROR DE COMPILACIÓN: ¿de qué objeto si no hay 'thi
s'?
        // metodoDeObjeto();      // ❌ ERROR DE COMPILACIÓN

        // Solo podemos acceder a miembros de objeto si referenciamos una instancia concret
a:
        EjemploContextos obj = new EjemploContextos();
        obj.variableDeObjeto = 20; // ✅ OK: indicamos explícitamente el objeto 'obj'
    }

    public void metodoDeObjeto() {
        variableDeClase = 10;     // ✅ OK: un objeto conoce a su clase
        metodoDeClase();          // ✅ OK
        this.variableDeObjeto = 5;// ✅ OK: acceso al propio objeto actual (this)
    }
}
```

5.3 Tabla Maestra: Combinaciones de `static` y `final` (Matriz 2×2)

Los modificadores `static` y `final` responden a dos preguntas completamente distintas pero complementarias:

- `static` responde a **«¿a quién pertenece?»**: sin `static` pertenece a cada objeto individual; con `static` pertenece a la clase y se comparte entre todas las instancias.
- `final` responde a **«¿puede cambiar de valor?»**: sin `final` puede reasignarse; con `final` solo admite una única asignación inicial.

Al cruzar ambos modificadores obtenemos cuatro combinaciones fundamentales en el diseño en Java:

	Sin `static` *(uno por cada objeto)*	**Con `static` *(uno compartido en la clase)***
Sin `final` *(puede cambiar)*	**1. Atributo de instancia normal** `private String nombre;` Cada objeto tiene el suyo y puede cambiar a lo largo de su vida.	**3. Variable de clase compartida** `private static int contador = 0;` Común a todos los objetos y su valor varía durante la ejecución.
Con `final` *(no puede cambiar)*	**2. Constante de instancia** `private final String dni;` Se asigna en el constructor de cada objeto y ya no cambia jamás para ese objeto.	**4. Constante global de clase** `public static final double IVA = 0.21;` Valor único, fijo y universal para toda la clase. Se escribe en `MAYUSCULAS`.

```
public class Factura {
    // 4. Constante de clase (static + final): única e inmutable para todo el programa
    public static final double IVA_GENERAL = 0.21;

    // 3. Variable de clase (static sin final): compartida pero mutable
    private static int totalFacturasEmitidas = 0;

    // 2. Constante de instancia (final sin static): distinta en cada factura, pero fija tra
s nacer
    private final int numeroFactura;

    // 1. Atributo de instancia normal (sin static ni final): propio de cada objeto y modifi
cable
    private String concepto;

    public Factura(String concepto) {
        totalFacturasEmitidas++;
        this.numeroFactura = totalFacturasEmitidas; // Se asigna una sola vez y queda sellad
o
        this.concepto = concepto;
    }
}
```

> **Cuándo NO usar `static`**
>
> No utilices variables `static` como «atajo» para compartir datos entre métodos sin pasarlos por parámetro. Una variable `static` mutable sin justificación real es, en la práctica, una **variable global encubierta** que rompe el encapsulamiento, acopla las clases entre sí y provoca errores gravísimos difíciles de rastrear. Usa `static` únicamente para **constantes (`public static final`)**, **contadores/registros de clase** o **métodos de utilidad pura** (como `Math.sqrt()`).

---

6. La Clase Raíz del Universo Java: `java.lang.Object`

En la jerarquía de tipos de Java, **toda clase desciende directa o indirectamente de la clase universal `java.lang.Object`**. Si no especificas la cláusula `extends` al declarar una clase, el compilador añade automáticamente `extends Object`.

Esto garantiza que cualquier objeto en Java herede un conjunto de métodos fundamentales:

Método en `java.lang.Object`	Propósito	¿Se puede / debe sobrescribir?
`String toString()`	Devuelve la representación textual del objeto. Por defecto devuelve `Clase@hashHex`.	**Sí (Recomendado siempre)**
`boolean equals(Object obj)`	Comprueba si dos objetos contienen la misma información semántica. Por defecto evalúa `this == obj`.	**Sí (Según identidad de negocio)**
`int hashCode()`	Devuelve el código hash entero del objeto para colecciones (`HashSet`, `HashMap`).	**Sí (Obligatorio si se sobrescribe `equals`)**
`protected Object clone()`	Crea y devuelve una copia superficial (*shallow copy*) del objeto (requiere `Cloneable`).	Se puede (se prefiere constructor de copia)
`final Class<?> getClass()`	Devuelve el objeto `Class` que representa la clase real en tiempo de ejecución.	**No (`final`)**
`final void notify() / notifyAll() / wait()`	Sincronización y coordinación de hilos concurrentes (*threads*).	**No (`final`)**
`protected void finalize()`	Invocado históricamente por el GC antes de destruir el objeto.	**Obsoleto / Prohibido desde Java 9+**

6.1 Sobrescritura (*Override*) y la Anotación `@Override`

La **sobrescritura** consiste en redefinir en una subclase un método heredado de su superclase para dotarlo de un comportamiento específico.

Para indicar formalmente al compilador que nuestra intención es sobrescribir un método, anteponemos la anotación `@Override`:

```
@Override
public String toString() { ... }
```

> **¿Por qué es indispensable usar @Override?**
> Si cometes una errata al escribir el nombre del método (por ejemplo `public String tostring()` o `public boolean equals(Cliente c)` en vez de `equals(Object obj)`), sin `@Override` el compilador creerá que estás sobrecargando o creando un método nuevo y tu clase no sustituirá el comportamiento heredado de `Object`. Con `@Override`, el compilador verifica la firma exacta y detiene la compilación informándote del error al instante.

6.2 Sobrescribir `toString()`: Representación Textual Legible

La implementación por defecto de `toString()` en la clase `Object` es:

```
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

Por ello, imprimir un objeto sin `toString()` produce una salida críptica como `es.ies.dam.banco.CuentaBancaria@7b4a2f`. Al sobrescribirlo:

```
@Override
public String toString() {
    return String.format("CuentaBancaria[IBAN='%s', Titular='%s', Saldo=%.2f€, Bloqueada=%
b]",
            this.iban, this.titular, this.saldo, this.bloqueada);
}
```

Ahora, al pasar el objeto a `System.out.println(miCuenta)` o concatenarlo con un `String` (`"Cuenta: " + miCuenta`), Java invoca automáticamente nuestro `toString()`:

```
System.out.println(miCuenta);
// Salida legible: CuentaBancaria[IBAN='ES912100...', Titular='Sofía Morales', Saldo=1450.50
€, Bloqueada=false]
```

6.3 Comparación de Objetos: Operador `==` vs. Método `equals()`

Este es el punto donde más programadores noveles tropiezan:

```
CuentaBancaria c1 = new CuentaBancaria("Carlos", "ES001");
CuentaBancaria c2 = new CuentaBancaria("Carlos", "ES001");

System.out.println(c1 == c2);      // ¡Imprime FALSE!
System.out.println(c1.equals(c2)); // Imprime TRUE (si sobrescribimos equals correctamente)
```

¿Por qué ocurre esto?

- **El operador `==`:** Compara **identidad física de referencia** (si ambas variables apuntan a la misma dirección física en el Heap). Como `c1` y `c2` fueron creados con dos sentencias `new` distintas, residen en posiciones de memoria diferentes; por tanto `c1 == c2` es `false`.
- **El método `equals()`:** Compara **igualdad de contenido o semántica de negocio** (si dos objetos representan la misma entidad real, por ejemplo dos cuentas con el mismo IBAN o dos cartas con el mismo palo y valor).

El Contrato Riguroso de `equals()` en Java

Según la especificación oficial de la JVM, cualquier implementación de `equals` debe cumplir cinco propiedades matemáticas:

1. **Reflexiva:** `x.equals(x)` debe devolver siempre `true`.
2. **Simétrica:** `x.equals(y)` debe devolver `true` si y solo si `y.equals(x)` devuelve `true`.
3. **Transitiva:** Si `x.equals(y)` y `y.equals(z)` son `true`, entonces `x.equals(z)` debe ser `true`.
4. **Consistente:** Múltiples invocaciones sobre objetos no modificados deben arrojar siempre el mismo resultado.
5. **Tratamiento de `null`:** `x.equals(null)` debe devolver siempre `false` (sin lanzar `NullPointerException`).

Implementación Canónica Paso a Paso de `equals()` (Java Moderno):

```
import java.util.Objects;

public class Cliente {
    private final String dni;
    private String nombre;
    private String email;

    public Cliente(String dni, String nombre, String email) {
        this.dni = Objects.requireNonNull(dni);
        this.nombre = nombre;
        this.email = email;
    }

    @Override
    public boolean equals(Object obj) {
        // 1. Comprobación rápida de identidad física (¿apuntan a la misma dirección?)
        if (this == obj) {
            return true;
        }

        // 2. Comprobación de null, tipo compatible y cast seguro con Pattern Matching
        if (!(obj instanceof Cliente other)) {
            return false;
        }

        // 3. Comparación de los atributos clave (Objects.equals protege frente a posibles n
ulls)
        return Objects.equals(this.dni, other.dni);
    }

    @Override
    public int hashCode() {
        // La regla de oro: si dos objetos son iguales por equals, DEBEN devolver el mismo h
ashCode
        return Objects.hash(this.dni);
    }
}
```

(Nota: Para comparar atributos primitivos dentro de `equals`, se usa `==` para `int`, `long`, `boolean` o `char`, y `Double.compare(this.precio, other.precio) == 0` para `double`).

6.4 El Contrato Indisociable con `hashCode()` y la Metáfora de los Cajones

El método `hashCode()` devuelve un número entero (`int`) que resume el contenido del objeto. Las colecciones basadas en tablas hash (como `HashSet` o `HashMap`, que estudiaremos en la Unidad 6) utilizan este número para organizar los objetos en **"cajones" (*buckets*)** de búsqueda ultrarrápida:

1. Cuando insertas un objeto en un `HashSet`, Java calcula primero su `hashCode()` para decidir en qué cajón guardarlo.
2. Cuando preguntas si un objeto ya existe (`.contains(obj)`), Java **no recorre toda la colección**: calcula su `hashCode()`, va directamente a ese cajón concreto y **solo dentro de ese cajón** compara los objetos usando `equals()`.

La Regla de Oro de equals y hashCode

- **Si `a.equals(b)` es `true`, entonces `a.hashCode()` y `b.hashCode()` DEBEN devolver obligatoriamente el mismo número entero.**
- *(El inverso no es obligatorio: dos objetos distintos pueden coincidir por azar en el mismo `hashCode` —colisión de hash—, en cuyo caso `equals()` los desempatará dentro del cajón).*

Si sobrescribes `equals()` pero olvidas sobrescribir `hashCode()`, dos objetos con el mismo DNI tendrán distinto `hashCode` (calculado por defecto según su dirección de memoria), caerán en cajones diferentes y `HashSet` jamás detectará que están duplicados. **Usa siempre exactamente los mismos atributos en `Objects.equals(...)` y en `Objects.hash(...)`.**

6.5 Generación de Métodos Canónicos con IntelliJ IDEA

En el desarrollo profesional nadie pica a mano el código de `equals()` y `hashCode()`, pues es fácil cometer errores sutiles de verificación de nulos:

1. Dentro de tu clase en IntelliJ IDEA, presiona `Alt + Insert` (o clic derecho → *Generate...*).
2. Selecciona `equals() and hashCode()`.
3. Elige la plantilla *IntelliJ Default* o *java.util.Objects.equals and hashCode*.
4. Marca las casillas de los campos que identifican de forma única a tu entidad (por ejemplo, el `dni` o el `iban`).
5. Pulsa *Finish*. El IDE generará el código canónico optimizado y libre de errores.

---

7. Diseño Avanzado de Clases, Clases Especiales y Patrones

El dominio de la orientación a objetos no concluye con la creación de clases ordinarias. La arquitectura de software moderna exige patrones de diseño contrastados y mecanismos para garantizar la inmutabilidad y la claridad semántica.

7.1 Clases Inmutables: El Peligro de las Referencias Mutables y la Copia Defensiva

Un **objeto inmutable** es aquel cuyo estado no puede alterarse en absoluto tras su instanciación. Ejemplos eminentes en el JDK son `String`, `Integer`, `LocalDate` y `BigDecimal`.

Ventajas Fundamentales de la Inmutabilidad:

1. **Seguridad frente a hilos (*Thread-Safety*):** En aplicaciones multihilo no se necesitan bloques de sincronización (`synchronized`) ni cerrojos, ya que ningún hilo puede corromper un estado de solo lectura.
2. **Ausencia de efectos colaterales:** Puedes pasar un objeto inmutable a cualquier método con la garantía absoluta de que regresará intacto.
3. **Fiabilidad en colecciones:** Su `hashCode()` nunca cambia tras insertarse en un `HashSet` o `HashMap`.

El Error Clásico: La Implementación "Ingenua" (*Naive*) frente a la Copia Defensiva

Muchos programadores creen que basta con poner `private final` y quitar los setters para que una clase sea inmutable. Observa qué ocurre cuando la clase contiene un atributo cuyo tipo es un **objeto mutable** (como un array `int[] notas`):

```
// ❌ IMPLEMENTACIÓN INGENUA (VULNERABLE): ¡Fuga de referencias!
public class AlumnoInmutableRotura {
    private final String nombre;
    private final int[] notas; // El array en sí es un objeto MUTABLE en el Heap

    public AlumnoInmutableRotura(String nombre, int[] notas) {
        this.nombre = nombre;
        this.notas = notas; // ¡PELIGRO 1! Guarda la misma referencia del array externo
    }

    public int[] getNotas() {
        return this.notas;  // ¡PELIGRO 2! Devuelve la referencia directa al array interno
    }
}
```

Si alguien hace `int[] misNotas = {8, 9}; AlumnoInmutableRotura a = new AlumnoInmutableRotura("Ana", misNotas);`, y acto seguido escribe `misNotas[0] = 0;` o `a.getNotas()[1] = 0;`, ¡habrá modificado las notas internas del supuesto objeto inmutable!

Las 4 Reglas de Oro para Blindar una Clase Inmutable:

1. Declarar la clase como **`final`** (`public final class ...`) para impedir que una subclase herede de ella y rompa la inmutabilidad.
2. Declarar todos los atributos como **`private final`**.
3. **No definir ningún método setter** ni métodos que alteren el estado (si se requiere una operación de cambio, debe devolver una **nueva instancia**).
4. **Realizar copias defensivas (`.clone()` o `Arrays.copyOf()`)** de cualquier objeto mutable tanto al **entrar** por el constructor como al **salir** por un getter:

```
import java.util.Arrays;

// ✅ IMPLEMENTACIÓN BLINDADA Y VERDADERAMENTE INMUTABLE
public final class AlumnoInmutable {
    private final int id;
    private final String nombre; // String ya es inmutable por naturaleza
    private final int[] notas;   // Array mutable: exige copia defensiva en entrada y salida

    public AlumnoInmutable(int id, String nombre, int[] notas) {
        this.id = id;
        this.nombre = (nombre != null) ? nombre : "Sin nombre";
        // COPIA DEFENSIVA DE ENTRADA (con .clone() o Arrays.copyOf):
        this.notas = (notas != null) ? notas.clone() : new int[0];
    }

    public int getId() { return this.id; }
    public String getNombre() { return this.nombre; }

    public int[] getNotas() {
        // COPIA DEFENSIVA DE SALIDA: devuelve un clon independiente
        return this.notas.clone();
    }

    @Override
    public String toString() {
        return "AlumnoInmutable[id=" + id + ", nombre='" + nombre + "', notas=" + Arrays.toS
tring(notas) + "]";
    }
}
```

7.2 Los `record` en Java Moderno (Java 16+, 21 LTS y 25 LTS)

En aplicaciones corporativas, una gran parte de las clases son meros **transportadores de datos inmutables** (*Data Carriers* o DTOs - *Data Transfer Objects*). Escribir manualmente constructores, getters, `equals()`, `hashCode()` y `toString()` genera decenas de líneas de código repetitivo (*boilerplate*).

Para solucionar esto, Java moderno introdujo los **`record`**: una forma concisa y declarativa de definir clases de datos inmutables:

```
// ¡Una única línea sustituye a 50 líneas de código tradicional!
public record CoordenadaGPS(double latitud, double longitud, String descripcion) {

    // Constructor compacto opcional para comprobación de invariantes y precondiciones
    public CoordenadaGPS {
        if (latitud < -90.0 || latitud > 90.0) {
            throw new IllegalArgumentException("Latitud fuera de rango [-90, 90]: " + latitu
d);
        }
        if (longitud < -180.0 || longitud > 180.0) {
            throw new IllegalArgumentException("Longitud fuera de rango [-180, 180]: " + lon
gitud);
        }
        if (descripcion == null) {
            descripcion = "Sin descripción";
        }
    }
}
```

¿Qué Genera Automáticamente la JVM para un `record`?

1. Una clase `final` inmutable que hereda de `java.lang.Record`.
2. Todos los campos declarados como componentes `private final`.
3. Un constructor canónico con todos los componentes.
4. Métodos accesores con el **mismo nombre que el componente, sin prefijo** `get` (por ejemplo: `punto.latitud()`, `punto.longitud()`).
5. Implementaciones canónicas perfectas de `equals()`, `hashCode()` y `toString()`.

7.3 El Patrón de Diseño Singleton (Inicialización Temprana vs. Perezosa)

El **Singleton** es un patrón de diseño creacional cuyo objetivo es garantizar que una clase tenga **una única instancia viva en toda la aplicación**, proporcionando un punto de acceso global a ella (por ejemplo, un gestor de configuración, un registro de logs o una caché central).

Se basa siempre en tres pilares:

1. **Constructor** `private` para impedir que nadie haga `new` desde el exterior.
2. **Atributo** `private static` del mismo tipo de la clase que guarda la referencia al objeto único.
3. **Método** `public static getInstancia()` que devuelve siempre ese único objeto.

Existen tres formas clásicas de implementarlo según cuándo y cómo se crea la instancia:

Variante A: Inicialización Perezosa (*Lazy Initialization*)

El objeto único no se crea al arrancar el programa, sino **solo en el instante en que alguien invoca** `getInstancia()` **por primera vez**:

```
public class Configuracion {
    // 1. Referencia estática inicialmente nula
    private static Configuracion instancia;
    private String idioma;

    // 2. Constructor privado: prohibido hacer 'new Configuracion()' desde fuera
    private Configuracion() {
        this.idioma = "es";
    }

    // 3. Método de acceso global: crea el objeto solo en la primera llamada
    public static Configuracion getInstancia() {
        if (instancia == null) {
            instancia = new Configuracion();
        }
        return instancia;
    }

    public String getIdioma() { return this.idioma; }
    public void setIdioma(String idioma) { this.idioma = idioma; }
}
```

Comprobación de identidad única:

```
Configuracion c1 = Configuracion.getInstancia();
Configuracion c2 = Configuracion.getInstancia();

c1.setIdioma("en");
System.out.println(c2.getIdioma()); // Imprime: "en"
System.out.println(c1 == c2);       // Imprime: true (son exactamente el mismo objeto en el
Heap)
```

Variante B: Inicialización Temprana (*Eager Initialization*) y Seguridad Multihilo

En aplicaciones con múltiples hilos (*threads*), la inicialización perezosa básica puede fallar si dos hilos entran a la vez en `if (instancia == null)`. Para garantizar seguridad total sin bloqueos:

- Podemos añadir `synchronized` a la cabecera (`public static synchronized Configuracion getInstancia()`), o bien:
- Crear la instancia directamente en una constante `private static final` (**Eager Initialization**) o mediante una clase interna estática (*Holder Pattern*), donde la propia JVM garantiza por diseño la creación atómica de una sola instancia:

```
public class ConfiguracionSistema {
    // Instancia única creada de forma segura al cargar la clase (Eager Initialization)
    private static final ConfiguracionSistema INSTANCIA = new ConfiguracionSistema();

    private String urlServidor = "https://api.empresa.es";

    private ConfiguracionSistema() {}

    public static ConfiguracionSistema getInstancia() {
        return INSTANCIA;
    }

    public String getUrlServidor() { return this.urlServidor; }
}
```

7.4 Enumeraciones (`enum`): De las Constantes Simbólicas a Objetos de Pleno Derecho

Imagina una empresa de alquiler de vehículos donde alquilar un coche cuesta `100.0 €` y una moto `50.0 €`.

- Si codificamos los tipos con números enteros (`1` para coche, `2` para moto) o constantes sueltas (`public static final int COCHE = 1;`), cualquier método `getPrecioAlquiler(int tipo)` nos obliga a validar manualmente que nadie pase un `99` o un `-4`.
- En Java, una **enumeración (`enum`) es una clase completa** donde cada constante declarada es, internamente, una instancia `public static final` de esa clase:

```
// Definir enum Vehiculo { COCHE, MOTO } equivale conceptualmente a:
// class Vehiculo {
//     public static final Vehiculo COCHE = new Vehiculo();
//     public static final Vehiculo MOTO  = new Vehiculo();
// }
```

Al ser una clase real, un `enum` puede tener **atributos `private final`, constructores privados y métodos getters o de negocio**:

```
public enum Vehiculo {
    // Al declarar cada constante se invoca implícitamente al constructor privado
    COCHE(100.0, "Turismo de 5 plazas"),
    MOTO(50.0, "Motocicleta urbana"),
    FURGONETA(140.0, "Vehículo industrial de carga");

    private final double precioDia;
    private final String descripcion;

    // El constructor de un enum es SIEMPRE privado
    Vehiculo(double precioDia, String descripcion) {
        this.precioDia = precioDia;
        this.descripcion = descripcion;
    }

    public double getPrecioDia() { return this.precioDia; }
    public String getDescripcion() { return this.descripcion; }
}
```

Métodos Nativos de todo `enum`:

- `Vehiculo.values()`: Devuelve un array `Vehiculo[]` con todas las constantes definidas.
- `Vehiculo.valueOf("COCHE")`: Devuelve la constante `Vehiculo.COCHE` a partir de su texto (o lanza `IllegalArgumentException` si no existe).
- `v.name()`: Devuelve el nombre exacto de la constante (`"COCHE"`).
- `v.ordinal()`: Devuelve el índice entero de posición (`0` para `COCHE`, `1` para `MOTO`, `2` para `FURGONETA`).

7.5 Anotaciones en Java (`@interface`): Metadatos para el Compilador y Tiempo de Ejecución

Las **anotaciones** son metadatos (como un *post-it* informático prefijado con `@`) que se incrustan sobre clases, métodos, atributos o parámetros. No alteran por sí mismas el flujo de instrucciones, pero proporcionan información vital al compilador, a herramientas de generación de código o al propio programa en tiempo de ejecución (vía *Reflection*).

Las 9 Anotaciones Estándar del Lenguaje Java:

1. **Anotaciones para el compilador (`java.lang`):**
   - `@Override`: Verifica en compilación que el método sobrescribe realmente a uno de su superclase o interfaz.
   - `@Deprecated`: Marca un método o clase como obsoleto para desaconsejar su uso (el IDE lo muestra tachado).
   - `@SuppressWarnings("...")`: Indica al compilador que silencie advertencias específicas conocidas.
   - `@FunctionalInterface`: Garantiza que una interfaz tenga exactamente un único método abstracto (para expresiones lambda).
   - `@SafeVarargs`: Suprime advertencias de seguridad en métodos con argumentos variables genéricos (`T...`).
2. **Meta-anotaciones (`java.lang.annotation` — sirven para configurar nuevas anotaciones):**
   - `@Target`: Define dónde se puede colocar la anotación (`ElementType.TYPE`, `FIELD`, `METHOD`, `PARAMETER`...).
   - `@Retention`: Define hasta qué fase sobrevive la anotación (`SOURCE`, `CLASS` o `RUNTIME` para leerla en ejecución).
   - `@Documented` e `@Inherited`: Incluyen la anotación en Javadoc y permiten que las subclases la hereden.

Creación y Procesamiento de una Anotación Propia con la API Reflection:

Cuando trabajes en el futuro con frameworks profesionales como *Spring Boot*, *Hibernate/JPA* o *JUnit*, verás anotaciones como `@Entity`, `@Column` o `@Test`. Así es como funcionan por dentro:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;

// 1. Definimos nuestra propia anotación @Imprimible
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME) // Disponible en tiempo de ejecución para Reflection
@interface Imprimible {
    boolean mayusculas() default false;
}

// 2. Aplicamos la anotación a los atributos de una clase
class LibroAnotado {
    @Imprimible(mayusculas = false)
    String titulo;

    @Imprimible(mayusculas = true)
    String autor;

    public LibroAnotado(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
    }
}

// 3. Procesamos los metadatos dinámicamente usando Reflection
public class ProcesadorAnotaciones {
    public static void imprimirCamposAnotados(Object objeto) throws IllegalAccessException {
        for (Field campo : objeto.getClass().getDeclaredFields()) {
            Imprimible anotacion = campo.getAnnotation(Imprimible.class);
            if (anotacion != null) {
                String valor = String.valueOf(campo.get(objeto));
                System.out.println(anotacion.mayusculas() ? valor.toUpperCase() : valor);
            }
        }
    }
}
```

---

8. Relaciones Básicas entre Objetos: Agregación Simple y Arrays de Objetos

Hasta ahora hemos creado clases individuales que contienen atributos de tipos primitivos (`int`, `double`, `boolean`) o tipos inmutables como `String`. No obstante, el auténtico poder del paradigma orientado a objetos surge cuando **los objetos colaboran, se combinan y se contienen unos a otros** para modelar fielmente la complejidad del mundo real.

8.1 El Principio «Tiene-un» (*Has-a*): Agregación y Composición Simple

En el diseño de software orientado a objetos, una clase puede tener como atributo una referencia a otra clase. Esta relación se conoce genéricamente como **asociación estructural** y responde a la pregunta semántica: **«¿Un objeto A *tiene un* objeto B?»**:

- Un `Vehiculo` *tiene un* `Motor`.

- Un `Alumno` *tiene una* `Direccion`.
- Una `Factura` *tiene un* `Cliente`.

```
public class Motor {
    private String tipoCombustible; // "Gasolina", "Diésel", "Eléctrico"
    private int potenciaCv;
    private boolean encendido;

    public Motor(String tipoCombustible, int potenciaCv) {
        this.tipoCombustible = tipoCombustible;
        this.potenciaCv = potenciaCv;
        this.encendido = false;
    }

    public void encender() { this.encendido = true; }
    public void apagar() { this.encendido = false; }
    public boolean isEncendido() { return encendido; }
    public int getPotenciaCv() { return potenciaCv; }
}

public class Vehiculo {
    private String matricula;
    private String marca;
    private Motor motor; // ATRIBUTO DE TIPO OBJETO: Agregación simple ("Tiene-un")

    public Vehiculo(String matricula, String marca, Motor motor) {
        if (matricula == null || motor == null) {
            throw new IllegalArgumentException("Matrícula y motor son obligatorios");
        }
        this.matricula = matricula;
        this.marca = marca;
        this.motor = motor; // Conecta el motor al vehículo
    }

    // El vehículo delega en su motor interno las operaciones mecánicas
    public void arrancar() {
        System.out.println("Poniendo en marcha el vehículo " + marca + "...");
        motor.encender();
    }

    public void detener() {
        System.out.println("Deteniendo el vehículo " + marca + "...");
        motor.apagar();
    }
}
```

Delegación de Responsabilidades

Fíjate en cómo `Vehiculo` no manipula directamente cables ni bujías: simplemente invoca `motor.encender()`. Esta técnica se denomina **delegación** y es la base de un código modular, donde cada clase es experta únicamente en su propia parcela de responsabilidad.

8.2 Gestión de Múltiples Objetos: Arrays de Objetos como Atributo de Clase

Antes de estudiar las estructuras dinámicas de la biblioteca de colecciones (`ArrayList`, `HashSet`) en la Unidad 6, en el desarrollo profesional con Java es vital aprender a modelar **clases contenedoras de dominio** que gestionan internamente una colección de objetos mediante un array convencional:

Imagina que necesitamos modelar un `Catalogo` que almacene hasta un máximo de `N` libros:

```
public class CatalogoLibros {
    private Libro[] libros;     // Array que almacena las referencias a los objetos
    private int totalLibros;    // Contador de cuántos libros reales hay almacenados

    public CatalogoLibros(int capacidadMaxima) {
        if (capacidadMaxima <= 0) {
            throw new IllegalArgumentException("La capacidad debe ser mayor que cero");
        }
        this.libros = new Libro[capacidadMaxima]; // Reserva el vector en el Heap
        this.totalLibros = 0;
    }

    // 1. ALTA SEGURA DE UN NUEVO OBJETO
    public boolean agregarLibro(Libro nuevoLibro) {
        if (nuevoLibro == null) {
            throw new IllegalArgumentException("No se puede agregar un libro nulo");
        }
        if (totalLibros >= libros.length) {
            return false; // Catálogo lleno
        }
        libros[totalLibros] = nuevoLibro;
        totalLibros++;
        return true;
    }

    // 2. BÚSQUEDA POR CRITERIO (ej. por título)
    public Libro buscarPorTitulo(String tituloBuscado) {
        if (tituloBuscado == null) return null;
        for (int i = 0; i < totalLibros; i++) {
            if (libros[i].getTitulo().equalsIgnoreCase(tituloBuscado)) {
                return libros[i]; // Retorna la referencia encontrada
            }
        }
        return null; // No encontrado
    }

    // 3. ELIMINACIÓN Y COMPACTACIÓN DEL ARRAY
    public boolean eliminarPorTitulo(String tituloBuscado) {
        for (int i = 0; i < totalLibros; i++) {
            if (libros[i].getTitulo().equalsIgnoreCase(tituloBuscado)) {
                // Desplaza todos los libros siguientes un hueco a la izquierda para no deja
r huecos nulos
                for (int j = i; j < totalLibros - 1; j++) {
                    libros[j] = libros[j + 1];
                }
                libros[totalLibros - 1] = null; // Limpia la última referencia para el Garba
ge Collector
                totalLibros--;
                return true;
            }
        }
        return false;
    }

    public int getTotalLibros() { return totalLibros; }
    public int getCapacidadMaxima() { return libros.length; }
}
```

8.3 Métodos Factoría Estáticos (*Static Factory Methods*)

Los **métodos factoría estáticos** son métodos `public static` que instancian y devuelven un objeto de su propia clase. Representan una alternativa de diseño muy superior a los constructores sobrecargados en diversas situaciones:

1. **Tienen nombres descriptivos:** Mientras que todos los constructores deben llamarse forzosamente igual que la clase, un método factoría explica claramente *cómo* se crea el objeto (ej. `Punto2D.desdeCartesianas(x, y)` frente a `Punto2D.desdePolares(radio, angulo)`).
2. **Evitan colisiones de firma:** Si dos formas de instanciar un objeto requieren los mismos tipos de parámetros (por ejemplo dos `double`), no es posible crear dos constructores distintos. La factoría estática resuelve esto con elegancia.
3. **No están obligados a crear una nueva instancia cada vez:** Pueden devolver instancias inmutables compartidas o precalculadas en caché (como hace `Integer.valueOf(int)` o `Boolean.valueOf(boolean)`).

```
public final class ColorRGB {
    private final int rojo;
    private final int verde;
    private final int azul;

    // Constantes predefinidas compartidas
    public static final ColorRGB ROJO = new ColorRGB(255, 0, 0);
    public static final ColorRGB VERDE = new ColorRGB(0, 255, 0);
    public static final ColorRGB AZUL = new ColorRGB(0, 0, 255);
    public static final ColorRGB BLANCO = new ColorRGB(255, 255, 255);
    public static final ColorRGB NEGRO = new ColorRGB(0, 0, 0);

    // Constructor privado: NADIE desde fuera puede hacer 'new ColorRGB(...)'
    private ColorRGB(int rojo, int verde, int azul) {
        this.rojo = rojo;
        this.verde = verde;
        this.azul = azul;
    }

    // FACTORÍA ESTÁTICA 1: Desde valores numéricos [0-255]
    public static ColorRGB fromRGB(int r, int g, int b) {
        if (r < 0 || r > 255 || g < 0 || g > 255 || b < 0 || b > 255) {
            throw new IllegalArgumentException("Componentes RGB deben estar entre 0 y 255");
        }
        return new ColorRGB(r, g, b);
    }

    // FACTORÍA ESTÁTICA 2: Desde una cadena hexadecimal ("#FF5733")
    public static ColorRGB fromHex(String hex) {
        if (hex == null || !hex.matches("^#?[0-9a-fA-F]{6}$")) {
            throw new IllegalArgumentException("Formato hexadecimal inválido: " + hex);
        }
        String limpio = hex.startsWith("#") ? hex.substring(1) : hex;
        int r = Integer.parseInt(limpio.substring(0, 2), 16);
        int g = Integer.parseInt(limpio.substring(2, 4), 16);
        int b = Integer.parseInt(limpio.substring(4, 6), 16);
        return new ColorRGB(r, g, b);
    }

    public int getRojo() { return rojo; }
    public int getVerde() { return verde; }
    public int getAzul() { return azul; }
}
```

8.4 Aplicación de Expresiones Regulares en la Validación de Objetos

En la Unidad 3 estudiamos en profundidad la sintaxis y el motor de **Expresiones Regulares (*Regex*)** de Java. En la Programación Orientada a Objetos, las expresiones regulares se convierten en la principal línea de defensa para **garantizar las invariantes de los objetos** en sus constructores, setters y `records`:

```
public record Usuario(String username, String email, String nif) {
    // Constructor compacto con validaciones rigurosas mediante Regex aprendidas en UD3
    public Usuario {
        if (username == null || !username.matches("^[a-zA-Z0-9_]{4,15}$")) {
            throw new IllegalArgumentException("Username inválido: debe ser alfanumérico ent
re 4 y 15 caracteres");
        }
        if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
            throw new IllegalArgumentException("Formato de correo electrónico no válido: " +
email);
        }
        if (nif == null || !nif.matches("^\\d{8}[A-HJ-NP-TV-Z]$")) {
            throw new IllegalArgumentException("Formato de DNI español incorrecto: " + nif);
        }
    }
}
```

Gracias a esta integración, ningún objeto `Usuario` podrá crearse en el Heap con un correo corrupto o un identificador ilegítimo, blindando la integridad de todo el sistema.

---

9. Organización Modular: Paquetes e Importaciones

A medida que una aplicación crece y cuenta con decenas o cientos de clases, es indispensable organizarlas en carpetas lógicas para evitar el caos y prevenir colisiones de nombres.

9.1 Concepto de Paquete (`package`)

Un **paquete** en Java cumple una doble función:

1. **Espacio de nombres (*Namespace*):** Permite que existan dos clases con el mismo nombre siempre que pertenezcan a paquetes diferentes (por ejemplo, `java.util.Date` frente a `java.sql.Date`).
2. **Estructura física en disco:** La sentencia de paquete debe coincidir **exactamente con la jerarquía de carpetas en el sistema de archivos**.

Convención Oficial de Nombres de Paquetes:

Se utiliza el **nombre de dominio inverso de la organización o centro educativo** en minúsculas puras, seguido del nombre del proyecto y el subdominio funcional:

```
package es.iesperenxisa.dam.gestionbancaria.modelo;
```

Físicamente en el disco duro, este fichero debe residir en la ruta:
`src/main/java/es/iesperenxisa/dam/gestionbancaria/modelo/CuentaBancaria.java`

9.2 Sentencias `import` e Importaciones Estáticas

Para utilizar una clase situada en otro paquete, debemos indicárselo al compilador mediante la instrucción **`import`**:

```
import java.util.Scanner;        // Importación explícita (Recomendada)
import java.time.LocalDate;

// Importación con comodín (importa todas las clases públicas del paquete, pero no sus subpa
quetes)
import java.util.*;

// Importación estática: permite invocar métodos y constantes estáticas directamente sin ant
eponer la clase
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

public class CalculoGeometria {
    public double calcularAreaCirculo(double r) {
        return PI * r * r; // Se usa PI directamente en vez de Math.PI
    }
}
```

> **El Paquete Implícito java.lang**
> Todas las clases del paquete fundamental **`java.lang`** (como `String`, `Math`, `System`, `Integer`, `Double`, `Object` o `Exception`) son importadas de forma completamente automática e invisible por el compilador en todos los ficheros de Java. Nunca necesitas escribir `import java.lang.String;`.

---

10. ⭐ Be the Code: Desafíos de Traza y Memoria

Un programador que domina la POO debe visualizar instantáneamente qué ocurre en la memoria Stack y Heap ante cada sentencia. Resuelve mentalmente los siguientes tres retos.

Desafío 1: Comparaciones de Identidad vs. Contenido y Paso por Referencia

Analiza minuciosamente el siguiente código y deduce qué imprime exactamente en la consola:

```
public class RetoPOO1 {
    static class Mensaje {
        String texto;
        Mensaje(String t) { this.texto = t; }
    }

    public static void alterar(Mensaje m1, Mensaje m2) {
        m1.texto = "Modificado";
        m2 = new Mensaje("Nuevo");
        m2.texto = "Cambiado";
    }

    public static void main(String[] args) {
        Mensaje a = new Mensaje("Original A");
        Mensaje b = new Mensaje("Original B");
        Mensaje c = a;

        alterar(a, b);

        System.out.println("1: " + a.texto);
        System.out.println("2: " + b.texto);
        System.out.println("3: " + (a == c));
        System.out.println("4: " + a.equals(c));
    }
}
```

▶ 🔍 Ver Resolución y Traza de Memoria

Desafío 2: Encadenamiento de Constructores y Atributos Estáticos

¿Cuál es la salida producida en la consola tras ejecutar este programa?

```
public class RetoPOO2 {
    static class ContadorInstancias {
        static int total = 0;
        int id;
        String nombre;

        ContadorInstancias(String n) {
            this(n, ++total);
            System.out.print("C1(" + this.nombre + ") ");
        }

        ContadorInstancias(String n, int i) {
            this.nombre = n;
            this.id = i;
            System.out.print("C2(" + this.id + ") ");
        }
    }

    public static void main(String[] args) {
        ContadorInstancias obj1 = new ContadorInstancias("Alfa");
        ContadorInstancias obj2 = new ContadorInstancias("Beta", 99);
        System.out.println("-> Total: " + ContadorInstancias.total);
    }
}
```

▶ 🔍 Ver Resolución y Traza

Desafío 3: La Trampa de la Mutabilidad Encubierta

Observa la siguiente clase y determina qué imprimirá la última línea:

```
public class RetoPOO3 {
    static class BovedaSegura {
        private final int[] claves;

        public BovedaSegura(int[] clavesEntrada) {
            this.claves = clavesEntrada; // ¡Alerta de diseño!
        }

        public int[] getClaves() {
            return this.claves; // ¡Alerta de diseño!
        }
    }

    public static void main(String[] args) {
        int[] misClaves = {1234, 5678};
        BovedaSegura boveda = new BovedaSegura(misClaves);

        misClaves[0] = 9999;
        boveda.getClaves()[1] = 0000;

        System.out.println(java.util.Arrays.toString(boveda.getClaves()));
    }
}
```

▶ 🔍 Ver Resolución y Explicación de Fuga de Referencia

---

11. ¡No Hay Preguntas Tontas!

---

1. ¿Por qué el constructor tiene que llamarse exactamente igual que la clase y no tiene tipo de retorno?

Porque el constructor no es una función ordinaria que calcula y devuelve un valor, sino una rutina especializada de inicialización de memoria gestionada directamente por el operador `new`. La JVM necesita una convención inequívoca para distinguir la creación de un objeto de una llamada a método. Si tuviera tipo de retorno (incluso `void`), el compilador creería que es un método cualquiera que casualmente comparte el nombre con la clase.

---

2. ¿Por qué no dejamos todos los atributos públicos y nos ahorramos escribir getters y setters?

Si dejas los atributos `public`, cualquier parte del programa puede asignar valores destructivos o ilegales en cualquier momento (como `cuenta.saldo = -999999;` o `alumno.nota = 450.0;`) sin que la clase pueda impedirlo ni enterarse. Al hacerlos `private` y usar `getters` y `setters`, el objeto mantiene el **control absoluto sobre sus invariantes**: puede validar datos, lanzar excepciones defensivas, calcular campos al vuelo o mantener auditorías de quién modificó la información.

---

3. Si pongo `final` en una variable objeto (`final Cuenta c = new Cuenta()`), ¿significa que su saldo no puede cambiar?

No. Este es uno de los equívocos más comunes. La palabra `final` en una variable de referencia significa que **la referencia (la dirección de memoria)** no puede cambiar jamás: nunca podrás reasignar `c = new Cuenta()`. Sin embargo, el objeto vivo en el Heap al que apunta `c` sigue siendo completamente mutable: puedes llamar a `c.ingresar(500)` y alterar su saldo libremente. Para que el estado interno sea inmutable, deben ser los atributos internos de la clase los que se declaren como `final`.

---

4. ¿Por qué `c1.equals(c2)` devuelve `false` por defecto si no lo sobrescribo?

Porque la implementación original de `equals()` en la clase raíz `java.lang.Object` se define literalmente como `return (this == obj);`. Es decir, si no sobrescribes el método en tu clase, `equals()` se comporta con la misma rigidez que el operador `==`, comparando únicamente si ambos objetos ocupan la misma dirección física en el Heap.

---

5. ¿Cuál es la diferencia conceptual entre una variable de instancia y una variable `static`?

- **Variable de instancia:** Pertenece a **un objeto concreto**. Si creas 100 objetos `CuentaBancaria`, habrá 100 variables `saldo` distintas en el Heap, cada una con el dinero de su titular.
- **Variable `static`:** Pertenece a **la clase global**. Existe una única celda de memoria en toda la JVM compartida por los 100 objetos (por ejemplo, el contador de cuántas cuentas existen en el banco).

---

6. ¿Por qué dentro de un método `static` no puedo utilizar la palabra reservada `this`?

Porque un método estático se ejecuta a nivel de clase (`NombreClase.metodo()`), con independencia de que existan o no objetos instanciados en el Heap. La palabra `this` representa la dirección del objeto que está ejecutando la llamada; en un contexto estático, no hay ningún objeto en ejecución, por lo que `this` carece por completo de significado físico.

---

7. ¿Qué es exactamente un `record` de Java y cuándo debo usarlo en lugar de una clase normal?

Un `record` es una clase especializada inmutable pensada para actuar como transportador transparente de datos. Debes usar un `record` cuando necesites agrupar un conjunto fijo de datos de solo lectura (como una coordenada, una fila leída de base de datos, un par clave-valor o un resultado de cálculo). Debes seguir usando una `class` ordinaria cuando tu entidad requiera estado mutable, herencia jerárquica con `extends` o encapsulamiento con lógica de negocio compleja.

---

8. ¿Por qué en una enumeración (`enum`) el constructor es obligatoriamente privado?

Porque el propósito de un `enum` es definir un conjunto exhaustivo y cerrado de instancias fijas conocidas en tiempo de compilación (`LUNES`, `MARTES`, etc.). Si el constructor fuera público, cualquier parte del programa podría ejecutar `new DiaSemana()` en tiempo de ejecución, creando días de la semana inexistentes y destruyendo la garantía de conjunto finito y seguro del tipo.

---

12. Resumen de la Unidad y Conexión con la Unidad 5

En esta Unidad 4 hemos dado el paso decisivo hacia la ingeniería del software orientada a objetos:

```
+-------------------------------------------------------------------------+
|                  NÚCLEO DE LA PROGRAMACIÓN ORIENTADA A OBJETOS          |
+-------------------+--------------------+------------------+-------------+
|  CLASES Y OBJETOS |  ENCAPSULAMIENTO   | MIEMBROS STATIC  | CLASE OBJECT|
| Molde en código   | Atributos private, | Variables de     | toString,   |
| vs instancias en  | getters/setters con| clase, constantes| equals y    |
| Heap con estado,  | validación, final  | globales y       | hashCode    |
| conducta y this.  | y records.         | utilidades.      | canónicos.  |
+-------------------+--------------------+------------------+-------------+
```

El Salto Hacia la Unidad 5: POO Avanzada

Ahora que sabemos crear clases robustas y encapsuladas, en la **Unidad 5: Programación orientada a objetos avanzada** exploraremos los otros dos grandes pilares de la POO:

- **Herencia (`extends`):** Cómo estructurar jerarquías de clases compartiendo atributos y métodos comunes mediante superclases y subclases.
- **Polimorfismo:** Cómo tratar a distintos objetos especializados bajo una misma categoría general.
- **Clases Abstractas e Interfaces (`interface` / `implements`):** El diseño formal de contratos y desacoplamiento en Java moderno.

---

13. Relación Curricular Oficial

Esta unidad cubre exhaustivamente los contenidos y criterios de evaluación prescritos en el currículo oficial del ciclo formativo **CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)** para el módulo **0485: Programación**:

- **RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
  - *CE a, d, f:* Identificación de bloques de clases, uso de operadores y comentarios de documentación Javadoc.
- **RA2. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.**
  - *CE a, b, c, d, e, f, g, h, i, j:* Fundamentos de POO, instanciación en el Heap, invocación de métodos y propiedades, uso de miembros `static`, constructores y depuración con IntelliJ IDEA.
- **RA3. Escribe y optimiza código estructurado aplicando los principios de la programación modular.**
  - *CE a, b, c, d:* Modularización en clases, paso de parámetros de objetos por valor en el Heap, sobrecarga de constructores y métodos, y recursividad.
- **RA4. Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.**
  - *CE a, b, c, d, f:* Sintaxis y estructura de clases, definición de atributos y métodos, creación de constructores canónicos y sobrecargados, control de visibilidad (`private` , `public` , `protected` , package-private) e instanciación de objetos en aplicaciones integradas.

---

14. Boletines de Ejercicios y Retos Prácticos

Para afianzar y llevar a la práctica las habilidades de diseño y codificación orientadas a objetos, dispones de una batería completa y unificada de 5 boletines:

Recurso	Nivel / Enfoque	Descripción
📝 **Boletín 04: Nivel Inicial**	Básico / Consolidación	16 ejercicios progresivos: creación de clases (`Libro` , `Rectangulo` , `Punto2D` , `Empleado`), atributos `private` vs `public` , referencias compartidas, `this(...)` , `static` y sobrecarga.
✅ **Boletín 04: Inicial Resuelto**	Soluciones Comentadas	Soluciones íntegras y razonadas con código Java 25 LTS, trazas de memoria y validaciones.
🚀 **Boletín 04: Nivel Intermedio**	Intermedio / Aplicado	16 ejercicios de arquitectura POO: `toString` / `equals` / `hashCode` , clases inmutables (`Coordenada` , `AlumnoInmutable`), `record` , Singleton (`ContadorGlobal` , `GestorConfiguracion`), `Racional` , arrays de objetos (`Circulo`), `StringBuilder` y Regex.
✅ **Boletín 04: Intermedio Resuelto**	Soluciones Comentadas	Soluciones completas con demostración de contratos de igualdad, copias defensivas y patrones de diseño.
🏆 **Boletín 04: Retos Orientados a Objetos (Extras)**	Avanzado / Arquitectura	8 grandes retos: `Fraccion` con MCD recursivo, `Mazo` de 40 cartas con Fisher-Yates, `ConjuntoEnteros` con varargs, clínica dietista con ficheros, máquina expendedora y referencias a CodeWars y Acepta el Reto.

DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.1: Nivel Inicial — Ejercicios Prácticos

Ejercicios fundamentales de clases, atributos privados, constructores, getters y setters, la referencia this y miembros estáticos en Java.

Bienvenido al boletín de nivel inicial de la **Unidad 4: Clases y objetos**.

Estos 16 ejercicios están diseñados para asentar los conceptos fundamentales de la Programación Orientada a Objetos (POO): definición de clases, encapsulación de atributos con visibilidad `private`, métodos accesores y mutadores con validaciones de invariantes, constructores canónicos y sobrecargados con delegación `this(...)`, paso de referencias en el Heap y uso del modificador `static`.

> **Consejo de Desarrollo con IntelliJ IDEA**
> Crea un proyecto en IntelliJ IDEA y coloca cada clase en su propio archivo `.java`. Practica la generación automática de constructores, getters y setters mediante el atajo `Alt + Insert` (o clic derecho → *Generate*...).

Ejercicio 1: La Clase `Libro`, Delegación con `this(...)` y Encapsulamiento Básico

Enunciado:
Diseña una clase llamada `Libro` que contenga los atributos privados: `titulo` (`String`), `autor` (`String`) y `numeroPaginas` (`int`).

- Define un constructor canónico que inicialice los tres atributos, validando que el título y el autor no sean nulos ni cadenas vacías, y que el número de páginas sea mayor o igual a 0 (lanzando `IllegalArgumentException` si no se cumplen).
- Añade un **constructor sin parámetros** que delegue en el constructor principal utilizando `this("Sin título", "Desconocido", 0)`.
- Implementa todos los métodos getters y setters con las mismas validaciones.
- Añade un método `public boolean esLargo()` que devuelva `true` si el libro tiene más de 300 páginas y `false` en caso contrario.
- En una clase `Main`, crea tres objetos `Libro`: `"Ficciones"` de `"Borges"` (200 páginas), `"1984"` de `"Orwell"` (328 páginas) y un tercer libro usando el constructor sin parámetros. Muestra por consola sus datos usando los getters, comprueba cuáles son largos con `esLargo()`, modifica el número de páginas de uno de ellos con su setter y vuelve a mostrarlo.

Ejercicio 2: Geometría y Contador Estático de Instancias con `Rectangulo`

Enunciado:
Crea una clase `Rectangulo` con dos atributos de instancia privados: `base` y `altura` (ambos de tipo `double`), y un atributo de clase privado `private static int numeroDeRectangulos = 0;`.

- El constructor debe rechazar valores menores o iguales a cero e incrementar `numeroDeRectangulos` cada vez que se instancie un rectángulo válido.

- Implementa los métodos `public double calcularArea()` (`base × altura`) y `public double calcularPerimetro()` (`2 × (base + altura)`).
- Implementa un método `public boolean esCuadrado()` que determine si la base y la altura son idénticas.
- Añade un método `public void escalar(double factor)` que multiplique ambas dimensiones por un factor positivo.
- Añade un método estático `public static int getNumeroDeRectangulos()` que devuelva cuántos rectángulos se han creado en total. Demuestra su funcionamiento en `main` creando 3 rectángulos y mostrando el contador global.

Ejercicio 3: La Clase `Circulo` y Constantes Matemáticas

Enunciado:
Diseña una clase `Circulo` con un atributo privado `radio` (double).

- El constructor debe asegurar que el radio sea positivo.
- Implementa `calcularArea()` y `calcularPerimetro()` haciendo uso de la constante estática oficial `Math.PI`.
- Proporciona un constructor por defecto sin parámetros que asigne un radio de `1.0` delegando en el constructor principal mediante `this(1.0)`.

Ejercicio 4: Conversor Térmico con la Clase `Termometro`

Enunciado:
Crea una clase `Termometro` que almacene internamente la temperatura en grados Celsius (`temperaturaCelsius` de tipo `double`).

- El constructor debe verificar que la temperatura no sea inferior al cero absoluto (-273.15 °C).
- Añade métodos para consultar la temperatura en escala Fahrenheit: `public double getFahrenheit()` y en escala Kelvin: `public double getKelvin()`.
- Añade métodos `incrementar(double delta)` y `decrementar(double delta)`.

Ejercicio 5: Coordenadas Cartesianas con la Clase `Punto2D`

Enunciado:
Modela un punto en el plano bidimensional con la clase `Punto2D`:

- Atributos privados: `x` e `y` (double).
- Constructor por defecto que sitúe el punto en el origen `(0.0, 0.0)` mediante `this(0.0, 0.0)`.
- Constructor parametrizado `public Punto2D(double x, double y)`.
- Método `public void desplazar(double dx, double dy)` que traslade las coordenadas.
- Método `public double distanciaAlOrigen()` que devuelva la distancia euclídea respecto al `(0,0)` (`√(x² + y²)`).

Ejercicio 6: La Máquina Cafetera

Enunciado:
Diseña una clase `Cafetera` para gestionar el servicio de café en una oficina:

- Atributos privados: `capacidadMaxima` (c.c. de café, por defecto 1000) y `cantidadActual` (c.c. de café que contiene).
- Constructor predeterminado: capacidad máxima 1000 c.c. y cafetera vacía (0 c.c.).
- Constructor con capacidad máxima dada (inicialmente vacía).
- Constructor con capacidad máxima y cantidad actual (si la cantidad actual supera la máxima, se ajusta al máximo).
- Métodos:
  - `llenarCafetera()`: llena la cafetera a su máxima capacidad.
  - `servirTaza(int capacidadTaza)`: descuenta el café servido. Si no hay suficiente para llenar la taza, sirve lo que quede y deja la cafetera a 0. Devuelve la cantidad real servida.
  - `vaciarCafetera()`: pone la cantidad actual a 0.
  - `agregarCafe(int cantidad)`: añade café sin sobrepasar la capacidad máxima.

Ejercicio 7: Cronómetro Simple con la Clase `Reloj`

Enunciado:
Modela la clase `Reloj` con tres atributos enteros privados: `hora` (0-23), `minuto` (0-59) y `segundo` (0-59).

- Constructor canónico con validación rigurosa de rangos.
- Constructor sin argumentos que inicialice el reloj a las `00:00:00`.
- Método `public void ticTac()` que adelante el reloj exactamente un segundo, gestionando adecuadamente los cambios de minuto, hora y el paso de medianoche (`23:59:59` → `00:00:00`).
- Método `public String getHoraFormateada()` que devuelva la hora en formato `HH:mm:ss` con dos dígitos por campo (p. ej. `"09:05:03"`).

Ejercicio 8: Identificadores Autoincrementales y Sueldo Protegido con `Empleado`

Enunciado:
Diseña una clase `Empleado` con:

- Un atributo estático privado `contadorSecuencial` inicializado en 1000 y un `private static String passwordMaestra = "RRHH2025";` común a toda la empresa.
- Un atributo de instancia inmutable `final int idEmpleado`.
- Atributos de instancia privados: `nombre` (String obligatorio), `edad` (int, debe ser siempre `>= 18`), `departamento` (String) y `sueldo` (double).
- Cada vez que se cree un nuevo empleado, el constructor debe asignarle automáticamente `++contadorSecuencial` y validar que el nombre no esté vacío y la edad sea `>= 18`.
- El sueldo solo podrá consultarse o modificarse mediante los métodos `getSueldo(String password)` y `setSueldo(double nuevoSueldo, String password)`, que comprueban primero si la contraseña coincide con `passwordMaestra` (lanzando `SecurityException` en caso contrario).

Ejercicio 9: La Clase `Contador` y Encadenamiento con `this`

Enunciado:
Implementa una clase `Contador` con un atributo entero privado `valor`.

- Constructor sin parámetros: inicializa en 0.

- Constructor con valor inicial.
- Constructor de copia: `public Contador(Contador otro)` que replique el valor de otra instancia.
- Métodos `incrementar()`, `decrementar()` e `incrementar(int cantidad)`.
- Implementa los métodos devolviendo la referencia `this` para posibilitar llamadas fluidas encadenadas: `contador.incrementar().incrementar(5).decrementar();`

Ejercicio 10: Array de Objetos Alumnos

Enunciado:
Utilizando la clase `Alumno` (con `nombre` y `nota` entre 0.0 y 10.0):

- En el método `main`, declara e inicializa un array de 5 alumnos con nombres y calificaciones distintas.
- Diseña un método estático `public static double calcularNotaMedia(Alumno[] lista)` que calcule la media del grupo.
- Diseña un método estático `public static Alumno obtenerMejorAlumno(Alumno[] lista)` que localice y devuelva la referencia al alumno con la calificación más alta.

Ejercicio 11: Control de Vehículo con la Clase `Coche`

Enunciado:
Modela una clase `Coche` con los atributos: `marca`, `modelo`, `motorEncendido` (boolean) y `velocidadActual` (int, en km/h).

- El constructor recibe marca y modelo; el coche arranca apagado y con velocidad 0.
- Métodos:
  - `arrancar()`: enciende el motor. No se puede arrancar si ya está encendido.
  - `apagar()`: apaga el motor, solo si la velocidad es 0 (no se puede apagar en movimiento).
  - `acelerar(int incremento)`: incrementa la velocidad si el motor está encendido, hasta un máximo de 200 km/h.
  - `frenar(int decremento)`: reduce la velocidad sin bajar de 0 km/h.

Ejercicio 12: Generador y Validador de Contraseñas

Enunciado:
Crea la clase `Password`:

- Atributos: `longitud` (por defecto 8) y `valor` (String con la contraseña).
- Constructor por defecto: genera una contraseña aleatoria de 8 caracteres.
- Constructor con longitud personalizada: genera una contraseña con la longitud indicada.
- Método `public boolean esFuerte()`: devuelve `true` si la contraseña contiene al menos 2 mayúsculas, 1 minúscula y 2 dígitos numéricos.
- Método `public void regenerar()`: genera un nuevo valor aleatorio manteniendo la longitud actual.

Ejercicio 13: Visibilidad `private` vs. `public` y Auditoría de Código con Errores

Enunciado:
Responde de forma razonada y corrige los siguientes casos prácticos:

1. **Cuestión conceptual:** ¿Qué diferencia existe entre declarar un atributo como `private` y declararlo como `public`? ¿Por qué en Programación Orientada a Objetos se recomienda declarar siempre los atributos como `private` y exponer getters/setters?
2. **Auditoría 1 (Contexto estático inválido):** Un compañero propone añadir a la clase `Empleado` el método `public static String getNombre() { return nombre; }` argumentando que «hace lo mismo para todos los objetos». Explica por qué no compila y por qué es conceptualmente erróneo.
3. **Auditoría 2 (Caza de errores en `Punto2D`):** Analiza el siguiente código defectuoso, enumera todos los fallos léxicos, sintácticos y de diseño que contiene (fíjate en el falso constructor `void`, el sombreamiento `x = x` sin `this`, la visibilidad y la impresión) y reescríbelo corregido:

```
Public class empleado {
    public double x, y;
    public void Punto2D(double x, double y) {
        x = x;
        y = y;
    }
}
```

Ejercicio 14: Traza de Referencias Compartidas en el Heap

Enunciado:
Sin ejecutar el programa, deduce qué mostrará exactamente por pantalla el siguiente código y explica paso a paso qué ocurre en la memoria **Stack** y en la memoria **Heap**:

```
Libro l1 = new Libro("Ficciones", "Borges", 200);
Libro l2 = l1;
l2.setTitulo("El Aleph");
System.out.println(l1.getTitulo());
System.out.println(l1 == l2);
```

A continuación, explica cómo tendríamos que crear `l2` si quisiéramos tener un segundo objeto `Libro` completamente independiente en memoria con los mismos datos iniciales que `l1`.

Ejercicio 15: La Clase `FormateadorBanner`: Encapsulamiento y Sobrecarga Orientada a Objetos

Enunciado:
Crea una clase `FormateadorBanner` para generar carteles y textos enmarcados en consola aplicando sobrecarga de métodos de instancia y variables de clase:

- Atributos de instancia privados: `caracterBorde` (`char`, por defecto `'*'`) y `anchoMinimo` (`int`, debe ser como mínimo 10).
- Atributo de clase privado: `private static int totalBannersGenerados = 0;`.
- Constructores:
  - Constructor por defecto que asigne `'*'` y ancho mínimo 30 delegando en el constructor parametrizado mediante `this('*', 30)`.
  - Constructor parametrizado `public FormateadorBanner(char caracterBorde, int anchoMinimo)`.
- Métodos sobrecargados de instancia:

  - `public void imprimirBanner(String texto)`: imprime el texto centrado entre dos líneas horizontales formadas por el `caracterBorde`, asegurando el ancho mínimo e incrementando el contador global `totalBannersGenerados`.
  - `public void imprimirBanner(String texto, char caracterEspecial)`: sobrecarga que utiliza temporalmente un carácter especial para ese cartel concreto, reutilizando la lógica interna.
  - `public void imprimirLineas(String... lineas)`: acepta un número variable de líneas (*varargs*) y las imprime todas dentro de un único recuadro coordinado.
- Método estático `public static int getTotalBannersGenerados()` para consultar el total global de carteles creados por cualquier instancia.
- En `main`, crea dos instancias (una con `'='` y otra por defecto), genera varios carteles y banners multilínea, y comprueba el contador estático.

Ejercicio 16: Modelado de Objetos con Estado: Las Clases `Dado` y `Cubilete`

Enunciado:
En lugar de funciones procedimentales sueltas, modela un simulador de juegos de azar aplicando Programación Orientada a Objetos:

1. Diseña la clase `Dado`:
   - Atributos privados: `numeroCaras` (entero, por defecto 6, validando que sea `>= 2`), `ultimoValor` (entero) y una instancia de `java.util.Random`.
   - Constructor por defecto (6 caras) y constructor parametrizado `public Dado(int numeroCaras)`.
   - Método `public int lanzar()`: genera un número aleatorio entre `1` y `numeroCaras`, actualiza `ultimoValor` y lo devuelve.
   - Getter `public int getUltimoValor()` y método `toString()` que devuelva `"Dado de [N] caras (última tirada: [V])"`.
2. Diseña la clase `Cubilete`:
   - Atributo privado: un array de dados `private Dado[] dados`.
   - Constructor `public Cubilete(int cantidadDados, int carasPorDado)`: inicializa el array con la cantidad indicada de objetos `Dado`.
   - Método `public int tirarTodos()`: lanza todos los dados del cubilete y devuelve la suma total obtenida.
   - Método `public int tirarHastaSuma(int sumaObjetivo)`: lanza repetidamente el cubilete hasta que la suma coincida con `sumaObjetivo` (validando que el objetivo sea matemáticamente posible), devolviendo cuántas tiradas han sido necesarias.
3. En `main`, crea un cubilete con 2 dados de 6 caras, calcula cuántas tiradas se necesitan para sacar un 12, muestra el estado de cada dado mediante sus métodos y prueba también un dado de 20 caras (`D20`).

> **¿Necesitas consultar las soluciones?**
> Revisa el código completo comentado en el Boletín 4.1 Resuelto.

DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.1: Nivel Inicial (Resuelto) — Soluciones Comentadas

Soluciones completas y comentadas de los 16 ejercicios de nivel inicial de la Unidad 4 en Java 25 LTS.

A continuación se presentan las soluciones comentadas de los 16 ejercicios iniciales de la Unidad 4, implementadas siguiendo los estándares de encapsulación, validación defensiva y buenas prácticas de Java 25 LTS.

Solución Ejercicio 1: La Clase `Libro`, Delegación con `this(...)` y Encapsulamiento Básico

```
public class Libro {
    private String titulo;
    private String autor;
    private int numeroPaginas;

    // Constructor canónico con validación
    public Libro(String titulo, String autor, int numeroPaginas) {
        setTitulo(titulo);
        setAutor(autor);
        setNumeroPaginas(numeroPaginas);
    }

    // Constructor sin parámetros que delega en el constructor principal mediante this(...)
    public Libro() {
        this("Sin título", "Desconocido", 0);
    }

    public String getTitulo() {
        return this.titulo;
    }

    public void setTitulo(String titulo) {
        if (titulo == null || titulo.trim().isEmpty()) {
            throw new IllegalArgumentException("El título del libro no puede ser nulo ni vac
ío.");
        }
        this.titulo = titulo.trim();
    }

    public String getAutor() {
        return this.autor;
    }

    public void setAutor(String autor) {
        if (autor == null || autor.trim().isEmpty()) {
            throw new IllegalArgumentException("El autor del libro no puede ser nulo ni vací
o.");
        }
        this.autor = autor.trim();
    }

    public int getNumeroPaginas() {
        return this.numeroPaginas;
    }

    public void setNumeroPaginas(int numeroPaginas) {
        if (numeroPaginas < 0) {
            throw new IllegalArgumentException("El número de páginas no puede ser negativo:
" + numeroPaginas);
        }
        this.numeroPaginas = numeroPaginas;
    }

    public boolean esLargo() {
        return this.numeroPaginas > 300;
    }
}
```

```
public class MainLibro {
    public static void main(String[] args) {
        Libro l1 = new Libro("Ficciones", "Borges", 200);
        Libro l2 = new Libro("1984", "Orwell", 328);
        Libro l3 = new Libro(); // "Sin título", "Desconocido", 0

        System.out.printf("Libro 1: '%s' de %s (%d págs.) -> ¿Es largo? %b%n",
                l1.getTitulo(), l1.getAutor(), l1.getNumeroPaginas(), l1.esLargo());
        System.out.printf("Libro 2: '%s' de %s (%d págs.) -> ¿Es largo? %b%n",
                l2.getTitulo(), l2.getAutor(), l2.getNumeroPaginas(), l2.esLargo());
        System.out.printf("Libro 3 (por defecto): '%s' de %s (%d págs.)%n",
                l3.getTitulo(), l3.getAutor(), l3.getNumeroPaginas());

        // Modificamos el número de páginas de l1 con su setter
        l1.setNumeroPaginas(224);
        System.out.println("Páginas actualizadas de '" + l1.getTitulo() + "': " + l1.getNume
roPaginas());
    }
}
```

Solución Ejercicio 2: Geometría y Contador Estático de Instancias con `Rectangulo`

```
public class Rectangulo {
    private static int numeroDeRectangulos = 0;

    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        if (base <= 0 || altura <= 0) {
            throw new IllegalArgumentException("La base y la altura deben ser mayores que ce
ro.");
        }
        this.base = base;
        this.altura = altura;
        numeroDeRectangulos++;
    }

    public static int getNumeroDeRectangulos() {
        return numeroDeRectangulos;
    }

    public double getBase() {
        return this.base;
    }

    public void setBase(double base) {
        if (base <= 0) throw new IllegalArgumentException("La base debe ser positiva.");
        this.base = base;
    }

    public double getAltura() {
        return this.altura;
    }

    public void setAltura(double altura) {
        if (altura <= 0) throw new IllegalArgumentException("La altura debe ser positiva.");
        this.altura = altura;
    }

    public double calcularArea() {
        return this.base * this.altura;
    }

    public double calcularPerimetro() {
        return 2 * (this.base + this.altura);
    }

    public boolean esCuadrado() {
        return Double.compare(this.base, this.altura) == 0;
    }

    public void escalar(double factor) {
        if (factor <= 0) {
            throw new IllegalArgumentException("El factor de escala debe ser positivo: " + f
actor);
        }
        this.base *= factor;
        this.altura *= factor;
    }

    public static void main(String[] args) {
```

```
        Rectangulo r1 = new Rectangulo(4, 3);
        Rectangulo r2 = new Rectangulo(5, 5);
        Rectangulo r3 = new Rectangulo(10, 2);

        System.out.println("Área r1: " + r1.calcularArea() + " | Perímetro r1: " + r1.calcul
arPerimetro());
        System.out.println("¿r2 es cuadrado?: " + r2.esCuadrado());
        System.out.println("Total de rectángulos creados: " + Rectangulo.getNumeroDeRectangu
los()); // 3
    }
}
```

Solución Ejercicio 3: La Clase `Circulo`

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio del círculo debe ser estrictamente
positivo.");
        }
        this.radio = radio;
    }

    // Constructor sin parámetros delegando en el canónico
    public Circulo() {
        this(1.0);
    }

    public double getRadio() {
        return this.radio;
    }

    public void setRadio(double radio) {
        if (radio <= 0) throw new IllegalArgumentException("Radio no válido.");
        this.radio = radio;
    }

    public double calcularArea() {
        return Math.PI * Math.pow(this.radio, 2);
    }

    public double calcularPerimetro() {
        return 2 * Math.PI * this.radio;
    }
}
```

Solución Ejercicio 4: Conversor Térmico con la Clase `Termometro`

```
public class Termometro {
    public static final double CERO_ABSOLUTO_CELSIUS = -273.15;
    private double temperaturaCelsius;

    public Termometro(double temperaturaCelsius) {
        setTemperaturaCelsius(temperaturaCelsius);
    }

    public double getTemperaturaCelsius() {
        return this.temperaturaCelsius;
    }

    public void setTemperaturaCelsius(double celsius) {
        if (celsius < CERO_ABSOLUTO_CELSIUS) {
            throw new IllegalArgumentException("Temperatura por debajo del cero absoluto: "
+ celsius);
        }
        this.temperaturaCelsius = celsius;
    }

    public double getFahrenheit() {
        return (this.temperaturaCelsius * 9.0 / 5.0) + 32.0;
    }

    public double getKelvin() {
        return this.temperaturaCelsius + 273.15;
    }

    public void incrementar(double delta) {
        setTemperaturaCelsius(this.temperaturaCelsius + delta);
    }

    public void decrementar(double delta) {
        setTemperaturaCelsius(this.temperaturaCelsius - delta);
    }
}
```

Solución Ejercicio 5: Coordenadas Cartesianas con `Punto2D`

```
public class Punto2D {
    private double x;
    private double y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Punto2D() {
        this(0.0, 0.0);
    }

    public double getX() { return this.x; }
    public void setX(double x) { this.x = x; }

    public double getY() { return this.y; }
    public void setY(double y) { this.y = y; }

    public void desplazar(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    public double distanciaAlOrigen() {
        return Math.sqrt(Math.pow(this.x, 2) + Math.pow(this.y, 2));
    }
}
```

Solución Ejercicio 6: La Máquina Cafetera

```
public class Cafetera {
    private int capacidadMaxima;
    private int cantidadActual;

    public Cafetera() {
        this(1000, 0);
    }

    public Cafetera(int capacidadMaxima) {
        this(capacidadMaxima, 0);
    }

    public Cafetera(int capacidadMaxima, int cantidadActual) {
        this.capacidadMaxima = Math.max(capacidadMaxima, 100);
        if (cantidadActual > this.capacidadMaxima) {
            this.cantidadActual = this.capacidadMaxima;
        } else {
            this.cantidadActual = Math.max(cantidadActual, 0);
        }
    }

    public void llenarCafetera() {
        this.cantidadActual = this.capacidadMaxima;
    }

    public int servirTaza(int capacidadTaza) {
        if (capacidadTaza <= 0) return 0;

        if (this.cantidadActual >= capacidadTaza) {
            this.cantidadActual -= capacidadTaza;
            return capacidadTaza;
        } else {
            int servido = this.cantidadActual;
            this.cantidadActual = 0;
            return servido; // Sirve lo que queda
        }
    }

    public void vaciarCafetera() {
        this.cantidadActual = 0;
    }

    public void agregarCafe(int cantidad) {
        if (cantidad > 0) {
            this.cantidadActual = Math.min(this.cantidadActual + cantidad, this.capacidadMax
ima);
        }
    }

    public int getCapacidadMaxima() { return this.capacidadMaxima; }
    public int getCantidadActual() { return this.cantidadActual; }
}
```

Solución Ejercicio 7: Cronómetro Simple con `Reloj`

```
public class Reloj {
    private int hora;
    private int minuto;
    private int segundo;

    public Reloj(int hora, int minuto, int segundo) {
        if (hora < 0 || hora > 23 || minuto < 0 || minuto > 59 || segundo < 0 || segundo > 5
9) {
            throw new IllegalArgumentException("Valores de tiempo fuera de límites.");
        }
        this.hora = hora;
        this.minuto = minuto;
        this.segundo = segundo;
    }

    public Reloj() {
        this(0, 0, 0);
    }

    public void ticTac() {
        this.segundo++;
        if (this.segundo == 60) {
            this.segundo = 0;
            this.minuto++;
            if (this.minuto == 60) {
                this.minuto = 0;
                this.hora++;
                if (this.hora == 24) {
                    this.hora = 0;
                }
            }
        }
    }

    public String getHoraFormateada() {
        return String.format("%02d:%02d:%02d", this.hora, this.minuto, this.segundo);
    }
}
```

Solución Ejercicio 8: Identificadores Autoincrementales y Sueldo Protegido con `Empleado`

```
import java.util.Objects;

public class Empleado {
    private static int contadorSecuencial = 1000;
    private static String passwordMaestra = "RRHH2025";

    private final int idEmpleado;
    private String nombre;
    private int edad;
    private String departamento;
    private double sueldo;

    public Empleado(String nombre, int edad, String departamento, double sueldo) {
        setNombre(nombre);
        setEdad(edad);
        this.idEmpleado = ++contadorSecuencial;
        this.departamento = (departamento != null) ? departamento : "General";
        this.sueldo = Math.max(sueldo, 0);
    }

    public static int getTotalEmpleados() {
        return contadorSecuencial - 1000;
    }

    public int getIdEmpleado() { return this.idEmpleado; }
    public String getNombre() { return this.nombre; }

    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del empleado es obligatorio.");
        }
        this.nombre = nombre.trim();
    }

    public int getEdad() { return this.edad; }

    public void setEdad(int edad) {
        if (edad < 18) {
            throw new IllegalArgumentException("El empleado debe ser mayor de edad (>= 18):
" + edad);
        }
        this.edad = edad;
    }

    public double getSueldo(String password) {
        verificarPassword(password);
        return this.sueldo;
    }

    public void setSueldo(double nuevoSueldo, String password) {
        verificarPassword(password);
        if (nuevoSueldo < 0) {
            throw new IllegalArgumentException("El sueldo no puede ser negativo.");
        }
        this.sueldo = nuevoSueldo;
    }

    private static void verificarPassword(String password) {
        if (!Objects.equals(passwordMaestra, password)) {
            throw new SecurityException("Acceso denegado: contraseña de RRHH incorrecta.");
```

```
        }
    }
}
```

Solución Ejercicio 9: La Clase `Contador` y Encadenamiento

```
public class Contador {
    private int valor;

    public Contador() {
        this(0);
    }

    public Contador(int valorInicial) {
        this.valor = valorInicial;
    }

    public Contador(Contador otro) {
        this(otro != null ? otro.valor : 0);
    }

    public Contador incrementar() {
        this.valor++;
        return this; // Permite method chaining
    }

    public Contador decrementar() {
        this.valor--;
        return this;
    }

    public Contador incrementar(int cantidad) {
        this.valor += cantidad;
        return this;
    }

    public int getValor() {
        return this.valor;
    }
}
```

Solución Ejercicio 10: Array de Objetos Alumnos

```
public class GestionAlumnos {
    public static class Alumno {
        private final String nombre;
        private final double nota;

        public Alumno(String nombre, double nota) {
            this.nombre = nombre;
            this.nota = nota;
        }

        public String getNombre() { return this.nombre; }
        public double getNota() { return this.nota; }
    }

    public static double calcularNotaMedia(Alumno[] lista) {
        if (lista == null || lista.length == 0) return 0.0;
        double suma = 0;
        for (Alumno a : lista) {
            suma += a.getNota();
        }
        return suma / lista.length;
    }

    public static Alumno obtenerMejorAlumno(Alumno[] lista) {
        if (lista == null || lista.length == 0) return null;
        Alumno mejor = lista[0];
        for (int i = 1; i < lista.length; i++) {
            if (lista[i].getNota() > mejor.getNota()) {
                mejor = lista[i];
            }
        }
        return mejor;
    }

    public static void main(String[] args) {
        Alumno[] clase = {
            new Alumno("Elena", 8.5),
            new Alumno("Marcos", 6.2),
            new Alumno("Sara", 9.8),
            new Alumno("David", 7.0),
            new Alumno("Lucía", 5.4)
        };

        System.out.printf("Nota media de la clase: %.2f%n", calcularNotaMedia(clase));
        Alumno top = obtenerMejorAlumno(clase);
        System.out.printf("Mejor alumno: %s con un %.1f%n", top.getNombre(), top.getNota());
    }
}
```

Solución Ejercicio 11: Control de Vehículo con la Clase `Coche`

```
public class Coche {
    private final String marca;
    private final String modelo;
    private boolean motorEncendido;
    private int velocidadActual;

    public Coche(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.motorEncendido = false;
        this.velocidadActual = 0;
    }

    public boolean arrancar() {
        if (this.motorEncendido) return false;
        this.motorEncendido = true;
        return true;
    }

    public boolean apagar() {
        if (!this.motorEncendido || this.velocidadActual > 0) {
            return false; // Peligro: no apagar en marcha
        }
        this.motorEncendido = false;
        return true;
    }

    public void acelerar(int incremento) {
        if (this.motorEncendido && incremento > 0) {
            this.velocidadActual = Math.min(this.velocidadActual + incremento, 200);
        }
    }

    public void frenar(int decremento) {
        if (decremento > 0) {
            this.velocidadActual = Math.max(this.velocidadActual - decremento, 0);
        }
    }

    public int getVelocidadActual() { return this.velocidadActual; }
    public boolean isMotorEncendido() { return this.motorEncendido; }
}
```

Solución Ejercicio 12: Generador y Validador de Contraseñas

```
import java.security.SecureRandom;

public class Password {
    private static final String CARACTERES = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrst
uvwxyz0123456789";
    private static final SecureRandom RANDOM = new SecureRandom();

    private int longitud;
    private String valor;

    public Password() {
        this(8);
    }

    public Password(int longitud) {
        this.longitud = Math.max(longitud, 4);
        regenerar();
    }

    public void regenerar() {
        StringBuilder sb = new StringBuilder(this.longitud);
        for (int i = 0; i < this.longitud; i++) {
            int idx = RANDOM.nextInt(CARACTERES.length());
            sb.append(CARACTERES.charAt(idx));
        }
        this.valor = sb.toString();
    }

    public boolean esFuerte() {
        int mayus = 0;
        int minus = 0;
        int digitos = 0;

        for (int i = 0; i < this.valor.length(); i++) {
            char c = this.valor.charAt(i);
            if (Character.isUpperCase(c)) mayus++;
            else if (Character.isLowerCase(c)) minus++;
            else if (Character.isDigit(c)) digitos++;
        }

        return mayus >= 2 && minus >= 1 && digitos >= 2;
    }

    public String getValor() { return this.valor; }
    public int getLongitud() { return this.longitud; }
}
```

Solución Ejercicio 13: Visibilidad `private` vs. `public` y Auditoría de Código con Errores

1. Diferencia entre `private` y `public`:

- Un atributo **`public`** es accesible y modificable directamente desde cualquier clase externa del programa (`libro.paginas = -50;`), dejando al objeto indefenso frente a estados inválidos y creando un fuerte acoplamiento con su estructura interna.
- Un atributo **`private`** solo es visible dentro de la propia clase que lo declara. Combinado con métodos públicos `get` y `set` (la **interfaz pública**), permite validar que los datos nunca tomen valores incoherentes

(principio de **encapsulamiento** y ocultación de la información).

2. Auditoría 1 (`public static String getNombre() { return nombre; }`):

- **No compila** (`non-static variable nombre cannot be referenced from a static context`).
- **Razón conceptual:** `nombre` es un atributo de instancia (cada empleado tiene su propio nombre en el Heap), mientras que un método `static` pertenece a la clase global y se ejecuta sin objeto (`this` no existe). La clase no puede adivinar el nombre de qué empleado concreto quieres devolver.

3. Auditoría 2 (Corrección de `Punto2D`):

- *Errores detectados en el código original:* `Public` con P mayúscula (error léxico), la clase se llamaba `empleado` en el fichero `Punto2D.java`, atributos `public` sin encapsular, el supuesto constructor `public void Punto2D(...)` tenía tipo de retorno `void` (convirtiéndolo en un método normal, no en un constructor), y `x = x; y = y;` asignaba los parámetros a sí mismos por sombreamiento al omitir `this.x = x;`.
- *Código corregido:*

```
public class Punto2D {
    private double x;
    private double y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public double getY() { return this.y; }
}
```

Solución Ejercicio 14: Traza de Referencias Compartidas en el Heap

Salida por pantalla:

```
El Aleph
true
```

Explicación paso a paso de la memoria:

1. `new Libro("Ficciones", "Borges", 200)` reserva un **único objeto** en el **Heap** y guarda su dirección en la variable `l1` del **Stack**.
2. `Libro l2 = l1;` **no duplica el objeto**, sino que copia la dirección de memoria de `l1` en `l2`. Ambas referencias apuntan exactamente al mismo objeto físico en el Heap.
3. Al ejecutar `l2.setTitulo("El Aleph")`, se modifica el atributo `titulo` de ese único objeto compartido. Por eso `l1.getTitulo()` devuelve `"El Aleph"` y `l1 == l2` evalúa a `true`.
4. **Cómo crear una copia independiente:** Tendríamos que instanciar un nuevo objeto en el Heap con `new`, ya sea pasando los getters (`Libro l2 = new Libro(l1.getTitulo(), l1.getAutor(), l1.getNumeroPaginas());`) o implementando un **constructor de copia** `public Libro(Libro otro)`.

Solución Ejercicio 15: La Clase `FormateadorBanner`: Encapsulamiento y Sobrecarga Orientada a Objetos

```
public class FormateadorBanner {
    private char caracterBorde;
    private int anchoMinimo;
    private static int totalBannersGenerados = 0;

    // Constructor canónico con validación
    public FormateadorBanner(char caracterBorde, int anchoMinimo) {
        if (anchoMinimo < 10) {
            throw new IllegalArgumentException("El ancho mínimo no puede ser menor a 10.");
        }
        this.caracterBorde = caracterBorde;
        this.anchoMinimo = anchoMinimo;
    }

    // Constructor por defecto delegando con this(...)
    public FormateadorBanner() {
        this('*', 30);
    }

    // 1. Método sobrecargado principal
    public void imprimirBanner(String texto) {
        imprimirBanner(texto, this.caracterBorde);
    }

    // 2. Sobrecarga con carácter especial
    public void imprimirBanner(String texto, char caracterEspecial) {
        String textoLimpio = (texto == null) ? "" : texto.trim();
        int longitudLinea = Math.max(this.anchoMinimo, textoLimpio.length() + 4);

        imprimirLineaBorde(longitudLinea, caracterEspecial);
        System.out.printf("%c %s %c%n", caracterEspecial, centrarTexto(textoLimpio, longitud
Linea - 4), caracterEspecial);
        imprimirLineaBorde(longitudLinea, caracterEspecial);

        totalBannersGenerados++;
    }

    // 3. Método con varargs para múltiples líneas
    public void imprimirLineas(String... lineas) {
        if (lineas == null || lineas.length == 0) {
            imprimirBanner("");
            return;
        }
        int maxLen = 0;
        for (String linea : lineas) {
            if (linea != null && linea.length() > maxLen) {
                maxLen = linea.length();
            }
        }
        int longitudLinea = Math.max(this.anchoMinimo, maxLen + 4);

        imprimirLineaBorde(longitudLinea, this.caracterBorde);
        for (String linea : lineas) {
            String l = (linea == null) ? "" : linea;
            System.out.printf("%c %-" + (longitudLinea - 4) + "s %c%n", this.caracterBorde,
l, this.caracterBorde);
        }
        imprimirLineaBorde(longitudLinea, this.caracterBorde);

        totalBannersGenerados++;
```

```
    }

    // Métodos auxiliares privados para no duplicar lógica
    private void imprimirLineaBorde(int longitud, char c) {
        for (int i = 0; i < longitud; i++) {
            System.out.print(c);
        }
        System.out.println();
    }

    private String centrarTexto(String texto, int espacio) {
        int padding = (espacio - texto.length()) / 2;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < padding; i++) sb.append(' ');
        sb.append(texto);
        while (sb.length() < espacio) sb.append(' ');
        return sb.toString();
    }

    public static int getTotalBannersGenerados() {
        return totalBannersGenerados;
    }

    public static void main(String[] args) {
        FormateadorBanner fb1 = new FormateadorBanner();
        FormateadorBanner fb2 = new FormateadorBanner('=', 40);

        fb1.imprimirBanner("BIENVENIDOS A 1º DAM");
        fb1.imprimirBanner("AVISO URGENTE", '!');
        fb2.imprimirLineas("SISTEMA DE GESTIÓN", "Módulo de Programación", "Java 25 LTS");

        System.out.println("Total de banners generados: " + FormateadorBanner.getTotalBanner
sGenerados());
    }
}
```

Solución Ejercicio 16: Modelado de Objetos con Estado: Las Clases `Dado` y `Cubilete`

```
import java.util.Random;

public class SimuladorDadosPOO {

    // 1. CLASE DADO: Entidad con estado y comportamiento propio
    public static class Dado {
        private final int numeroCaras;
        private int ultimoValor;
        private static final Random RANDOM = new Random();

        public Dado(int numeroCaras) {
            if (numeroCaras < 2) {
                throw new IllegalArgumentException("Un dado debe tener como mínimo 2 cara
s.");
            }
            this.numeroCaras = numeroCaras;
            this.ultimoValor = 1;
        }

        public Dado() {
            this(6); // Dado estándar de 6 caras
        }

        public int lanzar() {
            this.ultimoValor = RANDOM.nextInt(this.numeroCaras) + 1;
            return this.ultimoValor;
        }

        public int getNumeroCaras() { return numeroCaras; }
        public int getUltimoValor() { return ultimoValor; }

        @Override
        public String toString() {
            return String.format("Dado de %d caras (última tirada: %d)", numeroCaras, ultimo
Valor);
        }
    }

    // 2. CLASE CUBILETE: Gestiona un array de objetos Dado
    public static class Cubilete {
        private final Dado[] dados;

        public Cubilete(int cantidadDados, int carasPorDado) {
            if (cantidadDados <= 0) {
                throw new IllegalArgumentException("Debe haber al menos 1 dado en el cubilet
e.");
            }
            this.dados = new Dado[cantidadDados];
            for (int i = 0; i < cantidadDados; i++) {
                this.dados[i] = new Dado(carasPorDado);
            }
        }

        public int tirarTodos() {
            int suma = 0;
            for (Dado d : this.dados) {
                suma += d.lanzar();
            }
            return suma;
        }
```

```
        public int tirarHastaSuma(int sumaObjetivo) {
            int minimoPosible = this.dados.length;
            int maximoPosible = 0;
            for (Dado d : this.dados) maximoPosible += d.getNumeroCaras();

            if (sumaObjetivo < minimoPosible || sumaObjetivo > maximoPosible) {
                throw new IllegalArgumentException(String.format(
                        "Suma imposible. Rango válido: [%d..%d]", minimoPosible, maximoPosib
le));
            }

            int tiradas = 0;
            int suma;
            do {
                suma = tirarTodos();
                tiradas++;
            } while (suma != sumaObjetivo);

            return tiradas;
        }

        public void mostrarEstado() {
            for (int i = 0; i < dados.length; i++) {
                System.out.printf("  [Dado %d] %s%n", i + 1, dados[i]);
            }
        }
    }

    public static void main(String[] args) {
        // Prueba de un dado suelto de 20 caras (D20 para rol)
        Dado d20 = new Dado(20);
        System.out.println("Tirada D20: " + d20.lanzar());
        System.out.println(d20);

        // Prueba del cubilete con 2 dados de 6 caras
        Cubilete cubilete = new Cubilete(2, 6);
        System.out.println("\n--- Tirada inicial del cubilete ---");
        int sumaInicial = cubilete.tirarTodos();
        System.out.println("Suma obtenida: " + sumaInicial);
        cubilete.mostrarEstado();

        // Tirar hasta obtener un 12 (doble 6)
        System.out.println("\nLanzando hasta conseguir un 12...");
        int intentos = cubilete.tirarHastaSuma(12);
        System.out.printf("¡Conseguido en %d tiradas!%n", intentos);
        cubilete.mostrarEstado();
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.2: Nivel Intermedio — Ejercicios Prácticos

Ejercicios avanzados de orientación a objetos: composición, contratos equals y hashCode, inmutabilidad, records, enums, patrón Singleton y factorías estáticas en Java.

Bienvenido al boletín de nivel intermedio de la **Unidad 4: Clases y objetos**.

Este bloque de 16 ejercicios está diseñado para profundizar en la arquitectura y buenas prácticas del paradigma orientado a objetos: diseño de clases compuestas, implementación rigurosa de los contratos de `java.lang.Object` (`toString`, `equals`, `hashCode`), clases inmutables con copias defensivas, los modernos `record`, enumeraciones complejas con comportamiento, el patrón de diseño Singleton, procesamiento con `StringBuilder` y expresiones regulares.

> **Enfoque Arquitectural**
>
> Presta especial atención a la protección de datos: evita las fugas de encapsulamiento mediante copias defensivas (`.clone()` / `Arrays.copyOf()`) y asegura que cualquier clase que sobrescriba `equals()` sobrescriba de forma coordinada `hashCode()`.

Ejercicio 1: Sobrescritura Canónica de `toString()`, `equals()` y `hashCode()` en `Libro`

Enunciado:
Toma como base la clase `Libro` (`titulo`, `autor`, `paginas`):

- Sobrescribe `toString()` con `@Override` para que devuelva el formato `"Ficciones, de Borges (200 páginas)"`. Prueba a pasar directamente un objeto `Libro` a `System.out.println(libro)`.
- Sobrescribe `equals(Object o)` y `hashCode()` de forma que dos objetos `Libro` se consideren iguales si tienen exactamente el mismo `titulo` y el mismo `autor` (aunque tengan diferente número de páginas).
- En `main`, prueba explícitamente estos **tres casos obligatorios**:
  1. Dos objetos distintos con igual título y autor pero distintas páginas → `equals()` debe devolver `true`.
  2. Dos objetos con distinto título → `equals()` debe devolver `false`.
  3. Un objeto comparado consigo mismo (`a.equals(a)`) → debe devolver `true` (propiedad reflexiva).

Ejercicio 2: Composición de Objetos (`Motor` y `Vehiculo`)

Enunciado:
La composición (*"un objeto tiene otro objeto"*) es una de las relaciones más potentes de la POO:

- Crea la clase `Motor` con: `tipoCombustible` (String: Gasolina, Diésel, Eléctrico), `potenciaCv` (int) y `encendido` (boolean). Incluye métodos para encender, apagar y consultar estado.
- Crea la clase `Vehiculo` que contenga: `matricula` (String), `marca` (String) y un atributo de tipo objeto `Motor`.
- El constructor de `Vehiculo` debe instanciar o recibir su `Motor`.
- Proporciona métodos en `Vehiculo` para arrancar, detener y consultar la información completa delegando en su motor interno.

Ejercicio 3: La Clase `Racional` / `Fraccion` Inmutable con Simplificación MCD

Enunciado:
Diseña una clase inmutable llamada `Fraccion` (o `Racional`) definida por dos enteros `numerador` y `denominador` (distinto de cero):

- Incluye un constructor por defecto que genere el racional `1/1`, y un constructor parametrizado que lance `IllegalArgumentException` si `denominador == 0`, normalizando el signo y simplificando la fracción automáticamente mediante el **Máximo Común Divisor (algoritmo de Euclides)**.
- Implementa métodos de instancia `sumar(Fraccion otra)`, `restar(Fraccion otra)`, `multiplicar(Fraccion otra)` y `dividir(Fraccion otra)` que devuelvan siempre una **nueva instancia**, además de un método estático `Fraccion.sumar(Fraccion r1, Fraccion r2)`.
- Sobrescribe `toString()` (`"n/d"`) y `equals()` / `hashCode()`.

Ejercicio 4: Clase Inmutable Clásica `Coordenada` y `record CoordenadaGPS`

Enunciado:
Practica las dos formas de crear tipos inmutables de coordenadas en Java:

1. **Enfoque tradicional:** Crea una clase `public final class Coordenada` con atributos `private final double x` e `y`, sin setters, con getters y un método `public double distanciaA(Coordenada otra)` que calcule la distancia euclídea: `Math.sqrt(Math.pow(otra.x - this.x, 2) + Math.pow(otra.y - this.y, 2))`.
2. **Enfoque Java moderno:** Declara un `record CoordenadaGPS(double latitud, double longitud)` con constructor compacto que valide los rangos `[-90, 90]` y `[-180, 180]` y un método `distanciaKm(CoordenadaGPS destino)` mediante la fórmula de Haversine.

Ejercicio 5: El Patrón Singleton (`ContadorGlobal` y `GestorConfiguracion`)

Enunciado:
Implementa el patrón de diseño Singleton en dos variantes:

1. **Inicialización perezosa (*Lazy*):** Crea la clase `ContadorGlobal` con constructor `private`, instancia estática única y métodos `getInstancia()`, `incrementar()` y `getValor()`. Comprueba en `main` que incrementarlo desde dos referencias distintas (`c1` y `c2`) afecta al mismo contador y que `c1 == c2` es `true`.
2. **Inicialización temprana (*Eager*):** Crea la clase `GestorConfiguracion` que centralice un mapa de propiedades clave-valor (`setPropiedad`, `getPropiedad`).

Ejercicio 6: La Clase `Mochila` y Copias Defensivas (`.clone()` y `Arrays.copyOf()`)

Enunciado:
Diseña una clase `Mochila` que almacene un array `private String[] objetos`:

- Tanto en el constructor al recibir el array como en el getter `getObjetos()`, realiza una **copia defensiva** usando `.clone()` o `Arrays.copyOf()` para que nadie pueda alterar el array interno desde fuera.
- En `main`, crea un array `String[] equipo = {"Linterna", "Cantimplora"}`, pásalo al constructor de `Mochila`, modifica después `equipo[0] = "ROTO"` y altera también el array devuelto por `getObjetos()[1] = "VACÍO"`. Imprime el contenido real de la mochila para demostrar que sigue intacto.

Ejercicio 7: Enumeración Rica `NivelSeveridad`

Enunciado:
Modela una enumeración para un sistema de logging profesional llamada `NivelSeveridad`:

- Constantes: `DEBUG(1, "Depuración"), INFO(2, "Informativo"), WARN(3, "Advertencia"), ERROR(4, "Error Grave"), FATAL(5, "Fallo Crítico")`.
- Atributos privados: `int nivelNumerico` y `String etiqueta`.
- Constructor de enum parametrizado.
- Método `public boolean esCritico()`: devuelve `true` si el nivel es `ERROR` o `FATAL`.
- Método `public boolean esMayorOIgualQue(NivelSeveridad otro)`: compara si la severidad actual es igual o más prioritaria que otra.

Ejercicio 8: `CuentaBancaria` con Validación de Excepciones e Historial

Enunciado:
Crea una clase `CuentaBancaria` con `iban`, `titular`, `saldo` y un registro de las últimas 10 transacciones (`record Transaccion`):

- `ingresar(double cantidad)`: lanza `IllegalArgumentException` si `cantidad <= 0`.
- `retirar(double cantidad)`: lanza `IllegalArgumentException` con un mensaje descriptivo si `cantidad <= 0` o si `cantidad > saldo` (`"Saldo insuficiente"`).
- En `main`, envuelve las operaciones en un bloque `try-catch (IllegalArgumentException e)` intentando retirar más dinero del disponible y muestra el historial de movimientos.

Ejercicio 9: Validación Compacta con `record Usuario`

Enunciado:
Crea un `record Usuario(String username, String email, int edad)`:

- En el constructor compacto, valida:
  1. `username`: alfanumérico entre 4 y 15 caracteres (mediante regex `^[a-zA-Z0-9_]{4,15}$`).
  2. `email`: formato de correo válido con regex.
  3. `edad`: mayor o igual a 18 años.
- Si algún dato no cumple, lanza `IllegalArgumentException` con un mensaje explicativo.

Ejercicio 10: Enumeración de Divisas con Conversión Monetaria

Enunciado:
Crea un `enum Divisa`:

- Constantes: `EUR(1.0, "€"), USD(1.08, "$"), GBP(0.85, "£"), JPY(162.50, "¥")`.
- Cada divisa almacena su tasa de cambio respecto a 1.0 Euro base y su símbolo monetario.
- Diseña un método en el enum `public double convertirA(double cantidad, Divisa divisaDestino)` que transforme cualquier importe entre dos divisas cualesquiera pasando por el Euro como pivote.

Ejercicio 11: Encapsulación de Matriz con la Clase `Matriz2D`

Enunciado:
Crea una clase `Matriz2D` que encapsule una matriz bidimensional `double[][]`:

- Constructor con número de filas y columnas, rellenando con ceros.
- Constructor con datos `double[][] datosOriginales` aplicando copia defensiva profunda fila por fila.
- Métodos:
  - `public Matriz2D sumar(Matriz2D otra)`: devuelve una nueva matriz resultado (si las dimensiones coinciden).
  - `public Matriz2D transponer()`: devuelve una nueva matriz transpuesta.
  - `public double traza()`: suma de los elementos de la diagonal principal (solo si es cuadrada).
  - `toString()` formateando la matriz con saltos de línea y tabuladores.

Ejercicio 12: Factorías Estáticas (*Static Factory Methods*) con `ColorRGB`

Enunciado:
Crea una clase `ColorRGB`:

- Atributos `private final int rojo`, `verde`, `azul` (todos en el rango [0, 255]).
- **Constructor privado** para impedir la instanciación descontrolada con `new`.
- Constantes predefinidas: `public static final ColorRGB ROJO`, `VERDE`, `AZUL`, `BLANCO`, `NEGRO`.
- Métodos de factoría estáticos:
  - `public static ColorRGB fromRGB(int r, int g, int b)`: valida rangos [0, 255] y devuelve una instancia.
  - `public static ColorRGB fromHex(String hex)`: parsea cadenas como `"#FF5733"` o `"FF5733"`.
  - `public String toHex()`: devuelve la representación hexadecimal en formato `"#RRGGBB"`.

Ejercicio 13: Array de Objetos `Circulo` y Búsqueda del Mayor Área (`mayorArea`)

Enunciado:
Crea la clase `Circulo` con atributo `private double radio`, constructor, getter, `calcularArea()` y `toString()`.

- En `Main`, declara e inicializa un array de 5 objetos `Circulo` con radios distintos (`2.5`, `7.1`, `1.0`, `9.4`, `4.2`).
- Escribe un método estático `public static Circulo mayorArea(Circulo[] circulos)` que recorra el array y devuelva el objeto `Circulo` con mayor área (controlando que el array no sea nulo ni vacío).
- Muestra el círculo ganador y su área por consola.

Ejercicio 14: El Contrato `equals()` / `hashCode()`, la Metáfora de los Cajones y `Punto2D`

Enunciado:

1. **Razonamiento y demostración práctica:** ¿Por qué siempre que se sobrescribe `equals()` es obligatorio sobrescribir también `hashCode()`? ¿Qué ocurre exactamente con los "cajones" (*buckets*) cuando guardas dos objetos iguales según `equals()` en un `HashSet` si olvidaste sobrescribir `hashCode()`? Demuéstralo con código.

2. **Aplicación en `Punto2D`:** Añade a la clase `Punto2D(x, y)` el método `public Punto2D suma(Punto2D otro)`, sobrescribe `toString()` para que devuelva `"(x, y)"`, y sobrescribe `equals(Object obj)` y `hashCode()` anotados con `@Override` para que se pueda imprimir directamente `System.out.println(p1 + " + " + p2 + " = " + p3)` y comparar puntos por valor.

Ejercicio 15: De Clase Mutable `Alumno` a `AlumnoInmutable`

Enunciado:

1. Crea la clase `Alumno` con encapsulamiento: `int nia`, `String nombre` y `double[] notas`. Incluye un constructor con todos los datos, un constructor con solo `nombre` (que genere un `nia` aleatorio y array de 3 notas a `0.0`), getters/setters, `notaMedia()`, `toString()` y `equals()` / `hashCode()` (basados en el `nia`).
2. A continuación, crea la clase `AlumnoInmutable` que sea una versión 100% inmutable de `Alumno` (`final class`, atributos `private final`, sin setters y con copias defensivas `.clone()` del array `notas` en el constructor y en `getNotas()`).

Ejercicio 16: Sistema de Gestión de Inventario: Clases de Dominio, Invariantes y Contenedor de Objetos

Enunciado:
Diseña un sistema orientado a objetos para gestionar el catálogo y stock de un almacén informático mediante clases bien encapsuladas:

1. **Clase de entidad `Articulo`:**
   - Atributos privados: `codigo` (`String`, debe seguir el patrón `^[A-Z]{3}-\d{3}$`, ej. `"TEC-101"`), `descripcion` (`String` no nulo ni en blanco), `precio` (`double > 0`) y `stock` (`int >= 0`).
   - Constructor canónico con validación rigurosa de todas las invariantes (lanzando `IllegalArgumentException` ante datos erróneos).
   - Métodos getters y setters (con validación de datos en setters).
   - Métodos de negocio:
     - `public void reponer(int cantidad)`: incrementa el stock (validando que `cantidad > 0`).
     - `public boolean vender(int cantidad)`: si hay stock suficiente, descuenta las unidades y devuelve `true`; en caso contrario devuelve `false` sin modificar el stock.
   - Sobrescritura canónica de `equals(Object o)` y `hashCode()` basados exclusivamente en el `codigo`.
   - Sobrescritura de `toString()` con formato: `"Articulo[TEC-101: Teclado Mecánico | 59.99 € | Stock: 15]"`.
2. **Clase contenedora de dominio `GestorAlmacen`:**
   - Atributos privados: un array `private Articulo[] articulos` y un contador `private int totalArticulos`.
   - Constructor `public GestorAlmacen(int capacidadMaxima)`: inicializa el array con el tamaño indicado.
   - Métodos del almacén:
     - `public boolean registrarArticulo(Articulo a)`: comprueba que no sea nulo, que el almacén no esté lleno y que no exista ya un artículo con el mismo código (`equals`). Retorna `true` si se registró con éxito.
     - `public Articulo buscarPorCodigo(String codigo)`: busca por código y devuelve la referencia al artículo, o `null` si no existe.

- `public boolean eliminarArticulo(String codigo)`: localiza el artículo, lo elimina y **compacta el array hacia la izquierda** para evitar huecos intermedios, decrementando `totalArticulos`.
- `public double calcularValorTotalInventario()`: calcula y devuelve el valor económico total (`precio * stock`) de todas las existencias almacenadas.
- `public Articulo[] getArticulosDisponibles()`: devuelve una **copia defensiva** del array conteniendo exactamente los artículos registrados (sin posiciones nulas sobrantes).

3. En `main`, crea un almacén con capacidad para 5 productos, registra 3 artículos, realiza una venta válida y otra con stock insuficiente, consulta el valor total del inventario, busca un artículo por código, elimina uno y comprueba que el array queda perfectamente compactado.

> **¿Quieres revisar el código completo?**
> Consulta todas las implementaciones detalladas en el Boletín 4.2 Resuelto.

DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.2: Nivel Intermedio (Resuelto) — Soluciones Comentadas

Soluciones completas, comentadas y optimizadas de los 16 ejercicios de nivel intermedio de la Unidad 4 en Java 25 LTS.

A continuación se presentan las soluciones comentadas de los 16 ejercicios de nivel intermedio de la Unidad 4, haciendo énfasis en la inmutabilidad, el respeto a los contratos de `Object`, la modularidad y los patrones de diseño orientados a objetos.

Solución Ejercicio 1: Sobrescritura Canónica de `toString()`, `equals()` y `hashCode()` en `Libro`

```
import java.util.Objects;

public class Libro {
    private String titulo;
    private String autor;
    private int paginas;

    public Libro(String titulo, String autor, int paginas) {
        this.titulo = Objects.requireNonNull(titulo, "El título no puede ser nulo");
        this.autor = Objects.requireNonNull(autor, "El autor no puede ser nulo");
        this.paginas = Math.max(paginas, 0);
    }

    public String getTitulo() { return this.titulo; }
    public String getAutor() { return this.autor; }
    public int getPaginas() { return this.paginas; }

    @Override
    public String toString() {
        return this.titulo + ", de " + this.autor + " (" + this.paginas + " páginas)";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true; // 1. Misma referencia en el Heap
        if (!(o instanceof Libro otro)) return false; // 2. Null o clase incompatible
        // 3. Igualdad semántica por título y autor
        return Objects.equals(this.titulo, otro.titulo)
                && Objects.equals(this.autor, otro.autor);
    }

    @Override
    public int hashCode() {
        // Obligatorio usar exactamente los mismos campos que en equals()
        return Objects.hash(this.titulo, this.autor);
    }

    public static void main(String[] args) {
        Libro a = new Libro("Ficciones", "Borges", 200);
        Libro b = new Libro("Ficciones", "Borges", 350); // Mismo título y autor, distintas
páginas
        Libro c = new Libro("El Aleph", "Borges", 200);  // Distinto título

        System.out.println(a); // Imprime: Ficciones, de Borges (200 páginas)

        // Caso 1: Igual título y autor, distintas páginas -> true
        System.out.println("¿a.equals(b)? " + a.equals(b));
        // Caso 2: Distinto título -> false
        System.out.println("¿a.equals(c)? " + a.equals(c));
        // Caso 3: Reflexividad -> true
        System.out.println("¿a.equals(a)? " + a.equals(a));
    }
}
```

Solución Ejercicio 2: Composición (`Motor` y `Vehiculo`)

```
public class Motor {
    private final String combustible;
    private final int potenciaCv;
    private boolean encendido;

    public Motor(String combustible, int potenciaCv) {
        this.combustible = combustible;
        this.potenciaCv = Math.max(potenciaCv, 1);
        this.encendido = false;
    }

    public void encender() { this.encendido = true; }
    public void apagar() { this.encendido = false; }
    public boolean isEncendido() { return this.encendido; }
    public int getPotenciaCv() { return this.potenciaCv; }
    public String getCombustible() { return this.combustible; }
}
```

```
public class Vehiculo {
    private final String matricula;
    private final String marca;
    private final Motor motor; // Composición

    public Vehiculo(String matricula, String marca, Motor motor) {
        this.matricula = matricula;
        this.marca = marca;
        this.motor = java.util.Objects.requireNonNull(motor, "El motor es obligatorio");
    }

    public void arrancar() {
        if (!this.motor.isEncendido()) {
            this.motor.encender();
            System.out.println("Vehículo " + this.matricula + " ha arrancado su motor.");
        }
    }

    public void detener() {
        this.motor.apagar();
        System.out.println("Vehículo " + this.matricula + " ha apagado su motor.");
    }

    @Override
    public String toString() {
        return String.format("Vehiculo[%s %s | Motor: %d CV %s | Estado: %s]",
                this.marca, this.matricula, this.motor.getPotenciaCv(),
                this.motor.getCombustible(), this.motor.isEncendido() ? "ENCENDIDO" : "APAGA
DO");
    }
}
```

Solución Ejercicio 3: La Clase `Racional` / `Fraccion` Inmutable con Simplificación MCD

```
import java.util.Objects;

public final class Fraccion {
    private final int numerador;
    private final int denominador;

    // Constructor por defecto: racional 1/1
    public Fraccion() {
        this(1, 1);
    }

    public Fraccion(int numerador, int denominador) {
        if (denominador == 0) {
            throw new IllegalArgumentException("El denominador no puede ser cero.");
        }

        // Normalización de signos
        if (denominador < 0) {
            numerador = -numerador;
            denominador = -denominador;
        }

        // Simplificación por Máximo Común Divisor (Euclides recursivo)
        int divisor = mcd(Math.abs(numerador), denominador);
        this.numerador = numerador / divisor;
        this.denominador = denominador / divisor;
    }

    private static int mcd(int a, int b) {
        return b == 0 ? a : mcd(b, a % b);
    }

    public int getNumerador() { return this.numerador; }
    public int getDenominador() { return this.denominador; }

    public Fraccion sumar(Fraccion otra) {
        int num = (this.numerador * otra.denominador) + (otra.numerador * this.denominador);
        int den = this.denominador * otra.denominador;
        return new Fraccion(num, den);
    }

    public static Fraccion sumar(Fraccion r1, Fraccion r2) {
        return r1.sumar(r2);
    }

    public Fraccion restar(Fraccion otra) {
        int num = (this.numerador * otra.denominador) - (otra.numerador * this.denominador);
        int den = this.denominador * otra.denominador;
        return new Fraccion(num, den);
    }

    public Fraccion multiplicar(Fraccion otra) {
        return new Fraccion(this.numerador * otra.numerador, this.denominador * otra.denomin
ador);
    }

    public Fraccion dividir(Fraccion otra) {
        if (otra.numerador == 0) {
            throw new ArithmeticException("No se puede dividir entre una fracción con valor
cero.");
```

```
        }
        return new Fraccion(this.numerador * otra.denominador, this.denominador * otra.numer
ador);
    }

    @Override
    public String toString() {
        return this.numerador + "/" + this.denominador;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Fraccion f)) return false;
        return this.numerador == f.numerador && this.denominador == f.denominador;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.numerador, this.denominador);
    }
}
```

Solución Ejercicio 4: Clase Inmutable Clásica `Coordenada` y `record CoordenadaGPS`

```
// 1. Clase inmutable tradicional (final class + atributos private final + sin setters)
public final class Coordenada {
    private final double x;
    private final double y;

    public Coordenada(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public double getY() { return this.y; }

    public double distanciaA(Coordenada otra) {
        return Math.sqrt(Math.pow(otra.x - this.x, 2) + Math.pow(otra.y - this.y, 2));
    }

    @Override
    public String toString() {
        return "(" + this.x + ", " + this.y + ")";
    }
}
```

```
// 2. Enfoque moderno con record de Java
public record CoordenadaGPS(double latitud, double longitud) {

    public CoordenadaGPS {
        if (latitud < -90.0 || latitud > 90.0) {
            throw new IllegalArgumentException("Latitud fuera de rango [-90, 90]: " + latitu
d);
        }
        if (longitud < -180.0 || longitud > 180.0) {
            throw new IllegalArgumentException("Longitud fuera de rango [-180, 180]: " + lon
gitud);
        }
    }

    public double distanciaKm(CoordenadaGPS destino) {
        final double RADIO_TIERRA_KM = 6371.0;
        double dLat = Math.toRadians(destino.latitud() - this.latitud);
        double dLon = Math.toRadians(destino.longitud() - this.longitud);

        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
                   Math.cos(Math.toRadians(this.latitud)) * Math.cos(Math.toRadians(destino.
latitud())) *
                   Math.sin(dLon / 2) * Math.sin(dLon / 2);

        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return RADIO_TIERRA_KM * c;
    }
}
```

Solución Ejercicio 5: El Patrón Singleton (`ContadorGlobal` y `GestorConfiguracion`)

```
// 1. Singleton con Inicialización Perezosa (Lazy Initialization)
public class ContadorGlobal {
    private static ContadorGlobal instancia;
    private int valor;

    private ContadorGlobal() {
        this.valor = 0;
    }

    public static ContadorGlobal getInstancia() {
        if (instancia == null) {
            instancia = new ContadorGlobal();
        }
        return instancia;
    }

    public void incrementar() {
        this.valor++;
    }

    public int getValor() {
        return this.valor;
    }

    public static void main(String[] args) {
        ContadorGlobal c1 = ContadorGlobal.getInstancia();
        ContadorGlobal c2 = ContadorGlobal.getInstancia();

        c1.incrementar();
        c1.incrementar();
        c2.incrementar();

        System.out.println("Valor desde c1: " + c1.getValor()); // 3
        System.out.println("Valor desde c2: " + c2.getValor()); // 3
        System.out.println("¿Son el mismo objeto (c1 == c2)?: " + (c1 == c2)); // true
    }
}
```

```
// 2. Singleton con Inicialización Temprana (Eager Initialization)
import java.util.HashMap;
import java.util.Map;

public class GestorConfiguracion {
    private static final GestorConfiguracion INSTANCIA = new GestorConfiguracion();
    private final Map<String, String> propiedades;

    private GestorConfiguracion() {
        this.propiedades = new HashMap<>();
        this.propiedades.put("app.nombre", "SistemaDAM");
        this.propiedades.put("app.version", "2.5.0");
    }

    public static GestorConfiguracion getInstance() {
        return INSTANCIA;
    }

    public void setPropiedad(String clave, String valor) {
        this.propiedades.put(clave, valor);
    }

    public String getPropiedad(String clave, String valorPorDefecto) {
        return this.propiedades.getOrDefault(clave, valorPorDefecto);
    }
}
```

Solución Ejercicio 6: La Clase `Mochila` con Copias Defensivas (`.clone()` y `Arrays.copyOf()`)

```
import java.util.Arrays;

public class Mochila {
    private final String[] objetos;

    public Mochila(String[] objetos) {
        // Copia defensiva al entrar (evita que cambios en el array original alteren la moch
ila)
        this.objetos = (objetos != null) ? objetos.clone() : new String[0];
    }

    public String[] getObjetos() {
        // Copia defensiva al salir (evita que quien recibe el array modifique el interno)
        return this.objetos.clone();
    }

    public static void main(String[] args) {
        String[] equipo = {"Linterna", "Cantimplora", "Mapa"};
        Mochila m = new Mochila(equipo);

        // Intentamos sabotear la mochila desde fuera de dos formas distintas:
        equipo[0] = "ROTO";
        m.getObjetos()[1] = "VACÍO";

        // Comprobamos que el estado interno de la mochila sigue 100% intacto:
        System.out.println("Contenido seguro de la mochila: " + Arrays.toString(m.getObjetos
()));
        // Imprime: [Linterna, Cantimplora, Mapa]
    }
}
```

Solución Ejercicio 7: Enumeración Rica `NivelSeveridad`

```
public enum NivelSeveridad {
    DEBUG(1, "DEB", " [36m"),
    INFO(2, "INF", " [32m"),
    WARN(3, "WRN", " [33m"),
    ERROR(4, "ERR", " [31m"),
    FATAL(5, "FTL", " [35m");

    private final int nivelNumerico;
    private final String prefijo;
    private final String codigoColorAnsi;

    NivelSeveridad(int nivelNumerico, String prefijo, String codigoColorAnsi) {
        this.nivelNumerico = nivelNumerico;
        this.prefijo = prefijo;
        this.codigoColorAnsi = codigoColorAnsi;
    }

    public int getNivelNumerico() { return this.nivelNumerico; }
    public String getPrefijo() { return this.prefijo; }
    public String getCodigoColorAnsi() { return this.codigoColorAnsi; }

    public boolean esCritico() {
        return this.nivelNumerico >= 4;
    }

    public boolean esMayorOIgualQue(NivelSeveridad otro) {
        return this.nivelNumerico >= otro.nivelNumerico;
    }
}
```

Solución Ejercicio 8: `CuentaBancaria` con Validación de Excepciones e Historial

```
import java.time.LocalDateTime;
import java.util.Arrays;

public class CuentaBancaria {
    public record Transaccion(LocalDateTime fechaHora, String tipo, double importe, double s
aldoPosterior) {}

    private final String iban;
    private final String titular;
    private double saldo;
    private final Transaccion[] historial;
    private int contadorTransacciones;

    public CuentaBancaria(String iban, String titular, double saldoInicial) {
        if (titular == null || titular.trim().isEmpty()) {
            throw new IllegalArgumentException("El titular es obligatorio.");
        }
        if (saldoInicial < 0) {
            throw new IllegalArgumentException("El saldo inicial no puede ser negativo.");
        }
        this.iban = iban;
        this.titular = titular.trim();
        this.saldo = saldoInicial;
        this.historial = new Transaccion[10];
        this.contadorTransacciones = 0;
    }

    public void ingresar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a ingresar debe ser mayor que
0.");
        }
        this.saldo += cantidad;
        registrar("INGRESO", cantidad);
    }

    public void retirar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a retirar debe ser mayor que
0.");
        }
        if (cantidad > this.saldo) {
            throw new IllegalArgumentException("Saldo insuficiente. Saldo actual: " + this.s
aldo + "€");
        }
        this.saldo -= cantidad;
        registrar("RETIRADA", cantidad);
    }

    private void registrar(String tipo, double importe) {
        Transaccion t = new Transaccion(LocalDateTime.now(), tipo, importe, this.saldo);
        if (this.contadorTransacciones < 10) {
            this.historial[this.contadorTransacciones++] = t;
        } else {
            System.arraycopy(this.historial, 1, this.historial, 0, 9);
            this.historial[9] = t;
        }
    }

    public Transaccion[] getUltimosMovimientos() {
```

```
        return Arrays.copyOf(this.historial, this.contadorTransacciones);
    }

    public double getSaldo() { return this.saldo; }
    public String getTitular() { return this.titular; }

    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("ES01-2000", "Ana", 100.0);
        try {
            cuenta.ingresar(50.0);
            System.out.println("Saldo tras ingreso: " + cuenta.getSaldo() + "€");
            cuenta.retirar(200.0); // Lanzará IllegalArgumentException
        } catch (IllegalArgumentException e) {
            System.out.println("Operación rechazada: " + e.getMessage());
        }
    }
}
```

Solución Ejercicio 9: Validación Compacta con `record Usuario`

```
public record Usuario(String username, String email, int edad) {

    public Usuario {
        if (username == null || !username.matches("^[a-zA-Z0-9_]{4,15}$")) {
            throw new IllegalArgumentException("El username debe tener entre 4 y 15 caracter
es alfanuméricos.");
        }
        if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,6}$")) {
            throw new IllegalArgumentException("El formato del correo electrónico no es váli
do.");
        }
        if (edad < 18) {
            throw new IllegalArgumentException("El usuario debe ser mayor de edad (≥ 18 año
s): " + edad);
        }
    }
}
```

Solución Ejercicio 10: Enumeración de Divisas con Conversión

```
public enum Divisa {
    EUR(1.0, "€"),
    USD(1.08, "$"),
    GBP(0.85, "£"),
    JPY(162.50, "¥");

    private final double tasaRespectoEuro;
    private final String simbolo;

    Divisa(double tasaRespectoEuro, String simbolo) {
        this.tasaRespectoEuro = tasaRespectoEuro;
        this.simbolo = simbolo;
    }

    public double getTasaRespectoEuro() { return this.tasaRespectoEuro; }
    public String getSimbolo() { return this.simbolo; }

    public double convertirA(double cantidad, Divisa divisaDestino) {
        if (cantidad < 0) throw new IllegalArgumentException("La cantidad no puede ser negat
iva.");
        // Paso 1: Convertir a Euros base
        double euros = cantidad / this.tasaRespectoEuro;
        // Paso 2: Convertir de Euros a divisa destino
        return euros * divisaDestino.tasaRespectoEuro;
    }
}
```

Solución Ejercicio 11: Encapsulación de Matriz con `Matriz2D`

```
import java.util.Arrays;

public final class Matriz2D {
    private final double[][] datos;
    private final int filas;
    private final int columnas;

    public Matriz2D(int filas, int columnas) {
        if (filas <= 0 || columnas <= 0) throw new IllegalArgumentException("Dimensiones inv
álidas.");
        this.filas = filas;
        this.columnas = columnas;
        this.datos = new double[filas][columnas];
    }

    public Matriz2D(double[][] origen) {
        if (origen == null || origen.length == 0 || origen[0] == null) {
            throw new IllegalArgumentException("Matriz origen inválida.");
        }
        this.filas = origen.length;
        this.columnas = origen[0].length;
        this.datos = new double[filas][columnas];
        // Copia defensiva profunda
        for (int i = 0; i < filas; i++) {
            this.datos[i] = Arrays.copyOf(origen[i], columnas);
        }
    }

    public Matriz2D sumar(Matriz2D otra) {
        if (this.filas != otra.filas || this.columnas != otra.columnas) {
            throw new IllegalArgumentException("Las dimensiones deben ser idénticas para sum
ar matrices.");
        }
        Matriz2D res = new Matriz2D(this.filas, this.columnas);
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                res.datos[i][j] = this.datos[i][j] + otra.datos[i][j];
            }
        }
        return res;
    }

    public Matriz2D transponer() {
        Matriz2D t = new Matriz2D(this.columnas, this.filas);
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                t.datos[j][i] = this.datos[i][j];
            }
        }
        return t;
    }

    public double traza() {
        if (this.filas != this.columnas) throw new IllegalStateException("La traza solo se d
efine para matrices cuadradas.");
        double suma = 0;
        for (int i = 0; i < filas; i++) suma += this.datos[i][i];
        return suma;
    }
}
```

Solución Ejercicio 12: Factorías Estáticas con `ColorRGB`

```
import java.util.Objects;

public final class ColorRGB {
    public static final ColorRGB ROJO = new ColorRGB(255, 0, 0);
    public static final ColorRGB VERDE = new ColorRGB(0, 255, 0);
    public static final ColorRGB AZUL = new ColorRGB(0, 0, 255);
    public static final ColorRGB BLANCO = new ColorRGB(255, 255, 255);
    public static final ColorRGB NEGRO = new ColorRGB(0, 0, 0);

    private final int r;
    private final int g;
    private final int b;

    // Constructor estrictamente privado
    private ColorRGB(int r, int g, int b) {
        this.r = r;
        this.g = g;
        this.b = b;
    }

    public static ColorRGB fromRGB(int r, int g, int b) {
        validarRango(r);
        validarRango(g);
        validarRango(b);
        return new ColorRGB(r, g, b);
    }

    public static ColorRGB fromHex(String hex) {
        Objects.requireNonNull(hex);
        String limpio = hex.replace("#", "").trim();
        if (limpio.length() != 6) {
            throw new IllegalArgumentException("El código hexadecimal debe tener 6 dígitos:
" + hex);
        }
        int r = Integer.parseInt(limpio.substring(0, 2), 16);
        int g = Integer.parseInt(limpio.substring(2, 4), 16);
        int b = Integer.parseInt(limpio.substring(4, 6), 16);
        return new ColorRGB(r, g, b);
    }

    private static void validarRango(int v) {
        if (v < 0 || v > 255) {
            throw new IllegalArgumentException("Canal de color fuera de rango [0, 255]: " +
v);
        }
    }

    public String toHex() {
        return String.format("#%02X%02X%02X", this.r, this.g, this.b);
    }

    public int getR() { return this.r; }
    public int getG() { return this.g; }
    public int getB() { return this.b; }

    @Override
    public String toString() {
        return String.format("ColorRGB(%d, %d, %d -> %s)", this.r, this.g, this.b, toHex());
    }
}
```

Solución Ejercicio 13: Array de Objetos `Circulo` y Búsqueda del Mayor Área (`mayorArea`)

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio debe ser mayor que cero.");
        }
        this.radio = radio;
    }

    public double getRadio() {
        return this.radio;
    }

    public double calcularArea() {
        return Math.PI * this.radio * this.radio;
    }

    @Override
    public String toString() {
        return String.format("Circulo(radio=%.2f, área=%.2f)", this.radio, calcularArea());
    }

    public static Circulo mayorArea(Circulo[] circulos) {
        if (circulos == null || circulos.length == 0) {
            throw new IllegalArgumentException("El array de círculos no puede ser nulo ni va
cío.");
        }
        Circulo mayor = circulos[0];
        for (int i = 1; i < circulos.length; i++) {
            if (circulos[i].calcularArea() > mayor.calcularArea()) {
                mayor = circulos[i];
            }
        }
        return mayor;
    }

    public static void main(String[] args) {
        Circulo[] lista = {
            new Circulo(2.5),
            new Circulo(7.1),
            new Circulo(1.0),
            new Circulo(9.4),
            new Circulo(4.2)
        };

        Circulo ganador = mayorArea(lista);
        System.out.println("Círculo de mayor área: " + ganador);
    }
}
```

Solución Ejercicio 14: El Contrato `equals()` / `hashCode()`, la Metáfora de los Cajones y `Punto2D`

1. Razonamiento del contrato `equals()` / `hashCode()` y los cajones (*buckets*): Las estructuras basadas en tablas hash (`HashSet`, `HashMap`) organizan sus elementos en **cajones (*buckets*)** identificados por el número entero que devuelve `hashCode()`.

- Al insertar o buscar un objeto, la colección calcula primero su `hashCode()` para ir directamente a su cajón, y **solo compara con `equals()` los objetos que están dentro de ese mismo cajón**.
- Si sobrescribes `equals()` (por ejemplo, dos puntos con iguales coordenadas `x, y`), pero **olvidas sobrescribir `hashCode()`**, cada objeto heredará el `hashCode()` de `Object` (basado en su dirección de memoria física distinta). En consecuencia, dos objetos iguales según `equals()` tendrán distinto `hashCode()`, caerán en cajones diferentes y `HashSet` nunca detectará que son el mismo elemento, admitiendo duplicados o devolviendo `false` en `.contains()`.

2. Implementación completa de `Punto2D` con `suma()`, `toString()`, `equals()` y `hashCode()`:

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class Punto2D {
    private double x;
    private double y;

    public Punto2D() {
        this(0.0, 0.0);
    }

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public void setX(double x) { this.x = x; }
    public double getY() { return this.y; }
    public void setY(double y) { this.y = y; }

    public Punto2D suma(Punto2D otro) {
        return new Punto2D(this.x + otro.x, this.y + otro.y);
    }

    @Override
    public String toString() {
        return "(" + this.x + ", " + this.y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Punto2D otro)) return false;
        return Double.compare(this.x, otro.x) == 0
                && Double.compare(this.y, otro.y) == 0;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.x, this.y);
    }

    public static void main(String[] args) {
        Punto2D p1 = new Punto2D(3.0, 2.0);
        Punto2D p2 = new Punto2D(1.0, -1.0);
        Punto2D p3 = p1.suma(p2);

        System.out.println(p1 + " + " + p2 + " = " + p3); // (3.0, 2.0) + (1.0, -1.0) = (4.
0, 1.0)

        Set<Punto2D> conjunto = new HashSet<>();
        conjunto.add(new Punto2D(3.0, 2.0));
        System.out.println("¿Conjunto contiene (3.0, 2.0)?: " + conjunto.contains(p1)); // t
rue gracias a hashCode()
    }
}
```

Solución Ejercicio 15: De Clase Mutable `Alumno` a `AlumnoInmutable`

```
import java.util.Arrays;
import java.util.Objects;
import java.util.Random;

// 1. Versión Mutable encapsulada
class Alumno {
    private int nia;
    private String nombre;
    private double[] notas;

    public Alumno(int nia, String nombre, double[] notas) {
        this.nia = nia;
        this.nombre = Objects.requireNonNull(nombre);
        this.notas = (notas != null) ? notas.clone() : new double[3];
    }

    public Alumno(String nombre) {
        this(new Random().nextInt(900000) + 100000, nombre, new double[]{0.0, 0.0, 0.0});
    }

    public int getNia() { return this.nia; }
    public void setNia(int nia) { this.nia = nia; }
    public String getNombre() { return this.nombre; }
    public void setNombre(String nombre) { this.nombre = Objects.requireNonNull(nombre); }
    public double[] getNotas() { return this.notas; }
    public void setNotas(double[] notas) { this.notas = notas; }

    public double notaMedia() {
        if (this.notas == null || this.notas.length == 0) return 0.0;
        double suma = 0;
        for (double n : this.notas) suma += n;
        return suma / this.notas.length;
    }

    @Override
    public String toString() {
        return String.format("{nia: %d, nombre: '%s', notas: %s, media: %.2f}",
                this.nia, this.nombre, Arrays.toString(this.notas), notaMedia());
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Alumno otro)) return false;
        return this.nia == otro.nia;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.nia);
    }
}

// 2. Versión 100% Inmutable (final class + private final + sin setters + copias defensivas
.clone())
public final class AlumnoInmutable {
    private final int nia;
    private final String nombre;
    private final double[] notas;
```

```
    public AlumnoInmutable(int nia, String nombre, double[] notas) {
        this.nia = nia;
        this.nombre = Objects.requireNonNull(nombre);
        // Copia defensiva en el constructor
        this.notas = (notas != null) ? notas.clone() : new double[0];
    }

    public int getNia() { return this.nia; }
    public String getNombre() { return this.nombre; }

    public double[] getNotas() {
        // Copia defensiva en el getter
        return this.notas.clone();
    }

    public double notaMedia() {
        if (this.notas.length == 0) return 0.0;
        double suma = 0;
        for (double n : this.notas) suma += n;
        return suma / this.notas.length;
    }

    @Override
    public String toString() {
        return String.format("AlumnoInmutable{nia=%d, nombre='%s', notas=%s}",
                this.nia, this.nombre, Arrays.toString(this.notas));
    }
}
```

Solución Ejercicio 16: Sistema de Gestión de Inventario: Clases de Dominio, Invariantes y Contenedor de Objetos

```
import java.util.Arrays;
import java.util.Objects;

public class GestionInventarioAlmacen {

    // 1. CLASE DE ENTIDAD: ARTÍCULO
    public static class Articulo {
        private final String codigo; // Inmutable una vez asignado
        private String descripcion;
        private double precio;
        private int stock;

        public Articulo(String codigo, String descripcion, double precio, int stock) {
            if (codigo == null || !codigo.matches("^[A-Z]{3}-\\d{3}$")) {
                throw new IllegalArgumentException("Código inválido (formato esperado 'ABC-1
23'): " + codigo);
            }
            if (descripcion == null || descripcion.isBlank()) {
                throw new IllegalArgumentException("La descripción no puede estar vacía.");
            }
            if (precio <= 0) {
                throw new IllegalArgumentException("El precio debe ser estrictamente positiv
o: " + precio);
            }
            if (stock < 0) {
                throw new IllegalArgumentException("El stock no puede ser negativo: " + stoc
k);
            }
            this.codigo = codigo;
            this.descripcion = descripcion.trim();
            this.precio = precio;
            this.stock = stock;
        }

        public String getCodigo() { return codigo; }
        public String getDescripcion() { return descripcion; }
        public double getPrecio() { return precio; }
        public int getStock() { return stock; }

        public void setDescripcion(String descripcion) {
            if (descripcion == null || descripcion.isBlank()) {
                throw new IllegalArgumentException("Descripción vacía.");
            }
            this.descripcion = descripcion.trim();
        }

        public void setPrecio(double precio) {
            if (precio <= 0) {
                throw new IllegalArgumentException("Precio debe ser > 0.");
            }
            this.precio = precio;
        }

        public void reponer(int cantidad) {
            if (cantidad <= 0) {
                throw new IllegalArgumentException("La cantidad a reponer debe ser positiva:
" + cantidad);
            }
            this.stock += cantidad;
        }
```

```
        public boolean vender(int cantidad) {
            if (cantidad <= 0 || cantidad > this.stock) {
                return false; // Venta rechazada
            }
            this.stock -= cantidad;
            return true;
        }

        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Articulo otro)) return false;
            return Objects.equals(this.codigo, otro.codigo);
        }

        @Override
        public int hashCode() {
            return Objects.hash(this.codigo);
        }

        @Override
        public String toString() {
            return String.format("Articulo[%s: %s | %.2f € | Stock: %d]",
                    codigo, descripcion, precio, stock);
        }
    }

    // 2. CLASE CONTENEDORA: GESTOR DE ALMACÉN
    public static class GestorAlmacen {
        private final Articulo[] articulos;
        private int totalArticulos;

        public GestorAlmacen(int capacidadMaxima) {
            if (capacidadMaxima <= 0) {
                throw new IllegalArgumentException("La capacidad debe ser positiva.");
            }
            this.articulos = new Articulo[capacidadMaxima];
            this.totalArticulos = 0;
        }

        public boolean registrarArticulo(Articulo a) {
            if (a == null) return false;
            if (totalArticulos >= articulos.length) return false; // Lleno
            if (buscarPorCodigo(a.getCodigo()) != null) return false; // Ya existe

            articulos[totalArticulos++] = a;
            return true;
        }

        public Articulo buscarPorCodigo(String codigo) {
            if (codigo == null) return null;
            for (int i = 0; i < totalArticulos; i++) {
                if (articulos[i].getCodigo().equalsIgnoreCase(codigo)) {
                    return articulos[i];
                }
            }
            return null;
        }

        public boolean eliminarArticulo(String codigo) {
```

```
            for (int i = 0; i < totalArticulos; i++) {
                if (articulos[i].getCodigo().equalsIgnoreCase(codigo)) {
                    // Compactar hacia la izquierda
                    for (int j = i; j < totalArticulos - 1; j++) {
                        articulos[j] = articulos[j + 1];
                    }
                    articulos[totalArticulos - 1] = null; // Limpiar para Garbage Collector
                    totalArticulos--;
                    return true;
                }
            }
            return false;
        }

        public double calcularValorTotalInventario() {
            double total = 0.0;
            for (int i = 0; i < totalArticulos; i++) {
                total += articulos[i].getPrecio() * articulos[i].getStock();
            }
            return total;
        }

        // Copia defensiva del array disponible
        public Articulo[] getArticulosDisponibles() {
            return Arrays.copyOf(this.articulos, this.totalArticulos);
        }

        public int getTotalArticulos() { return totalArticulos; }
        public int getCapacidadMaxima() { return articulos.length; }
    }

    public static void main(String[] args) {
        GestorAlmacen almacen = new GestorAlmacen(5);

        Articulo a1 = new Articulo("TEC-101", "Teclado Mecánico RGB", 59.99, 15);
        Articulo a2 = new Articulo("RAT-202", "Ratón Óptico Ergonómico", 29.50, 20);
        Articulo a3 = new Articulo("MON-303", "Monitor 27 Pulgadas IPS", 189.00, 8);

        almacen.registrarArticulo(a1);
        almacen.registrarArticulo(a2);
        almacen.registrarArticulo(a3);

        System.out.println("Artículos registrados: " + almacen.getTotalArticulos());
        System.out.printf("Valor total del inventario: %.2f €%n", almacen.calcularValorTotal
Inventario());

        // Operaciones de venta
        System.out.println("\nVenta de 5 teclados: " + (a1.vender(5) ? "Éxito" : "Rechazad
a"));
        System.out.println("Venta de 50 monitores (insuficiente): " + (a3.vender(50) ? "Éxit
o" : "Rechazada"));
        System.out.println(a1);

        // Eliminación y compactación
        System.out.println("\nEliminando ratón (RAT-202): " + almacen.eliminarArticulo("RAT-
202"));
        System.out.println("Total artículos tras borrado: " + almacen.getTotalArticulos());

        // Listar copia defensiva
        System.out.println("\nListado actual del almacén:");
        for (Articulo a : almacen.getArticulosDisponibles()) {
```

```
            System.out.println(" - " + a);
        }
    }
}
```

DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.3: Retos Orientados a Objetos y Programación Competitiva

Retos avanzados de modelado orientado a objetos, algoritmos sobre estructuras de objetos y diseño de arquitecturas en Java 25 LTS.

Bienvenido al boletín de retos avanzados de la **Unidad 4: Clases y objetos**.

> 💡 ***Entrenamiento adicional recomendado:*** *Busca en CodeWars katas de nivel **7-6 kyu** etiquetadas con `Object-oriented Programming`, y en Acepta el Reto problemas de modelado y estructuración de datos. A continuación tienes **8 retos completos** diseñados con el mismo espíritu competitivo y de ingeniería de software, incluyendo pistas algorítmicas y soluciones íntegras comentadas en Java 25 LTS.*

Reto 1: La Baraja Española (40 cartas) y el Algoritmo de Fisher-Yates

Contexto y Planteamiento

Diseña un simulador completo de la baraja española de **40 cartas** (palos `"Oros"`, `"Copas"`, `"Espadas"` y `"Bastos"`; valores del `1` al `12` excluyendo el `8` y el `9`, o numerados del `1` al `10` para Sota, Caballo y Rey) aplicando tanto el enfoque clásico de clases (`Carta` y `Mazo`) como el enfoque moderno con `enum` y `record`:

1. Un `enum Palo` que defina los cuatro palos tradicionales (`OROS, COPAS, ESPADAS, BASTOS`).
2. Un `enum Valor` con los 10 valores de la baraja española y su puntuación numérica asociada.
3. Un `record Carta(Palo palo, Valor valor)` inmutable con un método `toString()` descriptivo (ej. `"As de OROS"`).
4. Una clase `Baraja` (o `Mazo`) que contenga el array de **40 objetos** `Carta`:
   - Debe inicializar las 40 combinaciones en el constructor (4 palos × 10 valores).
   - Método `public void barajar()`: reordena el array al azar mediante el **algoritmo in-place de Fisher-Yates / Knuth**, garantizando que todas las permutaciones sean equiprobables en tiempo lineal `O(n)` sin crear arrays auxiliares.
   - Método `public Carta robar()`: retira y devuelve la carta superior de la baraja (lanzando `IllegalStateException` si la baraja está agotada).
   - Método `public Carta[] repartir(int n)` (o `repartirMano(int n)`): devuelve un array con las primeras `n` cartas disponibles del mazo.

> **Pista para barajar (Algoritmo de Fisher-Yates)**
>
> Recorre el array de atrás hacia adelante (desde `i = cartas.length - 1` hasta `1`), y en cada posición intercambia la carta actual `cartas[i]` con una carta elegida en una posición aleatoria anterior o igual (`j` entre `0` e `i` inclusive).

▶ 💡 Ver Solución Comentada (Versión Moderna con Enum/Record y Versión Clásica)

Reto 2: Máquina Expendedora Automática con Algoritmo Voraz de Cambio

Contexto y Planteamiento

Diseña un sistema orientado a objetos para controlar una máquina expendedora automática con monedas reales:

1. `record Articulo(String codigo, String nombre, int precioCentimos, int stock)` (inmutable o con stock modificable).
2. Clase `MaquinaExpendedora`:
    - Gestiona un catálogo de artículos indexados por código.
    - Gestiona un saldo acumulado introducido por el cliente (en céntimos).
    - Método `public void insertarMoneda(int centimos)` (solo admite monedas de 5, 10, 20, 50 céntimos, 1€ y 2€).
    - Método `public Articulo comprar(String codigoArticulo)`: si el saldo es suficiente y hay stock, despacha el artículo, descuenta el stock y calcula el cambio restante.
    - Método `public int[] devolverCambio()`: descompone el cambio restante en el número mínimo de monedas de cada valor utilizando un **algoritmo voraz (*Greedy Algorithm*)** en denominaciones de 200, 100, 50, 20, 10 y 5 céntimos.

▶ 💡 Ver Solución Comentada

Reto 3: Sistema Bancario con Clientes y Múltiples Cuentas (Asociación Bidireccional)

Contexto y Planteamiento

Modela la relación entre un `Cliente` y sus `CuentasBancarias`:

- Un `Cliente` puede poseer hasta 3 cuentas bancarias distintas (ahorro, corriente, nómina).
- Método `vincularCuenta(CuentaBancaria c)` que asocie la cuenta al cliente evitando sobrepasar el límite de 3.
- Método `public double calcularPatrimonioTotal()` que compute la suma acumulada de saldos de todas las cuentas del cliente.
- Método `public static boolean transferir(CuentaBancaria origen, CuentaBancaria destino, double cantidad)`: operación atómica que verifique saldo, realice la retirada de origen y el abono en destino.

▶ 💡 Ver Solución Comentada

Reto 4: Calculadora de Polinomios Inmutables

Contexto y Planteamiento

Modela una clase inmutable `Polinomio` para manipular expresiones matemáticas del tipo $P(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$:

- El array interno de coeficientes almacena en el índice `i` el coeficiente del término de grado `i`.
- Constructor con array de coeficientes (realizando copia defensiva y recortando ceros superiores redundantes).
- Método `public double evaluar(double x)`: evalúa el valor numérico de `P(x)` mediante el **algoritmo de Horner** (`O(N)`).
- Método `public Polinomio sumar(Polinomio otro)`: devuelve un nuevo polinomio suma.
- Método `public Polinomio derivar()`: devuelve el polinomio derivada formal de `P(x)`.
- Sobrescribe `toString()` formateando algebraicamente la expresión matemática (ej. `"3.0x^2 - 4.0x + 5.0"`).

▶ 💡 Ver Solución Comentada

Reto 5: Simulación de Ecosistema en Tablero Matricial

Contexto y Planteamiento

Modela una simulación matricial de vida celular en un tablero de `FILAS × COLUMNAS`:

- `enum Especie { VACIA, HIERBA, CONEJO }`
- Cada casilla contiene un objeto `Organismo(Especie especie, int energia)`.
- En cada turno de la simulación:
  1. La `HIERBA` gana 1 punto de energía.
  2. El `CONEJO` pierde 1 punto de energía. Si su energía llega a 0, muere (se convierte en `VACIA`).
  3. Si un `CONEJO` tiene adyacente una casilla con `HIERBA`, se desplaza a ella, devora la hierba y suma su energía.
- Diseña la clase `TableroEcosistema` y simula 5 ciclos mostrando la evolución en consola.

▶ 💡 Ver Solución Comentada

Reto 6: Cola de Impresión Segura con Prioridades (Singleton + FIFO)

Contexto y Planteamiento

En sistemas operativos y servidores de red, el gestor de impresión (*Print Spooler*) debe ser único en toda la máquina (Patrón Singleton) y procesar trabajos ordenados según una cola de prioridad:

1. `record Documento(String id, String nombre, int paginas, int prioridad)` (prioridad 1 = Urgente, 2 = Normal, 3 = Baja).
2. Clase `SpoolerImpresion` (Singleton):
   - Constructor estrictamente privado.
   - Array interno de trabajos pendientes con capacidad máxima para 20 documentos.
   - Método `public boolean encolar(Documento doc)`: inserta el documento de forma ordenada según su prioridad (menor número = mayor prioridad) respetando el orden de llegada para igual prioridad.
   - Método `public Documento imprimirSiguiente()`: extrae y procesa el trabajo con máxima prioridad.
   - Método `public int getTrabajosPendientes()`.

▶ 💡 Ver Solución Comentada

Reto 7: Álgebra de Conjuntos con Varargs (`ConjuntoEnteros`) y Análisis de Complejidad

Contexto y Planteamiento

Implementa una estructura de datos propia llamada `ConjuntoEnteros` que represente un conjunto matemático finito de números enteros **sin elementos repetidos**, respaldado internamente por un array dinámico `int[]`:

1. **Constructor con `varargs`** (`public ConjuntoEnteros(int... elementos)`): crea el conjunto a partir de una secuencia arbitraria de enteros (o vacío si no se pasan argumentos), descartando automáticamente cualquier valor duplicado.
2. **Operaciones básicas y con `varargs`**:
   - `public int getTamano()` y `public boolean vacio()`.

- `public boolean pertenece(int... elementos)`: devuelve `true` si **todos** los elementos pasados por parámetro pertenecen al conjunto.
- `public boolean inserta(int... elementos)`: añade los elementos que aún no pertenezcan al conjunto, redimensionando el array interno solo en la cantidad exacta necesaria. Devuelve `true` si el conjunto ha cambiado (al menos un elemento nuevo insertado).
- `public boolean borra(int... elementos)`: elimina del conjunto todos los elementos indicados que estén presentes y compacta el array. Devuelve `true` si el conjunto se ha modificado.

3. **Operaciones algebraicas inmutables** (devuelven un nuevo `ConjuntoEnteros` sin alterar los operandos):
   - `public ConjuntoEnteros union(ConjuntoEnteros otro)`: elementos presentes en `this` o en `otro` (`A ∪ B`).
   - `public ConjuntoEnteros interseccion(ConjuntoEnteros otro)`: elementos comunes a ambos conjuntos (`A ∩ B`).
   - `public ConjuntoEnteros menos(ConjuntoEnteros otro)`: diferencia de conjuntos (`A \ B`), es decir, elementos de `this` que no pertenecen a `otro`.
4. **Identidad de `Object`**: sobrescribe `toString()` (ej. `"{1, 4, 7}"`), `equals(Object obj)` (dos conjuntos son iguales si tienen el mismo tamaño y exactamente los mismos elementos, **sin importar el orden** en que estén almacenados) y `hashCode()` (cuyo cálculo debe ser conmutativo para que el orden no altere el hash).
5. **Pregunta de reflexión algorítmica:** ¿Cuál es el coste temporal en notación asintótica (`O(...)`) de `pertenece`, `union`, `interseccion` y `equals` sobre un array desordenado frente a mantener el array interno ordenado y usar búsqueda binaria?

▶ 💡 Ver Solución Comentada y Análisis de Complejidad

Reto 8: Clínica Dietista (Cálculo de IMC) y Validador de Cuenta Bancaria Española (CCC)

Contexto y Planteamiento

Diseña un sistema integral de gestión de pacientes para una clínica de nutrición que combine el modelado de dominio con el algoritmo oficial de validación bancaria española:

1. **Clase `Paciente` y `enum Sexo { HOMBRE, MUJER }`**:
   - Atributos privados: `numPaciente` (código alfanumérico de 8 cifras + letra generado automáticamente al instanciar mediante un método privado `generaNumPaciente()`), `nombre`, `edad`, `sexo` (por defecto `MUJER`), `peso` (kg) y `altura` (metros), más su número de cuenta domiciliada `ccc`.
   - Constantes estáticas de diagnóstico nutricional: `INFRAPESO = -1`, `PESO_IDEAL = 0` y `SOBREPESO = 1`.
   - Tres constructores encadenados con `this(...)`: por defecto, con `(nombre, edad, sexo)` y completo con todos los parámetros.
   - Método `public int calcularIMC()`: calcula `IMC = peso / (altura²)` y devuelve `INFRAPESO` (`< 20`), `PESO_IDEAL` (`20` a `25` incluidos) o `SOBREPESO` (`> 25`).
   - Método `public boolean esMayorDeEdad()`: devuelve `true` si `edad >= 18`.
2. **Validador de Código Cuenta Cliente (`EEEE-SSSS-DC-CCCCCCCCCC`)**:
   - Método estático `public static boolean validarCCC(String ccc)` que verifique mediante expresión regular el formato de 20 dígitos (`4` de entidad, `4` de sucursal, `2` dígitos de control y `10` de número de cuenta) y compruebe matemáticamente los **dos dígitos de control (DC)** usando el algoritmo oficial de pesos módulo 11 (`{1, 2, 4, 8, 5, 10, 9, 7, 3, 6}`).

▶ 💡 Ver Solución Comentada